THÈSE

Pour obtenir le grade de

DOCTEUR DE L'UNIVERSITÉ GRENOBLE ALPES

École doctorale : EEATS - Electronique, Electrotechnique, Automatique, Traitement du Signal (EEATS)
Spécialité : Automatique - Productique
Unité de recherche : Laboratoire de conception et d'intégration des systèmes

Commande sous contraintes des systèmes non linéaires via platitude: théorie et expériences

Constrained control of nonlinear systems via flat mappings: theory and experiments

Présentée par :

Huu-Thinh DO

Direction de thèse :

Ionela PRODAN — Directrice de thèse
MAITRE DE CONFERENCES, GRENOBLE INP - UGA

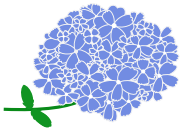
Rapporteurs :

Antoine GIRARD
DIRECTEUR DE RECHERCHE, CNRS DELEGATION ALPES
Ilya KOLMANOVSKY
FULL PROFESSOR, Michigan State University

Thèse soutenue publiquement le **11 décembre 2024**, devant le jury composé de :

Antoine GIRARD, DIRECTEUR DE RECHERCHE, CNRS DELEGATION ALPES	Président
Ilya KOLMANOVSKY, FULL PROFESSOR, Michigan State University	Rapporteur
Hugues MOUNIER, PROFESSEUR DES UNIVERSITES, Université Paris-Saclay	Examinateur
Florentina NICOLAU, MAITRESSE DE CONFERENCES, ENSEA	Examinatrice
Sylvain BERTRAND, INGENIEUR DE RECHERCHE, ONERA	Examinateur
Mirko FIACCHINI, CHARGE DE RECHERCHE HDR, CNRS DELEGATION ALPES	Examinateur
Franco BLANCHINI, FULL PROFESSOR, Università degli Studi di Udine	Examinateur

Acknowledgment



This little "journal" marks a significant milestone in my pursuit of both scientific knowledge and personal development. It is certain that I could never have gone this far without the help of many people. Let me take this opportunity to express my deepest gratitude to those whose support and encouragement build the foundation of my PhD.

First and needless to say, to Assoc. Prof. Ionela Prodan, my advisor, my career coach, and my friend: with your guidance, I am inspired and have grown so much personally and professionally. You are the one who granted me this PhD and the numerous opportunities that came with it. Thank you for always pushing me forward, thank you for always *kicking* me out of my comfort zone, and thank you for believing in me. It will cost another thesis to put into words my gratitude for you. It is a great honor and pleasure working with you. I would like to thank you simply for everything.

My PhD years would never have been as wonderful if they had not been filled with many insightful discussions, which literally come from all over the world. I am very grateful to Prof. Kolmanovsky and his group. Thank you for giving me the cutting-edge insights on reference governors. I thank also Prof. Stoican and his group for giving me concrete foundation set-based computation. I am as well truly indebted to the people who I am delighted to work with. I owe my scientific accomplishments to my advisor Assoc. Prof. Ionela Prodan, Prof. Franco Blanchini, Prof. Stefano Minani, Prof. Florin Stoican and Assoc. Prof. Florentina Nicolau. Thank you for all the fruitful discussions and for being patient with all my (silly) questions.

My sincere thanks are also due to the jury members who agreed to serve on the committee: Prof. Ilya Kolmanovsky, DR. CNRS Antoine Girard, Prof. Hugues Mounier, Cr. CNRS Mirko Fiacchini, Assoc. Prof. Florentina Nicolau, Prof. Franco Blanchini,

and Dr. Sylvain Bertrand. I am thankful for the time and expertise you have generously offered for my defense. I would especially want to thank the two reviewers, DR. CNRS Girard and Prof. Kolmanovsky, for carefully reading the thesis. Your comments considerably enhanced the manuscript's overall quality.

I am also grateful for all my lab members. The LCIS laboratory is one of the best places to do a PhD with excellent colleagues from all over the world. Being a part of LCIS has truly been a privilege. Thanks to all present and past members who make LCIS a special place, I wish you the best for your continuation.

I wish to give my very special thanks to all my friends. I want to thank Tien, Phuong, Huong, Hai, Triet, Vinh, and Vu for always listening to my messy thoughts; Vincent for being my personal tour guide all over the world; Maximilian, Bogdan, Theodor, and Tri for understanding my confusing enumerations of ideas, hopefully with no judgment; Khanh for being a thoughtful office-mate; co Le, anh Quang, chi Lan-Anh, chi Thuan, chi Hien and Bastien for all your 'emotional support' Vietnamese food. Thanks to Nguyen, Ngoc and Hieu for your comforting holidays at Gap, staying with you gives me so much energy. Thanks to all my Vietnamese friends who make Valence feel like home and give me confidence in every daily life aspect.

Finally, to my amazing family: thank you for just being there. Without you, I would not have made it here. I would like to thank my parents for trusting me, for being proud of me, and for loving me. I want to thank my sister, Trang, for looking after me, your caring reminds me of how lucky I am. I am grateful also for my little brother, thank you for taking care of our family, I am really proud of you.

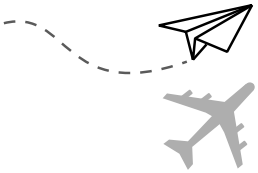
Let this thesis be my anchor, reminding me of our wonderful journey.

I'm grateful for everything.

December 2024, Valence, France.

Huu-Thinh Do
Đỗ Hữu Thịnh

Kính tặng bố mẹ!



"I walk slowly, but I never walk backwards."

Abraham Lincoln

Abstract

In today's landscape, despite advancements in powerful optimal control techniques, there remains a critical need for low-complexity controllers for nonlinear systems that can ensure stability and satisfy complex operating constraints. These controllers offer computationally efficient solutions, essential for real-time applications where accuracy, safety, and guaranteed performance are paramount.

To reduce the control complexity, one of the customary methods is to alleviate model nonlinearity. With this goal, the theory of differentially flat systems has proven advantageous by offering two key benefits: feedback linearization and differential parameterization. These benefits are achieved through a specific choice of output called the flat output and a linearizing coordinate transformation. While this approach simplifies the system dynamics, it often distorts state or input constraints in the new coordinates, complicating their validation with standard optimization methods. Furthermore, selecting the optimal flat output for control remains a significant challenge.

This thesis addresses these issues by developing methods to handle complex constraints and reduce computational complexity in constrained nonlinear control problems. It also extends beyond flat systems, utilizing Lyapunov-based arguments for control affine systems to ensure stability and constraint satisfaction.

To address complex constraints in the transformed coordinate system, the constraints are underapproximated into convex sets, linear inequalities, or mixed-integer (MI) constraints. For quadcopter position control, various inner approximations of these constraints are explored, enabling efficient control strategies such as implicit, explicit, and robust tube Model Predictive Control (MPC), as well as adaptive saturated control designs. These approaches reduce online complexity while maintaining stability, though they require trade-offs in terms of conservatism within the feasible domain.

Additionally, since linearity implies affinity in the control input, a method is developed to artificially generate a control Lyapunov function (CLF) using linear programming, which is then applied for online control. Stabilizing the system in this way reduces

computational overhead, with a trade-off of some optimality for performance. A novel concept of sub-control Lyapunov functions is introduced to formalize this compromise. Theoretical findings are validated through extensive simulations and experimental tests using a nano-drone testbed.

Moreover, to generalize the handling of complex constraints in the flat output space, Rectified Linear Neural Networks are employed to decompose the constraint set into MI linear constraints. This enables the use of established optimization-based strategies like MPC and CLF-based control. While this introduces non-convexity, simulations show that the constraints can still be efficiently managed by MI linear and MI quadratic programming solvers.

Regarding the selection of flat outputs, B-spline parameterization is applied to explore the optimal trajectory generation for an unmanned aerial vehicle model. Results indicate that different flat outputs result in varied cost and constraint formulations, producing trajectories with distinct characteristics. Moreover, with extended state observers for output feedback, it is shown that disturbances propagate differently depending on the flatness-based mappings, leading to minimal robust positive invariant sets of varying sizes. This lays the groundwork for future research of selecting the optimal flat output.

Through extensive experiments, the thesis bridges the gap between theory and practice by applying advanced control theory concepts to real-world motion planning scenarios. It demonstrates how theoretical results, such as those combining flatness, MPC, and CLF, can be effectively implemented, emphasizing their versatility and appeal in solving complex, practical problems across various domains.

Résumé

Malgré les progrès réalisés dans les approches de commande basées sur l'optimisation, il existe encore aujourd'hui un besoin crucial de contrôleurs à faible complexité pour les systèmes non linéaires capables d'assurer la stabilité et de satisfaire des contraintes opérationnelles complexes. Ces contrôleurs offrent des solutions efficaces sur le plan informatique, essentielles pour les applications en temps réel où la précision, la sécurité et des performances garanties sont importantes.

Afin de réduire la complexité de la commande, l'une des méthodes habituelles consiste à alléger la non-linéarité du modèle. Dans ce but, la théorie des systèmes différentiellement plats s'est avérée avantageuse en offrant deux avantages clés : la linéarisation par retour d'état et la paramétrisation différentielle. Ces avantages sont obtenus grâce à un choix spécifique de sortie appelé sortie plate et une transformation de coordonnées linéarisante. Bien que cette approche simplifie la dynamique du système, elle déforme souvent les contraintes d'état ou d'entrée dans les nouvelles coordonnées, compliquant leur validation avec les méthodes d'optimisation standard. De plus, le choix de la sortie plate optimale pour la conception de commande reste un défi important.

Cette thèse aborde ces problèmes en développant des méthodes pour gérer les contraintes complexes et réduire la complexité de calcul dans les problèmes de contrôle non linéaire contraints. Elle va également au-delà des systèmes plats, en utilisant des arguments basés sur Lyapunov pour les systèmes affines de contrôle afin d'assurer la stabilité et la satisfaction des contraintes.

Pour traiter les contraintes complexes dans le système de coordonnées transformé, les contraintes sont simplifiées en ensembles convexes, inégalités linéaires ou contraintes mixtes en nombres entiers (mixed integer–MI). Pour le contrôle de position des quadricoptères, diverses approximations internes de ces contraintes sont explorées, permettant des stratégies de contrôle efficaces telles que la commande prédictive (model predictive control–MPC) implicite, explicite et robuste, ainsi que des conceptions de commande saturée adaptative. Ces approches réduisent la complexité en ligne tout en préservant la stabilité, bien qu'elles nécessitent des compromis en termes de conservatisme dans le domaine réalisable.

De plus, puisque la linéarité implique l'affinité dans l'entrée, une méthode est développée pour générer artificiellement une fonction de Lyapunov contrôlée (control Lyapunov function–CLF) en utilisant la programmation linéaire, qui est ensuite appliquée pour la commande en ligne. La stabilisation du système de cette manière réduit le coût de calcul, avec un compromis en termes d'optimalité des performances. Un nouveau concept de sous-fonctions de Lyapunov contrôlées (sub-control Lyapunov function–SCLF) est introduit pour formaliser ce compromis. Les résultats théoriques sont validés par des simulations et des tests expérimentaux approfondis à l'aide d'un banc d'essai de nano-drones.

En outre, pour généraliser le traitement des contraintes complexes dans l'espace de sortie plate, des réseaux neuronaux linéaires rectifiés sont utilisés pour décomposer l'ensemble des contraintes en contraintes linéaires MI. Cela permet d'utiliser des stratégies d'optimisation établies comme le MPC et le contrôle basé sur la CLF. Bien que cela introduise de la non-convexité, les simulations montrent que les contraintes peuvent toujours être gérées efficacement par des solveurs de programmation linéaire MI et quadratique MI.

En ce qui concerne le choix des sorties plates, le paramétrage B-spline est appliqué pour explorer la génération optimale de trajectoires pour un modèle de véhicule aérien sans pilote. Les résultats indiquent que différentes sorties plates entraînent des formulations de coût et de contrainte variées, produisant des trajectoires aux caractéristiques distinctes. De plus, lors de l'utilisation d'estimateur d'état étendu, il est démontré que les perturbations se propagent différemment selon les transformations basées sur la platitude, résultant en des ensembles invariants positifs robustes minimaux de tailles variables. Ce résultat ouvre la voie à de futures recherches sur le choix optimal de la sortie plate dans la conception de la commande.

En résumé, à travers des expériences approfondies, la thèse comble l'écart entre la théorie et la pratique en appliquant des concepts avancés de théorie de commande à des scénarios de planification de mouvement réels. Elle démontre comment les résultats théoriques, tels que ceux combinant la platitude, le MPC et le CLF, peuvent être efficacement mis en œuvre, soulignant leur polyvalence et leur attrait dans la résolution de problèmes complexes et pratiques dans divers domaines.

Le Chapitre 1 sert de préambule à l'outil théorique principal du manuscrit : la théorie de la platitude différentielle. Bien que la définition formelle ne sera pas rappelée ici, les propriétés caractéristiques de la classe seront décrites de manière intuitive, couvrant des aspects allant de la planification de trajectoire à la conception de rétroaction. Cette description sera à son tour étayée par un bref aperçu des applications à large spectre de la théorie dans des scénarios réels. Les questions pertinentes liées au sujet seront également mises en évidence, établissant le cadre directeur pour le manuscrit. De plus, avec l'objectif clé de gérer les contraintes tout en maintenant une faible charge de calcul, l'utilisation de fonctions de Lyapunov contrôlées sera également abordée, étendant le contexte à une classe plus large de systèmes non linéaires. Conformément à ces orientations, ce chapitre présentera les contributions et l'organisation de la thèse.

Le Chapitre 2 offre un aperçu complet des avantages de la théorie des systèmes différentiellement plats. Outre l'explication de l'idée de base et des propriétés de la platitude, nous discutons également des travaux antérieurs de planification de mouvement dans ce domaine et des outils auxiliaires qui peuvent être utilisés pour exploiter la théorie. Les moyens de base comprennent les cadres de paramétrisation de courbe et le contexte théorique des ensembles à utiliser dans les approches basées sur l'optimisation.

Le Chapitre 3 abordera le problème des contraintes complexes causées par la linéarisation par rétroaction à partir de la platitude, en particulier pour l'application du quadricoptère. Ensuite, les réseaux de neurones seront explorés comme approximateur pour un espace de sortie plat compliqué en général. Dans le détail, le chapitre commence par une étude des contraintes sur la dynamique de translation du quadricoptère dans une architecture de contrôle hiérarchique. Ensuite, nous présentons un algorithme d'approximation du domaine déformé à l'aide de zonotopes paramétrés. Cette approximation permet la mise en œuvre MPC linéaire standard. De plus, l'approximation construite sera adaptée pour s'aligner avec le cadre MPC explicite afin d'accélérer le calcul en ligne, bien qu'au prix d'une approximation interne plus conservative. Ensuite, nous fournissons une fonction de saturation basée sur l'optimisation pour l'ensemble de contraintes non linéaire, permettant la conception d'un contrôle saturé explicite. Enfin, le chapitre se termine par l'exploration de l'application des réseaux de neurones comme outil polyvalent pour l'approximation des fonctions, en particulier dans le contexte d'un contrôleur de gestion des contraintes dans le système de coordonnées linéarisant.

Le Chapitre 4 étudie une technique de réduction de la complexité en temps réel avec le contrôle basé sur la CLF pour l'application du quadricoptère. Plus précisément, le MPC sera utilisé pour générer des trajectoires en boucle fermée stables en simulation. Ces trajectoires sont ensuite échantillonnées pour formuler un programme linéaire semi-infini (SILP) afin de paramétrer une candidate CLF. La CLF est ensuite validée expérimentalement via le problème de stabilisation du quadricoptère. Ensuite, pour formaliser les résultats validés expérimentalement, le concept de SCLF est introduit. Ce dispositif est ensuite suivi d'un algorithme plus efficace sur le plan informatique basé sur la technique des plans sécants, permettant l'application de systèmes de taille non triviale. Les résultats et les avantages des schémas sont validés à la fois en simulations et en expériences.

Le Chapitre 5 aborde les questions complémentaires qui se posent au cours de la thèse. Le chapitre comprend d'abord la robustesse des schémas MPC en adaptant la configuration caractérisée du quadricoptère au cadre MPC tube. Ensuite, en ce qui concerne le contrôle saturé, nous fournissons un schéma adaptatif pour identifier le gain satisfaisant pour le contrôleur, suivi de l'analyse de la robustesse du design. Les deux adaptations sont validées expérimentalement avec la plate-forme Crazyflie 2.1. Enfin, l'effet du changement de la sortie plate dans la conception de la trajectoire et du contrôle sera étudié via les simulations d'un modèle de véhicule aérien.

Le Chapitre 6 sert de conclusion à la thèse et explore les directions futures.

Contents

Abstract iv

Resumé vi

List of Figures xiv

List of Tables xix

List of Algorithms xxii

Notation xxiii

Acronyms xxv

1 Introduction 1

1.1 Low complexity constrained control 2

1.1.1 Differential flatness in planning and control 2

1.1.2 Beyond flatness: a CLF-based perspective 5

1.2 Thesis orientation 6

1.3 Thesis contributions 7

1.4 Thesis organisation 11

2 Curves, sets and flatness in optimization-based motion planning 13

2.1 Motivation . . . 14

2.2 Differentially flat systems . . . 14

2.2.1 Definition and properties . . . 14

2.2.2 Computation of the flat output and its non-uniqueness . . . 19

2.3 Flatness-based trajectory generation . . . 22

2.3.1 Parameterization tools . . . 24

2.3.2 Optimal trajectory generation with flatness . . . 29

2.4 Optimization-based control settings . . . 39

2.4.1 Sets and invariance notions . . . 40

2.4.2 Optimization-based control schemes . . . 46

2.5 Concluding remarks . . . 50

3 Constraint characterization for control in the flat space 51

3.1 Motivation . . . 53

3.2 The quadcopter's outer loop in the flat space . . . 56

3.2.1 Hierarchical control and constraint characterization . . . 56

3.2.2 Input constraints characterization in the flat output space . . . 59

3.2.3 Experimental platform description . . . 65

3.3 Computationally efficient control in the flat output space . . . 67

3.3.1 Implicit model predictive control . . . 67

3.3.2 Explicit model predictive control . . . 68

3.3.3 Explicit saturated control . . . 70

3.4 Simulations and experimental validation . . . 77

3.4.1 Implicit model predictive control . . . 78

3.4.2 Explicit model predictive control . . . 82

3.4.3 Explicit saturated control . . . 90

3.4.4 Discussions and comparisons 97
3.5 Towards a generalized constraint approximation with neural networks . . 99
3.5.1 Preliminaries and problem statement 99
3.5.2 Constraint characterization with MIP and applications in control . 101
3.5.3 Simulation study 106
3.6 Concluding remarks 110

4 Low complexity control synthesis with stability guaranteed 112
4.1 Motivation 113
4.2 Efficient control via LP-generated CLFs 115
4.2.1 Main idea and machinery 115
4.2.2 Admissible pair generator and control implementation 122
4.2.3 Application on quadcopter position control 124
4.2.4 Experimental validation 129
4.3 Real-time complexity reduction with sub-control Lyapunov functions . . . 133
4.3.1 Sub-Control Lyapunov Functions 134
4.3.2 Computation of a Sub-Control Lyapunov Function 141
4.3.3 Numerical examples and experimental validation 147
4.4 Concluding remarks 155

5 Insights on robustness and different flat outputs analysis 157
5.1 Motivation 159
5.2 Flatness-based tube MPC for quadcopter trajectory tracking 160
5.2.1 Robust Tube MPC formulation for a quadcopter 160
5.2.2 Experimental validation 163
5.3 Flatness-based adaptive saturated control for quadcopter outer loop . . . 168
5.3.1 Saturated control with adaptive gain 168

5.3.2 Robustness analysis . . . 171
5.3.3 Experimental validation . . . 174
5.4 Analysis of different flat outputs for the fixed-wing UAV model . . . 178
5.4.1 Various flat representations of the fixed-wing UAV model . . . 178
5.4.2 Trajectory generation using flatness . . . 180
5.4.3 Tracking control with feedback linearization . . . 183
5.4.4 Simulation result . . . 184
5.5 Robust flatness-based control for a fixed-wing UAV . . . 190
5.5.1 Output feedback for flat systems with extended state observers . . . 190
5.5.2 Application for a fixed-wing UAV . . . 194
5.5.3 Simulation results . . . 198
5.6 Concluding remarks . . . 202

6 Conclusions and future developments **203**
6.1 Conclusion . . . 203
6.2 Future developments . . . 205

Appendices **209**

A Appendix **209**
A.1 Largest box inscribed in $\mathcal{V}_c$ . . . 209
A.2 PWA-MPC formulation . . . 210
A.3 Proof of Remark 4.5 . . . 210
A.4 Feasibility-based bound tightening algorithm . . . 211

Bibliography **212**

List of Figures

1.1 Applications belonging to the class of differentially flat systems. 3
1.2 The brief descriptions and the dependencies between the thesis chapters. . 12

2.1 Planar fixed-wing UAV model and its variables' trajectories. 15
2.2 Trajectory of a differentially flat system in the original (left) and in the flat output space (right). 17
2.3 Relations between related classes of feedback linearizable systems. 18
2.4 Two different singular cases for different flat output parameterizations of the UAV model from the flat output (2.5) (left) and (2.18) (right). 21
2.5 Different possible roles of flatness in motion planning. 22
2.6 Infeasible (top) and feasible (bottom) trajectory of the UAV model. . . . 23
2.7 Interpretation of the conversion from the OCP (2.20) (top) to the curve parameterization (2.21) (bottom). 24
2.8 Bézier curve $\boldsymbol{z}(t) = [z_1(t), z_2(t)]^\top$ constructed from (2.26). 26
2.9 B-spline basis functions with the knot vector as in (2.31), $d = 3, n = 6$. . . 28
2.10 B-spline curve ($d = 3$) with the local support property and the polygons inscribing the curves. 29
2.11 The parameterization passing through way-points and avoiding collision at the collocation time. 34
2.12 Collision-free curve generation by excluding the control polygons from the obstacle. 35
2.13 Admissible set of $\boldsymbol{u}$ with the UAV model (2.4) in the original state (left) and the flat output space (right) 37
2.14 Constraint deformed in the flat output space and the set annexion problem for inner approximation. 38
2.15 Two equivalent representations of an ellipsoidal set with $\boldsymbol{c} = \boldsymbol{0}$. 41
2.16 Polytope and its dual representations. 42
2.17 Largest parameterized zonotope $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$ and ellipsoid $\mathcal{E}(\boldsymbol{P}, \rho)$ inscribed in a polytope $\mathcal{H}$. 44

3.1 General scheme of flatness-based feedback linearization control. 53
3.2 The decoupled dynamics of a quadcopter and its hierarchical control scheme. 56

3.3 General scheme for quadcopter outer-loop control in the flat output space. 58
3.4 Quadcopter's input constraints in the flat output space with $\psi = 0$ (rad), $g = T_{max}/1.45 = 9.81$ (m/s^2), $\phi_{max} = \theta_{max} = \pi/18$ (rad)). 60
3.5 Approximated constraint sets for the new input $\boldsymbol{v}$. 63
3.6 $\mathcal{V}_c$ in (3.14) and $\mathcal{V}_c^{\text{in}}$ in (3.22) with $s_1 = 7, s_2 = 3$, $g = T_{max}/1.45 = 9.81 m/s^2, \epsilon_{max} = 0.1745$ rad. 64
3.7 Hierarchical control architecture of the Crazyflie quadcopter. 65
3.8 LCIS lab experimental setup with optical motion capture system and the Crazyflie 2.1. 66
3.9 Explicit MPC scheme for the quadcopter position regulation. 69
3.10 Numerical result for the solution of (3.52) - a ball maximally inscribed in $\mathcal{V}_c$. 75
3.11 The projection of $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ into the subspace $(x, \dot{x})$ with different α. 76
3.12 Four proposed time-parameterized references. 79
3.13 RMS of tracking errors and computation time of the two controllers with different types of references (distinguished, respectively, by the subscript e and t under the name of the corresponding controller). 80
3.14 Computation time used for different references. 80
3.15 Crazyflie tracking error with flatness-based MPC and PWA-MPC (4 references) with $e_q \triangleq q^{\text{ref}} - q$, $q \in \{x, y, z\}$. 81
3.16 Input signals with their constraints (black dashed line). 81
3.17 Explicit MPC solution with different choices of N_p. 83
3.18 Comparison between the ExMPC for the proposed setup (Sce. 1), for the 6D model (Sce. 2) and the IMPC with the 6D model (Sce. 3). 84
3.19 Feasible static initial conditions of ExMPC (left - Sce. 1) and IMPC (right - Sce. 2) without state constraints. 86
3.20 The quadcopter position and input $\boldsymbol{u}_k$ (with limits shown in green solid lines) in experiment with ExMPC and IMPC. 87
3.21 Experimental trajectory of $\boldsymbol{v}(k)$ in Test 1, $N_p = 30$. 87
3.22 Computation time and RMS tracking errors with Explicit MPC and Implicit MPC in Test 1 (red) and Test 2 (blue). 88
3.23 Quadcopters' trajectory tracking with Explicit MPC (Square formation reference, dashed and color-coded with the corresponding drone). 89
3.24 Multiple quadcopters' trajectory tracking with Explicit MPC (Circular reference, dashed and color-coded with the corresponding drone). 89
3.25 The quadcopter's trajectory with different initial states (black points) implemented with different values of γ (projected into $(x, \dot{x})$ subspace). . . 91
3.26 Input $\boldsymbol{u}$ with its constraints in solid black line (left) and $\boldsymbol{v}$ (right) simulated with different values of γ. 92
3.27 Simulated trajectories in time with different values of γ, projected on the z axis. 92

3.28 The quadcopter's trajectories and the computation time of Algorithm 3.2 in comparison with the IPOPT solver ($\gamma = 15$) and FB-MPC method. . . 93
3.29 Ref. 1 (blue-dashed line) tracked with different values of γ. 95
3.30 Input $\boldsymbol{u}$ with different values of γ with the set-point tracking problem and its bounds $\mathcal{U}$ as in (3.5) (black dashed line). 95
3.31 Input $\boldsymbol{v}$ in the flat output space, with different values of γ in Ref. 1 tracking problem and its constraint set $\mathcal{V}_c$ as in (3.14) 96
3.32 Position tracking with different trajectories. 96
3.33 Input signal $\boldsymbol{u}$ in the tracking problem for Ref. 2, Ref. 3 and Ref. 4 with its bounds (black solid line). 97
3.34 FB-FL control with the MIP representation of the ANN. 100
3.35 Constraint characterization: from ReLU-ANN to MILP. 101
3.36 The formulation of the ReLU-ANN as MILP. 103
3.37 Stabilization with ANN-based constraint characterization. 107
3.38 Trajectory tracking with the MPC (3.84) for (3.86). 108
3.39 The input $u = \Phi_v(\cdot)$, the approximation $\Phi_{NN}(\cdot)$ and its predicted trajectories. 108
3.40 The displacement $x(t)$ and the computational cost w.r.t N_p. 109

4.1 Geometric interpretation of the stabilizing condition (4.6b) and the procedure to compute the CLF. 118
4.2 The surface of $V_i(\cdot)$ with different η and $\boldsymbol{F}$ (Case 1). 126
4.3 Generated CLF and its DOA for the first subsystem of (3.11) ($i = 1$). . . 127
4.4 Set-point tracking simulation with the first subsystem of (3.11) ($q = x$). . 127
4.5 Comparison between the control sampled from (4.27) (blue points) and the control (4.28) (pink surface) for Case 1, $i = 1$ or the x axis. 128
4.6 Admissible pairs (red) collected for Case 2. 128
4.7 Left: Set-point reference tracking experimented with the three scenarios. Right: The real input signals and their amplitude bound (dashed-dotted line). 131
4.8 Lyapunov function for Case 1 and 2 controllers in Scenario 1. 131
4.9 Position swapping with three Crazyflies using Case 2 controller. 132
4.10 The outer loop's input signals of the three quadcopters in Sce. 2 with the Case 2 controller. 132
4.11 Computation time in Scenario 1. 133
4.12 Pipeline for the SCLF construction based on cutting-plane technique. . . 145
4.13 Unit level set of some basis functions of the form (4.76) and a linear combination of them (red), with $\boldsymbol{x} \in \mathbb{R}^2, p = 2$. 147
4.14 The planar VTOL model 148
4.15 Largest violation found and build time (BT) for Bioreactor, VTOL and Drone model (top to bottom, respectively). 151

4.16 Snapshots of the SCLF candidates (left) and the verifier $\boldsymbol{\Omega}(\boldsymbol{\alpha}, \boldsymbol{x})$ as in (4.66) (right) during the execution of Algorithm 4.2 for the bioreactor model (4.77). . . . 151
4.17 Closed-loop trajectories (blue) and the level set of the SCLF (dotted black) of the bioreactor model. . . . 152
4.18 Planar VTOL stabilization with input constraints (the red box). . . . 153
4.19 Experiments for the quadcopter position tracking problem (3.10) with the generated SCLF. . . . 154
4.20 Suboptimal performance of the SCLF controller w.r.t. MPC in (4.59) for the VTOL model. . . . 154

5.1 Two references employed for the experimental validation. . . . 163
5.2 Two input $\boldsymbol{v}^{\text{ref}}$ of the two references and their bounding boxes. . . . 164
5.3 Effect of changing the gain K towards fast closed-loop convergence, projected into the first two axes of $\tilde{\boldsymbol{\zeta}}(k)$. . . . 165
5.4 Position tracking with the two references. . . . 166
5.5 Tracking error $e_q = q - q^{\text{ref}}, q \in \{x, y, z\}$ for Ref. 1 (left) and Ref. 2 (right). 166
5.6 Computation time for the proposed MPC scheme. . . . 167
5.7 Quadcopter's inputs for the tracking of Ref.1 and 2. . . . 167
5.8 Lyapunov function $V(\boldsymbol{\xi})$ (top), the evolution of $\gamma(t)$ and its upper bound, which is color-coded with μ in dashed-line (bottom). . . . 169
5.9 Real input $\boldsymbol{u}$ and the input $\boldsymbol{v}$ in the new coordinates. . . . 170
5.10 The system trajectories from various initial points, subject to disturbances (projected in $(x, \dot{x})$ subspace) . . . 172
5.11 Real input $\boldsymbol{u}$ and the new input $\boldsymbol{v}$ simulated with disturbances . . . 173
5.12 Set-point tracking errors ($e_q = q - q^{\text{ref}}, q \in \{x, y, z\}$) via different choices of adaptive gain μ (Test 1). . . . 175
5.13 Values of the Lyapunov function (top) and the evolution of $\gamma(t)$ (bottom) with different adaptive rate μ. . . . 176
5.14 Input $\boldsymbol{u}$ and $\boldsymbol{v}$ via different choices of adaptive gain μ. . . . 176
5.15 Position swapping with three Crazyflies (Test 2) . . . 177
5.16 $x_1(t)$, $x_2(t)$ and $x_3(t)$ with the 4 flat outputs. . . . 185
5.17 Input $u_1(t)$ and $u_2(t)$ with 4 flat outputs . . . 186
5.18 Position (x_1, x_2) planned with 4 flat outputs . . . 186
5.19 The the curves' length and objective function values in terms of the number of control points. . . . 187
5.20 Tracking error of $x_1(t), x_2(t)$ and $x_3(t)$ . . . 189
5.21 Extended state observer design for flat systems. . . . 192
5.22 Ultimate bounds tube projected in the Cartesian space. . . . 200
5.23 RMS of tracking errors under disturbances with different frequencies. . . . 200

6.1 Complexity reduction for nonlinear control: a flatness-based perspective. . 204

List of Tables

3.1 Parameters setup for $\mathcal{V}_c$ approximation . . . 63
3.2 Result specifications for $\tilde{\mathcal{V}}$ approximation . . . 64
3.3 Parameters for the invariant set construction . . . 76
3.4 Control parameters for the proposed scenarios . . . 79
3.5 Parameters for simulations and experimental tests . . . 84
3.6 Simulation results and numerical specifications . . . 85
3.7 Simulation result for $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35) . . . 90
3.8 CPU time of the proposed algorithm w.r.t IPOPT solver and FB-MPC approach . . . 91
3.9 Experiment specifications and results . . . 94
3.10 Comparison between the three controllers for system (3.10) in Chapter 3. 98
3.11 Numerical specifications . . . 107
3.12 Computation time of the simulations (ms). . . . 109

4.1 Parameters for point generator (4.27) with $\tau = 0.1$ (s), and the LP (4.6), $q \in \{x, y, z\}$, $N_p = 50$. . . . 125
4.2 Specifications of the LP (4.6) for the two cases . . . 129
4.3 Experiment results . . . 133
4.4 Numerical parameters used in the SCLF synthesis and online control . . . 149
4.5 Specifications and total build time (BT) for the three systems . . . 150
4.6 Comparison between SCLF-induced controller and MPC . . . 153

5.1 Controller's parameters and system's specification . . . 164
5.2 Experimental results and control parameters . . . 167
5.3 Controller's parameters for the quadcopter system . . . 170
5.4 Simulation parameters for robustness analysis . . . 174
5.5 Numerical parameters & results for test 2 . . . 175
5.6 Flat representations corresponding to different flat outputs. . . . 179
5.7 Data specification and computation results . . . 185
5.8 Simulation specifications . . . 188
5.9 Tracking error with different wind scenarios . . . 188
5.10 Simulation specifications . . . 199

5.11 Tracking performance with different controller and observer poles 199

List of Algorithms

3.1 $\mathcal{V}_c$ approximation procedure. 62

3.2 Compute the function $\text{sat}_\lambda(\boldsymbol{v})$ as in (3.33). 77

4.1 Offline CLF generation 120

4.2 SCLF construction with CP method 144

5.1 Tube MPC synthesis for quadcopter trajectory tracking 162

5.2 Robust MPC for trajectory tracking 162

A.1 Calculate L_j^k, U_j^k in (3.76) with FBBT. 211

Notation

STANDARD notations of the control literature will be used across the manuscript. In this part, we outline the mathematical conventions commonly used to enhance the consistency between chapters.

$\mathbb{R}$, $\mathbb{Z}$ and $\mathbb{N}$ denote the sets of real numbers, integers, and natural numbers, respectively. For $a, b \in \mathbb{Z}$, $\mathcal{I}_{a,b}$ denotes the set of integers in $[a, b]$, $\mathcal{I}_c \triangleq \mathcal{I}_{1,c}$. $\mathcal{N}(n, a, b)$ denotes a finite collection of n evenly sampled real numbers over the interval $[a, b] \subset \mathbb{R}$. For a set $\mathcal{X}$, $\partial\mathcal{X}$ and $\text{int}(\mathcal{X})$ denote its boundary and interior, respectively. For a polytope $\mathcal{B}$, $\text{Vert}(\mathcal{B})$ is the set of its vertices.

Matrices with appropriate dimensions will be noted with an uppercase, bold letter. $\boldsymbol{M} \succ 0$ and $\boldsymbol{M} \succeq 0$ imply that $\boldsymbol{M}$ is positive definite and semi-definite, respectively. Similarly, $\boldsymbol{M} \prec 0$ and $\boldsymbol{M} \preceq 0$ imply $-\boldsymbol{M} \succ 0$ and $-\boldsymbol{M} \succeq 0$, respectively. $\lambda_{min}(\boldsymbol{M})$, $\lambda_{max}(\boldsymbol{M})$ denote the smallest and largest eigenvalues of $\boldsymbol{M}$. For $\boldsymbol{W} \in \mathbb{R}^{m\times n}$, $\boldsymbol{W}_{ij}$ is its value at row i, column j while $\boldsymbol{W}_{i:}$ and $\boldsymbol{W}_{:j}$ collect all the matrices' entries on row i and column j, respectively. Furthermore, for a matrix $\boldsymbol{B}$, $\text{r}_i(\boldsymbol{B})$ returns its i-th row. $\oplus$ and $\ominus$ denote the Minkowski sum and Pontryagin difference, respectively. $\text{diag}(\cdot)$ returns a diagonally arranged matrix with the components given within.

A bold lowercase letter (such as $\boldsymbol{x}$) will serve as the vector's designation. For $\boldsymbol{x} \in \mathbb{R}^n$, if not particularly specified, x_i represents its ith entry and for $\boldsymbol{x}, \boldsymbol{y} \in \mathbb{R}^n$, $\boldsymbol{x} \leq \boldsymbol{y}$ means $x_i \leq y_i, \forall i \in \mathcal{I}_c$. Moreover, with a vector $\boldsymbol{q} \in \mathbb{N}^n$, $\boldsymbol{x}^{(\boldsymbol{q})} \triangleq [x_1^{(q_1)}, ..., x_n^{(q_n)}]^\top$. For a continuous signal $\boldsymbol{x}(t)$, $\boldsymbol{x}(t_0|it_s)$ signifies its predicted value, at the future time $t = t_0 + it_s$ upon the information available at $t = t_0$. For a discrete signal $\boldsymbol{x}(k)$, let $\boldsymbol{x}(k|k+i)$ be its predicted value, at discrete step $k + i$ upon the feedback measured at time step k.

Given a constant a, then a scalar function $\beta(\cdot) : [0, a) \to [0, \infty)$ is called a $\mathcal{K}$-class function if $\beta(0) = 0$, $\beta(\cdot)$ is continuous and strictly increasing. For $V(\boldsymbol{x}) : \mathbb{R}^n \mapsto \mathbb{R}$, $\nabla V(\boldsymbol{x})$ denotes the gradient vector of the function.

Notations for a general system

Notation	Description
$x^{(\ell)}(t)$	ℓ-order time derivative of $x(t)$, .i.e., $\mathrm{d}^\ell x(t)/\mathrm{d}t^\ell$
$\|\boldsymbol{x}\| = \|\boldsymbol{x}\|_2$	$\sqrt{\boldsymbol{x}^\top \boldsymbol{x}}$
$\|\boldsymbol{x}\|_{\boldsymbol{P}}$	$\sqrt{\boldsymbol{x}^\top \boldsymbol{P} \boldsymbol{x}}$
$\|\boldsymbol{x}\|_\infty$	infinity norm of vector $\boldsymbol{x}$
sat	standard saturation function
$\boldsymbol{I}$	identity matrix of appropriate dimension
$\boldsymbol{I}_n$	identity matrix of dimension n
$\mathbf{0}_{m\times n}$	zero matrix of dimension $m \times n$
$\mathbf{0}_n$	zero matrix of dimension $n \times n$
$\mathbf{0}$	zero vector
$\det(\boldsymbol{P})$	the determinant of matrix $\boldsymbol{P}$

Notations for the quadcopter system

Notation	Description
$\mathbf{p} = [\mathrm{p}_1\ \mathrm{p}_2\ \mathrm{p}_3]^\top$ $\triangleq [x\ y\ z]^\top$	position vector in $\mathbb{R}^3$ gathering the quadcopter's global position (m)
$\mathbf{p}_i \triangleq [\mathrm{p}_i\ \dot{\mathrm{p}}_i]^\top$	state vector in $\mathbb{R}^2$ gathering the position and velocity in p_i axis
$\boldsymbol{\xi} \triangleq [\mathbf{p}^\top\ \dot{\mathbf{p}}^\top]^\top$ $\boldsymbol{\zeta} \triangleq [\mathbf{p}_1^\top\ \mathbf{p}_2^\top\ \mathbf{p}_3^\top]^\top$	state vector in $\mathbb{R}^6$ of the translational dynamics gathering the position and velocity components of the three axes
$\boldsymbol{\eta} = [\phi\ \theta\ \psi]^\top$	angle vector gathering the roll, pitch and yaw angles (rad)
T	the normalized thrust generated by the propellers (m/s^2)
$\boldsymbol{u} = [T\ \phi\ \theta]^\top$	control input of the outer-loop including the normalized thrust, the roll and the pitch angles
$\boldsymbol{v} = [v_1\ v_2\ v_3]^\top$	the input of the exactly linearized dynamics

Acronyms

CLF - Control Lyapunov Function

DOF - Degree Of Freedom

ExMPC - Explicit MPC

FB-MPC - Flatness-based MPC

FBSC - Flatness-based saturated control

HJB - Hamilton-Jacobi-Bellman

IMPC - Implicit MPC

LMI - Linear Matrix Inequalities

LP - Linear Programming

LQR - Linear Quadratic Regulator

LTI - Linear Time Invariant

MILP - Mixed-Integer Linear Programming

MIP - Mixed-Integer Programming

MPC - Model Predictive Control

NP-hard - Non-Polynomial hard

ODE - Ordinary Differential Equation

PWA - Piece-wise Affine

QP - Quadratic Programming

RPI - Robust Positive Invariance

SCLF - Sub-Control Lyapunov Function

UAV - Unmanned Aerial Vehicle

mRPI - minimal Robust Positive Invariance

Chapter 1

Introduction

This chapter provides a preamble to the manuscript's main theoretical tool: the theory of differential flatness. While the formal definition will not be recalled here, the "trademark" properties of the class will be intuitively described, covering aspects from trajectory planning to feedback design. This description will, in turn, be upheld by a brief overview of the wide-ranging applications of the theory in real-world scenarios. Relevant questions related to the topic will also be highlighted, establishing the guiding framework for the manuscript. Furthermore, with the key objective of handling constraints while maintaining a low computational overhead, the use of control Lyapunov functions will also be addressed, extending the context for a broader class of nonlinear systems. In accordance with these directions, this chapter will outline the contributions and the organization of the thesis.

1.1 Low complexity constrained control 2
1.1.1 Differential flatness in planning and control 2
1.1.2 Beyond flatness: a CLF-based perspective 5
1.2 Thesis orientation 6
1.3 Thesis contributions 7
1.4 Thesis organisation 11

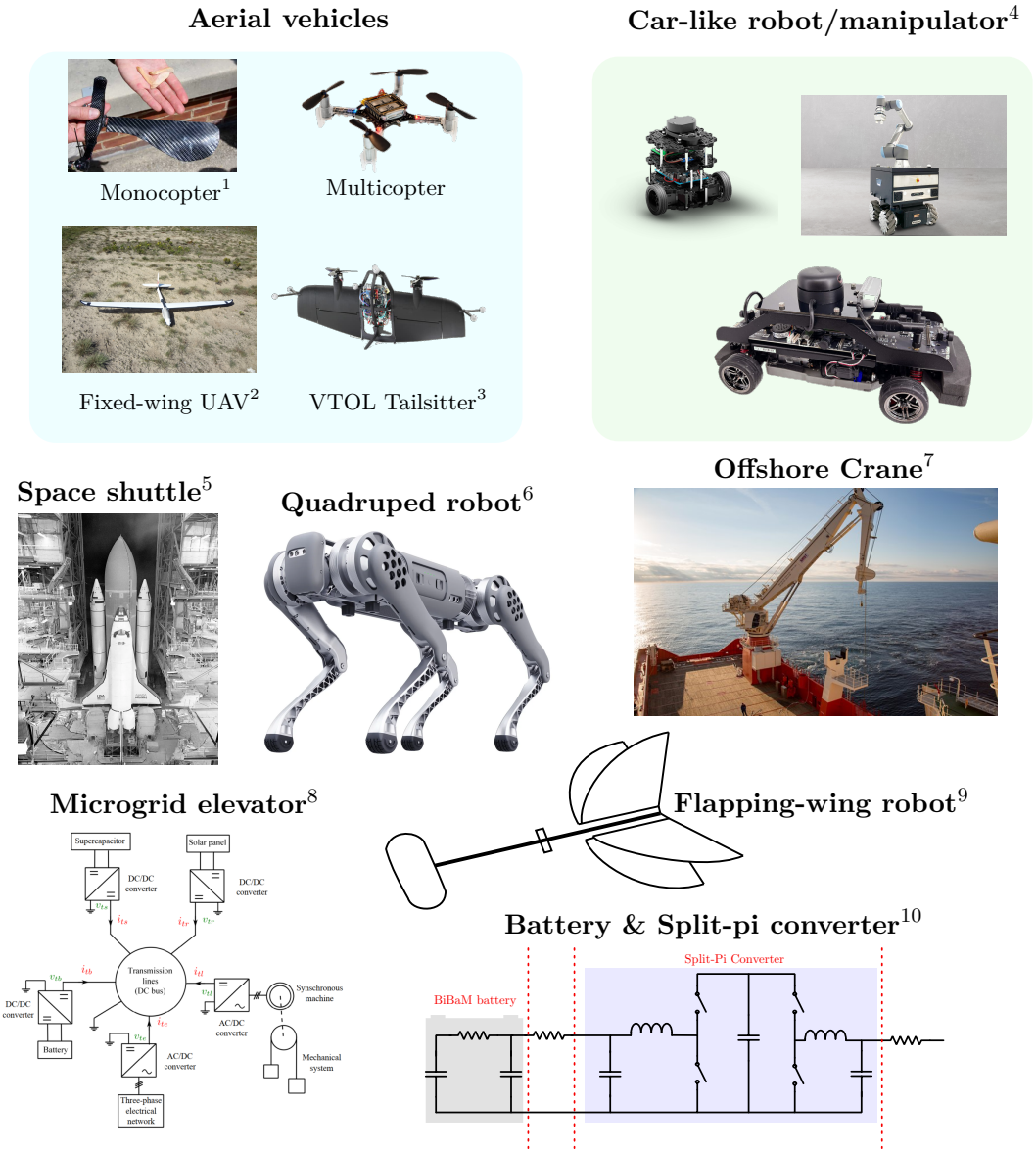
1.1 Low complexity constrained control

Ensuring constraint satisfaction is a crucial demand and ongoing issue for nonlinear control applications. Given the presence of robust optimization toolboxes, it is common practice to calculate an effective planning or control strategy using a constrained optimization problem. Nevertheless, the execution of these complex policies generally requires a substantial amount of real-time computing capacity, therefore restricting their practical applications. This shortcoming motivates the work presented in the manuscript, concentrating on lowering the computational overhead of optimization-based approaches while keeping essential guarantees such as stability, robustness, and constraint satisfaction. In general, the complexity of optimization-based strategies comes from the nonlinearity of the model, besides other aspects like dimensionality or the constraints' restrictiveness. To overcome this issue, the theory of differentially flat systems stands out as a promising medium thanks to its ability to linearize the model in closed-loop and eliminate the differential constraints in the optimal control problem. Furthermore, thanks to this linearization, the dynamics evidently fall into the class of control affine systems, allowing a further reduction in computation time via control Lyapunov functions (CLF). Practically, the immediate question revolves around the generality of the study or whether the class of differentially flat systems covers a limited range of applications. For this reason, we first present an overview of the ongoing research and applications using the concept, highlighting the viability of the dissertation. Then, a short motivation on the usefulness and potential of CLF will follow.

1.1.1 Differential flatness in planning and control

The theory of differentially flat systems, since its first introduction by Fliess et al. [1992], has been proven advantageous in solving the two main concerns of automatic control: *trajectory planning* and *tracking*, attracting an ever-growing attention in both the research and industry communities. This is because, the class covers a wide range of practical applications, such as autonomous robots or electrical systems (see Figure 1.1). To an equally important extent, systems in this class contain two important properties: *differential parameterization* and *dynamic feedback linearizability*.

The first property means that all the states and inputs variables can be algebraically expressed in terms of a special output (called the *flat output*) and a finite number of its derivatives. This expression is known as the *flat representation*. This definition implies that feasible trajectories of the system can be indirectly found via a proper time parameterization of the flat output [Levine, 2009]. As a result, the problem of planning a realizable trajectory and the associated control action is converted into parameterizing the flat output in time, avoiding the need to approximately solve any differential equations. In practice, via curve-fitting tools, exact and navigable paths or trajectories



[1] https://www.nbcnews.com/ and [Tang et al., 2024] [2] [Prodan et al., 2013]
[3] https://innovationorigins.com/en/revolutionary-flight-path-algorithm-takes-drones-to-new-heights/
[4] https://www.quanser.com/products/qcar/ and https://robotican.net/
[5] https://www.nasa.gov/ and [Morio, 2009] [8] [Pham et al., 2015]
[6] https://shop.unitree.com/products/unitree-b1 and [Li et al., 2021]
[7] https://www.smstequipment.com/offshore-cranes/ and [Schubert and Abel, 2023]
[9] [Qian et al., 2024] [10] [Zafeiratou et al., 2020]

FIGURE 1.1: Applications belonging to the class of differentially flat systems.

have been obtained for many complex systems. Indeed, the theory has been instrumental in the recent development in robotic systems, such as the project Mini Cheetah [Li et al., 2021] from MIT and NAVER LABS. The theory has also proven its usefulness with the application to the smart robotic exoskeleton (SREx), a robot-aided rehabilitation platform in physical therapy [Brahmi et al., 2021]. Validated via the ADVISOR simulator [Wipke et al., 1999], a development of battery charging in hybrid vehicle can be found in [Joševski and Abel, 2016]. Employing the model of the NASA "Discovery" space shuttle, a constrained atmospheric reentry planning software was developed in the work of [Morio, 2009, Louembet et al., 2009]. Regarding the class of aerial vehicles, many advanced applications benefit from the theory, such as a tail-sitter vertical takeoff and landing (VTOL) aircraft in [Mellinger and Kumar, 2011, Tal and Karaman, 2022], tethered drones in [Tognon and Franchi, 2020] or different configurations of aerial manipulators in the survey [Ollero et al., 2021]. Besides robotic systems, this implication of flatness has also been exploited for realistic profile generation in pragmatic applications such as roll-to-roll industrial web systems [Chang et al., 2012], electrical microgrids [Pham et al., 2015, Zafeiratou et al., 2020, Al-Ismail, 2021], steam power systems [Hanel and Lehner, 2013], geostationary satellites [Gazzino et al., 2017, Louembet et al., 2009, Farahani et al., 2016] or drug delivery [Guay et al., 2007]. In brief, it is important to highlight that with this technique, feasible trajectories of highly complex systems can be obtained in a simpler manner and avoid numerical error. Meanwhile, the second property suggests that flat systems are endogenous dynamic feedback linearizable [Levine, 2009]. This means that in a properly defined coordinate system, called *the flat output space*, flat systems evolve linearly. From a control theoretic viewpoint, this procedure can be regarded as the feedback linearization [Isidori, 1985] or model inversion-based strategy [Devasia, 2002]. This connection to classical control theory leads to many successfully validated results from industry to research. For example, various intensive studies and validations can be found for the multicopter family [Sun et al., 2022, Kumar et al., 2017, Tal and Karaman, 2020, Freire and Xu, 2023], terrestrial/aerial manipulators [Tang et al., 2010, Yüksel et al., 2016, Ryu and Agrawal, 2010], automotive driving [Wang et al., 2021, Tolosana, 2018, Ljungqvist, 2020, Guo et al., 2012], electrical systems [Laaziz et al., 2024, Thounthong et al., 2021, Zafeiratou et al., 2020] or chemical processes [Mounier and Rudolph, 1998, Utz and Kugi, 2012, Mahadevan et al., 2001]. To sum up, the key idea of flatness is:

> A flat system is equivalent to a linear controllable system[1].

With such an interpretation, planning and control of flat systems are directly connected to the framework of linear control theory. Yet, despite these remarkable number of applications, the benefit of flatness inevitably entails a conundrum cost. More specifically, the main difficulty hindering the application of the theory lies in the presence of constraints (e.g., state or input constraints). As the nonlinearity is canceled in the

[1]To explain better the definition of *equivalent* systems, we send the reader to the self-contained work of [Levine, 2009] with the original viewpoint of differential geometry.

system description, the operating constraints undergo the same transforming effect, and they are distorted into complicated restrictions in the new space. The challenge of such convoluted constraints has always been of interest, from the early days of flatness [Van Nieuwstadt, 1997, Faiz et al., 2001, Ross and Fahroo, 2006] up to the present [Beaver and Malikopoulos, 2024, Greco et al., 2022, Himmel et al., 2020, Mounier et al., 2019]. Concisely, to control or plan a constrained flat system, one has to face the question:

How to handle the convoluted constraints caused by flatness-induced mappings?

Furthermore, it is observed that each set of mappings results from an ansatz choice of the flat output and that this choice may not be unique [Levine, 2009, Kaminski et al., 2018]. Thus, another question which we will discuss is:

What is the optimal choice of flat output to exploit?

This question is relevant as it will affect the description of the linearizing mappings, the convoluted constraints, and hence the control or planning performance.

Apart from perceiving flat systems in the flat output space as linear dynamics, it is noticed that the linearized system can also be seen as a control affine system. This observation directly relates the problem to another constraint-handling tool: the CLF-based framework.

1.1.2 Beyond flatness: a CLF-based perspective

Adapted from the notion of Lyapunov functions [Liapounoff, 1907], control Lyapunov functions (CLF) have long been a powerful tool of analyzing and certifying stability [Artstein, 1983, Sontag, 1983]. From an implementation perspective, CLFs also give rise to computationally attractive control policies. Specifically, a CLF can be interpreted as an energy-like scalar function of whose time derivative is negative or whose value decreases along the system's trajectory. This implies that the information about a system's stability is encoded within the CLF. Therefore, the stabilization problem becomes that of finding a control strategy maintaining the negativity condition on the CLF's time derivative. This condition can be conveniently described by a single inequality. As a results, a steady stream of research has focused on finding analytical or explicit controllers to enforce this condition under various operating constraints [Sontag, 1989, Kidane et al., 2004, Yamashita et al., 2022, Leyva et al., 2023]. To avoid the exhaustive construction of explicit control laws, another approach involves solving the stabilizing constrained problem online using optimization solvers. For control affine systems, this online strategy remains relatively simple while allowing optimality to be considered through the cost function of the program [Freeman and Kokotovic, 1996, Pylorof and Bakolas, 2015].

A CLF provides a stability certificate for the control design and a computationally efficient description of the stabilizing input set.

However, as a trade-off for simplicity in online implementation, the offline construction of a CLF for an arbitrarily nonlinear system is not trivial, both computationally and analytically. For a comprehensive survey of computational methods for CLF construction, we refer to the review of Giesl and Hafstein [2015]. The noteworthy idea is that, although for nonlinear systems, the synthesis of CLFs is intractable with respect to nonlinearity and dimensionality, the construction for linear system is rather well-studied over decades [Blanchini and Miani, 1996, Lazar and Jokic, 2009a, Rubagotti et al., 2016]. This observation for developed results for linear systems, again, serves as the catalyst to study CLF-based control strategies in combination with flatness-induced linearization. In addition, with the application platform presented subsequently, this connection prompt us to experimentally validate the artificial construction of the CLFs, which is rarely implemented in practice in the literature. For these reasons, computational procedures for generating CLF will also be covered in the thesis, extending the subject to the class of control affine systems.

1.2 Thesis orientation

It is evident from the presented overview that the control of differentially flat systems is an extensive field with a wide range of tools and applications. In this work, we will focus on two key concerns: addressing complex constraints and analyzing the selection of flat outputs. Additionally, the extension to CLF-based design will be explored for constrained control affine systems. With this scope, the thesis orientation can be placed in the following contexts and theoretical foundations.

First, as we aim to take into account practical operating constraints in the control problem, it is customary to employ feedback strategies based on solving constrained optimization problems. By doing so, the constraints can be handled directly if properly characterized into manageable forms. For this purpose, it is necessary to describe the physical constraints mathematically. In this work, we choose to employ set-theoretic tools to model the problem, including both the systems and the constraints. In this manner, fundamental properties such as stability, robustness and constraint satisfaction can be analyzed formally.

Second, despite the large number of constrained flat systems, the scope of the thesis explores the case-study of quadcopters, leveraging the theoretically and experimentally established platform of Nguyen [2019] in the previous study of our research team. With respect to this setting, two essential differences can be noted as follows. Initially, the thesis regards the quadcopter as a flat system, rather than investigating the application as the particular subject as in [Nguyen, 2019]. With this direction, theoretical tools will be developed in a generalizable manner, rather than particularly for the vehicle's dynamics. This guiding principle leads us to the next distinction. Although different controllers will be proposed for the quadcopter, we would like to highlight that they will

be synthesized in the linearizing coordinates (the flat output space), where the dynamics is linearized in closed-loop. This idea sets our work apart from the previously mentioned work, which uses the original set of variables and handles directly the nonlinear dynamics. This novelty can be further clarified by the complexity of our control laws, including linear model predictive control (MPC) rather than nonlinear MPC.

Third, since the flatness-based linear representation allows the stabilization with CLFs, the control strategies in the manuscript will also be drawn upon the seminal framework of numerically synthesized CLFs [Blanchini and Miani, 1999a,b, Johansen, 2000]. More specifically, we align our work with the existing literature of developing computational techniques to generate CLFs based on the availability of an optimal controller [Ravanbakhsh and Sankaranarayanan, 2019, Mukherjee et al., 2022]. With this methodology, the online complexity will be reduced to a quadratic program at the expense of a performance sub-optimality gap with respect to the baseline control. Then, formal analysis and experimental validations will follow.

Finally, we would like to draw the readers' attention to our primary goal while using differential flatness in control: the real-time complexity reduction. This is because with the linearization from flatness, it is evident that the system description will become linear, and hence dynamically simpler. Motivated by this observation, the contributions of the dissertation will center around the tasks of characterizing other supporting elements such as a novel saturation function for the convoluted constraints, or an approximating procedure to obtain a convex subset of the complicated domain. Accordingly, classical solver-free linear design (saturated control) or well-studied optimization-based routine (MPC, CLF) can be leveraged to stabilize the original nonlinear dynamics.

1.3 Thesis contributions

Oriented on such an optimization-based framework, experimental platform and core interest, this thesis contributions can be summarized as the following.

Initially, the contribution can be placed along the development of control design for quadcopter systems, especially towards the direction of exploiting differential flatness such as in [Nguyen, 2019], [Greeff, 2022] and [Nguyen et al., 2024]. In this vein, the standard flatness-based feedback linearization will be applied, resulting in a linear systems subject to a non-convex constraint set. This convoluted domain will then be characterized to a nonlinear convex set [C3]. From this point, the design are divided into two directions.

- First, we continue to approximate the convex set by polytopes (i.e., linear inequalities). This approximation immediately gives us access to the powerful design of implicit MPC [C3], explicit MPC [J5] and tube MPC [C5] for linear systems. In this fashion, the nonlinear dynamics of the quadcopter can be controlled with

constraint satisfaction, stability, and robustness guarantees in computationally different efficient ways.

- Second, a specified saturation function for the new nonlinear convex constraint is derived based on an optimization problem. This optimization-based saturation function is then solved explicitly, enabling the employment of the so-called gradient-based control for linear system under input constraints [J1]. The particularity of this contribution is that the convoluted constraints now are handled by the new saturation function and there is no optimization problem that needs to be employed online. Then, an enhancement solving the parameter tuning problem for this scheme is also provided based on an adaptive technique, improving the performance of the saturated design [C4].

With both viewpoints, efficient controllers are proposed with different level of simplicity and flexibility. It is also worth mentioning that all the proposed designs have been experimentally validated via the Crazyflie 2.1 nanodrone platform.

Next, leveraging the characterization of the quadcopter model in the flat output space, we study and validate a technique to reduce real-time complexity of MPC via control Lyapunov function (CLF). The contributions in this direction can be summarized as:

- We employ MPC as a baseline controller to provide a stable closed-loop autonomous system. The trajectory of this system is then sampled to construct a CLF via appropriate parameterization. Ultimately, the CLF alone is used to control the system, significantly reducing the online computation compared to the original MPC routine. This approach is also experimentally validated through the previously established drone control setup [J6].
- To formalize the scheme and analyze the performance trade-off for complexity reduction, the concept of sub-control Lyapunov functions (SCLF) is introduced. This concept is validated for non-trivial size system in both comprehensive simulations and experiments [J2].

Additionally, exploring general questions related to flatness-based control, the manuscript provides the following contributions.

- In [J7], we examine the use of rectified linear unit neural networks to decompose the convoluted constraints caused by feedback linearization into mixed-integer linear constraints. The resulted formulation is then integrated into to CLF-based control and MPC setting to verify the effectiveness.
- In [C1] and [C2], we investigate the implications of changing the flat output in trajectory planning and control. Specifically, via the example of a fixed-wing unmanned aerial vehicle, we generate optimal profiles with different flat outputs to see

the effect on the trajectory generated, with respect to the curve's length, smoothness requirement and the complexity of the flat representations. Furthermore, it will also show that, with different flatness-induced linearizing laws, disturbances will propagate and have different effects on the closed-loop performance. The distinction is demonstrated via the minimal robust positive invariant set.

In alignment with the contributions, below is a list of papers that have been accepted, are under second-round review or have been submitted to various journals and international conferences. Additionally, the list includes talks presented at national SAGIP and GDR MACS activities each year, along with technical talks given at various international laboratories during mobility research visits:

Accepted journal papers:

[J7] **H.T. Do**, I. Prodan: On the constrained feedback linearization control based on the MILP representation of a ReLU-ANN. *IEEE Control Systems Letters*, DOI: https://doi.org/10.1109/LCSYS.2024.3411515, 2024 (presented also at the 63rd IEEE Conference on Decision and Control (CDC), Milano, Italy, 2024).

[J6] **H.T. Do**, F. Blanchini, S.Miani, I. Prodan: LP-generated Control Lyapunov Functions with application to multicopter control, *IEEE Transactions on Control Systems Technology* , DOI: https://doi.org/10.1109/TCST.2024.3396704, 2024.

[J5] **H.T. Do**, I. Prodan: Explicit flatness-based MPC design for indoor nano-quadcopter position tracking, *International Journal of Control*, DOI: https://doi.org/10.1080/00207179.2024.2340690, 2024.

Submitted journal papers:

[J4] **H. T. Do**, I. Prodan, F. Stoican and F. Nicolau: Differential Flatness in Optimization-Based Motion Planning, submitted to *Annual Reviews in Control* on January 11th 2025.

[J3] **H. T. Do**, I. Prodan and F. Stoican: On the application of ReLU-ANN for flatness-induced constrained control, submitted to *Nonlinear Dynamics* on January 2025.

[J2] **H.T. Do**, F. Blanchini, S.Miani, I. Prodan: Reducing real-time complexity via sub-control Lyapunov functions: from theory to experiments, submitted to *Automatica*, on August 2, 2024, (second round review).

[J1] **H.T. Do**, F. Blanchini, I. Prodan: A low computation saturated controller for a quadcopter position tracking with stability and convoluted input constraints guarantees, submitted to *European Journal of Control*, on September 5, 2023 (second round review).

Book chapter:

[BC1] **H.T. Do**, F. Blanchini, I. Prodan: Nonlinear constrained control systems, submitted to: *Encyclopedia of Systems and Control Engineering*, on October 15, 2024, to appear in 2025.

Accepted international conference papers:

[C5] **H. T. Do**, I. Prodan: Tube MPC via flatness for multicopter trajectory tracking, *2024 European Control Conference*, Stockholm, Sweden, 2024, DOI: https://doi.org/10.23919/ECC64448.2024.10591195

[C4] **H.T. Do**, F. Blanchini, I. Prodan: Enhancements on a Saturated Control for Stabilizing a Quadcopter: Adaptive and Robustness Analysis in the Flat Output Space, *2023 62nd IEEE Conference on Decision and Control (CDC)*, Singapore, Singapore, pp. 739-744, 2023, DOI: https://doi.org/10.1109/CDC49753.2023.10384084.

[C3] **H. T. Do**, I. Prodan: Indoor experimental validation of MPC-based trajectory tracking for a quadcopter via a flat mapping approach, *European Control Conference*, Romania, 2023, DOI: https://doi.org/10.23919/ECC57647.2023.10178260.

[C2] **H. T. Do**, F. Nicolau, F. Stoican and I. Prodan: Tracking control for a flat system under disturbances: a fixed-wing UAV example, *18th IFAC Workshop on Control Applications of Optimization*, Gif sur Yvette, France, 18–22 July 2022, DOI: https://doi.org/10.1016/j.ifacol.2022.09.058

[C1] **H. T. Do**, I. Prodan and F. Stoican: Analysis of alternative flat representations of a UAV for trajectory generation and tracking, *25th IEEE International Conference on System Theory, Control and Computing*, Iasi, Romania, 2021, DOI: http://doi.org/10.1109/ICSTCC52150.2021.9607073.

National congress and workshops:

[NC4] **H. T. Do**, I. Prodan: Flatness-based MPC : from ReLU Neural Networks to MIP constraints *Deuxième Congrès Annuel de la SAGIP*, Villeurbanne, France, 2024.

[NC3] **H. T. Do**, I. Prodan: On the implementation of an explicit MPC for a quadcopter via differential flatness *Premier Congrès Annuel de la SAGIP*, Marseille, France, 2023.

[NC2] **H.T. Do**, F. Blanchini, S.Miani, I. Prodan: Computer-generated Control Lyapunov Function via offline linear programming, *CT Commande Prédictive Non-Linéaire*, Valence, France, 2023.

[NC1] **H.T. Do**, F. Blanchini, S.Miani, I. Prodan: Tracking control for a class of flat system under disturbances, Insights on flatness-based MPC for a multicopter, *CT Commande Prédictive Non-Linéaire*, Paris, France, 2022.

International exchange seminars:

[P4] Constrained control via flat mappings: theory and experiments, November 2024, *Univ. of Michigan, Ann Arbor, Michigan, USA.*

[P3] Flatness-based constrained control for quadcopters, December 2023, *Univ. of Porto, Porto, Portugal.*

[P2] Benefits of differential flatness for optimal profile generation and tracking, April 2022, *POLITEHNICA Bucharest National Univ. of Science and Technology, Romania.*

[P1] Benefits of flatness in constrained optimization-based control for nonlinear autonomous systems, December 2021, *Univ. College London, London, United Kingdom.*

1.4 Thesis organisation

The remaining parts of the thesis are structured as following.

Chapter 2 provides a comprehensive look at the advantages of differentially flat system theory. Apart from explaining the basic idea and properties of flatness, we also discuss the earlier motion planning work in this area and the auxiliary tools that can be used to exploit the concept. The basic means include curve parameterization frameworks and set-theoretic background to be used in optimization-based approaches.

Chapter 3 will address the problem of convoluted constraints caused by feedback linearization from flatness, particularly for the quadcopter application. Then, neural networks will be explored as an approximator for a complicated flat output space in general. In detail, the chapter commences with a study of the constraints on the quadcopter's translational dynamics in a hierarchical control architecture. Subsequently, we present an algorithm for approximating the deformed domain using parameterized zonotopes. This approximation allows for the implementation of the standard linear Model Predictive Control (MPC). Moreover, the constructed approximation will be adapted to align with the explicit MPC framework in order to accelerate the online computation, albeit at the cost of a more conservative inner approximation. Next, we provide an optimization-based saturation function for the nonlinear constraint set, enabling the design of an explicit saturated control. Ultimately, the chapter finishes by exploring the application of neural networks as a versatile tool for approximating functions, specifically in the context of a constraint-handling controller in the linearizing coordinate system.

Chapter 4 studies a technique for reducing real-time complexity with CLF-based control with the quadcopter application. More specifically, MPC will be used to generate stable closed-loop trajectories in simulation. These trajectories are then sampled to formulate a semi-infinite linear program (SILP) to parameterize a CLF candidate. The CLF is then experimentally validated via the quadcopter stabilization problem. Next, to formalize the experimentally validated findings, the concept of the sub-control Lyapunov function is introduced. This setup is then followed by a more computationally efficient algorithm based on the cutting-plane technique, allowing the application of non-trivial size systems. The results and advantages of the schemes are validated in both simulations and experiments.

Chapter 5 addresses the complementary questions arising over the course of the thesis. The chapter first includes the robustification of the MPC schemes by adapting the characterized setup of the quadcopter to the tube MPC framework. Then, regarding the saturated control, we provide an adaptive scheme to identify the satisfactory gain for the controller, followed by the robustness analysis of the design. Both adaptations are experimentally validated with the Crazyflie 2.1 platform. Finally, the effect of changing the flat output in trajectory and control design will be investigated via the simulations of a fixed-wing UAV model.

Chapter 6 serves as the thesis' conclusion and delineates the future directions.

The descriptions and the dependencies of the chapters are illustrated in Figure 1.2.

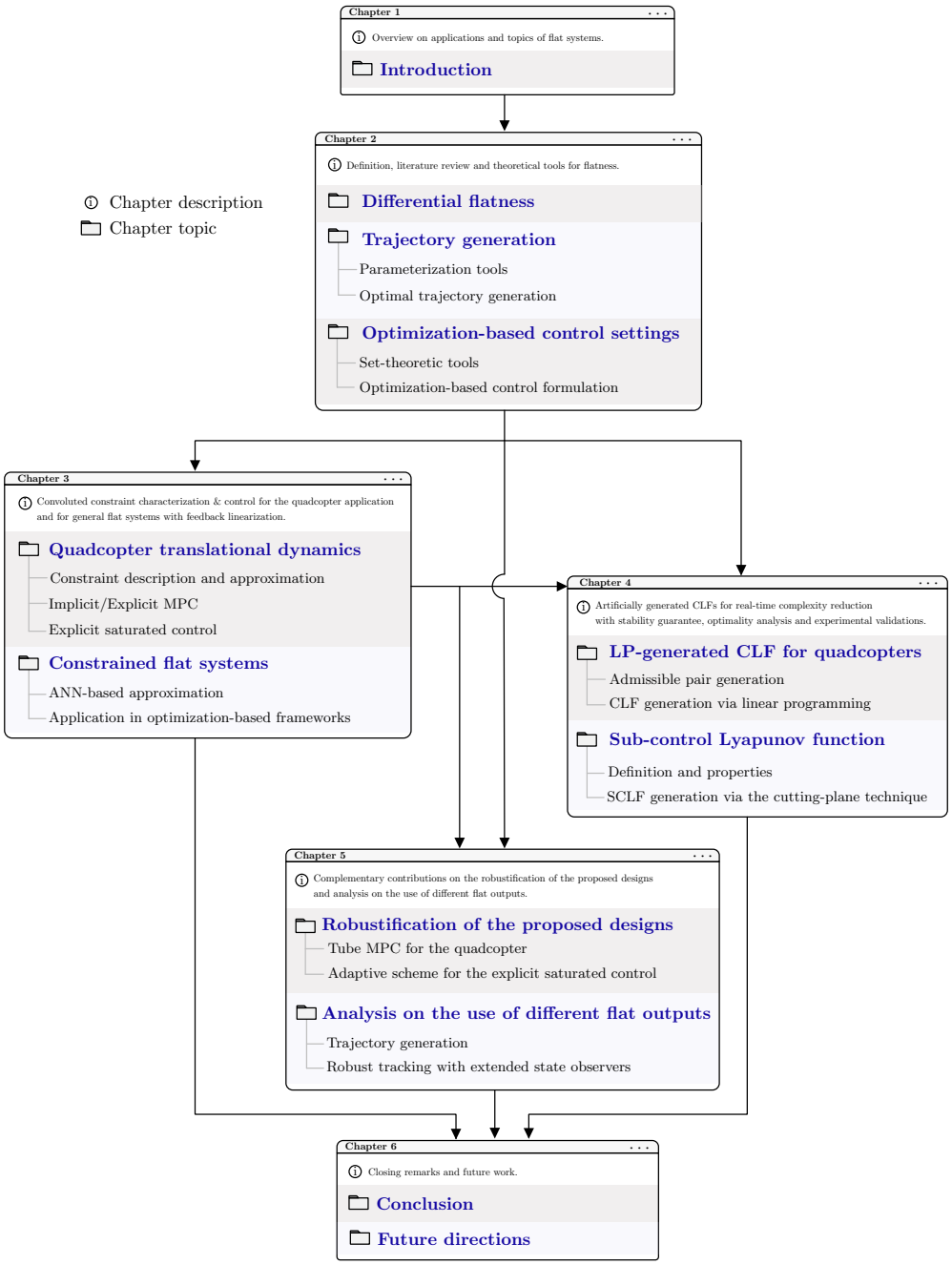


FIGURE 1.2: The brief descriptions and the dependencies between the thesis chapters.

Chapter 2

Curves, sets and flatness in optimization-based motion planning

This chapter provides an overview of the theory of differentially flat systems and its benefits. Beyond outlining the fundamental concepts and properties of flatness, it reviews relevant work in motion planning and discusses auxiliary tools to leverage the theory's advantages [J4]. This sets a solid foundation for the subsequent content of the manuscript and situates it within the broader literature.

2.1 Motivation . . . 14
2.2 Differentially flat systems . . . 14
2.2.1 Definition and properties . . . 14
2.2.2 Computation of the flat output and its non-uniqueness . . . 19
2.3 Flatness-based trajectory generation . . . 22
2.3.1 Parameterization tools . . . 24
2.3.2 Optimal trajectory generation with flatness . . . 29
2.4 Optimization-based control settings . . . 39
2.4.1 Sets and invariance notions . . . 40
2.4.2 Optimization-based control schemes . . . 46
2.5 Concluding remarks . . . 50

2.1 Motivation

With the benefit of simplifying the nonlinear dynamics through linearization, the theory of differentially flat systems has been extensively studied and applied during more than 20 years since its introduction. As underlined in the previous chapter, the idea of flatness encompasses a wide range of applications and engineering disciplines, generating numerous methodologies and research directions aimed at improving and exploiting the concept. Motivated by the lack of an exhaustive review on this topic, and given that the existing survey is limited to the work of [Soheil-Hamedani et al., 2016] for power systems, we aim to conduct a comprehensive review of flatness applications, their properties, and their role in control engineering, and in particular, motion planning. This review will identify open questions, trends and emerging research contexts, offering a structured basis for future development. Furthermore, for the manuscript, the research landscape established herein will serve as a foundation to better position our contributions. Finally, the goal of this chapter is also to provide a necessary theoretical background and numerical tools aligned with the subsequent chapters.

In what follows, we recall the definition of flatness and its useful properties in Section 2.2. Then, the related work exploiting these benefits, including feedback linearization and differential parameterization, will be analyzed in Section 2.3 and 2.4. With the methodologies considered, over the course of the review, theoretical frameworks and preliminaries for the manuscript are also presented. Finally, Section 2.5 concludes the chapter and identify possible future directions for using flatness in motion planning [J4].

2.2 Differentially flat systems

As stated earlier, this section provides a description of differentially flat systems and discusses their relevance in motion planning and control.

2.2.1 Definition and properties

A general differentially flat system can be described as the following.

Definition 2.1 (Differentially flat systems [Fliess et al., 1995])**.** *Consider the system governed by the ODE of the form:*

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}, \boldsymbol{u}) \tag{2.1}$$

with $\boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{u} \in \mathbb{R}^m$ *being the state and input vector, respectively.* $\mathbf{f} : \mathbb{R}^n \times \mathbb{R}^m \to \mathbb{R}^n$ *denotes the vector field. Then, the system* (2.1) *is called differentially flat if there exists*

a fictitious output $\boldsymbol{z} = [z_1, z_2, ..., z_m]^\top \in \mathbb{R}^m$ *of the form:*

$$\boldsymbol{z} = \Phi_z \left(\boldsymbol{x}, \boldsymbol{u}, \dot{\boldsymbol{u}}, \ddot{\boldsymbol{u}}, ..., \boldsymbol{u}^{(\boldsymbol{p})} \right), \tag{2.2}$$

for some $\boldsymbol{p} \in \mathbb{N}^m$*, such that* $\boldsymbol{z}$ *and a finite number of its derivatives can be used to algebraically parameterize the state and the input vector. Namely, with some* $\boldsymbol{q} \in \mathbb{N}^m$*:*

$$\boldsymbol{x} = \Phi_x \left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, ..., \boldsymbol{z}^{(\boldsymbol{q})} \right), \tag{2.3a}$$

$$\boldsymbol{u} = \Phi_u \left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, ..., \boldsymbol{z}^{(\boldsymbol{q}+1)} \right). \tag{2.3b}$$

Hereinafter, when (differentially) flat systems are considered, the expression (2.3) will be referred to as a *flat representation* of the original system (2.1). The corresponding duality between (2.2) and (2.3) has been proven useful in many stages of motion planning. To demonstrate the usefulness of this representation, let us accompany the subsequent discussions with the following example.

Example 2.1 (Planar fixed-wing UAV model). Consider the fixed-wing unmanned aerial vehicle (UAV) kinematic model described by the ODE [Beard and McLain, 2012]:

$$\dot{x}_1 = u_1 \cos x_3, \quad \dot{x}_2 = u_1 \sin x_3, \quad \dot{x}_3 = g(\tan u_2)/u_1. \tag{2.4}$$

where $\boldsymbol{x} = [x_1, x_2, x_3]^\top$ collects the positions and the heading (yaw) angle, respectively (see Figure 2.1). The two inputs $\boldsymbol{u} = [u_1, u_2]^\top$ contain the relative airspeed velocity (m/s) and the bank angle (rad), respectively and g is the gravitational acceleration.

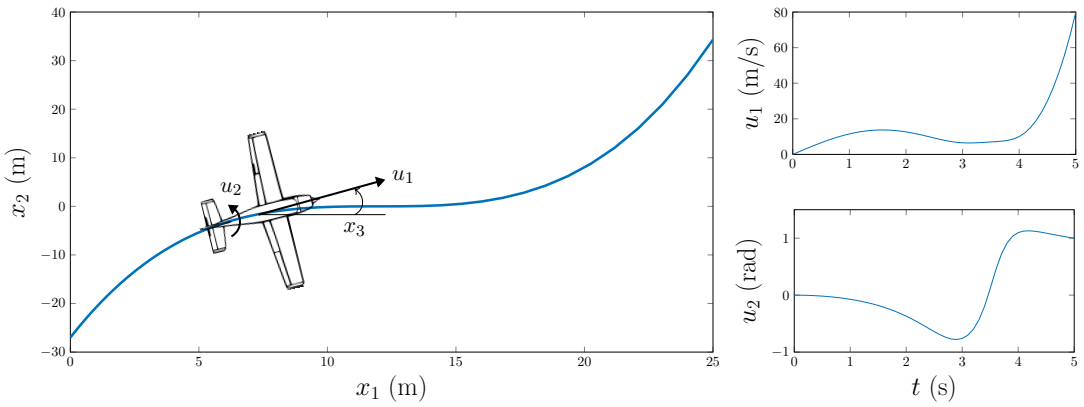


FIGURE 2.1: Planar fixed-wing UAV model and its variables' trajectories.

This system is differentially flat since all the states and inputs can be expressed in terms of a flat output $\boldsymbol{z} \in \mathbb{R}^2$ and a finite number of its derivatives. To see this, consider the flat output:

$$\boldsymbol{z} = \begin{bmatrix} z_1 & z_2 \end{bmatrix}^\top \triangleq \begin{bmatrix} x_1 & x_2 \end{bmatrix}^\top. \tag{2.5}$$

Then the flat representation of the system in the form (2.3) can be computed as:

$$\begin{aligned} x_1 &= z_1, x_2 = z_2, x_3 = \arctan(\dot{z}_2/\dot{z}_1), \\ u_1 &= \sqrt{\dot{z}_1^2 + \dot{z}_2^2}, \\ u_2 &= \arctan\left(g^{-1}(\dot{z}_1^2 + \dot{z}_2^2)^{-1/2}(\ddot{z}_2\dot{z}_1 - \ddot{z}_1\dot{z}_2)\right). \end{aligned} \tag{2.6}$$

With this example, it is noteworthy that the behavior of the entire system over time (involving five variables in $\boldsymbol{x}$ and $\boldsymbol{u}$) can be reconstructed from the two signals in $\boldsymbol{z}$. Namely, if one has a sufficiently smooth time trajectory of the two entries in $\boldsymbol{z}$, say, $\boldsymbol{z}(t) = [z_1(t), z_2(t)]^\top$, then a solution of $\boldsymbol{x}(t)$ and $\boldsymbol{u}(t)$ subject to the system of differential equations (2.4) can be recovered exactly via (2.6). For instance, via a simple parameterization:

$$z_1(t) = t^2,\ z_2(t) = (0.25t^2 - 3)^3, \tag{2.7}$$

the nominal trajectories of all variables constrained by the ODE (2.4) can be obtained via (2.6) (depicted in blue on Figure 2.1) without the need to solve any differential equations. This property of flat systems is highly relevant since, the exact solution of ODEs as in (2.1), called an *integral curve* [Levine, 2009], is generally not trivial to obtain and is usually approximated by numerical integration (e.g., Euler, Tustin or the more general Runge–Kutta methods). Thus, the primary practical use of flatness is to generate a feasible feedforward reference for the system, reducing the number of variables and eliminating the differential constraint (2.1).

> The flat representation can be exploited to obtain dynamically feasible trajectory without any numerical integration.

Beside the application to provide a feasible trajectory, the representation (2.3) also implies an important feature that is feedback linearizability. Namely, the system can be exactly linearized in closed-loop. This can be shown by having a closer look at the expression (2.6). Therein, it is suggested that, there exists a one-to-one relationship between a set of ordinary curves[1] (e.g., as in (2.7)) and the system's trajectory governed by (2.4). This observation was initially pointed out by Fliess et al. [1992], and interestingly, such a relation is mutual, or invertible. In other words, if the system is differentially flat, then there exists a bijective mapping transforming it into a set of m chains of integrators, with m being the dimension of the input $\boldsymbol{u}$ [Levine, 2009]. From a control-theoretic perspective, this correlation leads to the following outcome.

Proposition 2.1 ([Fliess et al., 1999a, Levine, 2009])**.** *A system is differentially flat if and only if it is endogenous dynamic feedback linearizable.*

[1] The curves that are not required to comply with any differential equation.

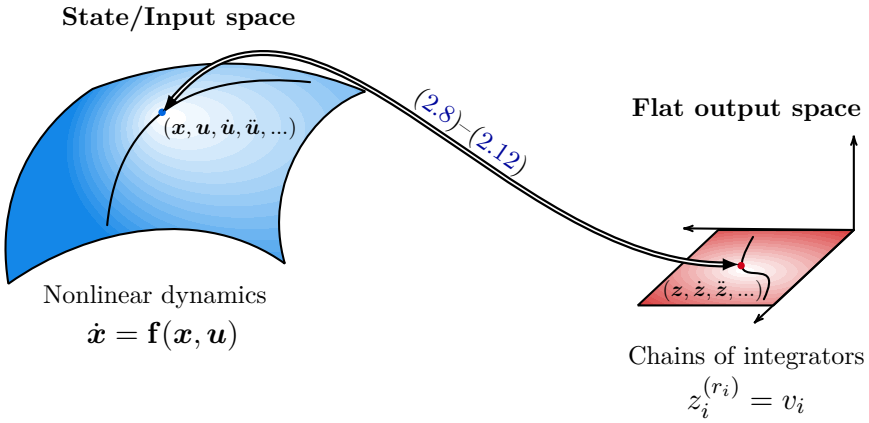


FIGURE 2.2: Trajectory of a differentially flat system in the original (left) and in the flat output space (right).

Formally, if system (2.1) is flat, there exists:

- an invertible endogenous[2] dynamic compensator of the form

$$\dot{\boldsymbol{\eta}} = \boldsymbol{\Xi}_\eta(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w}) \tag{2.8a}$$
$$\boldsymbol{u} = \boldsymbol{\Xi}_u(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w}) \tag{2.8b}$$

  with $\boldsymbol{\eta} \in \mathbb{R}^{n_\eta}$, $\mathbf{w} \in \mathbb{R}^m$ being the state and the control of the compensator, respectively;

- a diffeomorphism $\boldsymbol{\Xi}_\zeta$:

$$\boldsymbol{\zeta} = \boldsymbol{\Xi}_\zeta(\boldsymbol{x}, \boldsymbol{\eta}) \in \mathbb{R}^{n+n_\eta}, \tag{2.9}$$

  and invertible feedback transformation:

$$\boldsymbol{v} = \boldsymbol{\Xi}_v(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w}) \tag{2.10}$$

  with $\boldsymbol{v} \in \mathbb{R}^m$ and $\boldsymbol{\Xi}_v(\cdot)$ invertible with respect to $\mathbf{w}$, which renders the compensated system:

$$\begin{cases} \dot{x} &= \mathbf{f}(\boldsymbol{x}, \boldsymbol{\Xi}_u(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w})) \\ \dot{\boldsymbol{\eta}} &= \boldsymbol{\Xi}_\eta(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w}) \end{cases} \tag{2.11}$$

  into the linear time-invariant controllable system:

$$z_i^{(r_i)} = v_i, \ 1 \leq i \leq m, \tag{2.12}$$

  for m integers r_i such that $\sum_{i=1}^m r_i = n + n_\eta$.

[2]The word "endogenous" means that the vectors $\boldsymbol{\eta}$ and $\boldsymbol{v} = [v_1, v_2, ..., v_m]^\top$ are generated only with the system's original trajectory, i.e., from $\{\boldsymbol{x}, \boldsymbol{u}, \dot{\boldsymbol{u}}, \ddot{\boldsymbol{u}}, ...\}$ [Martin, 1994].

In this form, the control design task boils down to closing the loop for the trivial system (2.12) with the new input v_i, then finding back the real input $\boldsymbol{u}$ with (2.8).This procedure hence aligns with the idea of standard feedback linearization technique discussed in [Isidori, 1985]. A relations between feedback linearization properties and flatness is depicted in Figure 2.3.

Differentially flat systems can be linearized via endogenous dynamic feedback.

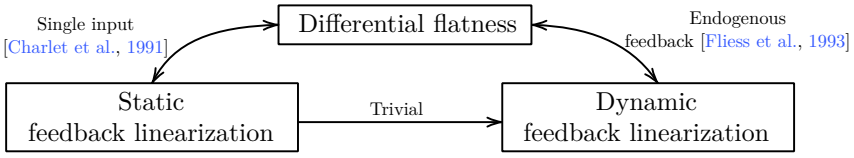


FIGURE 2.3: Relations between related classes of feedback linearizable systems.

Remark 2.1. The linear system (2.16) or generally (2.12), is also referred to as a *trivial system* or a system of Brunovsky canonical form, enjoying several properties such as controllability, observability or global stabilizability [Sontag and Sussmann, 1990]. □

Remark 2.2. Aligned with the framework of feedback linearization [Isidori, 1985], Proposition 2.1 implies that there exists a the linearizing coordinate change, or a *diffeomorphism*, under which the system evolves linearly as in (2.12). Such a coordinate change can be expressed as:

$$
\begin{aligned}
\boldsymbol{\zeta} &= \left[\begin{bmatrix} \Phi_{z,1}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \\ \cdots \\ \Phi_{z,1}^{(r_1-1)}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \end{bmatrix}^\top , \begin{bmatrix} \Phi_{z,2}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \\ \cdots \\ \Phi_{z,2}^{(r_2-1)}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \end{bmatrix}^\top , ..., \begin{bmatrix} \Phi_{z,m}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \\ \cdots \\ \Phi_{z,m}^{(r_m-1)}(\boldsymbol{x}, ..., \boldsymbol{u}^{(\boldsymbol{p})}) \end{bmatrix}^\top \right]^\top , \\
&= \left[\begin{bmatrix} z_1 \\ \cdots \\ z_1^{(r_1-1)} \end{bmatrix}^\top , \begin{bmatrix} z_2 \\ \cdots \\ z_2^{(r_2-1)} \end{bmatrix}^\top , \cdots , \begin{bmatrix} z_m \\ \cdots \\ z_m^{(r_m-1)} \end{bmatrix}^\top \right]^\top \\
\boldsymbol{v} &= [z_1^{(r_1)}, z_2^{(r_2)}, \cdots, z_m^{(r_m)}]^\top = \boldsymbol{\Xi}_v(\boldsymbol{x}, \boldsymbol{\eta}, \mathbf{w}),
\end{aligned}
\tag{2.13}
$$

where $\Phi_{z,i}(\cdot)$ denotes the ith row of the vector-valued function $\Phi_z(\cdot)$ in (2.2). Consequently, the system (2.12) can be rewritten as a concatenation of m chains of integrator, having equivalently the state space representation:

$$
\dot{\boldsymbol{\zeta}} = \boldsymbol{A}\boldsymbol{\zeta} + \boldsymbol{B}\boldsymbol{v}, \tag{2.14}
$$

where $\boldsymbol{A}, \boldsymbol{B}$ are constant matrices describing the dynamics (2.12). This coordinate system, hereinafter, will be referred to as the *flat output space* and $\boldsymbol{\zeta}, \boldsymbol{v}$ will be referred to as the flat state and flat virtual input[3], respectively. □

[3]This is not to be confused with the notion of *flat input* discussed in [Nicolau et al., 2020]

Particularly for the UAV example in (2.4), the dynamic feedback linearizing law can be deduced as:

$$\begin{cases} \dot{\boldsymbol{\eta}} &= w_1, \\ \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} &= \begin{bmatrix} \boldsymbol{\eta} \\ w_2 \end{bmatrix}, \end{cases} \quad \begin{cases} \boldsymbol{\zeta} &= \begin{bmatrix} x_1 & r\cos x_3 & x_2 & r\sin x_3 \end{bmatrix}^\top, \\ \boldsymbol{v} &= \begin{bmatrix} \cos x_3 & -r\sin x_3 \\ \sin x_3 & r\cos x_3 \end{bmatrix} \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}. \end{cases} \tag{2.15}$$

With (2.15), the linearized dynamics yields:

$$\ddot{z}_1 = v_1, \ddot{z}_2 = v_2, \tag{2.16}$$

or in a state-space form as in (2.14):

$$\dot{\boldsymbol{\zeta}} = \begin{bmatrix} 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 1 \\ 0 & 0 & 0 & 0 \end{bmatrix} \boldsymbol{\zeta} + \begin{bmatrix} 0 & 0 \\ 0 & 1 \\ 0 & 0 \\ 0 & 1 \end{bmatrix} \boldsymbol{v}. \tag{2.17}$$

Consequently, with the form of (2.16) or (2.17), the system becomes more manageable within the established control theory framework for linear systems [Sira-Ramirez and Agrawal, 2018].

So far, it has been shown that the practicality of flatness in control is immediately enabled after the choice of the flat output $\boldsymbol{z}$ of the form (2.2). However, the problem of how to formulate the function $\Phi_z(\cdot)$ in (2.2), or how to choose the flat output has not been addressed. For completeness, the next section will discuss the problem together with a brief review on the computation of flat output and current related open questions.

2.2.2 Computation of the flat output and its non-uniqueness

As suggested previously, the primary step to benefit from flatness is to "choose" the output resulting in the representation (2.3). This is usually the key step to verifying whether a system is flat. Generally, the problem of how to deduce a flat output remains open and only for certain classes was it addressed [Fliess et al., 1999b]. More specifically, for linear systems, the choice of flat output has been systematically summarized in [Sira-Ramirez and Agrawal, 2018, Levine, 2009] or [Suryawan, 2012]. Therein, it was shown that the flat output can be chosen as a proper linear combination of the states. It is also evident that for the class of robotic manipulators with computed-torque representation, the output can be chosen as the joint angles or the configuration coordinates. This is because with the so-called inverse dynamics transformation [Craig, 2004], all the states and inputs can be recovered by such a choice of output [Levine, 2009]. Thorough lists of flat outputs for different classes of electrical power systems are provided in the

survey [Soheil-Hamedani et al., 2016]. Exploiting the geometric representation of mechanical (Lagrangian) systems, catalogs for such a class can be found in [Murray et al., 1995, Martin et al., 2001]. Given a dynamical system implicitly described by equality constraints of the states and their time derivative, different algorithms were proposed in [Lévine, 2011, Franke and Röbenack, 2013] to transform the original system to the form of (2.12), revealing the linearizing coordinates and hence the formula for the flat output. A computational procedure for linear flat systems with delay is provided in [Morio et al., 2009a]. Other symbolic procedures can found in [Schlacher and Schöberl, 2007] or [Antritter and Verhoeven, 2010], to name a few.

Although the details on how to implement these algorithms will not be addressed in this work, the fact that they are built from different mathematical frameworks naturally triggers a question:

Will all of these existing algorithms produce the same solution?

Interestingly enough, the answer is negative. Without elaborating the details, let us revisit the UAV example (2.4) using the algorithm from [Franke and Röbenack, 2013], which provides an alternative vector of flat outputs:

$$\boldsymbol{z}^\star = \begin{bmatrix} z_1^\star & z_2^\star \end{bmatrix}^\top = \begin{bmatrix} x_1 \sin x_3 - x_2 \cos x_3, & x_3 \end{bmatrix}^\top, \tag{2.18}$$

After some computation, the state $\boldsymbol{x}$ can be expressed with $\boldsymbol{z}^\star, \dot{\boldsymbol{z}}^\star$ as:

$$\begin{aligned} x_1 &= \dot{z}_1^\star \dot{z}_2^{\star -1} \cos z_2^\star + z_1^\star \sin z_2^\star \\ x_2 &= \dot{z}_1^\star \dot{z}_2^{\star -1} \sin z_2^\star - z_1^\star \cos z_2^\star \\ x_3 &= z_2^\star. \end{aligned} \tag{2.19}$$

while the flat representation of the input $\boldsymbol{u}$ can be calculated by simply replacing $x_{1 \leq i \leq 3}$ in (2.19) and their time derivatives into (2.4).

Then, from different flat representations in (2.6) and (2.19), if the same advantages can be exploited, the question now is whether they imply different characteristics, and hence, different performance when being used for motion planning and control.

A preliminary answer for this concern can be immediately pointed out through the two flat representations (2.6) and (2.19) in the example. More specifically, each representation has different singular points in the new coordinate. For the former, it is the set of all points with $\dot{z}_1(t) = 0$ while for the latter, the trajectory is undefined where $\dot{z}_2^\star(t) = 0$, an example of the singular cases for both representations is given in Figure 2.4. Although in both scenarios, the pose of the vehicle is physically feasible, the parameterization with the two flat outputs is hindered by the definition of the representations (2.6) and (2.19).

Different flat outputs imply different benefits and drawbacks.

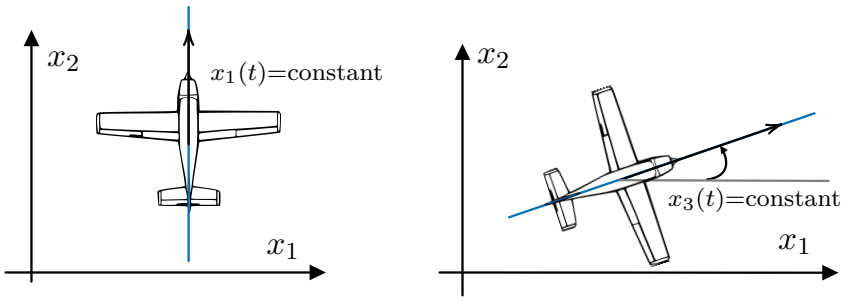


FIGURE 2.4: Two different singular cases for different flat output parameterizations of the UAV model from the flat output (2.5) (left) and (2.18) (right).

Furthermore, singular points present in one representation may be absent in the other, allowing them to be interchanged to avoid approaching their respective singularities. These are known as *apparent singularities*, leading to the question:

How can we effectively employ different flat outputs in control?

Although the question is relevant, analysis of the exploitation of different flat outputs has not been thoroughly studied in the literature, since deriving only one flat output is already considered challenging. A few particular efforts exploring this research gap can already be found for robotic systems. For instance, with the quadcopter model, Chang and Eun [2014] showed that there exist eight flat output vectors, creating a global atlas of flat outputs for the model. Therein, those different flat outputs were introduced to cover the entire configuration space. Correspondingly, with each region in the space, an associated dynamic feedback linearization law as in (2.8) was deduced, creating a globally singularity-free controller. In the context of generating a feasible trajectory for the same model, two different flat outputs [Nguyen, 2019] were used to reduce the complexity of the flat representation caused by trigonometric functions. The idea has been also discussed in [Kaminski et al., 2018] and [Kaminski and Ollivier, 2024] for a global planning of the so-called class of chained systems, with application to the unicycle and a twelve-dimensional aircraft model, respectively. These motivating case studies demonstrate that optimizing the use of different flat outputs is a promising direction, bridging the gap between nonlinear control and established solutions for linear systems.

To this point, the main properties of flatness discussed in the manuscript have been presented. These advantages of flat representation (2.3) and dynamic feedback linearization (2.8), are essentially two sides of the same coin, stemming from the equivalence of a system to a set of ordinary curves or chains of integrators. This attribute has undoubtedly been exploited in different stages of control design, from offline planning to online control [Levine, 2009, Sira-Ramirez and Agrawal, 2018]. In the remainder of this chapter, different axes of application will be elaborated along with the presentation of the theoretical tools used throughout of the dissertation.

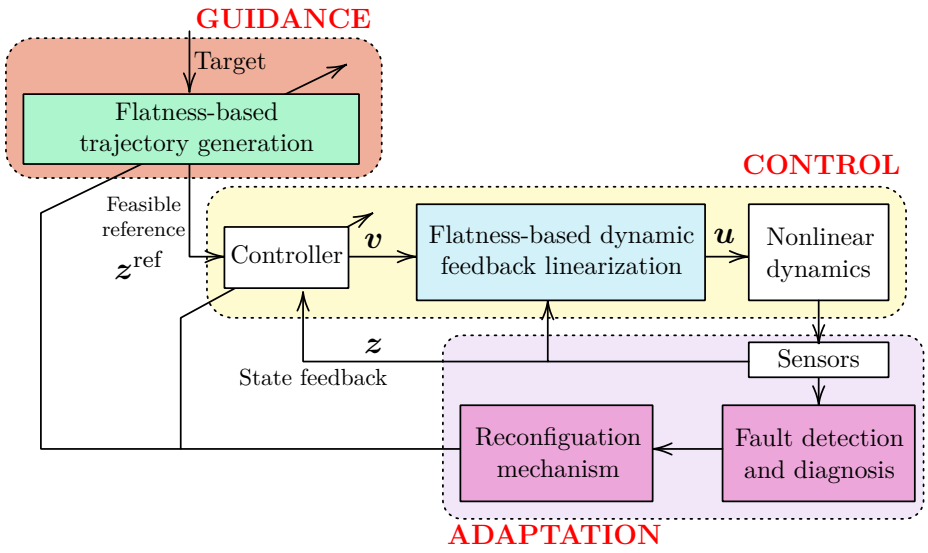


FIGURE 2.5: Different possible roles of flatness in motion planning.

The following sections of this chapter will review the use of flatness at various stages of motion planning as in Figure 2.5, along with pertinent theoretical and numerical tools. The first stage will be addressed as *guidance* where the flat representation of the states and inputs is integrated in curve-parameterization frameworks, leveraging the equivalence implication of flatness between the system's trajectory and ordinary curves. Second, control strategies induced by the dynamic feedback linearization law will be referred to at the *control* stage. Finally, the mechanism for managing uncertainties and/or unexpected events will be organized in the *adaptation* stage. This stage involves detecting fault or defects of the systems by comparing the nominal trajectory to the measured signals, and then providing reconfiguration decisions based on the system's healthiness. Although proven important in many applications [Shahin et al., 2016, Ortiz-Torres et al., 2020, Fischer and Deutscher, 2020], this step will not be discussed in the current manuscript to limit the scope of the discussion. A general scheme where flatness plays different roles in motion planning is depicted in Figure 2.5. The guidance and control phases will be discussed in the following sections.

2.3 Flatness-based trajectory generation

Trajectory or profile generation has always been a topic of importance in the control community since, with a proper reference signal, the controller computational burden or the closed-loop aggressive overshoot will be significantly reduced [LaValle, 2006]. In the context of optimal control, other performance metrics can also be taken into account in the trajectory generation stage, such as input effort, energy consumption, or transient time between two states. Conventionally, such tasks can be formulated as solving the

following optimal control problem (OCP):

$$\min_{\boldsymbol{u}(t)} J(\boldsymbol{u}) = \int_{t_0}^{t_f} \mathcal{L}(\boldsymbol{x}(t), \boldsymbol{u}(t))\mathrm{d}t, \tag{2.20a}$$

$$\boldsymbol{x}(0) = \boldsymbol{x}_0, \boldsymbol{x}(t_f) = \boldsymbol{x}_f, \tag{2.20b}$$

$$\boldsymbol{u}(0) = \boldsymbol{u}_0, \boldsymbol{u}(t_f) = \boldsymbol{u}_f, \tag{2.20c}$$

$$\boldsymbol{x}(t) \in \mathcal{X}, \boldsymbol{u}(t) \in \mathcal{U}, \tag{2.20d}$$

$$\dot{\boldsymbol{x}}(t) = \mathbf{f}(\boldsymbol{x}(t), \boldsymbol{u}(t)). \tag{2.20e}$$

The objective of the above problem is to find a time trajectory of the input signal $\boldsymbol{u}(t)$ in the time interval $t \in [t_0, t_f]$, driving the system from the initial state $\boldsymbol{x}_0$ to the final state $\boldsymbol{x}_f$ as in (2.20b)–(2.20c). Meanwhile, the constraints imposed consist of keeping the state and the input inside the sets $\mathcal{X}$ and $\mathcal{U}$ as in (2.20d), respectively. Finally, the differential constraint (2.20e) makes sure the trajectory generated is dynamically feasible, i.e., that it complies with the system's dynamics. This constraint is relevant since, without it, the generated reference may not be feasible for the system to follow. An example of an infeasible path for the UAV model (2.4) is depicted in red in Figure 2.6. This reference clearly violates the differential constraint $\dot{x}_1 \sin x_3 = \dot{x}_2 \cos x_3$ deduced from the dynamics (2.4) (although this is the shortest line connecting the two points). On the contrary, adding the dynamic constraint (2.20e) creates a more realistic solution in blue. Like this, we can also observe the system's behavior during its transition from state to state. However, in general, such a constraint is not trivial to handle and is usually approximated via discrete integration [Vinter, 2010, Karaman and Frazzoli, 2010].

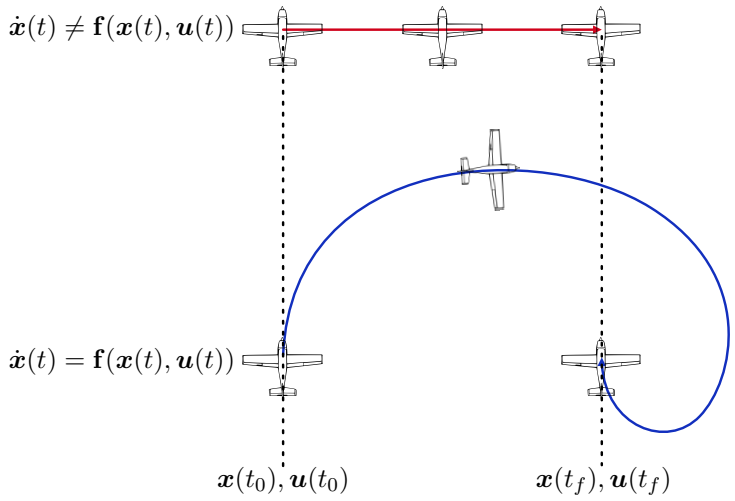


FIGURE 2.6: Infeasible (top) and feasible (bottom) trajectory of the UAV model.

Fortunately, this problem can be particularly simplified if the system accepts a flat representation (2.3). Namely, with a flat system, the OCP (2.20) can be reduced to

a static curve fitting problem without any dynamic constraint. This can be seen by replacing the flat representation (2.3) into (2.20):

$$\min_{\boldsymbol{z}(t)} J_z(\boldsymbol{z}(t)) = \int_{t_0}^{t_f} \mathcal{L}\left(\Phi_x(\boldsymbol{z}(t), ..., \boldsymbol{z}^{(q)}(t)), \Phi_u(\boldsymbol{z}(t), ..., \boldsymbol{z}^{(q+1)}(t))\right) \mathrm{d}t, \tag{2.21a}$$

$$\Phi_x(\boldsymbol{z}(t_0), ..., \boldsymbol{z}^{(q)}(t_0)) = \boldsymbol{x}_0, \Phi_x(\boldsymbol{z}(t_f), ..., \boldsymbol{z}^{(q)}(t_f)) = \boldsymbol{x}_f, \tag{2.21b}$$

$$\Phi_u(\boldsymbol{z}(t_0), ..., \boldsymbol{z}^{(q+1)}(t_0)) = \boldsymbol{u}_0, \Phi_u(\boldsymbol{z}(t_f), ..., \boldsymbol{z}^{(q+1)}(t_f)) = \boldsymbol{u}_f, \tag{2.21c}$$

$$\Phi_x(\boldsymbol{z}(t), ..., \boldsymbol{z}^{(q)}(t)) \in \mathcal{X}, \Phi_u(\boldsymbol{z}(t), ..., \boldsymbol{z}^{(q+1)}(t)) \in \mathcal{U}. \tag{2.21d}$$

In details, although the two problems (2.20) and (2.21) are equivalent, it can be pointed out that with the replacement of the flat representation, the differential condition (2.20e) always holds or becomes redundant. Therefore, in the formulation (2.21), such a constraint can be discarded. To this point, the OCP now becomes the task of finding a high-dimensional curve consistent with the boundary conditions (2.21b)–(2.21c) and the continuous time constraints (2.21d). The duality between the two problems is illustrated in Figure 2.7. This simplification, in turn, renders the OCP into a constrained curve parameterization, which can be managed by suitable curve fitting tools [Farin, 2014]. In the next section, curve parameterization frameworks frequently integrated with flatness will be introduced based on their level of complexity and geometric interpretation.

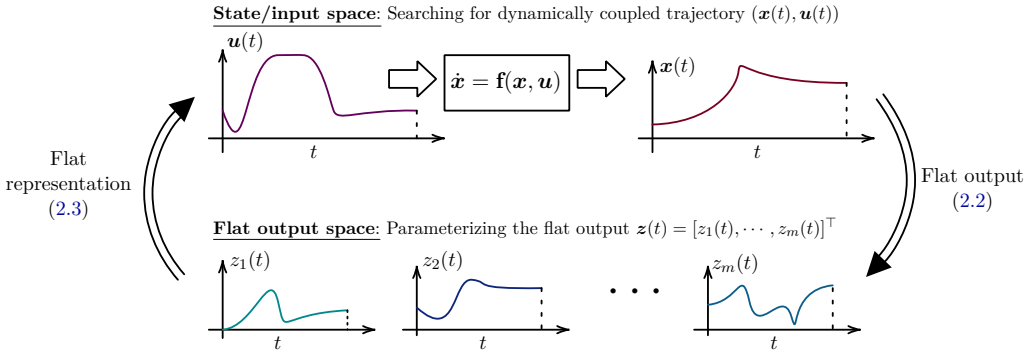


FIGURE 2.7: Interpretation of the conversion from the OCP (2.20) (top) to the curve parameterization (2.21) (bottom).

2.3.1 Parameterization tools

Typically, the parameterization of the ith component of the flat output vector $\boldsymbol{z}(t)$ will take the general form:

$$z_i(t) = \sum_{j=1}^{N} p_j \mathfrak{b}_j(t), \tag{2.22}$$

where the generated trajectory $z_i(t)$ is represented as a linear combination of N fixed basis functions $\mathfrak{b}_j(t), j \in \{1, ..., N\}$ with the coefficients p_j. This formulation, then reduces the problem (2.21) to finding a finite number of coefficients in p_j, yet at the expense of possible infeasibility if the basis $\mathfrak{b}_j(t)$ is too restrictive. Following such a setting, prevalent parameterization tools suitable for solving (2.21) will be discussed. These options are then employed subsequently to deduce satisfactory curves subject to typical problems of trajectory generation such as input saturation, collision, or obstacle avoidance. The subsection is, therefore, structured according to their particular representational power.

2.3.1.1 Polynomial parameterization

One of the most common tools to represent a smooth curve is the class of polynomial functions. In this framework, the trajectory of $\boldsymbol{z}(t)$ will be written as a linear combination of polynomial basis functions. Namely, the i-th flat output in (2.2) is adapted as follows:

$$z_i(t) = \sum_{j=1}^{d} a_{ij} t^{j-1}. \tag{2.23}$$

where $t \in [0,\, t_f]$ is the considered time interval and d is the degree of the polynomial associated with the coefficients a_{ij}. With a simple expression, this method has been widely used for reference generation transiting between setpoints [Beaver and Malikopoulos, 2024]. Although mathematically straightforward to compute, the evaluation of (2.23) will become sensitive to numerical error. This is because as the complexity of the curve increases (e.g., in the case of interpolation between multiple setpoints to avoid obstacles in robotic applications), the degree d must also increase. In these cases, even a minor numerical inaccuracy in t will be greatly magnified. For a more comprehensive analysis on this problem, we would like to refer to the specialized literature such as [Rose, 1982] or [Biagiotti and Melchiorri, 2008]. Moreover, this method is not commonly favored for physical applications such as robotic or automotive planning as the coefficients a_{ij} carry little geometric intuition. As an improvement for those specific drawbacks, another solution called Bézier curve will be introduced subsequently.

2.3.1.2 Bézier Curve

Indeed, though the matrix form of the Bézier curve is still a product of the basis function vector and the "coefficient" matrix, the representation of this curve gives the latter more geometric insight. To see this, consider a Bézier curve calculated as:

$$\boldsymbol{z}(t) = \sum_{j=0}^{d} \lambda_{j,d}(t) \boldsymbol{P}_j \quad t \in [0, 1] \tag{2.24}$$

where $\lambda_{j,d}(t)$ is the Bernstein polynomial [Bernštein, 1912]:

$$\lambda_{j,d}(t) = \frac{d!}{j!(d-j)!} t^j (1-t)^{d-j}, \quad t \in [0,1]. \tag{2.25}$$

and $\boldsymbol{P}_j \in \mathbb{R}^{m\times 1}$ are the curve's coefficients called *control points*. As an example, let us consider the control point matrix:

$$\boldsymbol{P}_1 = \begin{bmatrix} -1.5 \\ 1.5 \end{bmatrix}, \boldsymbol{P}_2 = \begin{bmatrix} 1 \\ 3.5 \end{bmatrix}, \boldsymbol{P}_3 = \begin{bmatrix} 1.5 \\ -1 \end{bmatrix}, \boldsymbol{P}_4 = \begin{bmatrix} 4 \\ -2 \end{bmatrix}, \boldsymbol{P}_5 = \begin{bmatrix} 5 \\ -1 \end{bmatrix}, \tag{2.26}$$

The curve $\boldsymbol{z}(t) = [z_1(t),\, z_2(t)]^\top$ and its parameterization are depicted in Figure 2.8.

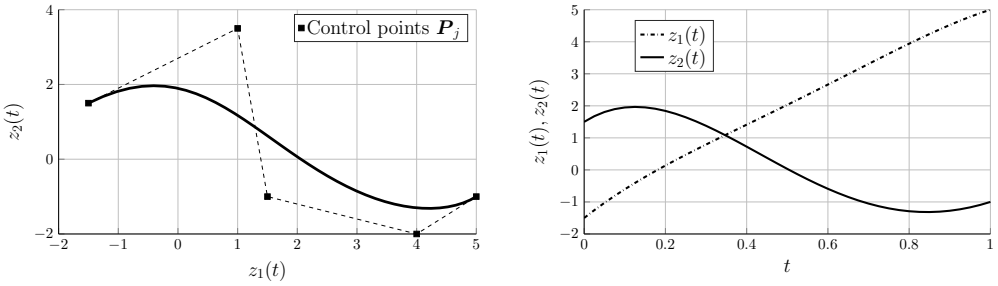


FIGURE 2.8: Bézier curve $\boldsymbol{z}(t) = [z_1(t), z_2(t)]^\top$ constructed from (2.26).

Remark 2.3. It is important to note that although the curve design procedure always takes place inside the "time" interval $t \in [0,\, 1]$, one can always change the time scale without changing the control points. This can be done by substituting $t = (t' - t_0)/(t_f - t_0)$ where $t' \in [t_0,\, t_f]$ is the new time variable.

As can be seen in Figure 2.8, the position of the control points directly shapes the resulting curve. For example, the initial and final control points coincide with the two extremities of the curve, or the two first and last control points define a line tangent to the generated curve. In other words, this parameterization method provides a geometrically more insightful way to construct a trajectory. This advantage can then be employed in a set-theoretic framework or rigid body trajectory generation with a constrained workspace [Galvane et al., 2015, Luis et al., 2020b]. Despite its wide application in position planning, the Bézier basis in (2.25) confronts the same numerical disadvantage as the polynomial basis in (2.23). Furthermore, in a scenario where the trajectory has to be reconfigured locally (e.g., to avoid obstacles), Bézier curve will not be suitable since it has a geometric characteristic called *global support*. Namely, changing the position of any control point $\boldsymbol{P}_j$ in (2.24) will alter the curve entirely [Suryawan, 2012]. This is particularly not desirable when we want to refine the curve in the vicinity where constraints are violated, rather than seeking a completely new set of coefficients. Therefore, a more advanced curve carrying the same advantages called *B-spline* will be introduced

in the next part. This parameterization exhibits both the advantage of local support and the computational simplicity of piecewise polynomials.

2.3.1.3 B-spline curve

Taking the typical form of a parameterized curve, a B-spline can be described by:

$$\boldsymbol{z}(t) = \sum_{i=0}^{N} \boldsymbol{P}_i \mathrm{B}_{i,d}(t) \triangleq \mathbf{P}\mathbf{B}_d(t); \quad t \in [t_0, t_f] \tag{2.27}$$

where $\boldsymbol{P}_i \in \mathbb{R}^m$ are the control points and $\mathrm{B}_{i,d}(t), i \in \{0, 1, ..., N\}$ are the basis functions, which can be written in matrix form as:

$$\begin{aligned} \mathbf{B}_d(t) &\triangleq \begin{bmatrix} \mathrm{B}_{0,d}(t) & \mathrm{B}_{1,d}(t) & \dots & \mathrm{B}_{N,d}(t) \end{bmatrix}^\top \\ \mathbf{P} &\triangleq \begin{bmatrix} \boldsymbol{P}_0 & \boldsymbol{P}_1 & \dots & \boldsymbol{P}_N \end{bmatrix}, \end{aligned} \tag{2.28}$$

where $N + 1$ is the a priori given number of control points, d is the B-spline order and the basis functions are recursively defined as [Lyche et al., 2018]:

$$\begin{aligned} \mathrm{B}_{i,1}(t) &= \begin{cases} 1 & \tau_i \leq t < \tau_{i+1}, \\ 0 & \text{otherwise} \end{cases} \\ \mathrm{B}_{i,d}(t) &= \frac{t - \tau_i}{\tau_{i+d-1} - \tau_i} \mathrm{B}_{i,d-1}(t) + \frac{\tau_{i+d} - t}{\tau_{i+d} - \tau_{i+1}} \mathrm{B}_{i+1,d-1}(t) \end{aligned} \tag{2.29}$$

for $d \in \mathbb{N}, d > 1$, $i = 0, 1, ..., N = n_{\mathbb{T}} - d - 1$ and $\frac{0}{0} := 0$. The knot time instants is collected in the set called *knot vector*:

$$\mathbb{T} = \{\tau_0, \tau_1, ..., \tau_{n_{\mathbb{T}}}\} \tag{2.30}$$

which has $n_{\mathbb{T}} + 1$ non-decreasing components, i.e., $\tau_k \leq \tau_{k+1}$.

Remark 2.4. The recursive definition of $\mathrm{B}_{i,d-1}(t)$ provides a continuous and piecewise polynomial in every interval $t \in [\tau_i, \tau_{i+1})$ (a.k.a., the piecewise structure of B-splines [Lyche et al., 2018]). This formulation is highly representative since, in this way, complex curves can be constructed by segments of relatively low-order polynomials, creating a numerically stable parameterization.

Depending on the application, the knot vector $\mathbb{T}$ is chosen differently. For example, with the application of curve interpolation, the set can be constructed via the data points in the space of the curve and its derivatives, resulting in a unique solution from a system of linear equations [Suryawan, 2012]. In the context of searching for an optimal trajectory among an infinite number of solutions, the set is typically chosen such that its d first and

last components are equal and the intermediate internal knots are chosen equidistantly:

$$\begin{cases} \mathbb{T} = \{\underbrace{\tau_0, \tau_1, ..., \tau_{d-1}}_{d \text{ equal knots}}, \underbrace{\tau_d, \tau_{d+1}, ..., \tau_N}_{N-d+1 \text{ internal knots}}, \underbrace{\tau_{N+1}, ..., \tau_{N+d}}_{d \text{ equal knots}}\} \\ \tau_0 = t_0, \tau_{N+d} = t_f. \end{cases} \tag{2.31}$$

The first and the last d equal knots implicitly ensure the end-point interpolation property up to the d derivatives of the parameterized signals (i.e., the last and the first control points align with the starting and ending points of the curve). This is relevant when it comes to solving the boundary conditions (2.21b) and (2.21c). An illustration of the basis functions formed by the above choice of knots is given in Figure 2.9.

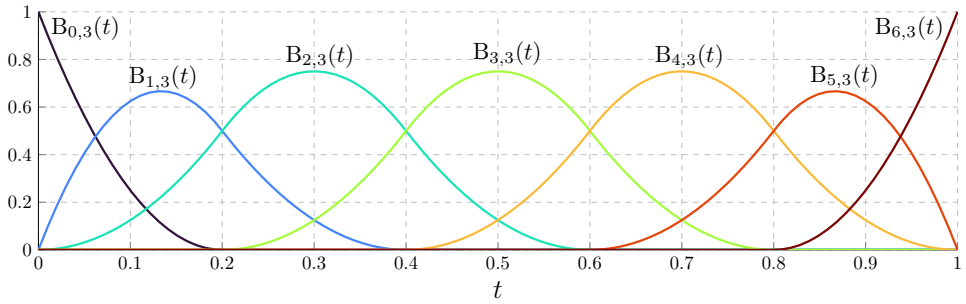


FIGURE 2.9: B-spline basis functions with the knot vector as in (2.31), $d = 3, n = 6$.

For illustration, let us construct a parameterized trajectory $\boldsymbol{z}(t) = [z_1(t), z_2(t)]^\top$ with time variable t and each components of $\boldsymbol{z}(t)$ is required to be 3-order derivable or $d = 3$. Given the control points $\mathbf{P}$, an example is depicted in Figure 2.10.

By construction, some interesting properties of B-spline can be pointed out as:

P1. the curve always lies inside the union of all convex polygon defined by every d successive control points (Figure 2.10);

P2. the derivatives of B-spline are also B-spline computed by as a linear combination of the pieces of higher degree on the same segment $[\tau_i, \tau_{i+1})$;

P3. Changing the value of the control point $\boldsymbol{P}_i$ will only alter the curve locally inside the interval $[\tau_i, \tau_{i+d+1})$ (see Figure 2.10).

With such advantages of geometric modeling, B-spline will be employed for all the trajectory generation phases in the manuscript.

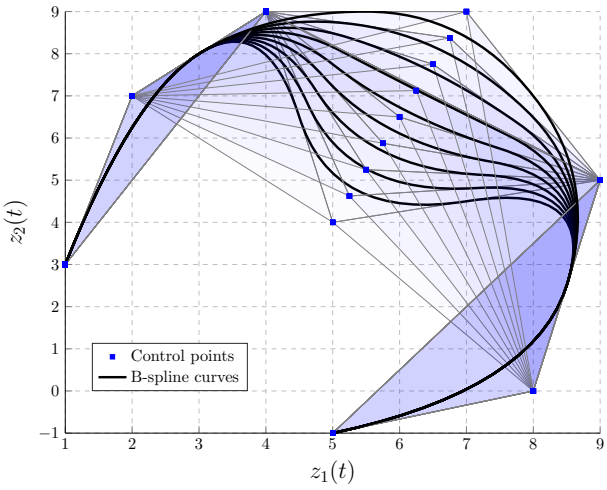


FIGURE 2.10: B-spline curve ($d = 3$) with the local support property and the polygons inscribing the curves.

2.3.1.4 Periodic parameterization

The final category of parameterization tool we would like to draw attention to is the class of periodic basis functions. As suggested by the name itself, they are exploited, particularly for flat systems orbiting periodically. A few notable applications can be found for the optimization of aircraft flight range or fuel consumption [Ogunbodede et al., 2019b, Varigonda et al., 2004], blood glucose concentration [Nandi and Singh, 2019], satellite docking [Farahani et al., 2016], or drug delivery [Guay et al., 2007]. For this class, the output is usually parameterized with the truncated Fourier series as:

$$\boldsymbol{z}(t) = \boldsymbol{P}_0^{off} + \sum_{i=1}^{L} \boldsymbol{P}_i^s \sin(i\omega t) + \boldsymbol{P}_i^c \cos(i\omega t) \tag{2.32}$$

where $\boldsymbol{P}_0^{off}$ is the offset value while $\boldsymbol{P}_i^s, \boldsymbol{P}_i^c \in \mathbb{R}^m$ denote the vector coefficients assigned to each frequency $i\omega, i \in \{1, ..., L\}$.

To this end, common parameterization tools have been presented. The next ingredients to define the optimization (2.21) contain the cost function $\mathcal{L}_z(\cdot)$ and the corresponding constraints. These elements will be described in the next section.

2.3.2 Optimal trajectory generation with flatness

In this part, flatness-based optimal trajectory generation studies will be reported and structured according to the employment of different control objectives (i.e., constraint

satisfaction and/or cost minimization). Indeed, since the dynamics constraints represented by the ordinary differential equations (ODE) (2.20e) are automatically satisfied via the differential parameterization of smooth curves in the flat output space, the main challenge resides in how to characterize the additional constraints to achieve a computationally tractable optimization problem. In this review, although we focus on motion planning applications, the adaptation to any flat system can be carried out in a similar manner. In what follows, we aim at listing the two most common costs: time and energy while the generated trajectory must satisfy operating (input or state) constraints; ensure safety conditions like collision or obstacle avoidance; and achieve the task assigned such as passing through predefined targets in the configuration space.

Objective: minimize **time/energy**
1. Way-point constraints;
2. Obstacle/Collision avoidance;
3. Operating constraints;

2.3.2.1 Objective/cost formulation

Herein, two common types of cost definition for the OCP (2.21) will be reported along with the research works that have examined them.

Total-time optimization

The objective of this formulation is to minimize the task execution time required. Since the time variable is unchanged via the coordinate transformation, the objective of minimizing the end-time can be handled similarly in standard OCP, i.e., without any manipulation related to the curve parameterization tools. The idea of time optimization is to include the end-time t_f in (2.21a) also as a variable to optimize, either by iteratively decreasing from a large value or directly solving it as a free variable penalized in the cost function, i.e., the cost in (2.21a) adopts the form:

$$\min J_z = J_z(t_f). \tag{2.33}$$

In [Guay et al., 2007], for the periodic curve employed therein, the log-barrier function was added to penalize state/input constraint violation, and as a result, a feasible (suboptimal) solution was derived iteratively with the modified cost without any hard constraints imposed. Time optimization is also taken into account as a weighted term in the objective function: $J_z(t_f) = \mu_{t_f} \log(t_f - \epsilon_{t_f})$ where $\mu_{t_f}, \epsilon_{t_f}$ are the tuning parameters. This objective can also be represented simply as $J_z(t_f) = t_f$ [Zhao et al., 2018, Hu and Mishra, 2017, Han et al., 2023] or in quadratic form, t_f^2 [Andrej and Meurer, 2018]. In the context of "discretizing the flat output space", another technique to achieve minimal end-time variable is to fix a sampling time t_s and increase the length of the investigated time interval $[t_0, t_0 + N_s t_s]$ by iteratively increasing the integer N_s. This technique

is relevant when operating constraints are considered. The procedure will be stopped when no violation of constraint is found. In [Zhang et al., 2020, Verscheure et al., 2008, Van Loock et al., 2013], from a geometric viewpoint, problem (2.21) can also be considered in a normalized timescale (a.k.a, path coordinate) $s \in [0, 1]$. Then, with the reformulation of the constraints with respect to the variable T, the optimal path can be generated and converted back to the original timescale.

Energy optimization

Meanwhile, to increase actuators' life expectancy or reduce battery/fuel consumption, the system trajectory is also constructed with the energy minimization objective. In the literature, the term *energy minimization* is usually referred to as an optimization with the cost written in terms of the input's trajectory or the integral of its absolute value over a predefined interval of time.

Generally, in the flat output space, handling such a cost is not an easy task. This is because even a simple (but meaningful) penalty function $\mathcal{L}(\cdot)$ in (2.20), such as $\mathcal{L}(\boldsymbol{x}(t), \boldsymbol{u}(t)) = \|\boldsymbol{u}(t)\|^2$ may pose a complicated expression after being transformed to its dual form in the flat output space as in (2.21a), i.e., $\mathcal{L}(\Phi_x(\cdot), \Phi_u(\cdot)) = \|\Phi_u(\cdot)\|^2$. Some particular efforts to characterize such convoluted costs can be found in the minimization of quadcopter's generated thrust [Stoican et al., 2024, Marguet et al., 2024b], manipulator's torque [Ross and Fahroo, 2006, Verscheure et al., 2008] or energy consumption of the electrical grid [Zafeiratou et al., 2020].

On the other hand, to sidestep this convolution of the cost via the mapping (2.3), it is typically preferred to reformulate a simpler cost for (2.21) directly in function of the flat output and its derivatives. One typical choice is to replace $J_z(\boldsymbol{z}(t))$ in (2.21a) with:

$$\min \tilde{J}_z(\boldsymbol{z}(t)) = \int_0^{t_f} \|\boldsymbol{z}^{(r)}(t)\|^2 \mathrm{d}t. \tag{2.34}$$

There are two reasons for this selection. Computationally, on one hand, this cost is simple to evaluate when the trajectory of $\boldsymbol{z}(t)$ accepts a parameterization form as in (2.22). For example, with the B-spline curve as in (2.27), the cost is simply quadratic with respect to the decision variable, or the control points in $\mathbf{P}$. To show that, first, it should be noted that B-spline allows computing exactly the time derivative of the signal as a linear combination of its lower-order basis functions, namely, with $\mathbf{B}_d(t)$ in (2.28):

$$\mathbf{B}_d^{(r)}(t) = \boldsymbol{M}_{r,d-r}\mathbf{B}_{d-r}(t), \tag{2.35}$$

where the constant matrix $\boldsymbol{M}_{r,d-r}$ is a *derivation matrix* that can be readily computed with r, d and the spline parameters (see Section 3 [Suryawan, 2012] for its explicit formulation); $\mathbf{B}_{d-r}(t)$ is the set of basis functions with order $d - r$. Then, the cost

function (2.34) yields [Stoican et al., 2015b]:

$$\begin{aligned}\min \tilde{J}(\boldsymbol{z}(t)) &= \min \int_0^{t_f} \|\boldsymbol{z}^{(r)}(t)\|^2 \mathrm{d}t = \min \int_0^{t_f} \left\| \sum_i \lfloor \boldsymbol{P}\boldsymbol{M}_{r,d-r} \rfloor_i B_{i,d-r}(t) \right\|^2 \mathrm{d}t \\ &= \min_{\boldsymbol{P}} \sum_{i,j} (\lfloor \boldsymbol{P}\boldsymbol{M}_{r,d-r} \rfloor_i)^\top \underbrace{\int_0^{t_f} B_{i,d-r}(t) B_{j,d-r}(t) \mathrm{d}t}_{\text{constant}} (\lfloor \boldsymbol{P}\boldsymbol{M}_{r,d-r} \rfloor_j).\end{aligned} \tag{2.36}$$

where for a matrix $\boldsymbol{E}$, $\lfloor \boldsymbol{E} \rfloor_i$ denotes its ith column.

On the other hand, the optimization of such a cost function implies somewhat the minimization of the input's exploitation or energy consumed, thanks to the physical meaning of the flat output in specific applications. In the literature, the idea was indeed employed in [Mellinger and Kumar, 2011, Liu et al., 2017] for quadrotors and VTOL fixed-wing aircraft [Tal et al., 2023]. Therein, the vehicle's snap (the fourth derivative of the flat output) and the second derivative of the yaw angle are minimized since these two quantities can be derived from the input's flat representation of the system. A particular indicator written in function of the flat output was also provided in [Murali et al., 2019] to maximize the visibility of the drone for surveillance. A similar effort for the quadrotor-payload system is introduced in [Zeng et al., 2020] with up to the six-order derivative of the load's position or in autonomous driving [Han et al., 2023] with the car's pose as the cost argument. Meanwhile, the minimization in terms of the trajectory's length was formulated as the integral of the Euclidean norm of the velocity (the flat output's first derivative) [Stoican et al., 2015b, Do et al., 2021]. A minimization problem of the dissipative energy for the DC-microgrid elevator system is studied in [Pham et al., 2015]. This energy function can be, again, written in the quadratic form of the control points.

Besides those preferable choices of the cost function, with the same parameterization tool, a particular objective formulation for vibration minimization in flexible mechanical systems was provided in [Tsuei and Milam, 2016]. Therein, Fourier transformation was employed to derive the cost function in the frequency domain. With a view to constructing fuel-efficient periodic references for UAV systems, Fourier series were also employed as basis functions in [Ogunbodede et al., 2019a, 2018].

2.3.2.2 Constraint formulation

The next component is the emblematic types of constraints in motion planning. Although different types of constraints will be discussed, the common intuition among them is that of retaining the high-dimensional curve of $\boldsymbol{z}(t)$ inside the allowed regions of the flat output space (or outside the interdicted sets). With each constraint, a brief description of these regions and how they are preferably implemented under the B-spline framework will be provided.

Way-point passing constraints

The first constraint to be dealt with is way-point passing, meaning the system needs to pass through some predefined configurations at a prescribed time. This constraint is one of the most fundamental elements for trajectory generation, since the OCP's boundary conditions also fall into this category. In practice, these constraints usually come after the task assignment stage (i.e., "deciding who goes where?" [Ioan et al., 2021]), forcing the system's output $\boldsymbol{y} = \boldsymbol{c}(\boldsymbol{x})$ to be in the right place at the right time, namely:

$$\boldsymbol{y}(t_k) = \boldsymbol{c}(\boldsymbol{x}(t_k)) = \boldsymbol{y}_k, \tag{2.37}$$

where $\boldsymbol{y}_k$ is the configuration that $\boldsymbol{y}$ has to adopt at the time $t = t_k$. Generally for flat system, this constraint is not straightforward to characterize. This is because with the flat representation (2.3), (2.37) becomes:

$$\boldsymbol{c}(\Phi_x(\boldsymbol{z}(t_k), \dot{\boldsymbol{z}}(t_k), ..., \boldsymbol{z}^{(q)}(t_k))) = \boldsymbol{y}_k, \tag{2.38}$$

for which the complexity depends on the mapping $\Phi_x(\cdot)$.

For this reason, among various choices of flat outputs from different algorithms, (see Section 2.2.2), one of the selection criteria is to pick the flat output with physically meaningful values, making the expression (2.38) simple. The most common case is where the output $\boldsymbol{y}$ can be expressed as the linear combination of the flat output:

$$\boldsymbol{y} = \mathbf{C}_y \boldsymbol{z}, \tag{2.39}$$

for some constant matrix $\mathbf{C}_y$, rendering (2.37) to:

$$\mathbf{C}_y \boldsymbol{z}(t_k) = \boldsymbol{y}_k. \tag{2.40}$$

By doing this, the constraint imposed on the parameterized curve, such as in the form of (2.22), will be a manageable linear equality constraint. Notable applications includes mainly mechanical or robotic systems where the 3D position or system's output is already included in the entries of the flat output $\boldsymbol{z}$, for example, unicycle-like systems [Guo et al., 2007, Hervagault et al., 2019], quadcopters [Mellinger and Kumar, 2011, Richter et al., 2016], industrial manipulators [Markus et al., 2017] or VTOLs [Tal et al., 2023]. Flatness, in this context, is useful since without worrying about dynamical feasibility of the reference, many constraints application-specific requirement can be added into the planning problem, such as object manipulation [Seo et al., 2017] or area coverage for structure inspection [Rousseau et al., 2019] cinematographic capture [Stoican et al., 2019].

Up to this point, the most basic optimization problem to generate a feasible reference can already be formulated. for example, in a simplest form, with the minimum length cost in (2.34) with $r = 2$ passing through a set of waypoints for some task completion. However, with only those ingredients, the generated reference is barely enough in practice, as

automated systems always have to navigate in complex and dynamic environments. Thus, the trajectory generation for completing tasks naturally extends to an equally crucial aspect: ensuring safety. In the context of motion planning, in the next part, we refer to safety as keeping the system's trajectory in a collision-free zone.

Obstacle/collision avoidance

In the flat output space containing $\boldsymbol{z}(t)$, where no dynamics are considered, the problem involves keeping the curve outside some interdicted regions, which has been extensively studied not only in motion planning but also in computer-aided design curve generation [Redon et al., 2004, Farin, 2014]. For navigation in two- or three-dimensional space, this problem is described by the exclusion of the trajectory $\boldsymbol{z}(t)$ from a convex shape, which, in general, is a non-convex constraint and difficult to handle. Depending on the computational power and theoretical tools at hand, different approaches can be used to deal with this constraints. In this thesis, since the main application is for quadcopters and robotic systems, the discussion will focus mainly on safety enforcement for systems with the position space of two or three dimension, in combination with the B-spline framework[4].

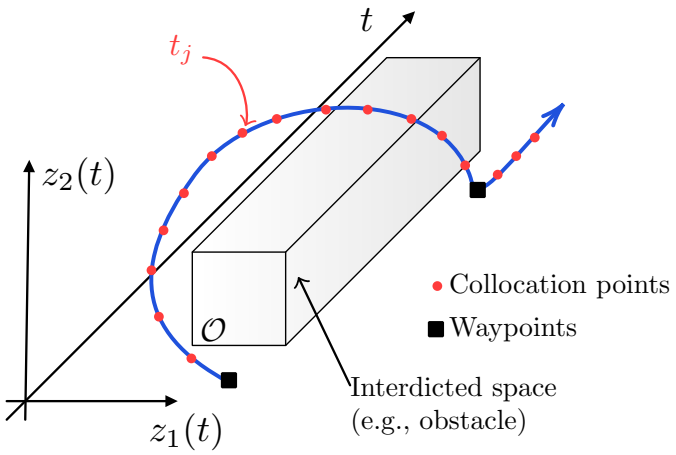


FIGURE 2.11: The parameterization passing through way-points and avoiding collision at the collocation time.

Consider, for example, in $\mathbb{R}^2$ a convex set of interdicted region $\mathcal{O} \subset \mathbb{R}^2$ and one needs to keep the curve $\boldsymbol{z}(t) = [z_1(t), z_2(t)]^\top$ outside from such set, i.e.,

$$\boldsymbol{z}(t) \notin \mathcal{O}, \forall t \in [t_0, t_f]. \tag{2.41}$$

In this form, the constraint is enforced as all time $t \in [t_0, t_f]$, creating infinitely many constraints. From this step, the treatment of this constraint is split into two main directions: sampling-based and continuous time approaches.

[4]Although being configured with B-spline, the discussion below can be easily adapted to any parameterization methods with the provided references.

The idea behind the former is simple. Instead of enforcing (2.41) continuously, the time interval $[t_0, t_f]$ is discretized into a finite one-dimensional grid, $\mathcal{T}_s = \{t_j\} \subset [t_0, t_f]$, called *collocation time*, then the constraint imposed can rewritten as:

$$\boldsymbol{z}(t) \notin \mathcal{O}, \forall t \in \mathcal{T}_s. \tag{2.42}$$

Intuitively, if the time interval is sufficiently sampled, the continuous time constraint (2.41) can be ensured by (2.42). However, the precise criteria for defining a good resolution of $\mathcal{T}_s$, which ensures (2.42) computationally feasible while also guarantees (2.41), is still a subject of ongoing discussion in the current literature. Some standard construction and adaptation strategy for the set $\mathcal{T}_s$ can found in [Mellinger and Kumar, 2011, Chen et al., 2016] or [Richter et al., 2016] for quadcopters, where iteratively, the parameterized curve was refined by adding additional mid-points where a collision with the obstacle is detected. This methodology to generate a collision-free trajectory is intuitively simple, but the implementation is rather intricate, since it requires a nest of different algorithms, including collision detection [Pan et al., 2012, Jiménez et al., 2001] and the iterative re-optimization of the trajectory.

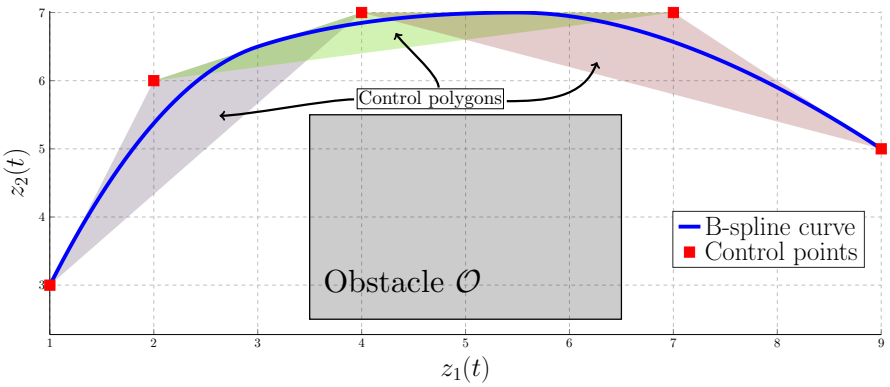


FIGURE 2.12: Collision-free curve generation by excluding the control polygons from the obstacle.

With a finite number of constraints, such a sampling-based approach provides a computationally tractable program. However, the guarantee for the satisfaction of condition (2.41) is not explicitly taken into account with such a methodology, motivating the latter with a geometric view point and a continuously satisfactory trajectory. More specifically, within the B-spline framework, the variable curve can be segmented into sections that are contained in a connected sequence of polytopes, called *control polygons* (see Figure 2.12). This segmentation renders the problem (2.41) into excluding these polygons, sequentially, out of the interdicted region $\mathcal{O}$. If the obstacle can be represented or over-approximated by a polytope, then many solutions are already available in the existing literature. For example, from a combinatorial viewpoint, the exclusion can be formulated as separating the control polygons from at least one facet of the obstacle,

creating a collection conditional constraints. These constraints can be efficiently handled within the mixed-integer programming (MIP) framework [Ioan et al., 2021, Prodan et al., 2019]. Avoiding using computationally heavy integer programs, different efforts have been made to characterize the knot vector and control points in (2.30), resulting in more computationally attractive software to generate trajectories with continuous time guarantee [Nguyen et al., 2023, Yu et al., 2024]. Another research direction to advance in this methodology is to tighten the control polygons towards the generated B-spline, creating a less conservative collision avoidance constraints, especially when more than one obstacles are considered [Dinh et al., 2024, Marguet et al., 2024a, Nguyen et al., 2023, Stoican et al., 2021].

To this point, a brief discussion on how to enforce the collision avoidance constraint (2.41) has been presented, with both discrete collocation in time and spline-based characterization in space. In truth, it is evident that the analysis has not fully covered the state-of-the-art techniques for such a spatial restriction. However, if we take away the context of the optimal control problem and concentrate exclusively on this constraint, the discussion will be eventually shifted to the construction of trivial curves (probably in 2D or 3D) and avoid unsafe regions. Since this is definitely not a novel problem, especially in computer-aided curve geometric design, we would like to conclude the revision related to the constraint (2.41) here and refer to existing surveys in the literature that address the problem geometrically, such as [Li et al., 2022, Sanchez-Ibanez et al., 2021] and [Ravankar et al., 2018].

As well, from the above observation, it is worth reminding that flatness has connected the planning of a dynamically constrained trajectory to a static generation of a parametric curve with which the two previous groups of constraints have been intensively studied. However, while converting the OCP to a static curve fitting problem, the flat mapping (2.3) also comes with a cost of an undesirably distorted coordinate system. The following part will explore various methods for managing such a complicated transformation.

Operating constraints

As introduced previously, the final and most challenging type of constraint to tackle in the flat output space will be discussed in this part. As in every problem exploiting coordinate changes, with flatness, although nonlinearity is canceled in the dynamics, all the meaningful descriptions of the system, such as actuator constraints or output limits, will be somewhat distorted. To demonstrate the unfavorable effect of the flat mapping (2.3), reconsider the model (2.4), but with a standard box constraint on the velocity and the bank angle:

$$\begin{bmatrix} \underline{u}_1 \\ -\bar{u}_2 \end{bmatrix} \leq \begin{bmatrix} u_1 \\ \tan u_2 \end{bmatrix} \leq \begin{bmatrix} \bar{u}_1 \\ \bar{u}_2 \end{bmatrix}, \tag{2.43}$$

with $\underline{u}_1 = 10, \bar{u}_1 = 26$ (m/s), $\bar{u}_2 = 0.61$. Then, while the dynamics (2.4) are linearized via the variable change (2.6), the constraints (2.43) become:

$$\begin{bmatrix} \underline{u}_1 \\ -\bar{u}_2 \end{bmatrix} \leq \begin{bmatrix} \sqrt{\dot{z}_1^2 + \dot{z}_2^2} \\ (\ddot{z}_2\dot{z}_1 - \ddot{z}_1\dot{z}_2)/\left(g\sqrt{\dot{z}_1^2 + \dot{z}_2^2}\right) \end{bmatrix} \leq \begin{bmatrix} \bar{u}_1 \\ \bar{u}_2 \end{bmatrix}. \tag{2.44}$$

A simplified sketch[5] of the feasible domain is given in Figure 2.13. This illustration suggests the main drawback of the flat formulation: the loss of convexity, which is definitely undesirable in solving optimization problems.

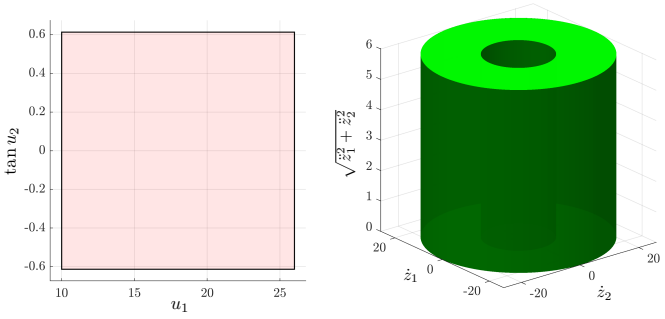


FIGURE 2.13: Admissible set of $\boldsymbol{u}$ with the UAV model (2.4) in the original state (left) and the flat output space (right)

Formally, with the general notations in Section 2.2.1, suppose that in the original coordinate, the state and input are constrained:

$$(\boldsymbol{x}, \boldsymbol{u}) \in \mathcal{X} \times \mathcal{U}. \tag{2.45}$$

Then, using the new coordinate (2.13), the feasible domain is defined as:

$$(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{C} = \{(\boldsymbol{\zeta}, \boldsymbol{v}) \text{ as in (2.13)} \ : \Phi_x\left(\boldsymbol{z}, ..., \boldsymbol{z}^{(q)}\right) \in \mathcal{X}, \Phi_u\left(\boldsymbol{z}, ..., \boldsymbol{z}^{(q+1)}\right) \in \mathcal{U}\}. \tag{2.46}$$

In fact, since the flat mapping does not have to maintain convexity, it is not generally clear that discarding the dynamic constraint in (2.21) will be worth the possible loss of convexity in terms of making the optimization problem easier [Ross and Fahroo, 2006]. Conversely, this also implies that if we have a convex description of the transformed feasible domain (2.46), flatness will definitely provide a computationally attractive program [Faulwasser et al., 2014].

Indeed, several efforts have been dedicated to providing a general inner approximation of $\mathcal{C}$ in (2.46). The earliest attempt identified by the authors is the work of [Faiz et al., 2001], where the idea is to find a maximal volume polytope contained in $\mathcal{C}$. With the

[5] One has $\left|(\ddot{z}_2\dot{z}_1 - \ddot{z}_1\dot{z}_2)/(g\sqrt{\dot{z}_1^2 + \dot{z}_2^2})\right| \leq \sqrt{\ddot{z}_1^2 + \ddot{z}_2^2}/g$, then (2.44) holds if $\sqrt{\ddot{z}_1^2 + \ddot{z}_2^2} \leq g\bar{u}_2$.

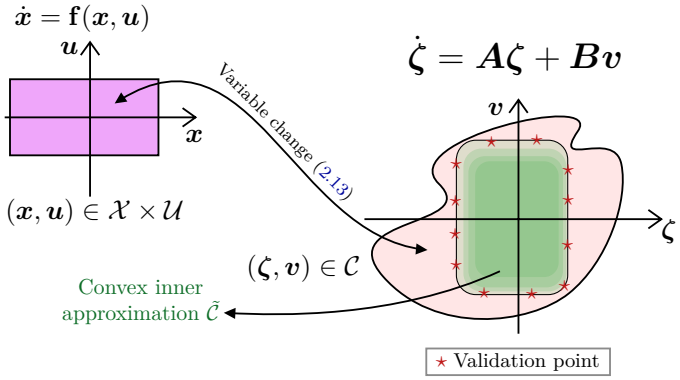


FIGURE 2.14: Constraint deformed in the flat output space and the set annexion problem for inner approximation.

vertices of the polytopes as the decision variables, the inner-approximation pertains to a semi-infinite program and is successfully applied to aircraft safe flight planning. This methodology is referred to as *set annexion* problem, where the convex inner approximation is obtained by inflating a predefined class of convex set (see Figure 2.14), namely by solving:

$$\max \mathrm{Vol}(\tilde{\mathcal{C}}), \tag{2.47a}$$

$$\tilde{\mathcal{C}} \subset \mathcal{C} \text{ as in } (2.46), \tag{2.47b}$$

where $\mathrm{Vol}(\tilde{\mathcal{C}})$ denotes the volume of $\tilde{\mathcal{C}}$. Still, solving such infinite program and verifying the condition $\tilde{\mathcal{C}} \subset \mathcal{C}$ in (2.47b) continuously are not trivial. Later, for an application of satellite slew maneuvering, Louembet et al. [2007] proposed both an inner and outer approximation for the domain. On one hand, the latter was heuristically treated with interval arithmetics, i.e., the new state-space of interest was gridded into box-type regions and checked individually. On the other hand, for the former, an ellipsoidal set $\tilde{\mathcal{C}}$ was inflated inside to constraint set $\mathcal{C}$. This choice of geometric set indeed provided a more tractable program, since the condition $\tilde{\mathcal{C}} \subset \mathcal{C}$ can be validated at the boundary points $\boldsymbol{b} \in \partial\tilde{\mathcal{C}}$, which can be finitely sampled with the parametric representation of a standard ellipsoid. Extending the same methodology towards a more flexible class of convex set, Morio et al. [2009b,c] parameterized the surface of high-dimensional super-ellipsoids to compute $\mathrm{Vol}(\tilde{\mathcal{C}})$ and verify (2.47b), providing a less conservative convex inner approximation of the constraint for an atmospheric reentry mission of space vehicles. Along the same line of using the ellipsoid family, the technique was recently formalized for flat systems with the convoluted constraint represented as semi-algebraic sets [Bekcheva, 2019, Greco et al., 2022] with applications to the unicycle and quadcopter control.

These attempts also mark the end of our survey on the characterization of the constrained flat output space with a view to generating feasible trajectories. As the next step in the scheme in Figure 2.5, an analysis on how flatness can be exploited in real-time or online

control will be provided, together with the necessary theoretical tools for the subsequent chapters.

2.4 Optimization-based control settings

In general, flatness-based control refers to a variety of methods using the properties of differentially flat systems to facilitate offline synthesis and online implementation.

Fundamentally, the linear equivalence of a flat system, as in (2.12) directly enables the utilization of design techniques from linear system theory to compute the virtual control $\boldsymbol{v}$ in the flat output space as in (2.14). Then, the real input in the original space will be recovered via the expression (2.3) and applied to the plant. Without the presence of constraints, the simplest approach is to apply a standard state-feedback method such as pole placement for the chains of integrators in (2.12) [Levine, 2009]. While the synthesis is straightforward thanks to the controllability of (2.14), practical implementation may prove difficult since the high-order derivatives of the flat output are not trivial to measure. To handle this difficulty, under the mild assumption of the measurement of only the flat output, many studies have exploited the observability of the linear system to build different types of estimators for flat systems, including the extended state observers [Yang and Huang, 2009] or flat filters [Sira-Ramírez et al., 2016]. These observers provide not only state evaluation but also disturbance estimation, building the so-called framework of active disturbance rejection for flat systems in the frequency domain (see the surveys [She et al., 2022, Sira-Ramirez et al., 2019]).

However, when practical applications are considered, operational constraints (e.g., input saturation or maximal output allowed) become more crucial and cannot be neglected. In this case, more sophisticated techniques have to be employed to take into account these constraints. One of the most efficient strategies is that of employing in real-time a constrained optimization program. With this direction, on one hand, in the original space, one intuitive solution is to linearly approximate the dynamics around a trajectory satisfying the ODE. Typically, this nominal trajectory is chosen as a static equilibrium point deduced from the zero of the vector field. Interestingly, for flat systems, since a feasible trajectory can be readily calculated from the trajectory generation (guidance) phase, the approximation can also be performed around such a parametric curve [Prodan et al., 2015a, Nguyen et al., 2017a], resulting in a linear parameter varying (LPV) representation of the dynamics [Heemels et al., 2001]. This method, although computationally attractive, exhibits some theoretical disadvantages. For instance, the LPV representation is only available locally around a fixed trajectory, making the control sensitive to large initial tracking error and the stability difficult to quantify [Do and Prodan, 2023]. On the other hand, migrating to the flat output space to exploit the linear dynamics (2.14) allows the control design without any approximation of the dynamics. The main difficulty now is how to characterize the convoluted constraints caused by the

transformation (2.13), as also mentioned in the previous section. In this direction, there are different approaches handling such constraints with optimization. Particularly for quadcopters, a convex inner approximation of the admissible virtual input is provided in [Greeff and Schoellig, 2018, Nguyen et al., 2020b], creating a standard control problem for a linear system with convex input constraint. This characterization, hence, serves as the stepping stone for many extended works for the system [Freire and Xu, 2023, Marguet et al., 2024a, Nguyen et al., 2024]. A relaxation on the constraint in the flat output space for the kinematic bicycle model is also addressed in [Freire and Xu, 2022], rendering the nonlinear optimal control problem into a convex cone program. To avoid directly dealing with the convoluted description of the constraints, different general methodologies have also been proposed based on approximation theory, such as learning the admissible strategy in the flat space [Sultangazin et al., 2023], stochastically approximating the flat mapping [Greeff and Schoellig, 2020] or directly representing the linearized dynamics via the past data [Alsalti et al., 2021]. These approaches provide theoretically sound design procedures, but they are difficult to scale due to the heavy computation required to construct either the approximated control policy, the flat mapping or the basis functions parameterizing the dynamics. Therefore, an efficient procedure to control constrained flat systems is still missing in the literature.

From the review presented above, it is clear that navigating in the constrained flat output space essentially requires formulating a tractable online optimization problem. By "tractable", we would like to refer to online problems which can be solved in a reasonable amount of time with available solver software. In other words, we will aim to break the problem down to common and manageable constraint descriptions.

With this goal in mind, let us briefly recall standard definitions of the set-theoretic elements, which later will serve as numerical tools to address different concerns in control, including constraint characterization or stability analysis. Subsequently, these notions give rise to the standard formulations of optimization-based control design.

2.4.1 Sets and invariance notions

Sets are fundamental to various aspects of control theory, including design specification, uncertainty, and constraints. To address the motion planning problem in the flat output space, the different type of sets will be exploited not only to characterize the convoluted constraints but also to analyze standard concerns in control such as stability and robustness characterization. Thus, in what follows, we will review standard formulations of sets and the and their connection to the stability of dynamical systems.

2.4.1.1 Common type of sets and their representations

One of the most used sets in control theory is the class of ellipsoids with their intuitive geometrical expression. More specifically, the definition of an ellipsoids can be given as the following.

Definition 2.2 (Ellipsoidal sets [Blanchini and Miani, 2008])**.** *In $\mathbb{R}^n$, given a vector $\boldsymbol{c}$, $\rho > 0$ and a positive definite matrix $\boldsymbol{P}$, an ellipsoid is described as:*

$$\mathcal{E}(\boldsymbol{c}, \boldsymbol{P}, \rho) = \{\boldsymbol{x} \in \mathbb{R}^n : (\boldsymbol{x} - \boldsymbol{c})^\top \boldsymbol{P}(\boldsymbol{x} - \boldsymbol{c}) \leq \rho\}. \tag{2.48}$$

For brevity, we denote also $\mathcal{E}(\boldsymbol{P}, \rho) = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{x}^\top \boldsymbol{P}\boldsymbol{x} \leq \rho\}$. The representation (2.48) describes a level set of a positive definite scalar function $\mathbb{R}^n \to \mathbb{R}_+ : \Psi_{\boldsymbol{P}}(\boldsymbol{x}) = \|\boldsymbol{x} - \boldsymbol{c}\|^2_{\boldsymbol{P}}$. Moreover, it is note worthy that $\mathcal{E}(\boldsymbol{c}, \boldsymbol{P}, \rho)$ can be geometrically parameterized as a linearly distorted unit ball, i.e.,

$$\mathcal{E}(\boldsymbol{c}, \boldsymbol{P}, \rho) = \mathcal{D}(\boldsymbol{c}, \boldsymbol{E}, \rho) = \{\boldsymbol{x} = \boldsymbol{c} + \sqrt{\rho}\boldsymbol{E}\boldsymbol{w}, \|\boldsymbol{w}\|_2 \leq 1\}, \tag{2.49}$$

where $\boldsymbol{E}^{-1} = \boldsymbol{P}^{1/2}$ is the square root of $\boldsymbol{P}$ and we denote, $\mathcal{D}(\boldsymbol{E}, \rho) \triangleq \mathcal{D}(\boldsymbol{0}, \boldsymbol{E}, \rho)$. While the first representation is strongly connected to Lyapunov stability of linear systems, the second form provides a way to represent the set's volume, i.e., volume of $\mathcal{E}(\boldsymbol{c}, \boldsymbol{P}, \rho)$ is propositional to $\det(\sqrt{\rho}\boldsymbol{E})$ [Boyd et al., 2004]. This later will prove convenient when it is required to find an ellipsoidal subset of a bounded domain. Figure 2.15 illustrates the ellipsoid in the form of (2.48) and (2.49) with $\boldsymbol{P} = \left[\begin{smallmatrix} 4 & -2 \\ -2 & 5 \end{smallmatrix}\right], \rho = 5, \boldsymbol{c} = \boldsymbol{0}$ and $\boldsymbol{E} = \left[\begin{smallmatrix} 0.5457 & 0.1213 \\ 0.1213 & 0.4851 \end{smallmatrix}\right]$.

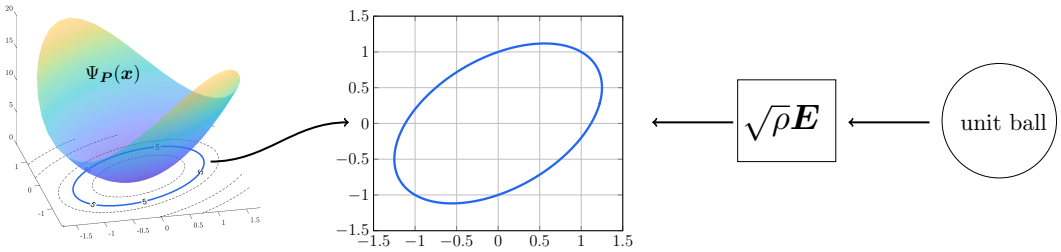


FIGURE 2.15: Two equivalent representations of an ellipsoidal set with $\boldsymbol{c} = \boldsymbol{0}$.

Despite being widely utilized in different applications [Kurzhanski and Vályi, 1997], ellipsoids are difficult to manipulate when combined with set operations such as convex hull or intersection. A more computationally appealing class circumventing this issue is the class of polytopes.

Definition 2.3 (Polytopes)**.** *In $\mathbb{R}^n$, given $\boldsymbol{H} \in \mathbb{R}^{n_h \times n}, \boldsymbol{k} \in \mathbb{R}^n$, then a set $\mathcal{H}$ is a polytope if it is a bounded set and can be described as:*

$$\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{H}\boldsymbol{x} \leq \boldsymbol{k}\}. \tag{2.50}$$

This representation is called half-space representation, or H-representation since it provides a description of a domain bordered by linear constraints (or half-spaces). Meanwhile, a polytope possesses as well a geometrical representation, i.e., the bounded polytope $\mathcal{H}$ in (2.50) can be equivalently defined as:

$$\mathcal{H} = \text{conv}\{\mathbf{v}_1, \mathbf{v}_2, ..., \mathbf{v}_d\} = \{\textstyle\sum_{i=1}^{d} \nu_i \mathbf{v}_i : \sum_{i=1}^{d} \nu_i = 1, \nu_i \geq 0\}, \tag{2.51}$$

where $\mathbf{v}_i, i \in \{1, ..., d\}$ are called the vertices of $\mathcal{H}$ and this representation is called vertex representation or V-representation of $\mathcal{H}$ (see Figure 2.16).

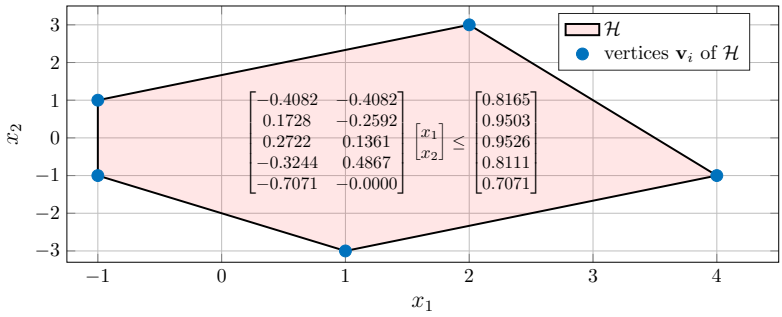


FIGURE 2.16: Polytope and its dual representations.

It can be seen that the domain defined inside a polytope is described by the intersection of n_h linear inequalities, providing a numerically simple representation when it comes to set calculation. In the context of finding a maximum volume inner approximation, the shortcoming of polytopes is the lack of an explicit formula for the volume. For this reason, we consider also a subclass of polytopes called zonotopes.

Definition 2.4. *In $\mathbb{R}^n$, given a center point $\boldsymbol{c}$ and a set of n_g vectors $\{\boldsymbol{g}_1, ..., \boldsymbol{g}_{n_g}\}$, then $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ is called a zonotope and can be described as [McMullen, 1971]:*

$$\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}) = \{\boldsymbol{c} + \textstyle\sum_{i=1}^{n_g} \beta_i \boldsymbol{g}_i : |\beta_i| \leq 1, \forall i \in \mathcal{I}_{n_g}\}, \tag{2.52}$$

with $\boldsymbol{G} = [\boldsymbol{g}_1, \boldsymbol{g}_2, ..., \boldsymbol{g}_{n_g}]$ gathering all the generators $\boldsymbol{g}_i$. A zonotope is, indeed, a centrally symmetric polytope.

Additionally, the following properties can be established. Consider the following set $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}))$ defined as:

$$\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})) \triangleq \{\boldsymbol{c} + \textstyle\sum_{i=1}^{n_g} \alpha_i \boldsymbol{g}_i, |\alpha_i| = 1\}. \tag{2.53}$$

Then, the set $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}))$ in (2.53) is a *finite* subset of $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ and contains all of its vertices [McMullen, 1971]. Consequently, for a convex set $\mathcal{X}$, the inclusion $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}) \subseteq \mathcal{X}$ holds if and only if $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})) \subset \mathcal{X}$. In this form, given a generator matrix $\boldsymbol{G}$ and the center $\boldsymbol{c}$, we can enumerate the elements of $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}))$ in (2.53) to have a set containing

all the vertices of $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$. This is relevant since if one need to inscribe a zonotope $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ inside the convex set, say $\mathcal{S}$, the constraint will become:

$$\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}) \subset \mathcal{S} \Leftrightarrow \mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})) \subset \mathcal{S}. \tag{2.54}$$

The necessary condition follows from $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})) \subset \mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ and the sufficient one is because $\mathcal{S}$ is convex, hence, the inclusion of the vertices of $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ implies that of the entire set.

Regarding the volume formula for the class, let us consider the family of *parameterized zonotopes* as the following [Ioan et al., 2019]:

$$\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c}) = \{\boldsymbol{c} + \textstyle\sum_{i=1}^{n_g} \alpha_i \delta_i \boldsymbol{g}_i : |\alpha_i| \leq 1\}, \tag{2.55}$$

where $\boldsymbol{G}$ is a given generator matrix and $\Delta = \text{diag}(\boldsymbol{\delta})$ denotes a diagonal matrix whose diagonal is collected in $\boldsymbol{\delta} = [\delta_1, ..., \delta_{n_g}]$. Then, its volume is explicitly written as:

$$\text{Vol}(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})) = \sum_{1 \leq k_1 < ... < k_d \leq n_g} \left|\det(\boldsymbol{G}^{k_1 ... k_d})\right| \prod_{k \in \{k_1, ..., k_n\}} \delta_k \tag{2.56}$$

where $\boldsymbol{G}^{k_1 ... k_d} \in \mathbb{R}^{d \times d}$ denotes the matrix formed by stacking the k_l-th column, $l \in \mathcal{I}_n$, of $\boldsymbol{G}$ together. In this manner, the problem of finding a maximum volume zonotope inscribed in a convex set, say $\mathcal{S}$, boils down to the solving of:

$$\begin{aligned} &\arg\max_{\Delta, \boldsymbol{c}} \text{Vol}(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})) \\ &\text{s.t } E(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})) \subset \mathcal{S}. \end{aligned} \tag{2.57}$$

In the same vein of finding a maximally inscribed convex inner approximation, another useful program to find the largest ellipsoid constrained in a polytope is recalled as the following.

Proposition 2.2. *(Largest ellipsoid constrained in a polytope [Boyd et al., 2004]) Consider a symmetric matrix $\boldsymbol{P} \succ 0$ and a polytope $\mathcal{H} = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{H}^\top \boldsymbol{x} \leq \boldsymbol{k}\}$, define an ellipsoid: $\mathcal{E}(\boldsymbol{P}, \rho) = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{x}^\top \boldsymbol{P} \boldsymbol{x} \leq \rho\}$. Then, the maximum $\varepsilon = \sqrt{\rho}$ such that $\mathcal{E} \subset \mathcal{H}$ can be found by solving:*

$$\begin{aligned} \varepsilon^* &= \arg\max_\varepsilon \ \log\det \mathfrak{B}(\varepsilon), \\ s.t\ \mathfrak{B}(\varepsilon) &= (\boldsymbol{V}\boldsymbol{S}^{1/2}\boldsymbol{U}^\top)^{-1}\varepsilon \text{ and } \|\mathfrak{B}(\varepsilon)\boldsymbol{H}_{i:}\|_2 \leq k_i, i = 1, ..., n_h, \end{aligned} \tag{2.58}$$

where $\boldsymbol{V}, \boldsymbol{S}, \boldsymbol{U}$ come from the singular value decomposition of $\boldsymbol{P}$ with $\boldsymbol{P} = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^\top$, and $\boldsymbol{H}_{i:}, k_i$ are the ith row of $\boldsymbol{H} \in \mathbb{R}^{n \times n_h}$ and $\boldsymbol{k} \in \mathbb{R}^{n_h}$, respectively.

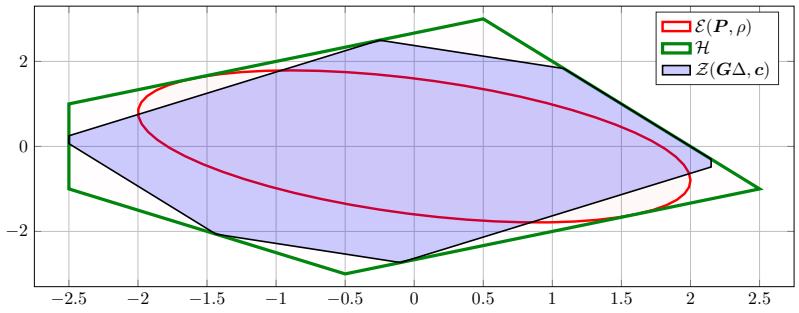


FIGURE 2.17: Largest parameterized zonotope $\mathcal{Z}(\boldsymbol{G}\Delta,\boldsymbol{c})$ and ellipsoid $\mathcal{E}(\boldsymbol{P},\rho)$ inscribed in a polytope $\mathcal{H}$.

By solving (2.57) and (2.58), respectively, an example of finding the largest parameterized zonotope as in (2.55) and the largest ellipsoid as in Proposition 2.2 is provided in Figure 2.17. The numerical specification are: $\mathcal{H} = \text{conv}\{\left[\begin{smallmatrix}-2.5\\1\end{smallmatrix}\right], \left[\begin{smallmatrix}0.5\\3\end{smallmatrix}\right], \left[\begin{smallmatrix}2.5\\-1\end{smallmatrix}\right], \left[\begin{smallmatrix}-0.5\\-3\end{smallmatrix}\right], \left[\begin{smallmatrix}-2.5\\-1\end{smallmatrix}\right]\}$, $\boldsymbol{P} = \left[\begin{smallmatrix}4&2\\2&5\end{smallmatrix}\right]$, $\boldsymbol{G} = \left[\begin{smallmatrix}1&-1&-2&0\\-2&-1&1&-1\end{smallmatrix}\right]$ while the solutions of the maximization problems yield $\rho = 12.8$, $\Delta = \text{diag}([0.5326\ 1.1244\ 0.3338\ 0.0925])$, $\boldsymbol{c} = [-0.1753 - 0.1168]^\top$.

So far, the basic classes of sets employed in this manuscript have been presented geometrically. Next, in association with a dynamical system, we present the notion of positive invariance, which will prove useful later to characterize the proposed control strategy.

2.4.1.2 Sets and the invariance of dynamical systems

Consider an autonomous system:

$$\dot{\boldsymbol{x}} = \bar{\mathbf{f}}(\boldsymbol{x}), \tag{2.59}$$

that is defined in a proper open set $\mathcal{O} \in \mathbb{R}^n$. Assume that there exists a globally defined solution of $\boldsymbol{x}(t), t \geq 0, \forall \boldsymbol{x} \in \mathcal{O}$. Then, the definition of positive invariance is provided as the following.

Definition 2.5 (Positively invariant set [Blanchini and Miani, 2008])**.** *The set $\mathcal{S} \subseteq \mathcal{O}$ is called positively invariant (PI) for the dynamics (2.59) if the globally defined solution of the ODE $\boldsymbol{x}(t) \in \mathcal{S}, t > 0$ for all $\boldsymbol{x}(0) \in \mathcal{S}$.*

This is to say, if the autonomous system starts inside a set $\mathcal{S}$, its future state will remain in $\mathcal{S}$ for all time. Then, for a control system, an indication of an ellipsoidal PI set can be given as follows.

Proposition 2.3 (Nagumo's invariance principle for ellipsoids)**.** *Consider the continuous time dynamics:*

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}, \boldsymbol{u}), \tag{2.60}$$

with $\boldsymbol{x} \in \mathbb{R}^n, \boldsymbol{u} \in \mathbb{R}^m$ *representing the state and the input vector, respectively. Then, the ellipsoid set* $\mathcal{E} = \{\boldsymbol{x} \in \mathbb{R}^n : \boldsymbol{x}^\top \boldsymbol{P}\boldsymbol{x} \leq \varepsilon\}$ *with* $\boldsymbol{P} \succ 0, \varepsilon > 0$ *is a PI set associated with the dynamics* (2.60) *if and only if:*

$$\boldsymbol{x}_b^\top \boldsymbol{P}\mathbf{f}(\boldsymbol{x}_b, \boldsymbol{u}) \leq 0, \forall \boldsymbol{x}_b \in \partial\mathcal{E}. \tag{2.61}$$

where $\boldsymbol{x}_b \in \partial\mathcal{E}$ *denotes the boundary points of the set* $\mathcal{E}$*.*

Definition 2.6 (Discrete time positive invariance [Blanchini and Miani, 2008])**.** *Similarly, for a discrete dynamical system governed by:*

$$\boldsymbol{x}(k+1) = \bar{\mathbf{f}}_d(\boldsymbol{x}(k)), \tag{2.62}$$

the set $\mathcal{S}$ *is positively invariant if and only if:*

$$\bar{\mathbf{f}}_d(\mathcal{S}) \subseteq \mathcal{S}. \tag{2.63}$$

The definitions above are relevant since, given a control law $\boldsymbol{u}(\boldsymbol{x})$, the resulting closed-loop can be characterized correspondingly. For the sake of later use, let us recall a standard linear feedback synthesis as the following.

Proposition 2.4 (Gradient-based control with a quadratic control Lyapunov function [Blanchini and Miani, 2008].)**.** *Consider the following linear time-invariant (LTI) system*

$$\dot{\boldsymbol{\xi}} = \boldsymbol{A}\boldsymbol{\xi} + \boldsymbol{B}\boldsymbol{v}, \tag{2.64}$$

with $\boldsymbol{\xi} \in \mathbb{R}^n, \boldsymbol{v} \in \mathbb{R}^m$*. Then, for any* $\alpha > 0$ *and* $\gamma \geq 1$*, by imposing a gradient-based control:*

$$\boldsymbol{v} = -\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}, \tag{2.65}$$

satisfying the condition of $P \succ 0$ *and:*

$$(\boldsymbol{A} - \boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P})^\top \boldsymbol{P} + \boldsymbol{P}(\boldsymbol{A} - \boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P}) \preceq -\alpha \boldsymbol{P}, \tag{2.66}$$

then the quadratic positive definite function:

$$V = \boldsymbol{\xi}^\top \boldsymbol{P}\boldsymbol{\xi}, \tag{2.67}$$

is a control Lyapunov function for the dynamics (2.64) *associated with the control* (2.65)*. As a result, this control setup ensures the exponential convergence of the system to the equilibrium point* $\boldsymbol{\xi} = \mathbf{0}, \boldsymbol{v} = \mathbf{0}$*.*

Next, the invariance condition for disturbed autonomous linear systems is also recalled, laying the theoretical foundation for the robustness analysis in the dissertation.

Proposition 2.5 (Invariance condition for autonomous disturbed system [Blanchini and Miani, 2008]:)**.** *Consider the autonomous system affected by additive disturbances:*

$$\dot{\boldsymbol{\xi}} = \bar{\boldsymbol{A}}\boldsymbol{\xi} + \boldsymbol{E}\boldsymbol{w}, \tag{2.68}$$

with $\boldsymbol{w}$ *denoting the disturbance constrained as* $\boldsymbol{w}^\top \boldsymbol{w} \leq 1$*. Then, the ellipsoid* $\mathcal{E}_w \triangleq \{\boldsymbol{x} : \boldsymbol{x}^\top \boldsymbol{Q}_w^{-1} \boldsymbol{x} \leq 1\}$ *is invariant under the dynamics* (2.68) *if, for some* $\beta > 0$*:*

$$\boldsymbol{Q}_w \bar{\boldsymbol{A}}^\top + \bar{\boldsymbol{A}}\boldsymbol{Q}_w + \beta \boldsymbol{Q}_w + \beta^{-1}\boldsymbol{E}\boldsymbol{E}^\top \preceq 0. \tag{2.69}$$

Then with the aim to take into account constraints during the control computation, the two common optimization-based control framework will be presented.

2.4.2 Optimization-based control schemes

Although there are several frameworks concerning about solving an optimization program online and handling constraints, we limit ourselves to the two control policies thanks to their ability to incorporate constraints explicitly.

2.4.2.1 Control Lyapunov function-based control

From the previous sections, it is clear that in the flat output space, the dynamics admits a linear representation (2.14). However, if we examine the linearized dynamics from a different perspective, we find that it is also a control-affine system, a well-studied class in the context of constrained control. This is because, given a Control Lyapunov Function (CLF), the condition forcing its time derivative negative reduces to a single linear constraint, making the search for a stabilizing control action relatively simple. To see this, let us recall the fundamental idea of a CLF-based strategy for control affine system as the following.

Suppose that the system of interest is affine in the control, or, the dynamics takes the form:

$$\dot{\boldsymbol{x}} = \mathbf{f}(x) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}, \text{ subject to } \boldsymbol{u} \in \mathcal{U}, \tag{2.70}$$

where $\mathcal{U}$ is a generic polytope as in Definition 2.3, and the objective is to stabilize the system at the origin $\boldsymbol{x} = \mathbf{0}, \boldsymbol{u} = \mathbf{0}$. Then, a CLF can be defined as following.

Definition 2.7 (Control Lyapunov Function [Blanchini and Miani, 2008])**.** *A positive definite smooth Lyapunov function,* $V(\boldsymbol{x})$*, is a CLF for* (2.70) *if there exists a function* $\boldsymbol{u} = \boldsymbol{\Xi}(\boldsymbol{x})$ *with values in* $\mathcal{U}$ *and*

$$\frac{\mathrm{d}V(\boldsymbol{x})}{\mathrm{d}t} = \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\Xi}(\boldsymbol{x})] \leq -\beta(V(\boldsymbol{x})), \tag{2.71}$$

for some $\mathcal{K}$-class function $\beta(\cdot)$. □

Then, it is immediate that if one has a CLF $V(\boldsymbol{x})$, then the set of stabilizing control action boils down to a state-dependent polytope:

$$\mathcal{U}_V(\boldsymbol{x}) = \mathcal{U} \bigcap \Big\{ \boldsymbol{u} : \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] \leq -\beta(V(\boldsymbol{x})) \Big\}. \tag{2.72}$$

Consequently, at each instant, given a state feedback $\boldsymbol{x}$, by searching the control inside $\mathcal{U}_V(\boldsymbol{x})$, asymptotic stability can be ensured. For instant, under the assumption of the non-emptiness of $\mathcal{U}_V(\boldsymbol{x})$ in (2.72), one common continuous QP controller is:

$$\boldsymbol{u} = \arg \min_{\boldsymbol{u} \in \mathcal{U}_V(\boldsymbol{x})} \|\boldsymbol{u} - \boldsymbol{u}(\boldsymbol{x})\|_2^2 \tag{2.73}$$

where $\boldsymbol{u}(\boldsymbol{x})$ is some desired reference control action [Grammatico et al., 2013].

The main challenge now is how to derive $V(\boldsymbol{x})$ numerically [Giesl and Hafstein, 2015]. Though it will be shown that this concern can be handled via linear programming in Chapter 4, it is worth recalling also that, one effective strategy to sidestep the a priori design of either the controller $\boldsymbol{\Xi}(\boldsymbol{x})$ in closed form or a CLF is the Model Predictive Control (MPC) approach, facilitating the offline design work.

2.4.2.2 Model Predictive Control

The idea behind MPC is to, at each instant, optimize the system trajectory with respect to an open-loop OCP. Namely, at each sampling time, given a state feedback $\boldsymbol{x}(k)$, the controller solves the following optimization problem:

$$\min V_f(\boldsymbol{x}(k|k+N_p)) + \sum_{i=0}^{N_p-1} V_s(\boldsymbol{x}(k|k+i), \boldsymbol{u}(k|k+i)), \tag{2.74a}$$

$$\boldsymbol{x}(k|0) = \boldsymbol{x}(k), \tag{2.74b}$$

$$\boldsymbol{x}(k|k+i+1) = \mathbf{f}_d(\boldsymbol{x}(k|k+i), \boldsymbol{u}(k|k+i)), \tag{2.74c}$$

$$\Theta_s(\boldsymbol{x}(k|k+i), \boldsymbol{u}(k|k+i)) \leq 0, k \in \{0, ..., N_p - 1\}, \tag{2.74d}$$

$$\Theta_f(\boldsymbol{x}(k|k+N_p) \leq 0, \tag{2.74e}$$

where t_s is the sampling time and N_p is the prediction horizon. $\Theta_s(\cdot,\cdot)$ and $\Theta_f(\cdot)$ represent the general stage and terminal constraint, respectively. $\mathbf{f}_d(\cdot,\cdot)$ denotes the discretized model of the system. Then the control law applied is:

$$\boldsymbol{u}_{mpc} = \boldsymbol{u}(k|k). \tag{2.75}$$

With a proper construction on the terminal constraint (2.74e) the stage cost $V_s(\cdot)$ and the terminal cost $V_f(\cdot)$ in (2.74a), the stability and feasibility of the system can be

guaranteed [Chen and Allgöwer, 1998, Mayne et al., 2000]. The standard synthesis for linear discrete system with ellipsoidal terminal region can be recalled as the following.

Proposition 2.6. *(Stabilizing MPC with ellipsoidal terminal region [Mayne et al., 2000]) Given the matrices* $\boldsymbol{Q} \succ 0, \boldsymbol{R} \succeq 0$ *and a constrained linear dynamics:*

$$\boldsymbol{\zeta}(k+1) = \boldsymbol{A}^d \boldsymbol{\zeta}(k) + \boldsymbol{B}^d \boldsymbol{v}(k) \text{ subject to } \boldsymbol{v}(k) \in \mathcal{V}, \tag{2.76}$$

where $\boldsymbol{\zeta}(k) \in \mathbb{R}^n, \boldsymbol{v}(k) \in \mathbb{R}^m$ *denote the state and input vector. The constant matrices* $\boldsymbol{A}^d, \boldsymbol{B}^d$ *represent the linear dynamics, and* $\mathcal{V}$ *is a polytopic constraint set. Consider the following procedure:*

- *Choose a feedback gain* $\boldsymbol{K}$ *such that* $\bar{\boldsymbol{A}} \triangleq \boldsymbol{A}^d + \boldsymbol{B}^d \boldsymbol{K}$ *is a Schur matrix;*
- *Choose* $\boldsymbol{M}$ *such that*
$$\boldsymbol{M} \succeq \boldsymbol{Q} + \lambda_{max}(\boldsymbol{R}) \boldsymbol{K}^\top \boldsymbol{K}; \tag{2.77}$$
- *Solve the Lyapunov equation (2.78) to find* $\boldsymbol{P}$*:*
$$\bar{\boldsymbol{A}}^\top \boldsymbol{P} \bar{\boldsymbol{A}} - \boldsymbol{P} + \boldsymbol{M} = 0; \tag{2.78}$$
- *Following Proposition 2.2 to find the largest ellipsoid:*
$$\mathcal{E}_f = \left\{ \boldsymbol{\zeta} : \boldsymbol{\zeta}^\top \boldsymbol{P} \boldsymbol{\zeta} \leq \varepsilon_f \right\}, \tag{2.79}$$
such that $\mathcal{E}_f \subset \mathcal{X} \triangleq \{\boldsymbol{\zeta} : \boldsymbol{K}\boldsymbol{\zeta} \in \mathcal{V}\}$.

Then, we have:

1. *The control* $\boldsymbol{v}(k) = \boldsymbol{K}\boldsymbol{\zeta}(k) \in \mathcal{V}, \forall \boldsymbol{\zeta}(k) \in \mathcal{E}_f$ *and also renders the ellipsoid* $\mathcal{E}_f$ *forward invariant for the system (2.76) i.e,*
$$\boldsymbol{\zeta}(k) \in \mathcal{E}_f \Rightarrow \boldsymbol{\zeta}_{k+1} \in \mathcal{E}_f, k \geq 0; \tag{2.80}$$
2. $V_f(\boldsymbol{\zeta}_{k+1}) + V_s(\boldsymbol{\zeta}(k), \boldsymbol{K}\boldsymbol{\zeta}(k)) - V_f(\boldsymbol{\zeta}(k)) \leq 0, \forall \boldsymbol{\zeta}(k) \in \mathcal{E}_f$
$$\text{with } \begin{cases} V_f(\boldsymbol{\zeta}(k)) \triangleq \|\boldsymbol{\zeta}(k)\|^2_{\boldsymbol{P}} \\ V_s(\boldsymbol{\zeta}(k), \boldsymbol{v}(k)) \triangleq \|\boldsymbol{\zeta}(k)\|^2_{\boldsymbol{Q}} + \|\boldsymbol{v}(k)\|^2_{\boldsymbol{R}}. \end{cases} \tag{2.81}$$

□

Proof. From the definition (2.79), $\boldsymbol{v}(k) = \boldsymbol{K}\boldsymbol{\zeta}(k) \in \mathcal{V}$, $\forall \boldsymbol{\zeta}(k) \in \mathcal{E}_f$ always holds. Next, with condition (2.78) and the inequality (2.77), we have:

$$\begin{aligned} \|\boldsymbol{\zeta}_{k+1}\|_{\boldsymbol{P}}^2 &= \boldsymbol{\zeta}(k)^\top \bar{\boldsymbol{A}}^\top \boldsymbol{P} \bar{\boldsymbol{A}} \boldsymbol{\zeta}(k) = \boldsymbol{\zeta}(k)^\top (\boldsymbol{P} - \boldsymbol{M}) \boldsymbol{\zeta}(k) \\ &\leq \left(1 - \frac{\lambda_{min}(\boldsymbol{M})}{\lambda_{max}(\boldsymbol{P})}\right) \boldsymbol{\zeta}(k)^\top \boldsymbol{P} \boldsymbol{\zeta}(k) \leq \|\boldsymbol{\zeta}(k)\|_{\boldsymbol{P}}^2. \end{aligned} \tag{2.82}$$

Then, if $\boldsymbol{\zeta}(k) \in \mathcal{E}_f$, $\|\boldsymbol{\zeta}_{k+1}\|_{\boldsymbol{P}}^2 \leq \|\boldsymbol{\zeta}(k)\|_{\boldsymbol{P}}^2 \leq \varepsilon_f$, or $\boldsymbol{\zeta}_{k+1} \in \mathcal{E}_f$, namely $\mathcal{E}_f$ is forward invariant for the closed-loop.

Finally, with the conditions in (2.77) and (2.78), we have:

$$\begin{aligned} &V_f(\boldsymbol{\zeta}_{k+1}) + V_s(\boldsymbol{\zeta}(k), \boldsymbol{K}\boldsymbol{\zeta}(k)) - V_f(\boldsymbol{\zeta}(k)) \\ &= \boldsymbol{\zeta}(k)^\top \bar{\boldsymbol{A}}^\top \boldsymbol{P} \bar{\boldsymbol{A}} \boldsymbol{\zeta}(k) + \boldsymbol{\zeta}(k)^\top (\boldsymbol{Q} + \boldsymbol{K}^\top \boldsymbol{R} \boldsymbol{K}) \boldsymbol{\zeta}(k) - \boldsymbol{\zeta}(k)^\top \boldsymbol{P} \boldsymbol{\zeta}(k) \\ &= -\boldsymbol{\zeta}(k)^\top (\boldsymbol{M} - \boldsymbol{Q} - \boldsymbol{K}^\top \boldsymbol{R} \boldsymbol{K}) \boldsymbol{\zeta}(k) \leq 0, \forall \boldsymbol{\zeta}(k) \in \mathbb{R}^n. \end{aligned} \tag{2.83}$$

□

Corollary 2.1. *From the setting of Proposition 2.6, consider the standard optimization problem at time step k [Mayne et al., 2000]:*

$$\min \sum_{i=0}^{N_p-1} V_s(\boldsymbol{\zeta}(k|k+i), \boldsymbol{v}(k|k+i)) + V_f(\boldsymbol{\zeta}(k|k+N_p)) \tag{2.84}$$

$$s.t \begin{cases} \boldsymbol{\zeta}(k|k+i+1) = \boldsymbol{A}^d \boldsymbol{\zeta}(k|k+1) + \boldsymbol{B}^d \boldsymbol{v}(k|k+1); \boldsymbol{\zeta}(k|k+N_p) \in \mathcal{E}_f & \text{(2.85a)} \\ \boldsymbol{v}(k|k+i) \in \mathcal{V}, i \in \{0, 1, ..., N_p - 1\}, & \text{(2.85b)} \end{cases}$$

where N_p denotes the prediction horizon, $\mathcal{E}_f$ is the terminal region; the set $\mathcal{V}$ describes the input constraints; $\boldsymbol{Q}, \boldsymbol{P} \succ 0, \boldsymbol{R} \succeq 0$ are weighting matrices. Then, by finding $\boldsymbol{P}$ and ε_f as in Proposition 2.6, the problem (2.84)-(2.85) is recursively feasible, and the origin $\boldsymbol{0}_{n\times 1}$ is asymptotically stable under the closed-loop system given by repetitively applying the first control action found in (2.84)-(2.85).

Proof. The properties achieved in Proposition 2.6 satisfy the conditions for feasibility and stability presented in Section 3.3 in [Mayne et al., 2000] for a standard MPC scheme. □

Remark 2.5. The existence of $\mathcal{E}_f$ in (2.79) simply requires the set $\mathcal{X}$ to include the origin as its interior point. In such case, there always exists a sufficiently small value of ε_f so that $\mathcal{E}_f$ is inscribed in $\mathcal{X}$.

The power of the MPC setting in general lies in its ability to explicitly take into account the operational restriction (2.74d) while ensuring the forecast behavior of the system is also constrained. However, the trade-off would be the heavy online computational overhead due to a significant amount of complex constraints in the optimization problem.

2.5 Concluding remarks

This chapter presented an overview of differentially flat systems and their applications in motion planning, including dynamically feasible trajectory generation and control design. In summary, the theory of flat systems provides an alternative perspective for handling nonlinear dynamics. Essentially, in the new coordinate, the system's time evolution will be equivalent to that of trivial dynamics (or chains of integrators) at the expense of a complicated description of the feasible domain. The crucial question is whether it is worthwhile to plan the trajectory or control the plant indirectly from such a convoluted space. Based on the literature, the answer is affirmative for a substantial number of applications cited previously.

Regarding the future direction of flatness-based control, different paths can be foreseen at different levels of generalization and connection to linear control theory. With the specific applications that have been successfully validated, the improvement may be accomplished by pushing further towards robustness, such as mitigations addressing adaptivity or observer design [Qian et al., 2024, Colombo and Girard, 2013]. This is because when uncertainty or disturbances arise, the nonlinearity-canceling effect of the nominal mapping could deteriorate, leading to a negative impact on performance. Next, with the system where a feasible domain has been convexified, important progress may be expected in the cooperation of state-of-the-art programming techniques, further optimizing the controller's real-time computational footprint [Greeff et al., 2022, Akbari and Greeff, 2024]. Additionally, needless to say, an important issue concerns the generalization of how the convoluted constraint in the flat space should be characterized or approximated via a more powerful approximator, such as neural networks [Grimstad and Andersson, 2019] or the Gaussian process [Jeloka et al., 2024, Hall et al., 2023]. At last, since flatness serves as a pivotal link between nonlinear and linear control design, we can directly apply the expanding research on linear systems, such as data-driven or model-free control [Berberich et al., 2022, De Persis et al., 2023], to the linearized dynamics, incorporating the innovative features from the linear framework into the nonlinear design [Alsalti et al., 2021].

So far, this chapter has provided a basic understanding of differential flatness, its applications, and the theoretical tools employed to support its properties. By integrating these concepts with a broad review of related literature, we establish the foundation for the subsequent chapters, which will explore the identified research gaps in flatness-based control, covering both specific applications and general methods. In the next part, a case study of quadcopter control in the flat output space will be thoroughly investigated. With the presence of convoluted constraints caused by linearization, the feasible domain will be characterized into convex inner approximation, giving rise to the application of efficient controllers for linear systems subject to convex constraints. This case study will be followed by an attempt to generally approximate the convoluted constraint with neural networks.

Chapter 3

Constraint characterization for control in the flat space

This chapter focuses on tackling the convoluted constraints, particularly for the quadcopter application via different approximation and design techniques. Then, an effort to explore neural networks as an approximator for a general convoluted flat output space will follow. The chapter starts by analyzing the constraints of the quadcopter's translational dynamics within a hierarchical architecture. With feasible domain distorted by feedback linearization, we provide a comprehensive procedure to approximate it using parameterized zonotopes. This approximation then enables the application of the MPC strategy. Additionally, the constructed framework is adapted to explicit MPC, significantly reducing computational overhead. Ultimately, the convexity of the nonlinear set is exploited, providing a low-cost explicit saturated control. The chapter concludes with an exploration of neural networks as a more general approximator, resulting in a controller that guarantees constraint satisfaction in the linearizing coordinate. These contributions can be briefly highlighted as follows:

i) we investigate the translational dynamics of the quadcopter and describe the convoluted constraint set in the flat output space [J1];

ii) we provide an procedure to approximate the feasible domain, resulting in a polytopic description of the new input set [C3];

iii) we formulate both implicit and explicit MPC schemes based on the constructed approximation, showing the complexity reduction for the online program [J5];

iv) we derive an explicit solution for saturating the input on the surface of the nonlinear constraint set, creating a solver-free satisfactory solution [J1];

v) we generalize the idea of approximating the constraint in the flat space by using rectified linear unit neural networks [J7].

3.1 Motivation . . . 53
3.2 The quadcopter's outer loop in the flat space . . . 56
3.2.1 Hierarchical control and constraint characterization . . . 56
3.2.2 Input constraints characterization in the flat output space . . . 59
3.2.3 Experimental platform description . . . 65
3.3 Computationally efficient control in the flat output space . . . 67
3.3.1 Implicit model predictive control . . . 67
3.3.2 Explicit model predictive control . . . 68
3.3.3 Explicit saturated control . . . 70
3.4 Simulations and experimental validation . . . 77
3.4.1 Implicit model predictive control . . . 78
3.4.2 Explicit model predictive control . . . 82
3.4.3 Explicit saturated control . . . 90
3.4.4 Discussions and comparisons . . . 97
3.5 Towards a generalized constraint approximation with neural networks . . . 99
3.5.1 Preliminaries and problem statement . . . 99
3.5.2 Constraint characterization with MIP and applications in control . 101
3.5.3 Simulation study . . . 106
3.6 Concluding remarks . . . 110

3.1 Motivation

Flatness-based feedback linearization (FB-FL) technique, although being investigated for decades [Fliess et al., 1993, Greeff and Schoellig, 2020, Marguet et al., 2023, Freire and Xu, 2023], still attracts considerable attention in both industry and research community thanks to its connection from nonlinear design to the well-developed control synthesis for linear systems. The general idea of FB-FL includes a flatness-induced variable transformation as in (2.13) that reveals the linear structure of the original nonlinear dynamics in new coordinates, reducing the control design to closing the loop for the linearized dynamics (see Figure 3.1). However, the advantage does not generally hold when operating constraints are considered (such as input and state constraints). This is because, while the nonlinearity is cancelled in the dynamics, the constraints are rendered nonlinear and state-dependent, which poses challenges in both theoretical analysis and real-time implementation.

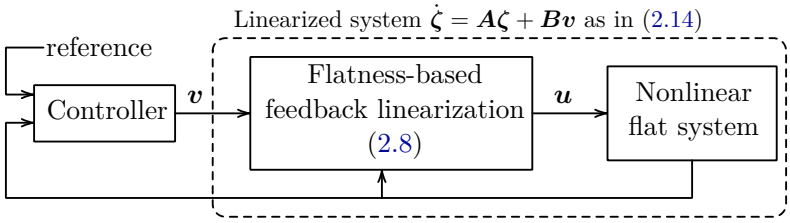


FIGURE 3.1: General scheme of flatness-based feedback linearization control.

For flat systems specifically and feedback linearizable systems in general, several studies have been conducted, ranging from particular applications to general methodologies. On one hand, to provide rigorous stability guarantees by exploiting the constraints' structural characteristics (e.g., shapes, convexity, or explicit bounds), specific investigations have been carried out: for inverted pendulum with nested control [Ibanez and Frias, 2008], linear matrix inequality-based design for wheeled mobile robots [Tiriolo and Lucia, 2023], or polytope-based constraint characterization for quadcopters [Nguyen et al., 2024, Do and Prodan, 2023, Greeff and Schoellig, 2018]. Although these methods were proven effective, the problem of generally converting the convoluted constraints into an online, tractable program remains open. On the other hand, with the ability to handle constraints, MPC constitutes a promising candidate for navigation in the new coordinates. However, although the prediction model is linear in the new coordinate, the constraints enforced are not trivial to solve. This is because, over a prediction horizon, they are defined implicitly by the decision variables (the predicted states and inputs), creating a computationally intensive program online. To relax this setting and achieve real-time operability, approximation-based solutions have been proposed, creating control laws with the complexity of quadratic programs (QP). The most common technique computes the actual constraint via state feedback and extends it constantly to the whole prediction horizon [Nevistic and Del Re, 1994, Kandler et al., 2012]. In a less conservative manner, predictions from the previous time step are employed to approximate

the exact actual constraints [Kurtz and Henson, 1998]. While both of these solutions demonstrate computational efficiency, they pose challenges in terms of stability analysis development [Kurtz and Henson, 1998]. This issue persists even under the assumption of online feasibility, as the proof of constraint satisfaction is limited to the applied input rather than the entire predicted state or input trajectory. Recently, probabilistic approaches have been used to learn the constraints via Gaussian processes [Hall et al., 2023, Greeff et al., 2021], leveraging the affine structure in single-input flat systems to formulate a controller as a second-order cone program.

Lying within the existing landscape, we concentrate on two main directions: enhancing the current study with lower computational complexity and generalizing a control-handling program for the linearized system. With this aim, the present chapter can be separated into two main parts.

The quadcopter outer loop design: Firstly, we dedicate the next three sections (Section 3.2–3.4) to focusing specifically on the quadcopter outer-loop control, which is the experimental platform considered in this work. This dynamics is indeed a well-developed system with a thorough description of the operating constraints in the flat output space (see [Nguyen et al., 2018] for example). However, they are either taken into account offline or sidestepped via a conservative inner approximation. For instance, in [Freire and Xu, 2023, Sun et al., 2022], the constraints are imposed only during the trajectory generation phase (or prior to the flight). Thus, a dynamically-constrained feedforward reference was given to the vehicle in addition to the standard feedback control to improve accuracy and follow aggressive trajectories. Meanwhile, the other direction includes stabilizing the linear model by conservatively sketching the new constraints through some linear approximation and subsequently handling them online. More specifically, by proposing a hyper box inscribed in the feasible domain, the system was governed under the MPC framework, where a constrained optimization problem is solved in-flight to regulate the system [Greeff and Schoellig, 2018, Lu et al., 2017, Mueller and D'Andrea, 2013]. Furthermore, in [Nguyen et al., 2019, 2020b], although the state prediction is computed via the quadcopter's nonlinear model, its linearized representation was indirectly employed to ensure the feasibility and stability of the nonlinear MPC approach via the existence of a feasible local controller. An computationally improved design without terminal ingredients was later studied in [Gomaa et al., 2022]. Yet, as a price of solving a nonlinear optimization problem online, these methods have a high computational demand, thus leading to long sampling time. Inspired by the classical approach in [Teel, 1992], the idea of controlling the exactly linearized model was also developed in a nested control fashion for the quadcopter's position control [Xie and Veitch, 2020, Nguyen et al., 2020a], which guarantees the constraint satisfaction by appropriately saturating the amplitude of the virtual input. This design was validated to be effective and computationally efficient, since the online calculation requires only algebraic computation and the standard saturation operators. However, the nonlinear constraints were, again, sidestepped by a conservative inner approximation. Hence, in the present contribution, we tackle this

limitation by providing a less conservative inner approximation, giving rise to the implementation of standard implicit and explicit MPC (IMPC and ExMPC, respectively). Additionally, the standard saturated control design is also adapted to the new nonlinear constraint set, creating a computationally low-cost stabilizing control for the model. The theoretical formulations and experimental validations of such control strategies will be given in Section 3.3 and Section 3.4, respectively.

> Section 3.2–3.4 handle the constrained control problem of the quadcopter's outer loop with IMPC, ExMPC and saturated control design.

General constraint approximation via neural network: Secondly, in our pursuit of constructing a robust and general approximation of the constraints and rendering the online routine tractable, Section 3.5 will explore the application of the Rectified Linear Unit (ReLU) Artificial Neural Network (ANN). The focus is on characterizing the intricate constraints induced by FB-FL. We opt for this neural network structure due to its versatile nature as a universal approximator and, most importantly, its reprensentability in the form of Mixed-Integer (MI) linear constraints [Grimstad and Andersson, 2019, Fischetti and Jo, 2018]. This configuration offers the advantage in that it can seamlessly integrate with other linear constraints, including those derived from Control Lyapunov Function (CLF) or Control Barrier Function (CBF) principles [Li et al., 2023]. Moreover, within the MPC framework, the constraints can be rigorously imposed along the entire prediction horizon without resorting to additional approximations. Notably, the MI constraints retain linearity, enabling efficient resolution through Mixed-Integer Program (MIP) solvers [CPLEX, 2009]. To the best of the authors' knowledge, this application is novel to the literature, offering a theoretically comprehensive solution to the problem of intricate constraints associated with FB-FL. Different from the approaches in related works [Hall et al., 2023, Greeff et al., 2021], our constraint reformulation, while specifically tackling uncertainty-free problems, exhibits versatility in its applicability to multi-input systems. This adaptability is achievable through an approximation that maintains a bounded error via the ANN.

> Section 3.5 uses ReLU-ANN to approximate the convoluted constraints caused by linearizing coordinate change.

In what follows, we will expound upon the theoretical findings by presenting both simulations and experiments to emphasize the contributions presented in this chapter. Finally, we will draw conclusions and finish by comparing the approaches that were previously described.

3.2 The quadcopter's outer loop in the flat space

Herein, the dynamics of quadcopter will be recapitulated, with two layers of control called the outer and inner loop. These loops handle the vehicle's translational and rotational dynamics, respectively. Then, with a specific focus on the outer loop, the constraint will be characterized in the flat output space and approximated via zonotopes and polytopes, giving rise to the synthesis of computationally low-cost designs for this layer in the subsequent sections.

3.2.1 Hierarchical control and constraint characterization

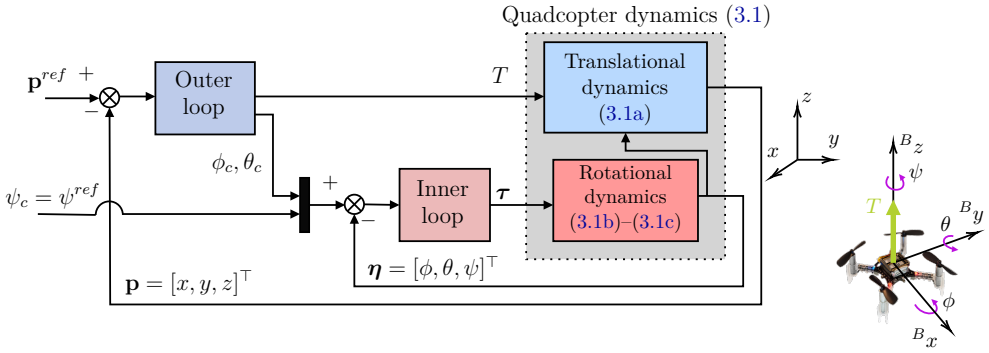


FIGURE 3.2: The decoupled dynamics of a quadcopter and its hierarchical control scheme.

The dynamics of a quadcopter can be recalled as [Formentin and Lovera, 2011]:

$$\ddot{\mathbf{p}} = -g\boldsymbol{e}_z + T\mathbf{R}(\boldsymbol{\eta})\boldsymbol{e}_z, \tag{3.1a}$$

$$\boldsymbol{\omega} = \boldsymbol{W}(\boldsymbol{\eta})\dot{\boldsymbol{\eta}}, \tag{3.1b}$$

$$\boldsymbol{J}\dot{\boldsymbol{\omega}} = -\boldsymbol{\omega} \times (\boldsymbol{J}\boldsymbol{\omega}) + \boldsymbol{\tau}, \tag{3.1c}$$

where $\mathbf{p} = [x\ y\ z]^\top$ gathers the drone's position in the global frame and $\boldsymbol{\eta} = [\phi\ \theta\ \psi]^\top$ collects the roll, pitch, yaw angles, respectively. $\boldsymbol{e}_z = [0\ 0\ 1]^\top$, $T \in \mathbb{R}$ is the normalized thrust acting along the ${}^B z$ axis of the local frame attached to the drone (see Figure 3.2). $\boldsymbol{\tau} = [\tau_\phi\ \tau_\theta\ \tau_\psi]^\top \in \mathbb{R}^3$ denotes the actuator torques affecting the rotation around the axis of the roll, pitch and yaw angles. In a quasi-stationary flight settings, we assume that the aerodynamic forces are always in the direction of the z axis. Centrifugal force and disturbance (e.g., drag) are neglected. $\boldsymbol{J}$ denotes the inertia tensor of the system and $\boldsymbol{W}(\boldsymbol{\eta})$ is defined as:

$$\boldsymbol{W}(\boldsymbol{\eta}) = \begin{bmatrix} 1 & 0 & -\sin\theta \\ 0 & \cos\phi & \sin\phi\cos\theta \\ 0 & -\sin\phi & \cos\phi\cos\theta \end{bmatrix}. \tag{3.2}$$

The rotation matrix $\mathbf{R}(\boldsymbol{\eta})$ is given as:

$$\mathbf{R}(\boldsymbol{\eta}) = \begin{bmatrix} \cos\theta\cos\psi & \sin\phi\sin\theta\cos\psi - \cos\phi\sin\psi & \cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi \\ \cos\theta\sin\psi & \sin\phi\sin\theta\sin\psi + \cos\phi\cos\psi & \cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi \\ -\sin\theta & \sin\phi\cos\theta & \cos\phi\cos\theta \end{bmatrix}.$$

In the literature, the whole dynamics (3.1) can be directly controlled [Cowling et al., 2007]. However, as can be seen from (3.1), with the natural decoupling of the dynamics, the control problem can also be devided into two hierarchical subproblems. This cascade design includes position and attitude control (outer loop and inner loop, respectively) to handle the translational and the rotational dynamics of the vehicle, respectively [Zagaris et al., 2004, Bertrand et al., 2011]. Thanks to this separation, the work load for control design is significantly relieved by considering the two fully-actuated layers separately. Several validated results provided for both loops can be found in [Nascimento and Saska, 2019, Cao and Lynch, 2016] and the references therein. With this architecture, different quadcopter autopilot brands and their specifications are reviewed in [Foehn et al., 2022].

In details, the structure of the hierarchical control can be described as the following. At the outer loop, the controller will compute the control thrust T and the control roll, pitch angles (ϕ_c, θ_c). These angles then are gathered with the desired yaw angle ψ_c to be the reference of the inner loop. Then, based on the error between the real angles in $\boldsymbol{\eta}$ and the desired values ϕ_c, θ_c, ψ_c, the inner loop will provide the stabilizing control torque $\boldsymbol{\tau}$. This torque vector joins with T to become the input of the vehicle. An illustration of this architecture is provided in Figure 3.2.

In this work, due to the availability of the experimental platform, we limit ourselves on the design of the outer loop. For this reason, we suppose that the inner loop satisfies the following assumption.

Assumption 3.1. In the hierarchical scheme, the attitude controller (inner loop) is assumed to be capable of tracking the control roll and pitch angles (ϕ, θ) given by the position control loop, i.e.:

$$\phi_c = \phi, \theta_c = \theta \text{ and } \psi_c = \psi. \tag{3.3}$$

Then, the system dynamics (3.1) becomes [Nguyen et al., 2020a]:

$$\ddot{\mathbf{p}} = \begin{bmatrix} T(\cos\phi\sin\theta\cos\psi + \sin\phi\sin\psi) \\ T(\cos\phi\sin\theta\sin\psi - \sin\phi\cos\psi) \\ T\cos\phi\cos\theta - g \end{bmatrix} \triangleq \boldsymbol{h}_\psi(\boldsymbol{u}), \tag{3.4}$$

where $\mathbf{p} = [\mathrm{p}_1\ \mathrm{p}_2\ \mathrm{p}_3]^\top \triangleq [x, y, z]^\top$ collect the drone's position in 3D and the yaw angle ψ is assumed to be known by measurement. $\boldsymbol{u} = [T,\ \phi,\ \theta]^\top \in \mathbb{R}^3$ collects the input vector

of the system. Denote also $\mathcal{U} \subset \mathbb{R}^3$ the constraint set for $\boldsymbol{u} \in \mathbb{R}^3$:

$$\mathcal{U} = \{\boldsymbol{u} : 0 \leq T \leq T_{max}, |\phi| \leq \phi_{max}, |\theta| \leq \theta_{max}\} \tag{3.5}$$

where $T_{max} > 0, (\phi_{max}, \theta_{max}) \in (0; \pi/2)^2$ are constant bounds of the inputs.

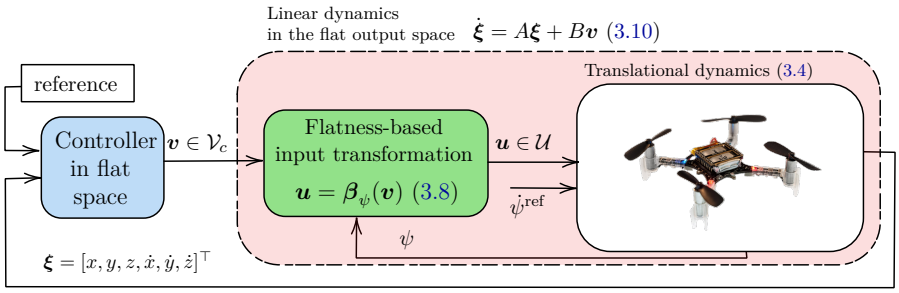


FIGURE 3.3: General scheme for quadcopter outer-loop control in the flat output space.

The model (3.4) of the quadcopter is known to be differentially flat [Nguyen et al., 2017b, Sun et al., 2022]. Namely, all state and input variables can be differentially parameterized by a flat output, and a finite number of its derivatives as in (2.3). Indeed, by choosing the flat output as $\mathbf{p}$ in (3.4), the flat representation of the form (2.3) can be given as:

$$x = \mathrm{p}_1, y = \mathrm{p}_2, z = \mathrm{p}_3, \tag{3.6a}$$

$$T = \sqrt{\ddot{\mathrm{p}}_1^2 + \ddot{\mathrm{p}}_2^2 + (\ddot{\mathrm{p}}_3 + g)^2}, \tag{3.6b}$$

$$\phi = \arcsin\left((\ddot{\mathrm{p}}_1 \sin\psi - \ddot{\mathrm{p}}_2 \cos\psi)/T\right), \tag{3.6c}$$

$$\theta = \arctan\left((\ddot{\mathrm{p}}_1 \cos\psi + \ddot{\mathrm{p}}_2 \sin\psi)/(\ddot{\mathrm{p}}_3 + g)\right). \tag{3.6d}$$

Then by exploiting (3.6b)–(3.6d), the linearizing feedback can be deduced as:

$$T = \sqrt{v_1^2 + v_2^2 + (v_3 + g)^2}, \tag{3.7a}$$

$$\phi = \arcsin\left((v_1 \sin\psi - v_2 \cos\psi)/T\right), \tag{3.7b}$$

$$\theta = \arctan\left((v_1 \cos\psi + v_2 \sin\psi)/(v_3 + g)\right), \tag{3.7c}$$

or in a more compact form:

$$\boldsymbol{u} = \boldsymbol{\beta}_\psi(\boldsymbol{v}), \tag{3.8}$$

where $\boldsymbol{v} = [v_1, v_2, v_3]^\top \in \mathbb{R}^3$ is the new input of the transformed system. Then, under the condition $v_3 \geq -g$ and the mapping (3.8), the system (3.4) yields:

$$\ddot{\mathrm{p}}_i = v_i, \quad i \in \{1, 2, 3\}. \tag{3.9}$$

or, in the state-space form:

$$\dot{\boldsymbol{\xi}} = \boldsymbol{A}\boldsymbol{\xi} + \boldsymbol{B}\boldsymbol{v}, \tag{3.10}$$

with $\boldsymbol{\xi} = [\mathbf{p}^\top, \dot{\mathbf{p}}^\top]^\top = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^\top \in \mathbb{R}^6$ and $\boldsymbol{A} = \begin{bmatrix} \mathbf{0}_3 & \boldsymbol{I}_3 \\ \mathbf{0}_3 & \mathbf{0}_3 \end{bmatrix}$, $\boldsymbol{B} = \begin{bmatrix} \mathbf{0}_3 \\ \boldsymbol{I}_3 \end{bmatrix}$.

Remark 3.1. It can be seen that the linearized system (3.9) includes three separated double integrators. This interpretation will later prove useful when the 6D control problem for (3.10) is decoupled into that of three 2D systems in (3.9). For this purpose, let us define two other equivalent descriptions of the linearized dynamics as the following. The state-space form of (3.9) is denoted as:

$$\dot{\mathbf{p}}_i = \boldsymbol{A}_{2D}\mathbf{p}_i + \boldsymbol{B}_{2D}v_i, i = \{1, 2, 3\}, \tag{3.11}$$

where $\mathbf{p}_i = [\mathrm{p}_i \ \dot{\mathrm{p}}_i]^\top$ from (3.4), $\boldsymbol{A}_{2D} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right], \boldsymbol{B}_{2D} = [0 \ 1]^\top$. Next, to highlight the representation of three concatenated double integrators, the dynamics (3.10) is rearranged into:

$$\dot{\boldsymbol{\zeta}} = \boldsymbol{A}_{6D}\boldsymbol{\zeta} + \boldsymbol{B}_{6D}\boldsymbol{v}, \tag{3.12}$$

where $\boldsymbol{\zeta} = [\mathbf{p}_1^\top \ \mathbf{p}_2^\top \ \mathbf{p}_3^\top]^\top \in \mathbb{R}^6$ and the matrices are $\boldsymbol{A}_{6D} = \mathrm{diag}(\boldsymbol{A}_{2D}, \boldsymbol{A}_{2D}, \boldsymbol{A}_{2D})$, $\boldsymbol{B}_{6D} = \mathrm{diag}(\boldsymbol{B}_{2D}, \boldsymbol{B}_{2D}, \boldsymbol{B}_{2D})$. As a result, along the manuscript, the four descriptions (3.9), (3.10), (3.11) and (3.12) refer to the same system and will be used correspondingly depending on the context.

A general scheme for the outer loop design based on the feedback linearization law (3.8) is given in Figure 3.3. With the dynamics (3.10) and the feedback law (3.7), in the next subsection, the constraints on $\boldsymbol{v}$ will be analyzed to provide a context for the controller designs developed later.

3.2.2 Input constraints characterization in the flat output space

To lay the ground work for the control design, in this part, the new nonlinear constraint set in the flat output space will be described. Furthermore, exploiting the convexity of the set, a procedure to approximate such a set will be then provided.

3.2.2.1 Convoluted constraint description

To ensure $\boldsymbol{u} \in \mathcal{U}$, we define the constraint set for the new in input $\boldsymbol{v}$ in (3.10) as:

$$\mathcal{V} = \{\boldsymbol{v} \in \mathbb{R}^3 : \boldsymbol{\beta}_\psi(\boldsymbol{v}) \in \mathcal{U} \text{ as in (3.5)}\}. \tag{3.13}$$

Next, we show by a counterexample that $\mathcal{V}$ as in (3.13) is a non-convex set. More specifically, $\mathcal{V}$ contains the two points $\boldsymbol{v}_\pm = \boldsymbol{h}_\psi([T_{max}, \pm\phi_{max}, \pm\theta_{max}]^\top)$ but it does not contain their midpoint $0.5(\boldsymbol{v}_- + \boldsymbol{v}_+)$. Thus, the non-convexity hinders us from embedding the control (3.10) into any convex optimization based framework. Second, the description of $\mathcal{V}$ requires a constant update of the yaw angle ψ, which is usually time-varying in practical applications. This requirement is evidently demanding in terms of

computation. For those reasons, $\mathcal{V}$ is rarely exploited in the literature. Indeed, as an alternative that is independent of the yaw angle, a convex subset of $\mathcal{V}$, denoted as $\mathcal{V}_c$, is frequently employed, as [Freire and Xu, 2023, Nguyen et al., 2020a]:

$$\mathcal{V}_c = \left\{ \boldsymbol{v} \in \mathbb{R}^3 : \begin{bmatrix} v_1^2 + v_2^2 + (v_3+g)^2 - T_{max}^2 \\ \sqrt{v_1^2+v_2^2} - (v_3^2+g)\tan\epsilon_{max} \end{bmatrix} \leq 0, \epsilon_{max} \triangleq \min(\theta_{max}, \phi_{max}) \right\}. \quad (3.14)$$

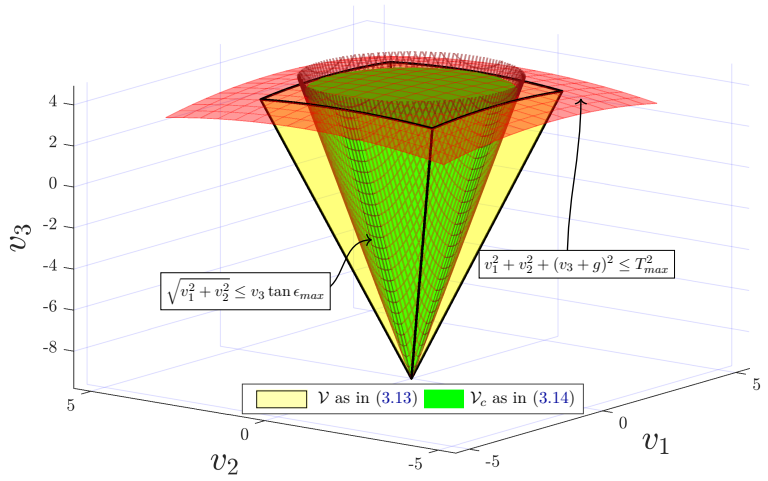


FIGURE 3.4: Quadcopter's input constraints in the flat output space with $\psi = 0$ (rad), $g = T_{max}/1.45 = 9.81$ (m/s^2), $\phi_{max} = \theta_{max} = \pi/18$ (rad)).

In detail, the inclusion of $\mathcal{V}_c$ in (3.14), inside $\mathcal{V}$ as in (3.13) can be shown by considering the following inequalities (see Figure 3.4). Firstly, for the constrained angles, by applying Cauchy-Schwarz inequality in (3.7b) and (3.7c), we always have:

$$|\sin\phi| \leq \sin\epsilon(\boldsymbol{v}) \text{ and } |\tan\theta| \leq \tan\epsilon(\boldsymbol{v}), \quad (3.15)$$

where $\epsilon(\boldsymbol{v}) \triangleq \arctan(\sqrt{(v_1^2+v_2^2)/(v_3+g)^2})$ interprets as a $\boldsymbol{v}$-dependent angle upper bounding the roll ϕ and the pitch angle θ. Then, with the assumption $|\theta|, |\phi| \leq \pi/2$ as in (3.5) and by imposing $\epsilon(\boldsymbol{v}) \leq \epsilon_{max} \triangleq \min(\theta_{max}, \phi_{max})$, we can ensure $[|\theta| - \theta_{max}, |\phi| - \phi_{max}]^\top \leq 0$ thanks to the local monotonicity of $\tan(\cdot)$ and $\sin(\cdot)$ functions. Secondly, for the constrained thrust T, it is straightforward that if $v_1^2+v_2^2+(v_3+g)^2 - T_{max}^2 \leq 0$ holds, we have $0 \leq T \leq T_{max}$. Therefore, $\mathcal{V}_c$ is a subset of $\mathcal{V}$. Moreover, the convexity of $\mathcal{V}_c$ can be proven by showing that $\mathcal{V}_c$ is the intersection of two convex sets: a ball described by $v_1^2+v_2^2+(v_3+g)^2 - T_{max}^2 \leq 0$ and a convex cone given by $\sqrt{v_1^2+v_2^2} \leq (v_3^2+g)\tan\epsilon_{max}$. An illustration of $\mathcal{V}$ and $\mathcal{V}_c$ is given in Figure 3.4 .

Remark 3.2. Let us underline that, the new input constraint set $\mathcal{V}_c$ does not appear in the standard form of the component-wise saturation on the signal magnitude [Hu and Lin, 2001, Blanchini, 1990, Wang and Liu, 2018], but in a convoluted nonlinear representation.

This concern is the motivation to adapt the conventional saturation function towards a specified version particularized for the set $\mathcal{V}_c$, which will be introduced in the Section 3.3.3.

The problem has now been reduced to the control of the linear dynamics (3.10) subject to a convex constraint $\mathcal{V}_c$ as in (3.14) (i.e., designing the blue block in Figure 3.3). With this description, in the next part, we provide a non-conservative polytopic approximation of the set.

3.2.2.2 Zonotopic and polytopic inner approximation

Herein, to further embed the problem into the context of linear control, we seek to approximate $\mathcal{V}_c$ by linear constraints, or a polytope. With the idea of approximating the set by inflating a geometric object and achieving the largest volume inscribed, ellipsoids are typically employed owing to their volume's explicit expression [Boyd et al., 2004]. However, with ellipsoids, few advantages can be of use both geometrically and computationally if employed with MPC. Hence, let us exploit the benefits of zonotopes for which we not only have the volume's explicit formula but also obtain linear constraints.

i) Zonotopic approximation

Exploiting the framework of parameterized zonotope described in Section 2.4.1, we construct an optimization problem to find the largest zonotope from the family of $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$ as in (2.55) constrained in the convex set $\mathcal{V}_c \subset \mathbb{R}^3$:

$$(\boldsymbol{\delta}, \boldsymbol{c})^* = \arg\max_{\boldsymbol{\delta}, \boldsymbol{c}} \mathcal{C}(\boldsymbol{\delta}), \tag{3.16a}$$

$$\text{s.t } \boldsymbol{v} \in \mathcal{V}_c, \forall \boldsymbol{v} \in \mathbb{E}(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})), \tag{3.16b}$$

$$\boldsymbol{v}^{\text{int}} \in \mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c}), \tag{3.16c}$$

where $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c}) \subset \mathbb{R}^3$ denotes the zonotope formed by the given n_g generators in $\boldsymbol{G} \in \mathbb{R}^{3\times n_g}$ centered at $\boldsymbol{c}$; $\boldsymbol{\delta} = [\delta_1, ..., \delta_{n_g}] \in \mathbb{R}^{n_g}$ is a scaling factor magnifying the original zonotope $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$. $\mathcal{C}(\boldsymbol{\delta})$ denotes the volume of $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$ computed as in (2.56) with $\Delta \triangleq \text{diag}(\boldsymbol{\delta})$. Next, $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})) \subset \mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$ is *finite* and contains all vertices of $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$, and can be enumerated as in (2.53). This condition (3.16b) ensures the inclusion $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c}) \subset \mathcal{V}_c$ as previously discussed. Finally, $\boldsymbol{v}^{\text{int}} \in \text{int}(\mathcal{V}_c)$ is a user-defined point towards which the resulting set is allowed to expand. Specifically, since zonotopes are symmetric, it is impossible for them to expand freely inside $\mathcal{V}_c$. Therefore, progressively imposing the condition (3.16c) with different $\boldsymbol{v}^{\text{int}}$ helps us obtain several zonotopes reaching to specific "corners" of $\mathcal{V}_c$. Then the final approximated set is achieved as the convex hull of all the zonotopes deduced from those choices of $\boldsymbol{v}^{\text{int}}$.

For instance, let us denote $\mathcal{I}$ the sets of choice containing some $\boldsymbol{v}^{\text{int}} \in \text{int}(\mathcal{V}_c)$. One candidate can be enumerated as in (3.17) which contains $N_0 + 1$ points taken between

two extreme ones of $\mathcal{V}_c$: $[0, 0, -g]^\top$ and $[0, 0, T_{max} - g]^\top$:

$$\mathcal{I} = \{\boldsymbol{v}_0^{\text{int}}, ..., \boldsymbol{v}_{N_0}^{\text{int}}\} \tag{3.17}$$

with $\boldsymbol{v}_k^{\text{int}} = [0, 0, 1]^\top((1 - k/N_0)T_{max} - g)$. Then, for each $0 \leq k \leq N_0$ as in (3.17), we obtain from (3.16) a parameterized zonotope $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$, assigned as $\mathcal{S}_v^k$. Finally, the resulting approximation set is computed as:

$$\mathcal{S}_v = \text{Conv}\left\{\mathcal{S}_v^0, ..., \mathcal{S}_v^{N_0}\right\}. \tag{3.18}$$

The procedure is summarized in Algorithm 3.1.

Algorithm 3.1: $\mathcal{V}_c$ approximation procedure.

Input: Generator $\boldsymbol{G}$ as in (3.16) and $\mathcal{I}$ as in (3.17).
Output: The approximation set $\mathcal{S}_v$ as in (3.18) of $\mathcal{V}_c$.

for $k = 0$ **to** N_0 **do**
  $\boldsymbol{v}_k^{\text{int}} \leftarrow [0, 0, 1]^\top((1 - k/N_0)T_{max} - g)$;
  Solve the optimization problem (3.16) for $(\boldsymbol{\delta}, \boldsymbol{c})$;
  $\mathcal{S}_v^k \leftarrow \mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c})$ with $\Delta = \text{diag}(\boldsymbol{\delta})$;
end
$\mathcal{S}_v \leftarrow \text{Conv}\left\{\mathcal{S}_v^0, ..., \mathcal{S}_v^{N_0}\right\}$ as in (3.18).

Remark 3.3. Although the semi-definite condition (3.16b) is straightforward to construct, it comes with a shortcoming: the computational cost rises exponentially with the number of generators, n_g, since the number of elements in $\mathbb{E}(\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c}))$ is 2^{n_g}. This drawback is indeed burdensome, especially for the approximation of high dimensional sets, because one needs a sufficiently large number of generators to have a good basis zonotope $\mathcal{Z}(\boldsymbol{G}, \boldsymbol{c})$ to be scaled with $\boldsymbol{\delta}$.

For illustration, we examine two scenarios with $N_0 = 2$ and $N_0 = 25$ for (3.17). The numerical results and parameters setup are given in Figure 3.5 and Table 3.1, respectively.

For comparison, we examine other constraint sets for quadcopter in the literature, which are constructed as follows.

- In [Nguyen et al., 2020b], a box-type subset of $\mathcal{V}_c$ was introduced as:

$$\mathcal{P}_v = \{\boldsymbol{v} \in \mathbb{R}^3 : |v_i| \leq \bar{v}_{ip}, i \in \{1, 2, 3\}\} \tag{3.19}$$

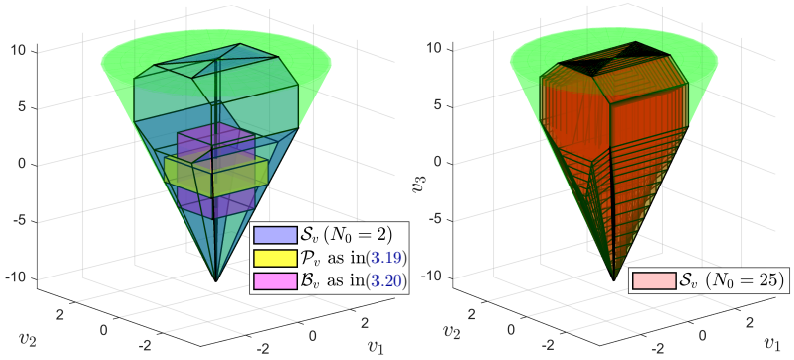


FIGURE 3.5: Approximated constraint sets for the new input $\boldsymbol{v}$.

with the constant $\bar{v}_{ip}$ satisfying the conditions:

$$\begin{cases} \bar{v}_{3p} < g;\ \bar{v}_{1p}^2 + \bar{v}_{2p}^2 \leq (-\bar{v}_{3p} + g)^2 \tan^2 \epsilon_{max} \\ \sqrt{\bar{v}_{1p}^2 + \bar{v}_{2p}^2 + (\bar{v}_{3p} + g)^2} \leq T_{max}. \end{cases}$$

- The approximated origin-centered constraint set, in [Mueller and D'Andrea, 2013, Greeff and Schoellig, 2018], described as:

$$\mathcal{B}_v = \{\boldsymbol{v} \in \mathbb{R}^3 : |v_i| \leq \bar{v}_{ib}, i \in \{1, 2, 3\}\} \tag{3.20}$$

can be similarly found by employing (3.16) with $\boldsymbol{G} = \boldsymbol{I}$ and $\boldsymbol{v}^{\text{int}} = [0, 0, 0]^\top$.

TABLE 3.1: Parameters setup for $\mathcal{V}_c$ approximation

Symbols	Values
g, T_{max}	9.81; 19.62 m/s^2
θ_{max}, ϕ_{max}	0.1745 rad (10^o)
$\boldsymbol{G}$	$\begin{bmatrix} 1 & 0 & 0 & 0 & 0 \\ 0 & 1 & 0 & 1 & -1 \\ 0 & 0 & 1 & 2 & 2 \end{bmatrix}$
$\bar{v}_{1p} = \bar{v}_{2p} = \bar{v}_{3p}$	1.0875
$[\bar{v}_{1b}, \bar{v}_{2b}, \bar{v}_{3b}]$	$[0.815, 0.815, 3.270]$

The illustration of the aforementioned sets are also shown in Figure 3.5 with their numerical values given in Table 3.1 and 3.2.

As depicted in Figure 3.5, our approach provides an improved approximation for the input $\boldsymbol{v}$, with respect to the literature.

TABLE 3.2: Result specifications for $\tilde{\mathcal{V}}$ approximation

	$\mathcal{S}_v, N_0 = 2$	$\mathcal{S}_v, N_0 = 25$	$\mathcal{P}_v$	$\mathcal{B}_v$
Volume	122.57	125.27	10.29	17.39
No. of vertices	28	226	8	8
No. of inequalities	20	216	6	6

Remark 3.4. It is noteworthy that Algorithm 3.1 can be adapted to approximate any convex set $\mathcal{S}$ by simply sampling the interior $\text{int}(\mathcal{S})$ by a list of interior points $\mathcal{I}$. Furthermore, the feasibility the optimization problem (3.16) is always ensured since by definition of interior points, there always exist some scaling vector $\boldsymbol{\delta}$ small enough so that $\mathcal{Z}(\boldsymbol{G}\Delta, \boldsymbol{c}) \subset \mathcal{S}$.

Algorithm 3.1 can be used to obtain a polytopic inner-approximation of convex sets.

To this end, a generic procedure to approximate convex set via zonotopes has been proposed. Next, we provide a particular linear approximation of $\mathcal{V}_c$ in (3.14) via the parameterization of its surfaces.

ii) A particular polytopic approximation of $\mathcal{V}_c$

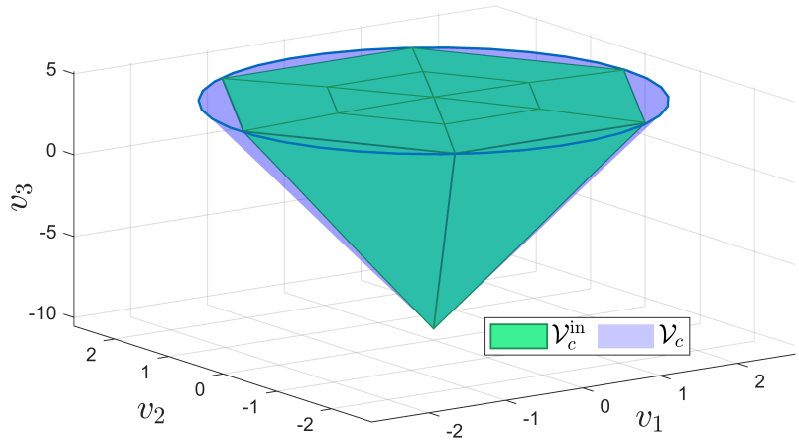


FIGURE 3.6: $\mathcal{V}_c$ in (3.14) and $\mathcal{V}_c^{\text{in}}$ in (3.22) with $s_1 = 7, s_2 = 3$, $g = T_{max}/1.45 = 9.81 m/s^2, \epsilon_{max} = 0.1745\ rad.$

It follows directly from the convexity of $\mathcal{V}_c$ that given a finite collection of boundary points $\mathcal{V}_b \subset \mathcal{V}_c$, $\text{conv}\{\mathcal{V}_b\} \subset \mathcal{V}_c$. In this manner, a less conservative polytopic approximation $\text{conv}\{\mathcal{V}_b\}$ can be deduced by collecting boundary points into $\mathcal{V}_b$. With this idea, it can be seen from (3.14) that, $\mathcal{V}_c$ contains a sphere of radius T_{max} and a convex cone characterized by ϵ_{max}. After some simple manipulation, the intersection of these two

bounding surfaces can be represented implicitly by the circle:

$$\mathcal{R}_v = \{\boldsymbol{v} = [v_1\ v_2\ v_3]^\top : v_3 = v_3^\star, v_1^2 + v_2^2 = R^\star\}. \tag{3.21}$$

with $R^\star \triangleq T_{max} \sin \epsilon_{max}$, $v_3^\star \triangleq T_{max} \cos \epsilon_{max} - g$. Hence, by parameterizing this intersection between the two surfaces, the boundary point of the disk and the vertex $[0, 0, -g]^\top$, the set $\mathcal{V}_c$ can be approximated as:

$$\mathcal{V}_c^{\text{in}} = \text{conv} \left\{ \begin{matrix} [0, 0, -g]^\top, [R^\star \cos \alpha_i, R^\star \sin \alpha_i, v_3^\star]^\top, \\ [r \cos \alpha_i, r \sin \alpha_i, \sqrt{T_{max}^2 - r^2} - g]^\top \end{matrix} \right\} \tag{3.22}$$

with $\alpha_i \in \mathcal{N}(s_1, 0, 2\pi), r \in \mathcal{N}(s_2, 0, R^\star)$ for some integer $s_1, s_2 > 2$ (the larger s_1 and s_2, the more closely $\mathcal{V}_c^{\text{in}}$ approaches $\mathcal{V}_c$). An illustration the set is provided in Figure 3.6.

So far, the exactly linearized model and the approximations of the constraint in the new coordinate have been presented. Next, before proceeding to the control design, let us introduce the experimental platform in Laboratoire de Conception et d'Intégration des Systèmes (LCIS) with the Crazyflie 2.1 nano-drone.

3.2.3 Experimental platform description

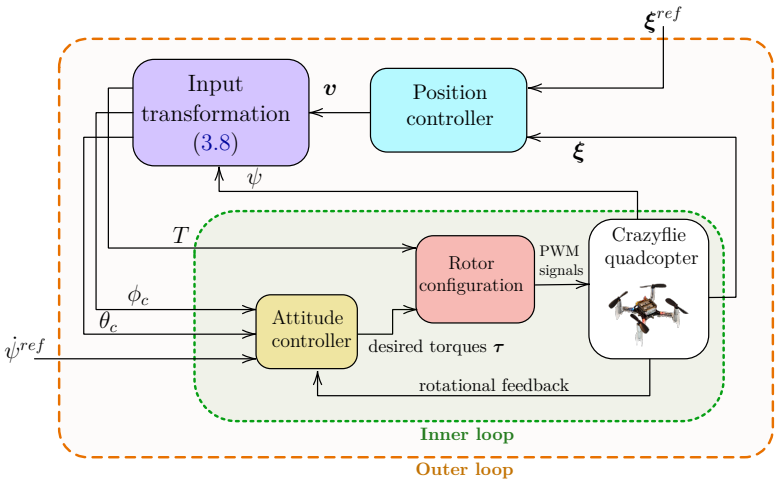


FIGURE 3.7: Hierarchical control architecture of the Crazyflie quadcopter.

For the experiments, a nano-drone Crazyflie 2.1 platform will be employed. This aerial drone provides the access to its outer loop controller, which requires exactly the input $\boldsymbol{u} = [T, \phi, \theta]^\top$ as in (3.4) together with the desired angular rate for the yaw angle (for our scenario, this variable $\dot{\psi}^{\text{ref}}$ is considered as 0). The packet containing $\boldsymbol{u}$ and $\dot{\psi}^{\text{ref}} = 0$ will be computed via the proposed controller on a ground station computer and sent to the Crazyflie via a PA USB radio dongle. Note that, before being sent

to the drone, the normalized thrust $T\,(m/s^2)$ needs to be converted to the thrust unit of the Crazyflie platform, ranging from 0 to 65535. Details on the conversion can be found on the documentation of Bitcraze AB team with the system identification fitting model [Förster, 2015]. The implementation is carried out using the *Commander* class provided in the API reference with Python 3.9. The computed parameters are sent to an onboard attitude controller, which calculates the desired torque necessary for the tracking. Together with such values, the PWM signals are computed accordingly via an appropriate rotor configuration [Luis and Ny, 2016, Giernacki et al., 2017] and applied to the motors. The experimental adaptation of Figure 3.2 is provided in Figure 3.7. Therein, for the attitude controller, two cascade control loops are integrated to stabilize the rotational dynamics [Giernacki et al., 2017]: i) the reference for the angular rates $\dot{\phi}^{ref}, \dot{\theta}^{ref}$ of the roll, pitch angles are computed via a PID controller stabilizing the errors between the measured and the desired values received from the position controller (the proposed outer-loop); ii) these rate references combined with $\dot{\psi}^{ref}$ will be tracked by another PID loop, which results in the desired torques in the direction of the roll, pitch and yaw angles. These torques and the thrust T then are linearly transformed into PWM signals via a so-called *Control Mixer* specified for the "X"-shaped configuration of the nanodrone. Detailed description and operational parameters can be found in the technical report [Luis and Ny, 2016]. Other comparisons between different rotor configurations (e.g. "+"-shaped configuration) were studied in [Idrissi et al., 2022].

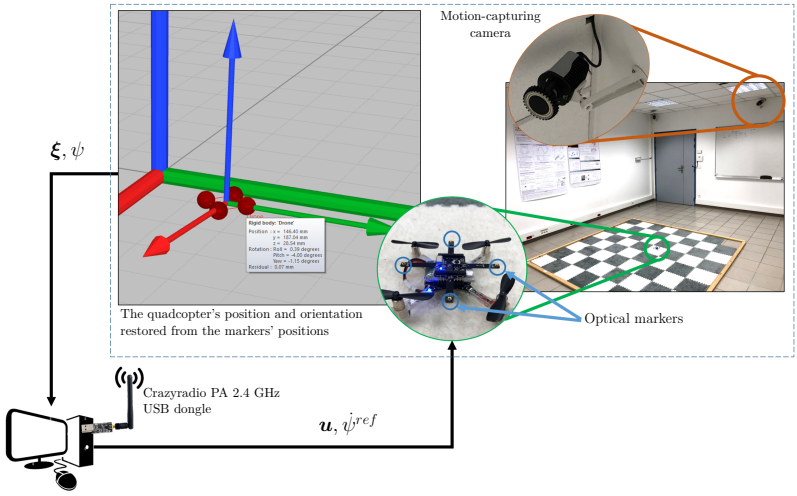


FIGURE 3.8: LCIS lab experimental setup with optical motion capture system and the Crazyflie 2.1.

For the feedback sensors, an indoor optical motion capture system is employed. More specifically, 8 infrared cameras from Qualisys [Senior, 2004] are used to capture the position and orientation of a rigid body identified with a collection of reflective markers which are glued on the quadcopter. Then, based on the positions of the markers captured

by the cameras, the motion of the drone are estimated by Qualisys Track Manager software and sent to the ground station computer in real time via TCP/IP. The cameras' capture rate is chosen at 120Hz for the experiments. An illustration of our experimental setup is given in Figure 3.8.

With the above theoretical analysis and experimental setting, three different control strategies will be provided in the next section.

3.3 Computationally efficient control in the flat output space

So far, the quadcopter model has been exactly linearized to (3.10) and the convoluted constraint is approximated by a polytope, or linear inequalities (as in (3.18) or (3.22)). This setting fits into the synthesis of the well-developed linear MPC framework. Thus, we first explore the existing MPC solutions, which include implicit and explicit MPC. These strategies will be followed by an explicit saturated control adapted from a standard gradient-based solution.

3.3.1 Implicit model predictive control

For digital implementation, we proceed with the discretization of (3.10) as follows. With $\boldsymbol{u}_k = \boldsymbol{\beta}_\psi(\boldsymbol{v}_k)$ in (3.8), the system (3.10) can now be discretized as:

$$\boldsymbol{\xi}(k+1) = \boldsymbol{A}_d\boldsymbol{\xi}(k) + \boldsymbol{B}_d\boldsymbol{v}(k), \tag{3.23}$$

subject to $\boldsymbol{v}(k) \in \mathcal{S}_v$ in (3.18) with $\boldsymbol{A}_d = \begin{bmatrix} \boldsymbol{I}_3 & t_s\boldsymbol{I}_3 \\ \boldsymbol{0}_3 & \boldsymbol{I}_3 \end{bmatrix}$, $\boldsymbol{B}_d = [t_s^2\boldsymbol{I}_3/2,\ t_s\boldsymbol{I}_3]^\top$ are from the 4th order Runge-Kutta discretization of (3.10) with the sampling time t_s. Thus, to validate the inner approximation in Section 3.2.2.2, in the trajectory tracking application, we solve online the following QP over the prediction horizon of N_p steps:

$$\underset{\boldsymbol{v}(k|k+i)}{\arg\min} \sum_{i=0}^{N_p-1} \left\|\boldsymbol{\xi}(k|k+i) - \boldsymbol{\xi}^{\text{ref}}(k+i)\right\|_{\boldsymbol{Q}}^2 + \left\|\boldsymbol{v}(k|k+i) - \boldsymbol{v}^{\text{ref}}(k+i)\right\|_{\boldsymbol{R}}^2 \tag{3.24a}$$

$$\text{s.t: } \begin{cases} \boldsymbol{\xi}(k|k+i+1) = \boldsymbol{A}_d\boldsymbol{\xi}(k|k+i) + \boldsymbol{B}_d\boldsymbol{v}(k|k+i), \\ \boldsymbol{v}(k|k+i) \in \mathcal{S}_v \text{ in (3.18)}, i \in \{0, ..., N_p-1\} \end{cases} \tag{3.24b}$$

with $\boldsymbol{\xi}^{\text{ref}}(k), \boldsymbol{v}^{\text{ref}}(k)$ gathering the reference signal of the state $\boldsymbol{\xi}$ and input $\boldsymbol{v}$ at time step k. Then the first value of the solution sequence (i.e, $\boldsymbol{v}(k|k)$) will be used to compute the real control $\boldsymbol{u}(k) = \boldsymbol{\beta}_\psi(\boldsymbol{v}(k))$, which then is applied to the quadcopter. Note that to align with the results published in [C3], we also refer to the control in (3.24) as flatness-based MPC. However, when it is put next to the explicit MPC (ExMPC) setup, all

the MPC problems solved online (including (3.24)) will be referred to as implicit MPC (IMPC).

Remark 3.5. In the flat output space, by using our convex approximation of the feasible domain, the MPC design employs both a linear model and linear constraints. Hence, within this framework, other properties (e.g, stability, robustness), either in discrete or continuous time, become more accessible for investigation [Mayne et al., 2006, Blanchini et al., 2003]. Note that these advantages do not exist for all flat systems, especially for those with the flat output different from the states (or output) of interest [Zafeiratou et al., 2020, Do et al., 2021], hence, complicating the theoretical guarantees when switching from one space to the other.

The online MPC routine is now reduced to a standard QP problem. To this end, aiming for complexity reduction, the natural question is whether it can be solved explicitly via the multi-parametric QP (mpQP) solvers and provide a faster controller. The answer is positive, yet at a price of, again, another constraint approximation. The next part will present how explicit MPC can be applied for this drone control problem.

3.3.2 Explicit model predictive control

Previously, the problem of nonlinearity has been tackled thanks to the mapping (3.8) and the approximation (3.22). With the complexity of a QP, in principle, the framework of linear Explicit MPC (ExMPC) now can be applied to compute explicitly the optimal control for the six-dimensional (6D) system (3.23) with the constraint $\boldsymbol{v}(k) \in \mathcal{V}_c^{\text{in}}$ as in (3.22). However, as notoriously known for ExMPC, the dimensionality burden is significant for systems of high dimension [Lee, 2011]. In other words, the explicit solution is exponentially costly in the offline construction time, data storage capacity required and the online look-up time, with respect to the dimension.

To overcome this challenge, it is noteworthy that in the new coordinate, the linear dynamics (3.9) indeed contains three independent double integrators as in (3.11). This representation will be exploited to decouple the controller into three sub-problems. More specifically, the input constraints for $\boldsymbol{v}$ will be approximated by a box-type constraint, giving rise to the explicit MPC formulation for each double integrator in (3.11). Comparison and simulation with the solution of the classical 6D model will be provided. A summary of the control scheme is given in Figure 3.9.

3.3.2.1 ExMPC formulation

Consider the RK4 discretization of (3.11):

$$\mathbf{p}_i(k+1) = \boldsymbol{A}_{2D}^d \mathbf{p}_i(k) + \boldsymbol{B}_{2D}^d v_i(k), \tag{3.25}$$

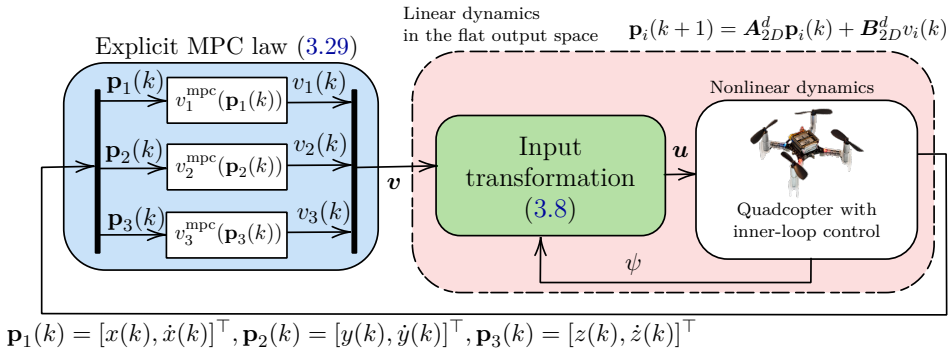


FIGURE 3.9: Explicit MPC scheme for the quadcopter position regulation.

with $\boldsymbol{A}^d_{2D} \triangleq \left[\begin{smallmatrix} 1 & t_s \\ 0 & 1 \end{smallmatrix}\right], \boldsymbol{B}^d_{2D} \triangleq [\,0.5t_s^2 \;\; t_s\,]^\top$ and the sampling time t_s. Given a box in $\mathbb{R}^3$ described as:

$$\mathcal{B}_v = \left\{\boldsymbol{v} \in \mathbb{R}^3 : |v_i| \leq \bar{v}_i, i \in \{1,2,3\}\right\} \subset \mathcal{V}_c \text{ as in (3.14)}, \tag{3.26}$$

then, without loss of generality, we decouple the position tracking problem derived from (3.9) into the three following MPC regulation problems:

$$\min \sum_{j=0}^{N_p-1} V_s(\mathbf{p}_i(k|k+j), v_i(k|k+j)) + V_f(\mathbf{p}_i(k|k+N_p)) \tag{3.27a}$$

$$\text{s.t:} \begin{cases} \mathbf{p}_i(k|k+j+1) = \boldsymbol{A}^d_{2D}\mathbf{p}_i(k|k+j) + \boldsymbol{B}^d_{2D}v_i(k|k+j), \\ \mathbf{p}_i(k|k+j) \in \mathcal{X}_i,\ j \in \{0,...,N_p-1\}, \\ |v_i(k|k+j)| \leq \bar{v}_i, \text{ as in (3.26)}, \mathbf{p}_i(k|k+N_p) \in \mathcal{X}_{f,i},\ i \in \{1,2,3\}, \end{cases} \tag{3.27b}$$

where, for each $i \in \{1,2,3\}$, $\mathbf{p}_i \in \mathbb{R}^2$ and v_i are the state and the input of the system (3.11). $\mathcal{X}_i \triangleq \{\mathbf{p}_i \in \mathbb{R}^2 : |\mathbf{p}_i| \leq [\bar{\mathrm{p}}_i, \dot{\bar{\mathrm{p}}}_i]^\top\}$ is the rectangular state constraint defining the workspace of the vehicle; $\mathcal{X}_{f,i} \subseteq \mathcal{X}_i$ denotes the terminal constraint set which is associated with the terminal cost $V_f(\cdot)$ to ensure both stability and feasibility [Mayne et al., 2000]. In details, the state cost $V_s(\cdot)$ and terminal cost $V_f(\cdot)$ are chosen as:

$$V_s(\mathbf{p}_i, v_i) = \|\mathbf{p}_i\|^2_{\boldsymbol{Q}} + \|v_i\|^2_{\boldsymbol{R}};\ \ V_f(\mathbf{p}_i) = \|\mathbf{p}_i\|^2_{\boldsymbol{P}}, \tag{3.28}$$

with $\boldsymbol{R} \succ 0, \boldsymbol{Q} \succeq 0$ and $\boldsymbol{P} \succeq 0$ are the user-designed weighting matrix defining the optimization problem (3.27a). For the sake of simplicity, in (3.27a), we employ the same choice of weighting $\boldsymbol{P}, \boldsymbol{Q}, \boldsymbol{R}$ for all $i \in \{1,2,3\}$.

Denote $\boldsymbol{v}_i^*(\mathbf{p}_i(k)) \triangleq [v_i^*(k|k), ..., v_i^*(k|k+N_p)]^\top$ the optimizer of (3.27). Then, the control applied to system (3.25) is given as:

$$v_i(k) = v_i^{\text{mpc}}(\mathbf{p}_i(k)) \triangleq v_i^*(k|k). \tag{3.29}$$

Moreover, with the system's linear representation (3.25), the solution for the QP (3.27) can be rewritten in its dual form via the Karush-Kuhn-Tucker (KKT) conditions. Then, depending on the state $\mathbf{p}_i(k)$, a subset of constraints (3.27b) becomes active, on which is formulated a critical region. Over this activated domain, the constrained optimum for (3.27) can be explicitly parameterized by $\mathbf{p}_i(k)$ as [Bemporad et al., 2002]:

$$\underset{l\in\{1,\dots,\bar{l}_i\}}{\boldsymbol{v}_i^*(\mathbf{p}_i(k))} = \boldsymbol{F}_{i,l}\mathbf{p}_i(k) + \boldsymbol{\mu}_{i,l}, \text{ if } \mathbf{p}_i(k) \in \mathcal{R}_{i,l}, \tag{3.30}$$

where $\mathcal{R}_{i,l} = \{\mathbf{p}_i : \boldsymbol{A}_{i,l}\mathbf{p}_i \leq \boldsymbol{b}_{i,l}\} \subset \mathbb{R}^2$ denotes the l-th polyhedral critical region, and the constant parameters $\boldsymbol{F}_{i,l}, \boldsymbol{\mu}_{i,l}$ are the associated optimal parameters in such a region. $\bar{l}_i$ denotes the total number of critical regions for each double integrator in (3.25).

Remark 3.6. It is noteworthy that since $\mathcal{V}_c$ has the origin in its interior, there is always some subset $\mathcal{B}_v$ in the form of (3.26) with $\bar{v}_i$ being sufficiently small. Moreover, as one candidate, the maximum volume box $\mathcal{B}_v$ inscribed in $\mathcal{V}_c$, can be found by inflating a zonotope via an optimization problem presented in Section 3.2.2.2, since, $\mathcal{B}_v$ is, indeed, also a zonotope with the center at the origin. Furthermore, analytical solution for the largest $\mathcal{B}_v$ inscribed in $\mathcal{V}_c$ are formulated in the Appendix A.1.

To this point, the control of the nonlinear system (3.4) has been reduced to solving an online MPC as in (3.24) or a search in the look-up tables as in (3.30). Next, we show that the controller can be embedded in the standard saturated control as well.

3.3.3 Explicit saturated control

Herein, a specialized saturated controller for the constraint set $\mathcal{V}_c$ is introduced, along with the stability and constraint satisfaction guarantees.

3.3.3.1 Controller characterization

With the goal to stabilize the system (3.10) under constraints (3.14), a saturated controller is introduced as follows.

Proposition 3.1. *Consider a ball*

$$\mathcal{B}(\rho) = \{\boldsymbol{v} \in \mathbb{R}^3 : \|\boldsymbol{v}\|_2^2 \leq \rho\} \ s.t\ \mathcal{B}(\rho) \subseteq \mathcal{V}_c \ \textit{as in}\ (3.14), \tag{3.31}$$

some scalars $\alpha > 0, \gamma \geq 1$, *a matrix* $\boldsymbol{P} \succ 0$ *satisfying the condition* (2.66), *and the saturated controller defined as:*

$$\boldsymbol{v} = \text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}), \tag{3.32}$$

with $\text{sat}_\lambda(\boldsymbol{v})$ *given by:*

$$\text{sat}_\lambda(\boldsymbol{v}) = \begin{cases} \boldsymbol{v} \text{ if } \boldsymbol{v} \in \mathcal{V}_c, \\ \lambda^*(\boldsymbol{v})\boldsymbol{v} \text{ if } \boldsymbol{v} \notin \mathcal{V}_c \end{cases} \tag{3.33a}$$

$$\text{with } \lambda^*(\boldsymbol{v}) = \arg\max_{\lambda \boldsymbol{v} \in \mathcal{V}_c} \lambda. \tag{3.33b}$$

The following results hold for the system (3.10) *under the controller* (3.32)*:*

1. *The control action* $\boldsymbol{v}$ *as in* (3.32) *is contained in* $\mathcal{V}_c$*, i.e.,*

$$\boldsymbol{v} = \text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) \in \mathcal{V}_c \, \forall \boldsymbol{\xi} \in \mathbb{R}^6, \gamma \geq 1. \tag{3.34}$$

2. *The following ellipsoidal set is rendered invariant:*

$$\mathcal{E}(\boldsymbol{P}, \varepsilon) = \{\boldsymbol{\xi} \in \mathbb{R}^6 : \|\boldsymbol{\xi}\|_{\boldsymbol{P}}^2 \leq \varepsilon\}, \tag{3.35}$$

where the largest $\varepsilon > 0$ *is found by solving:*

$$\varepsilon^* = \arg\max_{\varepsilon} (\varepsilon) \tag{3.36a}$$

$$s.t \begin{bmatrix} \boldsymbol{P} & \boldsymbol{0}_{6\times 1} \\ \boldsymbol{0}_{1\times 6} & -\varepsilon \end{bmatrix} - \tau \begin{bmatrix} \boldsymbol{P}\boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P} & \boldsymbol{0}_{6\times 1} \\ \boldsymbol{0}_{1\times 6} & -\rho \end{bmatrix} \succeq 0, \tau \geq 0. \tag{3.36b}$$

Moreover, the system (3.10) *achieves exponential stability with domain of attraction* $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ *as in* (3.35)*.*

Proof. The constraint satisfaction follows immediately from condition (3.33). Next, to prove the stability, we show that the quadratic function associated with $\boldsymbol{P}$ is decreasing (its derivative is negative) inside $\mathcal{E}(\boldsymbol{P}, \varepsilon)$. Then, the set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ is indeed positively invariant. Hence, once the system is inside the set, its stability and constraints are respected.

Firstly, let us proceed to prove the stability of the system inside $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ by explaining the choice of ε as in (3.36). More specifically, by solving (3.36b), it can be shown that the containment of the state $\boldsymbol{\xi}$ inside the ellipsoid $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ implies that $-\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi} \in \mathcal{B}(\rho)$ as in (3.31), i.e.,

$$\boldsymbol{\xi} \in \mathcal{E}(\boldsymbol{P}, \varepsilon) \Rightarrow -\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi} \in \mathcal{B}(\rho) \text{ as in (3.31)}. \tag{3.37}$$

Indeed, rewriting (3.37) explicitly yields:

$$\|\boldsymbol{\xi}\|_{\boldsymbol{P}}^2 \leq \varepsilon \Rightarrow \|\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}\|_2^2 \leq \rho \tag{3.38}$$

Then by applying the $\mathcal{S}$-procedure [Boyd et al., 1994], the condition is (3.38) equivalently rewritten as:

$$\exists \tau \geq 0 \text{ s.t: } \begin{bmatrix} \boldsymbol{P} & \mathbf{0}_{6\times 1} \\ \mathbf{0}_{1\times 6} & -\varepsilon \end{bmatrix} - \tau \begin{bmatrix} \boldsymbol{P}\boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P} & \mathbf{0}_{6\times 1} \\ \mathbf{0}_{1\times 6} & -\rho \end{bmatrix} \succeq 0. \tag{3.39}$$

Therefore, the conditions (3.37)-(3.38) can be assured by solving (3.36). Moreover, from the definition (3.33), $\forall \gamma \geq 1$ we can state:

$$\gamma \lambda^*(\gamma \boldsymbol{v}) \geq 1, \forall\ \boldsymbol{v} \neq \mathbf{0}_{3\times 1}, \boldsymbol{v} \in \mathcal{V}_c. \tag{3.40}$$

The explanation is twofold. First, one has $\gamma^{-1} \in \ \Delta \triangleq \{\lambda : \gamma\lambda \boldsymbol{v} \in \mathcal{V}_c\}$ when $\boldsymbol{v} \in \mathcal{V}_c$. Second, by definition, $\lambda^*(\gamma \boldsymbol{v}) = \max \Delta$, hence $\lambda^*(\gamma \boldsymbol{v}) \geq \gamma^{-1}$, leading to (3.40).

This property later will serve as an ingredient to show the non-positivity of the Lyapunov function's derivative inside the set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35). More specifically, with the fact that $\boldsymbol{P}$ satisfies the condition (2.66), consider the the Lyapunov function:

$$V(\boldsymbol{\xi}) \triangleq \boldsymbol{\xi}^\top \boldsymbol{P} \boldsymbol{\xi}. \tag{3.41}$$

Then, under the controller (3.32), the time derivative of (3.41) yields:

$$\dot{V}(\boldsymbol{\xi}) = 2\boldsymbol{\xi}^\top \boldsymbol{P}(\boldsymbol{A}\boldsymbol{\xi} + \boldsymbol{B}\text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi})). \tag{3.42}$$

According to the definition of the saturation function in (3.33), the closed-loop behavior inside $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ is dependent on the two following cases:

Unsaturated input: If the control input (3.32) is not saturated, we have:

$$\boldsymbol{v} = \text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) = -\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}. \tag{3.43}$$

Thus, the stability is straightforward, since the equation (3.42) now yields:

$$\dot{V}(\boldsymbol{\xi}) \leq -\alpha V(\boldsymbol{\xi}),\ \forall \boldsymbol{\xi} \in \mathcal{E}(\boldsymbol{P}, \varepsilon). \tag{3.44}$$

Saturated input: If saturated, the control input (3.32) is computed as:

$$\boldsymbol{v} = \text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) = -\lambda^*(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi})\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}. \tag{3.45}$$

Then, taking into account (2.66), equation (3.42) becomes:

$$\begin{aligned} \dot{V}(\boldsymbol{\xi}) &= 2\boldsymbol{\xi}^\top \boldsymbol{P}\boldsymbol{A}\boldsymbol{\xi} + 2\boldsymbol{\xi}^\top \boldsymbol{P}\boldsymbol{B}\text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) \\ &\leq \boldsymbol{\xi}^\top(-\alpha \boldsymbol{P} + 2\boldsymbol{P}\boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P})\boldsymbol{\xi} + 2\boldsymbol{\xi}^\top \boldsymbol{P}\boldsymbol{B}\text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) \\ &\leq -\alpha\|\boldsymbol{\xi}\|_{\boldsymbol{P}}^2 - 2\|\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}\|_2^2(\gamma\lambda^*(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) - 1). \end{aligned} \tag{3.46}$$

Meanwhile, by replacing $\boldsymbol{v} = -\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}$ into the condition (3.40), one achieves:

$$\gamma\lambda^*(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}) \geq 1 \;\; \forall \boldsymbol{\xi} \in \mathcal{B}_{\boldsymbol{P}}(\varepsilon) \setminus \{\boldsymbol{0}_{6\times 1}\}. \tag{3.47}$$

As a consequence, (3.46) yields:

$$\dot{V}(\boldsymbol{\xi}) \leq -\alpha \|\boldsymbol{\xi}\|^2_{\boldsymbol{P}} = -\alpha V(\boldsymbol{\xi}) \;\; \forall \boldsymbol{\xi} \in \mathcal{E}(\boldsymbol{P}, \varepsilon). \tag{3.48}$$

Therefore, from (3.44) and (3.48), $V(\boldsymbol{\xi})$ is shown to be exponentially decreasing inside $\mathcal{E}(\boldsymbol{P}, \varepsilon)$:

$$\dot{V}(\boldsymbol{\xi}) \leq -\alpha V(\boldsymbol{\xi}). \tag{3.49}$$

Secondly, the Nagumo's invariance condition as in Proposition 2.3 for the set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ can be evidently proven since, particularly in this case, the condition (2.61) yields:

$$\boldsymbol{\xi}_b^\top \boldsymbol{P}(\boldsymbol{A}\boldsymbol{\xi}_b + \boldsymbol{B}\text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi}_b)) = \dot{V}(\boldsymbol{\xi}_b)/2 \leq 0, \tag{3.50}$$

which holds $\forall \boldsymbol{\xi}_b \in \partial\mathcal{E}(\boldsymbol{P}, \varepsilon)$ because $\dot{V}(\boldsymbol{\xi}) < 0$, $\forall \boldsymbol{\xi} \in \mathcal{E}(\boldsymbol{P}, \varepsilon)$ as proven previously for the stability. Furthermore, condition (3.47) along the invariance property ensures asymptotic stability with domain of attraction $\mathcal{E}(\boldsymbol{P}, \varepsilon)$. □

With the above demonstration, we summarize the synthesis procedure for the controller (3.32) as the following:

1. Choose $\rho > 0$, such that $\mathcal{B}(\rho) \subset \mathcal{V}_c$ as in (3.31);
2. Choose $\alpha > 0$ and solve (2.66) for a stabilizing symmetric matrix $\boldsymbol{P}$;
3. With any choice $\gamma \geq 1$, the ellipsoid $\mathcal{B}(\boldsymbol{P}, \varepsilon)$ as in (3.35) is guaranteed to be positively invariant.

Remark 3.7. It is apparent that no eigenvalue of the matrix $\boldsymbol{A}$ in (3.10) has positive real part, and that the pair $(\boldsymbol{A}, \boldsymbol{B})$ is stabilizable. Hence, as stated in [Sontag and Sussmann, 1990, Saberi et al., 2012], for any fixed value of $\rho > 0$, there always exists a control law which makes the equilibrium points $(\boldsymbol{\xi} = \boldsymbol{0}, \boldsymbol{v} = \boldsymbol{0})$ globally asymptotically stable. Thus, with a proper choice of α, one can enlarge the invariant set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ defined in (3.35) arbitrarily just with the nominal controller satisfying:

$$\boldsymbol{v} = -\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi} \in \mathcal{B}(\rho) \forall \boldsymbol{\xi} \in \mathcal{E}(\boldsymbol{P}, \varepsilon). \tag{3.51}$$

However, having a large domain of attraction along with the linear controller (3.51) produces very poor local performance, namely, close to the steady state. To tackle such drawback, we scaled up the nominal control (3.51) with $\gamma > 1$ and introduced the saturation function proposed in (3.33) to maintain continuity, invariance and constraint satisfaction.

Remark 3.8. It is also noteworthy that the saturation function $\text{sat}_\lambda(\boldsymbol{v})$ as in (3.33a) is Lipschitz continuous due to the convexity of $\mathcal{V}_c$ given in (3.14). This property allows one to practically implement the controller without any chattering, singularities or discontinuous actuation. Moreover, input saturation in the literature is typically considered as a nonlinear component, which complicates the stability analysis and causes the system to behave unexpectedly or undesirably [Hu et al., 2002a, Hu and Lin, 2001]. Yet, in this paper, the novel saturation function for the quadcopter proposed in (3.33) helps to secure the constraint satisfaction. Even when the input $\boldsymbol{v}$ is saturated on the boundary of the set $\mathcal{V}_c$ as in (3.14), the stability is still maintained thanks to the controller characterization proposed previously.

Up to this point, a control strategy ensuring both exponential stability and constraint satisfaction has been presented. In the next subsection, we illustrate the synthesis procedure and provide some insights concerning the effect of different parameter values.

3.3.3.2 Invariant set characterization

As stated above, the proposed design process will be carried out to illustrate its feasibility and to analyze the parameters' effect. With the first step, instead of empirically choosing a small ball $\mathcal{B}(\rho)$ as in (3.31) inscribed in $\mathcal{V}_c$, one candidate can be found as follows.

Proposition 3.2. *The largest ball, of the form* (3.31) *and inscribed in* $\mathcal{V}_c$ *as in* (3.14)*, can be computed by solving the following semi-definite programming problem:*

$$\rho^* = \arg\max_{\rho} \rho \tag{3.52}$$

$$s.t \begin{cases} \begin{bmatrix} \boldsymbol{I}_3 & \boldsymbol{0}_{3\times 1} \\ \boldsymbol{0}_{1\times 3} & -\rho \end{bmatrix} - \tau_1 \begin{bmatrix} \boldsymbol{I}_3 & \bar{\boldsymbol{g}} \\ \bar{\boldsymbol{g}}^\top & \bar{\boldsymbol{g}}^\top \bar{\boldsymbol{g}} - T_{max}^2 \end{bmatrix} \succeq 0 & \text{(3.53a)} \\ \begin{bmatrix} \boldsymbol{I}_3 & \boldsymbol{0}_{3\times 1} \\ \boldsymbol{0}_{1\times 3} & -\rho \end{bmatrix} - \tau_2 \begin{bmatrix} \boldsymbol{H} & \boldsymbol{H}^\top \bar{\boldsymbol{g}} \\ \bar{\boldsymbol{g}}^\top \boldsymbol{H} & \bar{\boldsymbol{g}}^\top \boldsymbol{H} \bar{\boldsymbol{g}} \end{bmatrix} \succeq 0 & \text{(3.53b)} \\ \begin{bmatrix} \boldsymbol{I}_3 & \boldsymbol{0}_{3\times 1} \\ \boldsymbol{0}_{1\times 3} & -\rho \end{bmatrix} - \tau_3 \begin{bmatrix} \boldsymbol{0}_3 & -\boldsymbol{\mu} \\ -\boldsymbol{\mu}^\top & -g \end{bmatrix} \succeq 0, \tau_i \geq 0, i \in \{1,2,3\} & \text{(3.53c)} \end{cases}$$

with $\boldsymbol{H} = \text{diag}(1, 1, -\tan\epsilon_{max}), \bar{\boldsymbol{g}} = [0, 0, g]^\top$ *and* $\boldsymbol{\mu} = [0, 0, 1/2]^\top$.

Proof. Rewriting the formulation of $\mathcal{B}(\rho)$ in (3.31), we need the following condition to hold:

$$\text{if } \boldsymbol{v} \in \mathcal{B}(\rho) \Rightarrow \boldsymbol{v} \in \mathcal{V}_c \tag{3.54}$$

or explicitly:

$$\text{if } \|\boldsymbol{v}\|_2^2 \leq \rho \Rightarrow \begin{cases} (\boldsymbol{v}+\bar{\boldsymbol{g}})^\top (\boldsymbol{v}+\bar{\boldsymbol{g}}) \leq T_{max}^2 \\ (\boldsymbol{v}+\bar{\boldsymbol{g}})^\top \boldsymbol{H} (\boldsymbol{v}+\bar{\boldsymbol{g}}) \leq 0 \\ 2\boldsymbol{\mu}^\top \boldsymbol{v} + g \geq 0. \end{cases} \tag{3.55}$$

Then, by applying the $\mathcal{S}$-procedure [Boyd et al., 1994], we rewrite (3.55) as in (3.53), hence, completing the proof. □

An illustration for the optimization problem (3.52) under the condition (3.53) is provided in Figure 3.10 and Table 3.3.

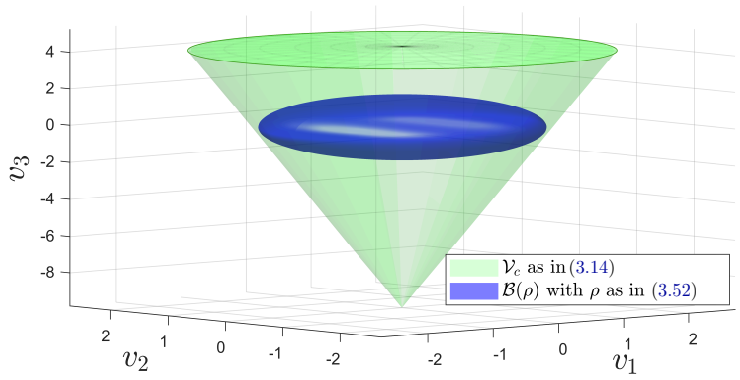


FIGURE 3.10: Numerical result for the solution of (3.52) - a ball maximally inscribed in $\mathcal{V}_c$.

Next, to ensure closed-loop stability, it is required to compute a stabilizing matrix $\boldsymbol{P}$ satisfying condition (2.66). By setting $\boldsymbol{Q} = \boldsymbol{P}^{-1}$, pre and post multiplying the result with $\boldsymbol{Q}$, the condition (2.66) yields:

$$\boldsymbol{Q}\boldsymbol{A}^\top + \boldsymbol{A}\boldsymbol{Q} - 2\boldsymbol{B}\boldsymbol{B}^\top \preceq -\alpha \boldsymbol{Q}; \;\; \boldsymbol{Q} \succ 0, \tag{3.56}$$

which is, again, an LMI. Then finally, with the solution of (3.56), the invariant set as in (3.35) is characterized by $\boldsymbol{P} = \boldsymbol{Q}^{-1}$ and ε found in (3.36), finalizing the synthesis procedure.

For the sake of illustration, the simulations are depicted with different values for $\alpha > 0$ (see Figure 3.11). The numerical results were computed with Yalmip Toolbox [Lofberg, 2004] together with MATLAB 2021b. The specifications for the simulation are provided in Table 3.3.

As expected, the invariant set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ can be enlarged by tuning down the value of $\alpha > 0$. The downside for this expansion is the reduction of the convergence speed, which can be interpreted from the differential inequalities (3.44) and (3.48). Therefore, a crucial point

TABLE 3.3: Parameters for the invariant set construction

Symbols	Values
$T_{max} = 1.45g$	14.2245 m/s^2
$\theta_{max}, \phi_{max}, \epsilon_{max}$ as in (3.14)	0.1745 rad (10^o)
ρ in (3.31) computed with (3.52)–(3.53)	2.9019

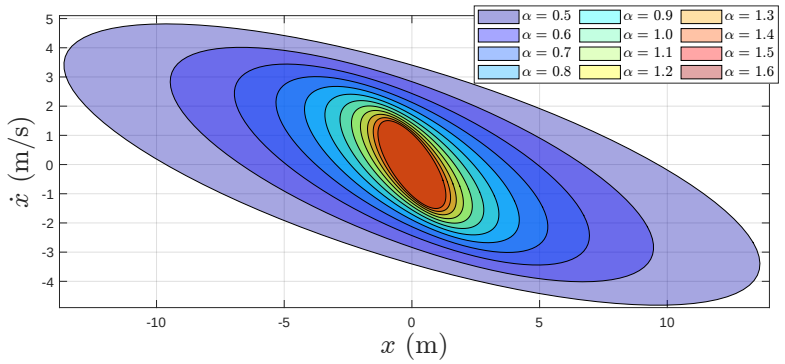


FIGURE 3.11: The projection of $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ into the subspace $(x, \dot{x})$ with different α.

of concern to develop in the future studies is how to best trade the closed-loop performance for the extent of the region of attraction. Besides, computationally, the costliest step of the control implementation is the evaluation of $\lambda^*(\boldsymbol{v})$ as in (3.33b) when the input is saturated. One straightforward solution is to employ a reliable optimization solver during the flight to iteratively solve for the result. However, taking advantage of the representation of $\mathcal{V}_c$, an explicit solution can be derived for the controller implementation as shown subsequently.

3.3.3.3 Implementation algorithm

In view of the explicit computation of the proposed saturation function, recall its definition $\text{sat}_\lambda(\boldsymbol{v})$ from (3.33):

$$\text{sat}_\lambda(\boldsymbol{v}) = \begin{cases} \boldsymbol{v} \text{ if } \boldsymbol{v} \in \mathcal{V}_c \\ \lambda^*(\boldsymbol{v})\boldsymbol{v} \text{ if } \boldsymbol{v} \notin \mathcal{V}_c. \end{cases} \tag{3.57a}$$

$$\text{where } \lambda^*(\boldsymbol{v}) = \underset{\lambda \boldsymbol{v} \in \mathcal{V}_c}{\arg\max}\ (\lambda). \tag{3.57b}$$

Since (3.57b) is a convex optimization problem, the value of $\lambda^*(\boldsymbol{v})$ can be derived directly from the candidate minimizers contained in $\mathcal{M}(\boldsymbol{v})$ (found from the Karush–Kuhn–Tucker

necessary conditions [Boyd et al., 2004]) as the following:

$$\mathcal{M}(\boldsymbol{v}) = \left\{ \frac{-g}{v_3}, \frac{a_0}{v_3}, \frac{-b_1 \pm \sqrt{b_1^2 - 4a_1c_1}}{2a_1}, \frac{-b_2 \pm \sqrt{b_2^2 - 4a_2c_2}}{2a_2} \right\} \tag{3.58}$$

$$\text{with} \begin{cases} a_0 &= T_{max}\cos\epsilon_{max} - g,\ a_1 = v_1^2 + v_2^2 - v_3^2\tan^2(\epsilon_{max}), \\ b_1 &= -2\tan^2(\epsilon_{max})v_3 g,\ c_1 = -\tan^2(\epsilon_{max})g^2, \\ a_2 &= v_1^2 + v_2^2 + v_3^2,\ b_2 = 2v_3 g,\ c_2 = g^2 - T_{max}^2. \end{cases}$$

With this finite set of only 6 components, the computation of $\lambda^*(\boldsymbol{v})$ is reduced to a simple search for a maximum real number:

$$\lambda^*(\boldsymbol{v}) = \max \mathcal{M}(\boldsymbol{v}) \cap (0;1) \tag{3.59}$$

Note that $\mathcal{M}(\boldsymbol{v})$ as in (3.58) may contain complex numbers that have to be disregarded, since both inputs $\boldsymbol{v}$ and $\boldsymbol{u}$ are necessarily real. Then, (3.59) can be implemented by a low-cost search for a maximum value inside $\mathcal{M}(\boldsymbol{v}) \cap (0;1)$ (e.g, a bubble sort). Briefly, the function (3.57a) (or (3.33a)) can be embedded with the following algorithm:

Algorithm 3.2: Compute the function $\text{sat}_\lambda(\boldsymbol{v})$ as in (3.33).

Input: Variable $\boldsymbol{v}$, and constants $T_{max}, \epsilon_{max}, g$.
Output: The result of $\text{sat}_\lambda(\boldsymbol{v})$ as in (3.33)

```
if v ∉ V_c then
    Enumerate 6 components of M(v) as in (3.58);
    λ*(v) ← max{M(v) ∩ (0;1)};
    v ← λ*(v)v;
return v;
```

In this manner, there is no need for a numerical solver to stabilize system (3.10) with the nonlinear constraint set $\mathcal{V}_c$ in (3.14).

The above explicit formulation of the saturation specified for $\mathcal{V}_c$ in (3.14) also concludes this section of control design. So far, three control approaches have been synthesized including implicit MPC, explicit MPC and saturated control in Section 3.3.1, 3.3.2 and 3.3.3, respectively. In the next section, we carry out both simulation study and experimental tests to validate and evaluate the efficiency of the proposed controllers.

3.4 Simulations and experimental validation

This section presented the simulations and experimental validations for the control strategies proposed previously.

3.4.1 Implicit model predictive control

To validate the inner approximation proposed in Section 3.2.2.2 and highlight the advantage of controlling the drone in the flat output space, let us consider the two following MPC settings:

Flatness-based MPC (FB-MPC): First, we employ the implicit MPC proposed in (3.24) with the approximated polytope $\mathcal{S}_v$ in (3.18);

Piece-Wise Affine MPC (PWA-MPC): Then, for comparison, this method is adopted from [Prodan et al., 2015a] with the linearization of the model via Taylor series and the linear MPC setup. Details on the implementation are given in the Appendix A.2.

3.4.1.1 Scenarios of trajectories

To identify benefits and drawbacks of the two methods, we provide some scenarios of trajectories. In those trajectories, the flat output $\mathbf{p}$ in (3.10) will be parameterized in time, which leads to the complete nominal reference of the system thanks to the representation (3.6). The trajectories are described as:

• Ref. 1: The trajectory generation is adopted from Section 2.3, with the minimal length optimal B-spline parameterized curve (order 8) passing the way-points w_k at $t_k\,(s)$:

$$w_k \in \left\{ \begin{matrix} (0;0;3.5);(3;-3;4);(6;0;7.5);(6;3;8);(3;6;8); \\ (0;6;8);(-3;3;8);(-3;0;5);(0;0;3.5) \end{matrix} \right\} \times 10\ (cm)$$
$$t_k = (k-1) \times 30/8\,(s), k \in \{1,2,...,9\}.$$

With this method, the curve's parameters are chosen so that all the states and inputs respect their constraints, giving a favorable reference to them to follow.

• Ref. 2: To make the reference more aggressive, set points are given under sequences of step functions.

• Ref. 3: Next, we adopt the circular trajectory in [Greeff and Schoellig, 2018] as:

$$\mathrm{p}_1^{\mathrm{ref}}(t) = 0.5\cos\omega t, \mathrm{p}_2^{\mathrm{ref}}(t) = 0.5\sin\omega t$$
$$\mathrm{p}_3^{\mathrm{ref}}(t) = 0.3(m), \omega = 0.3\pi.$$

• Ref. 4: Finally, we adapt the arbitrary sinusoidal trajectory given in [Garcia et al., 2017], describing as:

$$\mathrm{p}_1^{\mathrm{ref}}(t) = 0.5\cos\omega t, \mathrm{p}_2^{\mathrm{ref}}(t) = 0.5\sin\omega t$$
$$\mathrm{p}_3^{\mathrm{ref}}(t) = 0.5\sin 0.5\omega t + 0.5(m), \omega = \pi/15.$$

Illustration of the reference trajectories are given in Fig. 3.12.

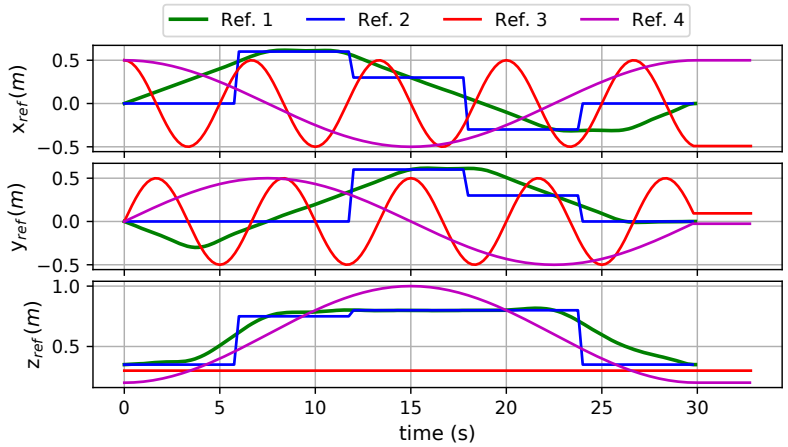


FIGURE 3.12: Four proposed time-parameterized references.

3.4.1.2 Experimental results

The experiments are conducted in the setting described in Section 3.2.3. During the experiment, the desired ψ angle was set to 0. The experiments' parameters are listed in Table 3.4 while the video is available at: https://youtu.be/1a1K6R6__3s.

Computationally, the sampling times were chosen according to the execution time with different trajectories. With the feasible Ref. 1, the input references $\boldsymbol{v}^{\text{ref}}(k), \boldsymbol{u}^{\text{ref}}(k)$ can be nominally defined for the system's dynamics, hence speeding up the search for optimal solutions in both methods. Consequently, the sampling time can be chosen only as $t_s = 0.1s$, while, the remaining three trajectories demand much higher time for the initial search, resulting in larger sampling time ($\geq 0.2s$, see Fig. 3.14).

TABLE 3.4: Control parameters for the proposed scenarios

		$\boldsymbol{Q}$	$\boldsymbol{R}$	$t_s(s)$	N_p
Ref.1	FB-MPC	$\text{diag}(35\boldsymbol{I}_2, 50, 5\boldsymbol{I}_3)$	$\boldsymbol{I}_3$	0.1	20
	PWA-MPC	$\text{diag}(35\boldsymbol{I}_2, 50, 5\boldsymbol{I}_3)$	$\text{diag}(5, 75\boldsymbol{I}_2)$		
Ref.2	FB-MPC	$\text{diag}(50\boldsymbol{I}_3, 5\boldsymbol{I}_3)$	$5\boldsymbol{I}_3$	0.25	20
	PWA-MPC	$\text{diag}(50\boldsymbol{I}_3, 5\boldsymbol{I}_3)$	$\text{diag}(5, 75\boldsymbol{I}_2)$		
Ref.3	FB-MPC	$\text{diag}(180\boldsymbol{I}_3, 10\boldsymbol{I}_3)$	$5\boldsymbol{I}_3$	0.2	10
	PWA-MPC	$\text{diag}(50\boldsymbol{I}_3, 5\boldsymbol{I}_3)$	$\text{diag}(5, 80\boldsymbol{I}_2)$		
Ref.4	FB-MPC	$\text{diag}(90\boldsymbol{I}_3, 5\boldsymbol{I}_3)$	$5\boldsymbol{I}_3$	0.25	20
	PWA-MPC	$\text{diag}(35\boldsymbol{I}_2, 50, 5\boldsymbol{I}_3)$	$\text{diag}(5, 75\boldsymbol{I}_2)$		

In terms of performance, Fig. 3.13 and 3.15 show the root-mean-square (RMS) and the tracking errors of the two controllers, respectively, in the four references. Expectedly, although requiring more computation time, the FB-MPC can be considered better while being put next to the well-known PWA-MPC with centimeters of tracking error.

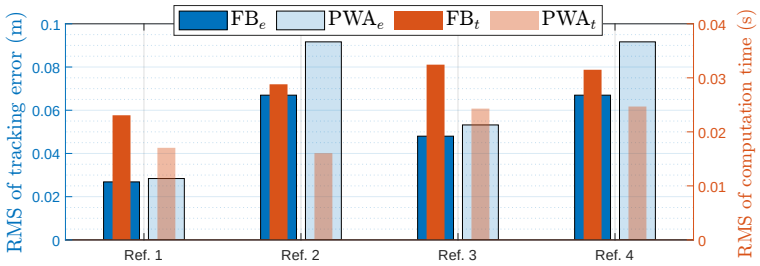


FIGURE 3.13: RMS of tracking errors and computation time of the two controllers with different types of references (distinguished, respectively, by the subscript e and t under the name of the corresponding controller).

In details, one shortcoming of FB-MPC is that it demands slightly more execution time than the PWA-MPC in practice. This can be explained by showing the complexity of the optimization problems. Particularly, both the PWA-MPC and FB-MPC as in (3.24) are QP problems. However, the constraint set $\mathcal{S}_v$ in (3.18) is geometrically complex, compared to the box $\mathcal{U}$ in (3.5) with more vertices or inequalities. The effect can also be seen in Fig. 3.13 with a roughly constant gap in computation time necessary for the two methods.

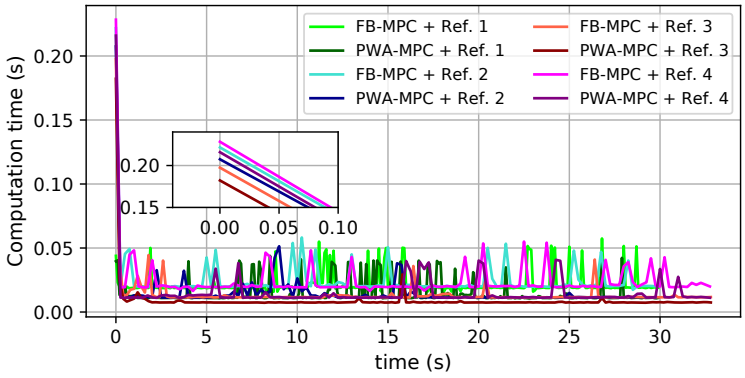


FIGURE 3.14: Computation time used for different references.

Yet, since PWA-MPC depends on the approximation of the model along the trajectory, its efficiency is reliant on the reference's quality, hence making the method more vulnerable to uncertainty than our proposed FB-MPC. Indeed, while with Ref. 1, both controllers achieve fairly good tracking (See Fig. 3.13), with Ref. 2, large oscillations in tracking error are observed with PWA-MPC compared to that of the FB-MPC (see Fig. 3.15). Moreover, despite being constructed via an approximated input constraint,

the FB-MPC always shows an equivalently reliable performance without saturating the input, in comparison with the PWA-MPC, as in Fig. 3.16.

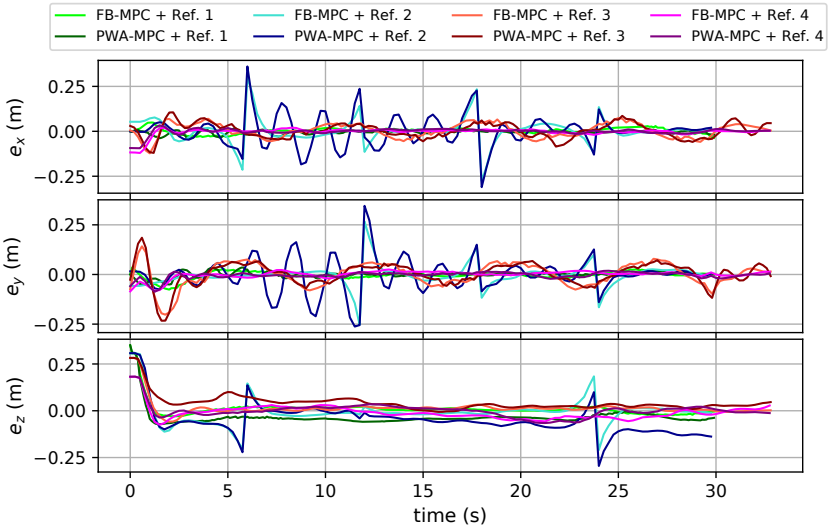


FIGURE 3.15: Crazyflie tracking error with flatness-based MPC and PWA-MPC (4 references) with $e_q \triangleq q^{\text{ref}} - q,\ q \in \{x, y, z\}$.

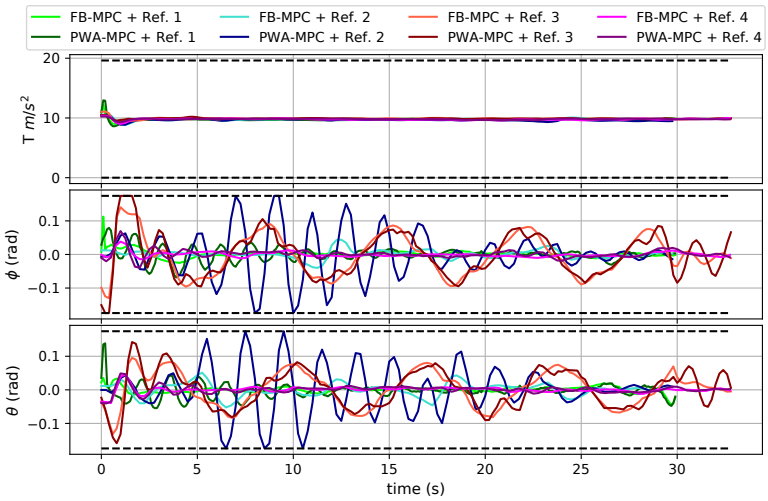


FIGURE 3.16: Input signals with their constraints (black dashed line).

Furthermore, due to the fact that there is no approximation in our model, the performance of the proposed FB-MPC surpasses its approximation-based contestant in [Garcia et al., 2017] with Ref. 3. Finally, although being constructed in the similar framework of flatness-based MPC, with Ref. 4, our improved performance is apparent thanks to the less conservative constraint set $\mathcal{S}_v$ as opposed to the box-type set in [Greeff and Schoellig, 2018].

With the successful validation of the MPC program (3.24), we wish to advance to the explicit MPC validation, confirming the real-time complexity reduction, stability and constraint satisfaction guarantees.

3.4.2 Explicit model predictive control

The content of this part is twofold. First, we analyze the compromise of the box-type approximation over the computational advantage of the ExMPC. Second, we validate the proposed ExMPC in comparison with the standard stabilizing IMPC. The experiment video can be found at https://youtu.be/u7PsNDheIR4.

3.4.2.1 Simulation study

Regarding the explicit MPC formulation, with the subset $\mathcal{B}_v$ as in (3.26), we have sidestepped the problem of dimensionality brought about by the 6D description of the model (3.10). To demonstrate such a computational trade-off and to better select the parameters for the proposed scheme, let us proceed by carrying out a simulation study for both approaches as the following scenarios:

- *Scenario 1* (Sce. 1): First, we employ the proposed MPC law generated from (3.27). The weighting $\boldsymbol{P}$ is computed from the algebraic Riccati equation. The terminal constraint set $\mathcal{X}_{f,i}$ in (3.27) is constructed with the polytopic maximal positive invariant (MPI) set with a local controller $v_i^{\text{loc}}(\mathbf{p}_i(k)) \triangleq \boldsymbol{K}\mathbf{p}_i(k)$ chosen from the LQR for $(\boldsymbol{A}_{2D}^d, \boldsymbol{B}_{2D}^d, \boldsymbol{Q}, \boldsymbol{R})$, ensuring the stability and feasibility of system (3.25) under the control (3.29) [Mayne et al., 2000].

- *Scenario 2* (Sce. 2): Herein, we concatenate[1] the dynamics (3.25) and formulate the following MPC:

$$\underset{\boldsymbol{v}(k|k+j)}{\arg\min} \sum_{j=0}^{N_p-1} \|\boldsymbol{\zeta}(k|k+j)\|_{\underline{\boldsymbol{Q}}}^2 + \|\boldsymbol{v}(k|k+j)\|_{\underline{\boldsymbol{R}}}^2 + \|\boldsymbol{v}(k|k+N_p)\|_{\underline{\boldsymbol{P}}}^2, \tag{3.60}$$

$$\text{s.t} \begin{cases} \boldsymbol{\zeta}(k|k+j+1) = \boldsymbol{A}_{6D}^d \boldsymbol{\zeta}(k|k+j) + \underline{\boldsymbol{B}}\boldsymbol{v}(k|k+j), \\ \boldsymbol{v}(k|k+j) \in \mathcal{V}_c^{\text{in}} \text{ as in (3.22)}, \boldsymbol{\zeta}(k|k+j) \in \underline{\mathcal{X}}, \boldsymbol{\zeta}_{k+N_p} \in \underline{\mathcal{X}}_f, \end{cases}$$

where $\boldsymbol{\zeta}_k \triangleq [\mathbf{p}_{1,k}^\top \mathbf{p}_{2,k}^\top \mathbf{p}_{3,k}^\top]^\top \in \mathbb{R}^6$ collects the position and velocity of the drone; $\boldsymbol{A}_{6D}^d = \text{diag}(\boldsymbol{A}_{2D}^d, \boldsymbol{A}_{2D}^d, \boldsymbol{A}_{2D}^d)$; $\underline{\boldsymbol{B}} = \text{diag}(\boldsymbol{B}_{2D}^d, \boldsymbol{B}_{2D}^d, \boldsymbol{B}_{2D}^d)$, $\boldsymbol{A}_{2D}^d, \boldsymbol{B}_{2D}^d$ are from (3.25); $\underline{\mathcal{X}} = \{\boldsymbol{\zeta}_k : \mathbf{p}_i(k) \in \mathcal{X}_i, i \in \{1,2,3\}\}$ is the same state constraints chosen as in (3.27b). Similarly, the matrices $\underline{\boldsymbol{Q}} \triangleq \text{diag}(\boldsymbol{Q}, \boldsymbol{Q}, \boldsymbol{Q})$ and $\underline{\boldsymbol{R}} \triangleq \text{diag}(\boldsymbol{R}, \boldsymbol{R}, \boldsymbol{R})$

[1] The concatenation is simply a variable rearrangement of the system (3.23).

are chosen from choice of weighting for the problem (3.27). Lastly, the terminal ingredients $\underline{\boldsymbol{P}}, \underline{\mathcal{X}}_f$ are computed similarly as in Sce. 1. In other words, for the sake of comparison, the cost function, the state constraints and the terminal ingredients (the local controller, the quadratic terminal cost) are identically chosen with respect to Scenario 1. The explicit MPC law is then generated in the same fashion for the optimization problem (3.60). The main difference in this control synthesis is that the non-conservative approximation $\mathcal{V}_c^{\text{in}}$ as in (3.22) is employed, as opposed to $\mathcal{B}_v$ as in (3.26) for Sce. 1.

- *Scenario 3* (Sce. 3): In this case, the implicit MPC law (3.60) is implemented by solving the problem online with IPOPT solver [Wächter and Biegler, 2006] in CasAdi interface [Andersson et al., 2019].

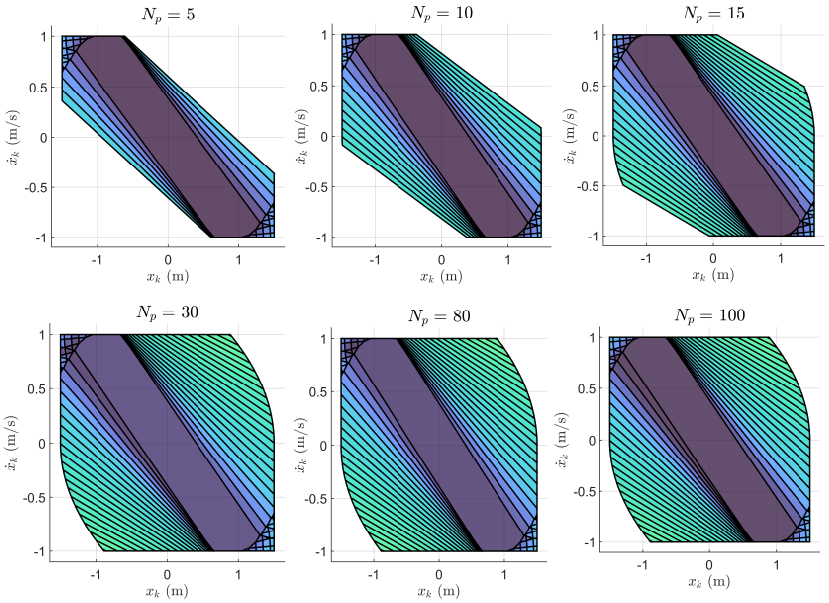


FIGURE 3.17: Explicit MPC solution with different choices of N_p.

The simulations are conducted for an interval of 60 seconds. The critical regions are calculated with the MPT3 toolbox on Matlab 2021b [Herceg et al., 2013] and stored as a `.mat` file data. The initial position for both scenarios are: $\mathbf{p}_1(0) = [1.25, -0.8]^\top$, $\mathbf{p}_2(0) = [0, 0.2]^\top$, $\mathbf{p}_3(0) = [0.5, 0.2]^\top$ while the tuning adopts the values: $\boldsymbol{Q} = \text{diag}(50, 5), \boldsymbol{R} = 10$, resulting in the corresponding value of $\boldsymbol{P} = \begin{bmatrix} 524.37 & 223.75 \\ 223.75 & 225.97 \end{bmatrix}$, $\boldsymbol{K} = -[2.0003, 2.0978]$. The parameter setup is given in Table 3.5.

Discussion: Figure 3.17 presents how the critical regions expand and cover the state space with respect to different choices of the prediction horizon N_p. It is noticeable that, for a fixed choice of state constraints $\mathcal{X}_i$, after a certain value of N_p, the number of

TABLE 3.5: Parameters for simulations and experimental tests

Parameters	Values
$T_{max}; \epsilon_{max}$ in (3.22)	$1.45g$ m/s^2; 0.1745 (rad)
Sampling time t_s	100 ms
$\bar{v}_1, \bar{v}_2, \bar{v}_3$ as in (3.26)	0.8154,0.8154,3.27
$\bar{\mathrm{p}}_1, \bar{\mathrm{p}}_2, \bar{\mathrm{p}}_3$ for $\mathcal{X}_i$ in (3.27b)	1.5,1.5,1.5 (m)
$\bar{\dot{\mathrm{p}}}_1, \bar{\dot{\mathrm{p}}}_2, \bar{\dot{\mathrm{p}}}_3$ for $\mathcal{X}_i$ in (3.27b)	1,1,1.5 (m/s)

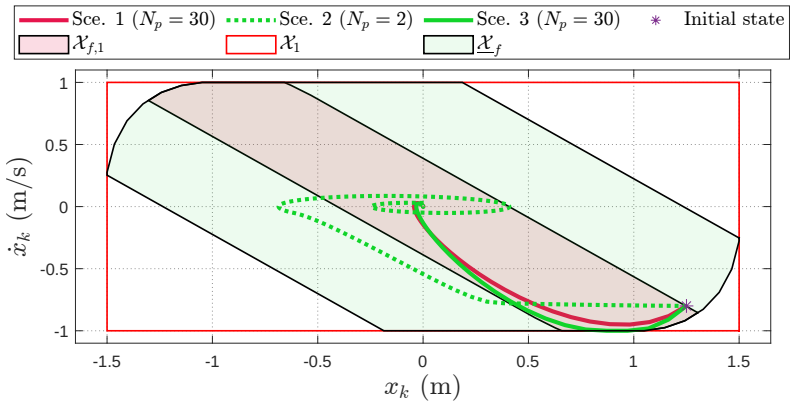


FIGURE 3.18: Comparison between the ExMPC for the proposed setup (Sce. 1), for the 6D model (Sce. 2) and the IMPC with the 6D model (Sce. 3).

critical regions converges to a fixed value. This can also be seen in the result in Table 3.6 for Scenario 1. After $N_p > 30$, the number of regions in each of the three axes remains constant. Besides, even reaching the limit value, the computational requirement for data storage is relatively small. Meanwhile, with just 2 steps of prediction, the 6D explicit MPC solution in Scenario 2 accounts for a memory space that is 234 times larger than that of a 100-N_p in Scenario 1. Although regarded as an offline effort, the enumeration of all these regions required roughly 24 hours, in comparison with less than 3 minutes for the entire Scenario 1. Moreover, with an effortful online search (sequentially checking all the regions), the computational burden of the 6D explicit MPC is exposed via the runtime required (more than 13 times the sampling time t_s). This performance can be considered impractical for real-time implementation, highlighting the computational advantage of (3.27) over (3.60). Note that during the implementation, the sequential search among the critical regions was not turned off while the system enters the invariant terminal region. Therefore, the computational strength of the ExMPC scheme necessarily comes from the proposed decoupling settings.

Furthermore, regarding the closed-loop performance for the same chosen parameters, it is certain that the Implicit MPC (IMPC) in Scenario 3 will outperform the ExMPC in Scenario 1 due to its less conservative input power (between $\mathcal{B}_v$ in (3.26) and $\mathcal{V}_c^{\text{in}}$ in (3.22), respectively). This advantage is not only shown by smaller root-mean-square

TABLE 3.6: Simulation results and numerical specifications

	Number of regions	Size (MB)	N_p	Avg. CPU time	RMS errors (cm)
Sce.1	(99,99,11)	0.02	5	0.1563 ms	18.906
	(103,103,11)	0.11	30	0.3385 ms	18.906
	(103,103,11)	0.33	80	0.2864 ms	18.906
	(103,103,11)	0.41	100	0.1980 ms	18.906
Sce.2	49897	96.0	2	1318.5 ms	109.5
Sce.3	-	-	5	4.72 ms	11.46
	-	-	30	11.63 ms	11.319
	-	-	80	27.05 ms	11.319
	-	-	100	33.08 ms	11.319

(RMS) errors[2], but also indicated via the larger terminal invariant set (see Figure 3.18). With the interpretation as a safe region ensuring constraint satisfaction and asymptotic stability, the terminal region $\underline{\mathcal{X}}_f$ for Scenario 3 (as well as Scenario 2) encloses a larger domain in the state space in comparison with Scenario 1. This indicates that, for a fixed choice of initial condition, both Scenario 2 and Scenario 3 will require a smaller size of prediction horizon to guarantee the system's origin convergence and recursive feasibility. However, despite this advantage, the more efficient offline computation and implementation of Scenario 2 still remain open for further investigation from both combinatorial and geometric standpoints [Mihai et al., 2022, Xiu and Zhang, 2020]. It is important also to highlight that the controllers for both Scenario 1 and 2 exhibit a symmetric description, for example, in the constraint sets ($\mathcal{B}_v$ in (3.26) or $\mathcal{V}_c^{\text{in}}$ in (3.22)). This observation aligns with the theoretical framework presented by Danielson and Borrelli [2014], suggesting a potential complexity reduction in ExMPC representation and hence, real-time implementation. Lastly, this conservativeness of Scenario 1 exposes another limitation concerning its feasibility. More specifically, since the size of the terminal set for a fixed horizon directly affects the domain of attraction [Brunner et al., 2015], it is certain that the feasible domain will be significantly limited when the set $\mathcal{B}_v$ in (3.26) is employed as apposed to $\mathcal{V}_c^{\text{in}}$. To demonstrate this drawback, the state constraints in Scenario 1 and 2 were disregarded and static feasible initial conditions for both cases were collected by checking the position space on a 20×20 grid sampled on $x, y, z \in [-20, 20]\,(m)$. The simulation result is depicted in Figure 3.19. As expected, for a fixed horizon of $N_p = 30$, Sce. 1 shows considerably less feasible initial conditions, while the polytopic approximation $\mathcal{V}_c^{\text{in}}$ in Sce. 2 enjoys more control power and, hence, a larger set of feasible states.

Remark 3.9. With the intention of adapt the stabilization problem (3.27) for trajectory tracking problem in the experiment, it is assumed that the reference trajectory satisfies:

[2] with the position $\mathbf{p}(k) = [\mathrm{p}_1(k), \mathrm{p}_2(k), \mathrm{p}_3(k)]^\top$ as in (3.4) and the desired reference $\mathbf{p}^{\text{ref}}(k) = [\mathrm{p}_1^{\text{ref}}(k), \mathrm{p}_2^{\text{ref}(k)}, \mathrm{p}_3^{\text{ref}}(k)]^\top$, the RMS error was computed as: RMS error $= \frac{1}{3}\sum_{i=1}^{3}\sqrt{N_{sim}^{-1}\sum_{k=0}^{N_{sim}}(\mathrm{p}_i(k)^{\text{ref}} - \mathrm{p}_i(k))^2}$ with N_{sim} denoting the final step of the test.

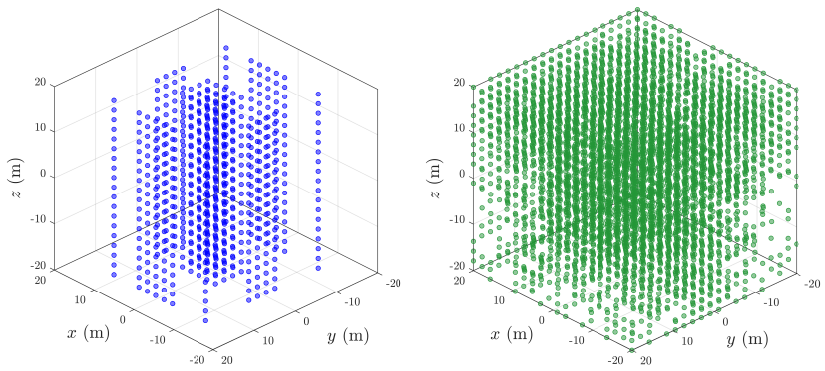


FIGURE 3.19: Feasible static initial conditions of ExMPC (left - Sce. 1) and IMPC (right - Sce. 2) without state constraints.

1. $\mathbf{p}_i^{\text{ref}}(k+1) = \boldsymbol{A}_{2D}^d \mathbf{p}_i^{\text{ref}}(k) + \boldsymbol{B}_{2D}^d v_i^{\text{ref}}(k)$ and $\boldsymbol{v}_k^{\text{ref}} \in \mathcal{V}_c^{\text{ref}}$, where $\mathbf{p}_i(k)^{\text{ref}}, \boldsymbol{v}^{\text{ref}}(k)$ denote the reference for the state $\mathbf{p}_i(k)$ and the input $\boldsymbol{v}(k)$, respectively. $\mathcal{V}_c^{\text{ref}}$ is a time-invariant set enclosing the reference signal $\boldsymbol{v}_k^{\text{ref}}$.

2. The polyhedral set $\Delta\mathcal{V}_c^{\text{in}} \triangleq \mathcal{V}_c^{\text{in}} \ominus \mathcal{V}_c^{\text{ref}}$ contains the origin as its interior point.

Then, with $\Delta\mathbf{p}_i(k) \triangleq \mathbf{p}_i(k) - \mathbf{p}_i^{\text{ref}}(k)$, the tracking error yields:

$$\Delta\mathbf{p}_i(k+1) = \boldsymbol{A}_{2D}^d \Delta\mathbf{p}_i(k) + \boldsymbol{B}_{2D}^d \Delta v_i(k), \tag{3.61}$$

with the constraint $\Delta\boldsymbol{v}(k) = \boldsymbol{v}(k) - \boldsymbol{v}(k)^{\text{ref}} \in \Delta\mathcal{V}_c^{\text{in}}$. Consequently, by finding a box inside $\Delta\mathcal{V}_c^{\text{in}}$, the dynamics (3.61) can be stabilized by the control synthesis for $\Delta\boldsymbol{v}(k)$ with the similar explicit MPC setup presented previously. Finally, the reference tracking control can be implemented as:

$$\boldsymbol{v}(k) = \Delta\boldsymbol{v}(k) + \boldsymbol{v}^{\text{ref}}(k). \tag{3.62}$$

3.4.2.2 Experimental validation

For the validation, we consider the two following tests.

- *Test 1:* With the objective of tracking the set-point at $x_f = 0.6, y_f = 0.6, z_f = 0.8$(m), the ExMPC computed in (3.30) is used. Moreover, a comparison between different choices of prediction horizon is given. A classical IMPC as in (3.60) is also tested in the same setup to highlight the simplicity of our formulation. IPOPT solver is employed with CasADi Python.

- *Test 2:* Herein, four quadcopters will be controlled sequentially with the presented method to track two groups of predefined trajectories: i) *Circular reference:* (Ref. 1) The four drones pursue a circular motion while maintaining a constant altitude of 1m; ii) *Square formation reference:* (Ref. 2) The four drones will follow a square formation trajectories while maintaining a one-meter altitude. The trajectory generation solution was adopted from minimum length B-spline parameterization framework [Prodan et al., 2019].

In the experiments, weighting matrices in Section 3.4.2.1 are readopted.

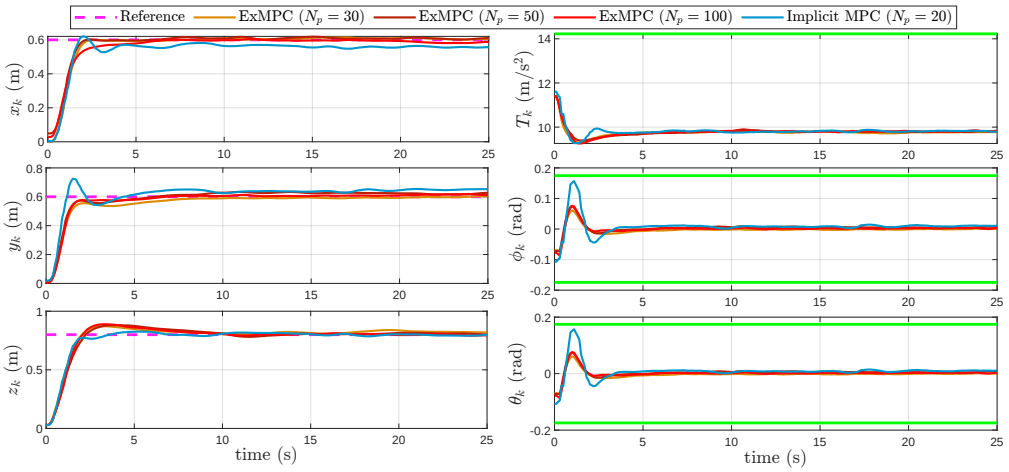


FIGURE 3.20: The quadcopter position and input $\boldsymbol{u}_k$ (with limits shown in green solid lines) in experiment with ExMPC and IMPC.

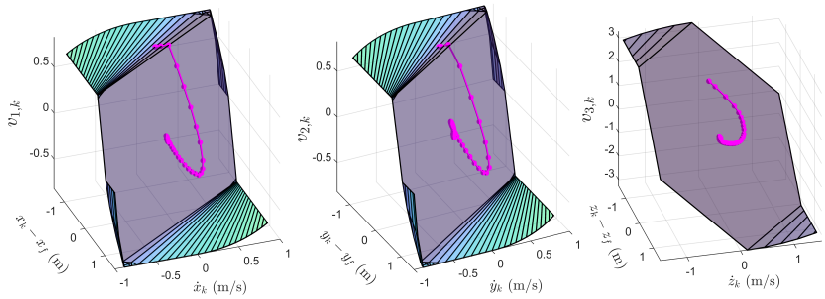


FIGURE 3.21: Experimental trajectory of $\boldsymbol{v}(k)$ in Test 1, $N_p = 30$.

3.4.2.3 Experimental results

Figure 3.20 depicts the tested results for the Crazyflie quadcopter, where the tracking objective and input constraints are both respected. The constraint satisfaction can also be shown by the containment of $\boldsymbol{v}(k)$ in $\mathcal{B}_v$ in (3.26) on Figure 3.21. Therein, it can be

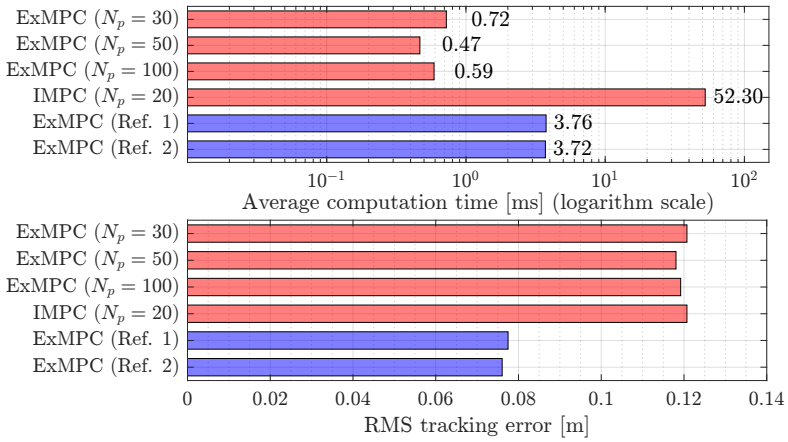


FIGURE 3.22: Computation time and RMS tracking errors with Explicit MPC and Implicit MPC in Test 1 (red) and Test 2 (blue).

seen that all the ExMPC setups provide notably less input exploitation as a consequence of the conservative approximation of $\mathcal{B}_v$ in (3.26) with respect to $\mathcal{V}_c^{\text{in}}$ in (3.22). However, this difference does not necessarily imply slower convergence or a smaller overshoot. This is because the trade-off of control power for simplicity in these schemes contrarily provides the ability to have a solution with much longer prediction horizon, resembling the infinite horizon solution. Next, the computational advantage of the proposed scheme can also be observed via the execution time for each controller in Figure 3.22. Therein, the IMPC demands virtually 80 times more of the computation time in Test 1, although the tracking performance between the two controllers appears to be commensurate. Even when there are four drones being controlled in Test 2, the computation time of the ExMPC (around 4ms) is still lower than that of the IMPC tracking with a single drone in Test 1 (52.3ms). Beside the simplicity in digital implementation, qualitative comparisons in computational burden can also be addressed with the NMPC approaches proposed with the same theoretical guarantees in the literature [Nguyen et al., 2020b, Gomaa et al., 2022] (≈10-100ms) while the ExMPC requires under 1ms. This simplicity allows only the proposed setup to be implemented with four drones in a centralized manner without exceeding the chosen sampling time.

Moreover, although there are no collision avoidance constraints imposed, the experiments show the controller's reliability to closely follow a predefined trajectory with a low computational cost, input constraint satisfaction, and stability guaranteed. This satisfactory tracking opens possible implementations in a hierarchical structure where collision-free formation is obtained via online curve parameterization or consensus-based guidance [Luis et al., 2020a] and dynamical constraints are guaranteed at the lower level. Furthermore, from an implementation viewpoint, although restrictive, the fast feasible decision, prediction and collection of active constraints from the proposed setup also show a promising direction where they are exploited in a warm-start procedure to reduce online computational effort within the IMPC settings [Zeilinger et al., 2011].

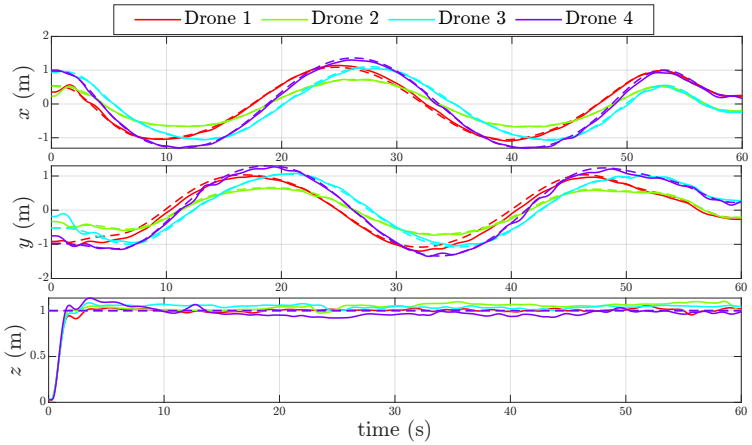


FIGURE 3.23: Quadcopters' trajectory tracking with Explicit MPC (Square formation reference, dashed and color-coded with the corresponding drone).

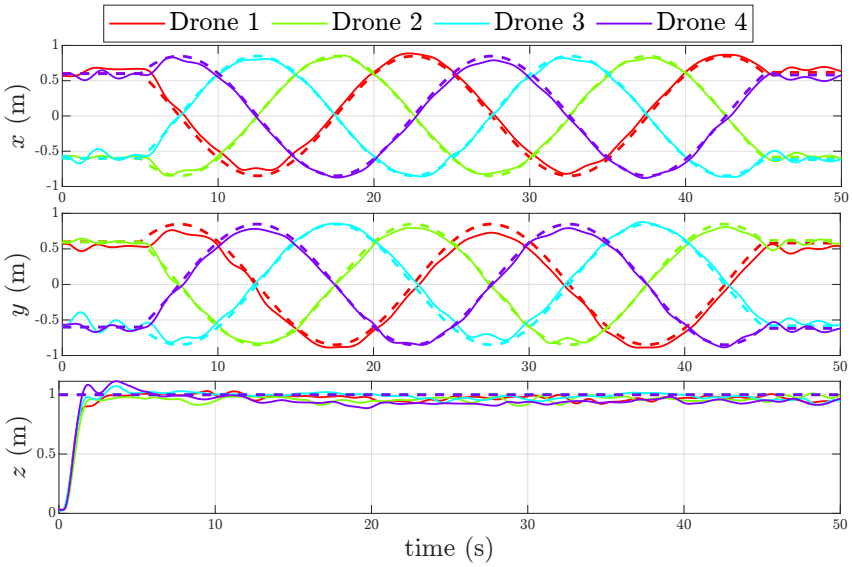


FIGURE 3.24: Multiple quadcopters' trajectory tracking with Explicit MPC (Circular reference, dashed and color-coded with the corresponding drone).

3.4.3 Explicit saturated control

The next validation is for the saturated control designed in Section 3.3.3. Through different simulation tests and comparisons, stability, constraint satisfaction, and complexity reduction will be verified. Then, experimental results show the applicability of nanodrone control. The experiment video can be found at https://youtu.be/cgXNKBvSBRM.

3.4.3.1 Simulation study

TABLE 3.7: Simulation result for $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35)

ε as in (3.35)	3.8692
$\boldsymbol{P} = \boldsymbol{Q}^{-1}$ as in (3.56)	$\begin{bmatrix} 0.2109\boldsymbol{I}_3 & 0.2813\boldsymbol{I}_3 \\ 0.2813\boldsymbol{I}_3 & 0.7500\boldsymbol{I}_3 \end{bmatrix}$

Simulation study: i) We first illustrate the invariance of $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35) regardless of the choice $\gamma \geq 1$ with the simulation of an origin-tracking problem. The trajectories' projection are given in Figure 3.25. In this simulation, the parameters in Table 3.3 were employed with $\alpha = 0.75$ to solve (3.56), resulting in an invariant ellipsoid $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ characterized by $\boldsymbol{P}$ and ε given in Table 3.7; ii) With the initial point $\boldsymbol{\xi}(0) = [-3.77, -0.46, -3.60, 0.94, -0.24, 2.31]^\top$, we validate also our Algorithm 3.2 with the numerical solution for the saturation function (3.33a) provided by the IPOPT solver [Wächter and Biegler, 2006] implemented with Yalmip MATLAB Toolbox. Furthermore, as a comparison in the computation cost, the implicit flatness-based MPC (FB-MPC) setup as in (3.24) is adopted and assigned to the controller (the blue block) in Figure 3.3:

$$\underset{\boldsymbol{v}(k|k+j)}{\arg\min} \quad \sum_{j=0}^{N_p-1} \|\boldsymbol{\xi}(k|k+j)\|^2_{\boldsymbol{Q}_{mpc}} + \|\boldsymbol{v}(k|k+j)\|^2_{\boldsymbol{R}_{mpc}} \tag{3.63}$$

$$\text{s.t: } \boldsymbol{\xi}(k|k+j+1) = \boldsymbol{A}_d\boldsymbol{\xi}(k|k+j) + \boldsymbol{B}_d\boldsymbol{v}(k|k+j), \boldsymbol{v}(k|k+j) \in \mathcal{V}_c^{\text{in}} \text{ as in (3.22)}.$$

where the state are predicted with the discretized model of (3.10) as in (3.23), and the discretization time $t_s = 0.1$(s); $\mathcal{V}_c^{\text{in}}$ is the tight inner approximation of $\mathcal{V}_c$ in (3.14); $N_p = 10$ is the prediction horizon and the weighting matrices are chosen as $\boldsymbol{Q}_{mpc} = \text{diag}(50\boldsymbol{I}_3, 5\boldsymbol{I}_3), \boldsymbol{R}_{mpc} = 5\boldsymbol{I}_3$.

Firstly, as can be seen from Figure 3.25, the control guarantees the invariance of the set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$. The same confirmation can be made in all other subspaces during the simulation. Moreover, the validity of Algorithm 3.2 can also be seen in Figure 3.26 where the new input $\boldsymbol{v}$ is always secured inside the constraint set $\mathcal{V}_c$. Finally, the effect of the choice for γ can also be distinguished from both Figure 3.27 and Figure 3.26. More specifically, when $\gamma = 1$, the controller is never saturated due to the property (3.40) as proven before. At the same time, when $\gamma > 1$, the damping effect appears to be increased

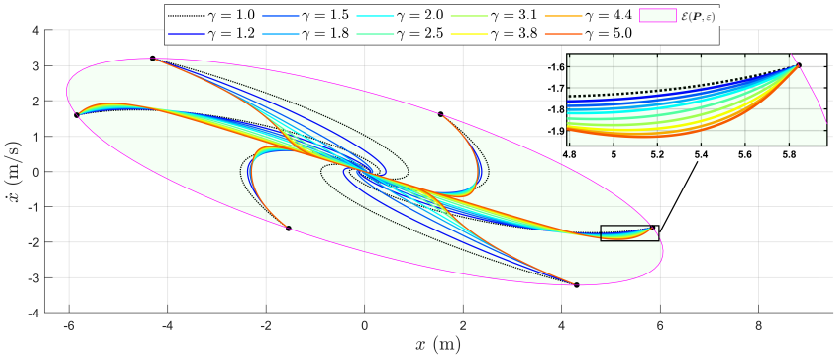


FIGURE 3.25: The quadcopter's trajectory with different initial states (black points) implemented with different values of γ (projected into $(x, \dot{x})$ subspace).

together with γ. However, as can be seen from Figure 3.27, choosing a larger value for γ does not necessarily imply a faster convergence, hence, giving us an indication not to abusively augment the value of this scalar while designing the controller.

Remark 3.10. From a practical viewpoint, the most influential factor in choosing γ relies on the well-known condition of the different frequency responses of the quadcopter's two hierarchical control loops [Nguyen et al., 2020a, Intwala and Parikh, 2015]. In detail, it is required that the vehicle's inner loop (attitude control) responses considerably faster than its outer loop (position control), to decouple the two problems and solve them hierarchically. Therefore, although the effect cannot be seen in the simulation, it is important to choose the gain γ in (3.32) appropriately to keep the convergence speed of the position control loop from exceeding the that of the attitude controller. In the next section, the undesirable behavior caused by the high gain γ will also be demonstrated via the experiments.

TABLE 3.8: CPU time of the proposed algorithm w.r.t IPOPT solver and FB-MPC approach

Control method	Average CPU time (s)	Min (s)	Max (s)	Optimization problem
FBSC (IPOPT)	0.0162	0.0147	0.0851	QCQP
FBSC (Alg. 1)	1.63E-05	4.30E-06	0.0059	max operation
FB-MPC	0.015	0.0131	0.0248	QP

Besides, it is also noticeable from Figure 3.28 and Table 3.8 that Algorithm 3.2 generates the correct trajectory as the IPOPT solver does, but the consumed computation time for our solution is significantly reduced (-98% in average CPU time) compared to that of the solver. This complexity reduction can be explained by the change from a Quadratically Constrained Quadratic Programming (QCQP) problem in (3.33) to a Linear Programming (LP) search for the maximum value described in Algorithm 3.2.

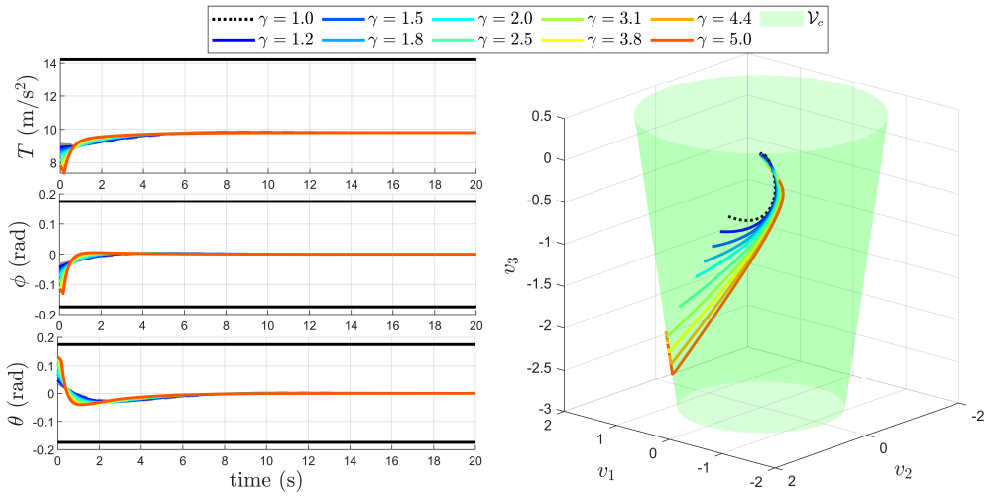


FIGURE 3.26: Input $\boldsymbol{u}$ with its constraints in solid black line (left) and $\boldsymbol{v}$ (right) simulated with different values of γ.

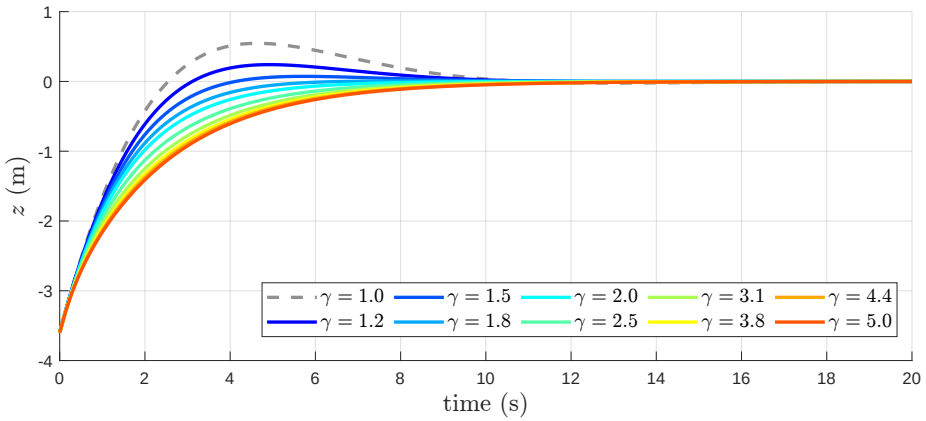


FIGURE 3.27: Simulated trajectories in time with different values of γ, projected on the z axis.

This advantage can also be highlighted when it is compared to the Quadratic Programming (QP) problem (3.63) within the MPC settings. For more details, in Table 3.8, it can be seen that compared to the MPC, the FBSC solution with our algorithm significantly reduces the average computation time from 1.5×10^{-2} (s) to 1.63×10^{-5} (s). The advantage is also shown via the structural nature of the online program. With MPC, a processor equipped with an optimization solver is commonly required to solve the quadratic program online while with our explicit solution, the control can be computed via a closed-form representation as in (45) and can be effortlessly embedded in a microcontroller. These observations, hence, confirm not only the validity of our saturation component but also the computational advantage of the proposed solution (3.58).

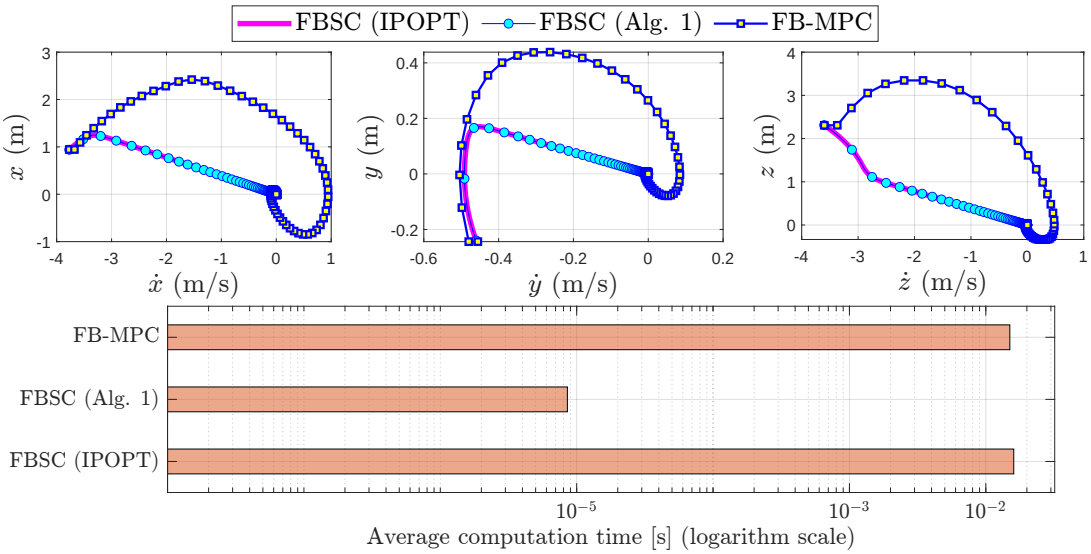


FIGURE 3.28: The quadcopter's trajectories and the computation time of Algorithm 3.2 in comparison with the IPOPT solver ($\gamma = 15$) and FB-MPC method.

Next, to illustrate the applicability of our controller and highlight its advantages, experimental tests are carried out and compared with the results of existing methods in the literature. Analysis considering the practical choice of the γ gain mentioned in Remark 3.10 will also be provided.

3.4.3.2 Experimental scenarios

Next, to examine the performance of the controller (3.32), the following tracking scenarios are introduced as follows.

- Ref. 1: In this scenario, with a fixed choice of $\boldsymbol{P} = \boldsymbol{Q}^{-1}$ satisfying (3.56), the proposed controller will be employed to track a static point $\boldsymbol{\xi}^{\text{ref}} = [0.3, 0.3, 0.8, 0, 0, 0]^\top$. We consider various values for γ to analyze the impact of such constant on the quadcopter's tracking performance in real implementation.
- Ref. 2: For comparison, we adopt the time varying reference used for validation in [Santoso et al., 2019] where a hybrid PID-fuzzy controller was used to stabilize the linearly approximated system around its equilibrium point.
- Ref. 3: We also consider the smooth trajectory generated via B-spline parameterization [Prodan et al., 2019], and tracked in an MPC setup.
- Ref. 4: Furthermore, for comparison, we select also the circular trajectory applied in [Greeff and Schoellig, 2018] where the authors applied the model predictive

control for the linear system (3.10) in the flat output space but with a conservative box-type constraints as opposed to $\mathcal{V}_c$ as in (3.14). For this last scenario, the flat output $\mathbf{p}$ is parameterized as:

$$\mathrm{p}_1^{\mathrm{ref}}(t) = 0.5\cos\omega t + 0.2;\ \mathrm{p}_2^{\mathrm{ref}}(t) = 0.5\sin\omega t$$
$$\mathrm{p}_3^{\mathrm{ref}}(t) = 0.3(m);\ \omega = 0.3\pi.$$

Finally, in all the scenarios, we simply employ the stabilizing matrix $\boldsymbol{P} = \boldsymbol{Q}^{-1} = \begin{bmatrix} 0.9766\boldsymbol{I}_3 & 0.7813\boldsymbol{I}_3 \\ 0.7813\boldsymbol{I}_3 & 1.2500\boldsymbol{I}_3 \end{bmatrix}$ for the controller (3.32) as a solution of (3.56) while choosing $\alpha = 1.25$, so that the resulted invariant set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35) covers the drone's initial position. The video of the experiment is available at https://youtu.be/0YcEM280gts.

TABLE 3.9: Experiment specifications and results

	Ref. 1	Ref. 2	Ref. 3	Ref. 4
Average computation time (s) $\times 10^{-4}$	1.164	1.321	1.327	1.323
RMS of tracking error (cm)	10.03	1.99	2.57	3.89
γ as in (3.32)	5	4.5	4.5	4.5
α; $\boldsymbol{P}$ as in (3.32)	1.25; $\begin{bmatrix} 0.9766\boldsymbol{I}_3 & 0.7813\boldsymbol{I}_3 \\ 0.7813\boldsymbol{I}_3 & 1.2500\boldsymbol{I}_3 \end{bmatrix}$			
Sampling time (s)	0.075			

3.4.3.3 Experimental results and discussion

Overall, as can be seen in Figure 3.29, 3.32 and Table 3.9, all four trajectories were tracked properly with the root-mean-square (RMS) of the tracking errors under 10 centimeters. Moreover, the input constraints are always respected in all scenarios and the computational cost is considerably low compared to the other constraint-handling controllers with stability guarantees found in the literature [Greeff and Schoellig, 2018, Nguyen et al., 2020b, Nguyen and Prodan, 2021]. Further details on the experimental results and our proposed controller performance are given in the following.

Firstly, the tracking result for Ref. 1 is depicted in Figure 3.29 while the corresponding input signals in the original space and in the flat output space are given in Figure 3.30 and 3.31, respectively. As expected from the simulation in Figure 3.27, after increasing to a certain value of $\gamma > 1$ in (3.32), not only does the settling time not decrease, but also an unwanted oscillating effect appears (Figure 3.29). While this effect is not apparent in the simulation, it can be explained as the outcome of the high gain controller that awakes the "parasitic" dynamics existing within the vehicles (e.g. the rotational/attitude dynamics in the hierarchical scheme). Therefore, it is practically verified that γ does not need

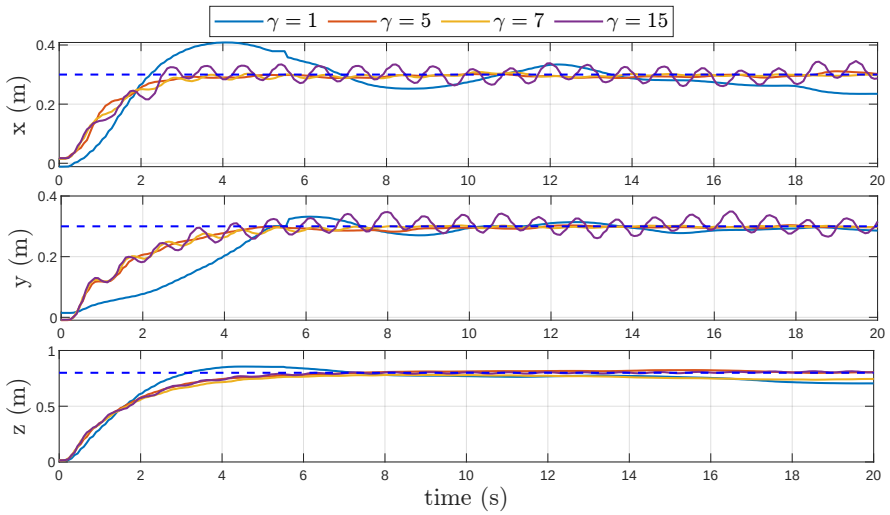


FIGURE 3.29: Ref. 1 (blue-dashed line) tracked with different values of γ.

to be chosen excessively large to achieve a favorable performance. Besides, from Figure 3.30 and 3.31, the effectiveness of the proposed saturation function is again validated via the restraint $\boldsymbol{v} \in \mathcal{V}_c$ given in (3.14), or consequently, the constraint satisfaction $\boldsymbol{u} \in \mathcal{U}$ given in (3.5).

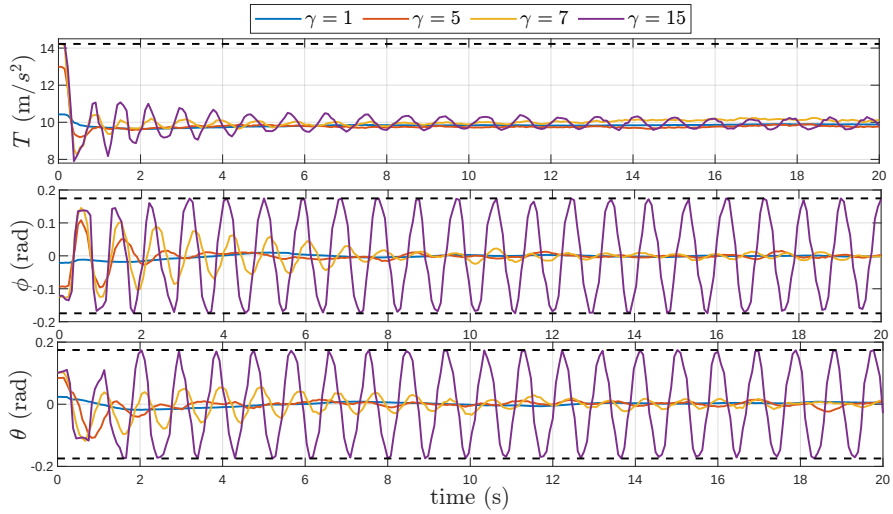


FIGURE 3.30: Input $\boldsymbol{u}$ with different values of γ with the set-point tracking problem and its bounds $\mathcal{U}$ as in (3.5) (black dashed line).

Next, it is also notable that the average computation time rests approximately at 0.1 milliseconds (see Table 3.9). In terms of computing time, this result surpasses previously mentioned controllers in the literature (around 5 to 50 milliseconds and above), which also guarantee stability and constraint satisfaction [Greeff and Schoellig, 2018, Nguyen

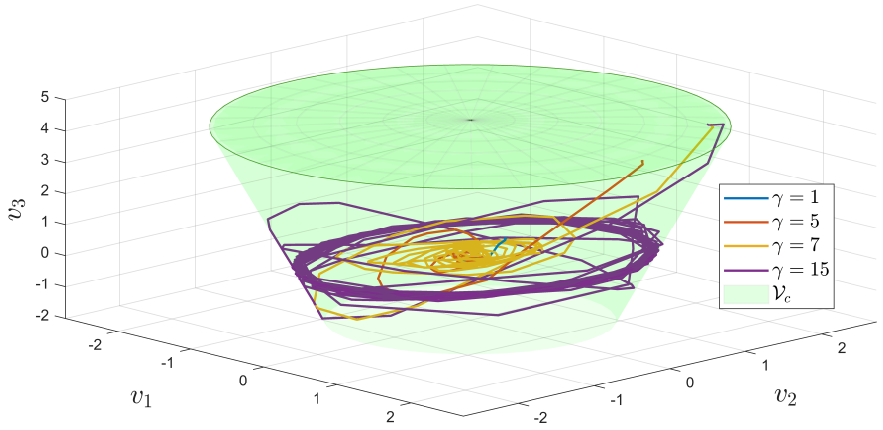


FIGURE 3.31: Input $\boldsymbol{v}$ in the flat output space, with different values of γ in Ref. 1 tracking problem and its constraint set $\mathcal{V}_c$ as in (3.14) .

et al., 2020b, Nguyen and Prodan, 2021, Gomaa et al., 2022]. This satisfactory performance comes from the fact that we provide an explicit formula of the controller from computing the feedback input to saturating such signal via Algorithm 3.2. Although in this experiment, all the control computation is executed remotely in the ground station computer, these results appear promising for embedding the controller directly on the drone's microcontroller thanks to the simplicity of the proposed scheme.

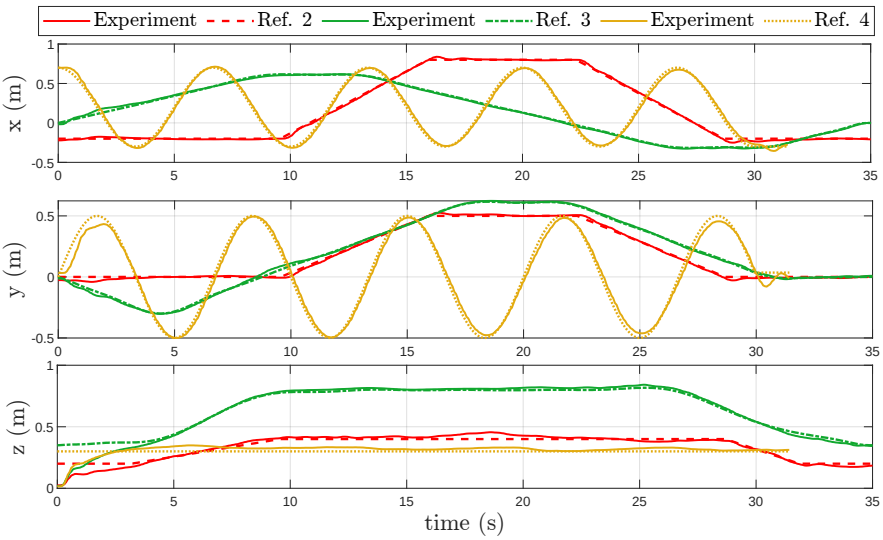


FIGURE 3.32: Position tracking with different trajectories.

Regarding the performance, the tracking errors provided by the proposed method can be considered commensurate to the FB-MPC, the well-known piece-wise affine MPC

(where the model is approximated along the trajectory) [Do and Prodan, 2023], or the fuzzy-based method proposed in [Santoso et al., 2019].

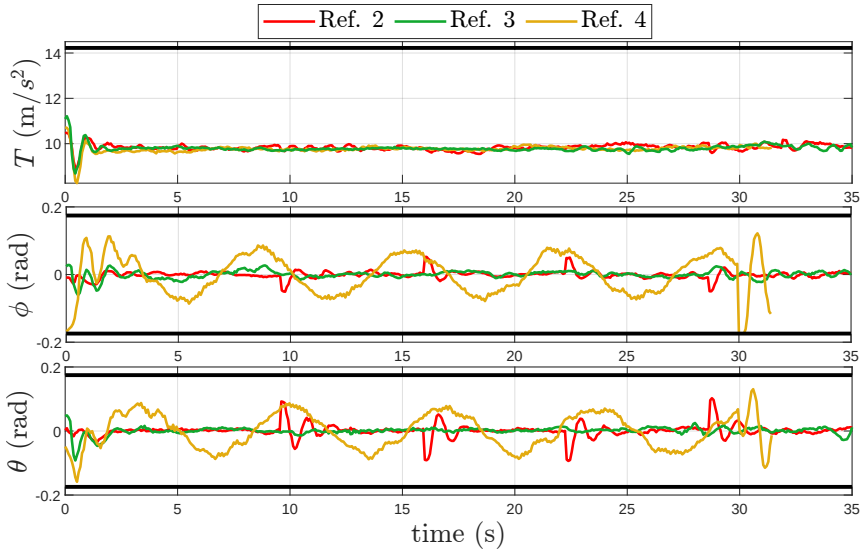


FIGURE 3.33: Input signal $\boldsymbol{u}$ in the tracking problem for Ref. 2, Ref. 3 and Ref. 4 with its bounds (black solid line).

Finally, as mentioned above, the selection of the high gain γ plays also an important role in the overall stability of the system. For these analyses, we refer to Chapter 5 which characterizes robustness and deals with the gain selection problem via an adaptive scheme.

These experimental validations conclude our three attempts to design a computationally low-cost outer loop of the quadcopter. Next, let us summarize the three control algorithms with insights on there real time performance.

3.4.4 Discussions and comparisons

So far, the constraint set $\mathcal{V}$ in (3.13) in the flat output space of the quadcopter translational dynamics has been investigated. This is characterized by a nonlinear convex subset, called $\mathcal{V}_c$ as in (3.14). Then different state-of-the-art control approaches have been adapted to stabilize the system (3.10) in the flat output space.

More specifically, standard online MPC or implicit MPC was first formulated in (3.24) or (3.27) to create a receding horizon control scheme via a polytopic inner approximation of $\mathcal{V}_c$, called $\mathcal{V}_c^{\text{in}}$ as in (3.22). Although the resulted optimization problem is a quadratic program, the explicit solution cannot be computed in real-time due to a significant size of the look-up table. This problem is due to the system investigated is in six dimension, resulting in a huge look-up table of the optimization's minimizers. To sidestep this

problem, the equivalence of three double integrators as in (3.9) of the system is employed with a box-type inner approximation of $\mathcal{V}_c$, called $\mathcal{B}_v$ as in (3.26). This representation allows us to decouple the 6D MPC routine into three 2D ones and achieve real-time operability with three explicit MPC solutions.

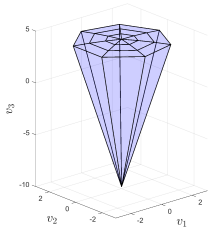
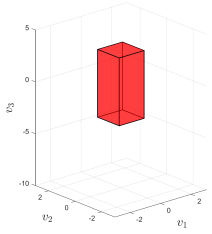
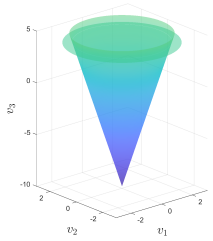
	IMPC	ExMPC	Explicit saturated control
Formulation	(3.24) or (3.60)	(3.27)	(3.32)
Input constraint set	$\mathcal{V}_c^{\text{in}}$ in (3.22) 	$\mathcal{B}_v$ in (3.26) 	$\mathcal{V}_c$ in (3.14) 
Complexity	quadratic program	3 look-up tables	max operation
Solver	`quadprog`	none	none
Avg. CT	15.4676 ms	0.0247 ms	0.0012 ms

TABLE 3.10: Comparison between the three controllers for system (3.10) in Chapter 3.

Finally, from a geometric viewpoint, a particularized explicit saturated function for the set $\mathcal{V}_c$ was formulated. This function allows us to synthesize an efficient gradient-based control with a quadratic control Lyapunov function for the system (3.10), requiring no external solvers or any other algorithm to ensure constraint satisfaction.

A summary of the input constraints and their illustration is given in Table 3.10. Therein, we also report the result of the simulation with initial points taken from a $5 \times 5 \times 5$ grid of the box $|x|, |y|, |z| \leq 1$ (m). From this, it is evident that controlling the system in the flat output space significantly reduces the computation time (CT). Even with the most complex method, IMPC, the control requires only around 15 ms. As discussed in the validation of ExMPC, this controller is essentially a program of three point-location problems in 2D, which can be effortlessly solved in real-time, even with a sequential search. Finally, the simplest form of the controller is the explicit saturated control in (3.32). This is because the saturation function, specified for the nonlinear constraint set $\mathcal{V}_c$ in (3.14), has been solved explicitly in Section 3.3.3.3, resulting in a constraint-handling controller with around 0.001 ms of calculation.

From the analysis of the three approaches, it can be seen that the computational simplicity is achieved by sacrificing either optimality or extensibility. Specifically, while the explicit saturated control particularly designed for the set $\mathcal{V}_c$ offers the highest speed, it is difficult to adapt to additional constraints, such as state or safety constraints. This restriction is somewhat alleviated with ExMPC, as demonstrated in the validation tests, where state constraints can also be accounted for during the offline synthesis phase.

With IMPC, more actuator power can be exploited with the least conservative polytopic inner approximation of the constraint, resulting in a larger feasible domain for the online control. Additionally, since the controller is formulated as an optimization problem, features such as safety or robustness can be incorporated by directly modifying the cost and the constraints, e.g., using a barrier function formulation or robust MPC design [Wu et al., 2019, Zeng et al., 2021].

In summary, thanks to the characterization in the flat output space, the outer loop control has been computationally simplified. Depending on the available computing power and design specifications, different strategies can be selected to stabilize the linearized dynamics.

Thus far, a particular case of constrained flat systems has been investigated over the last three sections. In the next section, we explore the effectiveness of neural networks in generalizing an approximation procedure for constraints in the flat output space.

3.5 Towards a generalized constraint approximation with neural networks

In this part, in the vein of approximating the convoluted constraints in the flat output space to obtain a tractable optimization-based control, we investigate the application of rectified linear unit neural networks (ReLU-ANN) to characterize the feasible set. Comprehensive simulation tests will also be provided to affirm the theoretical findings.

3.5.1 Preliminaries and problem statement

Herein, we study the class of differentially flat systems that are linearizable via endogenous feedback. For the sake of convenience, let us specify the context of constrained flat systems with feedback linearization as the following.

Consider the nonlinear flat system governed by:

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}, \boldsymbol{u}), \text{ s.t } \boldsymbol{u} \in \mathcal{U}, \tag{3.64}$$

where $\boldsymbol{x} \in \mathbb{R}^{n_x}, \boldsymbol{u} \in \mathbb{R}^m$ denote the state and input vectors, respectively. $\mathbf{f} : \mathbb{R}^{n_x} \times \mathbb{R}^m \to \mathbb{R}^{n_x}$ denotes the dynamics, which is assumed to be Lipschitz continuous. For $\boldsymbol{u}_{max} > \mathbf{0}$, the input constraints are as follows:

$$\mathcal{U} \triangleq \{\boldsymbol{u} \in \mathbb{R}^m : |\boldsymbol{u}| \leq \boldsymbol{u}_{max}\}. \tag{3.65}$$

Suppose that, for system (3.64), there exists a coordinate change and an input transformation that linearize the model in closed-loop. Consequently, the system's dynamics

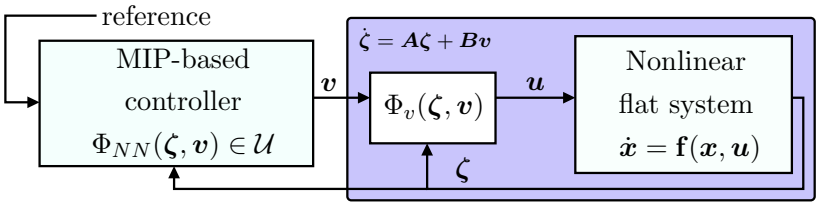


FIGURE 3.34: FB-FL control with the MIP representation of the ANN.

become linear in the Brunovsky canonical form, as in (2.14):

$$\dot{\boldsymbol{\zeta}} = \boldsymbol{A}\boldsymbol{\zeta} + \boldsymbol{B}\boldsymbol{v}, \tag{3.66}$$

where $\boldsymbol{A}, \boldsymbol{B}$ describe chains of integrators. $\boldsymbol{\zeta} \in \mathbb{R}^{n_\zeta}, \boldsymbol{v} \in \mathbb{R}^m$ are the new state and input variables, respectively. Note that linearization does not have to preserve the state dimension. Since, in the case of certain differentially flat systems [Fliess et al., 1993, Nicolau and Respondek, 2017], a state prolongation is required to deduce the linearizing law (a.k.a., dynamic feedback linearization). Denote the linearizing mapping from the virtual input $\boldsymbol{v}$ to the input $\boldsymbol{u}$ as:

$$\boldsymbol{u} = \Phi_v(\boldsymbol{\zeta}, \boldsymbol{v}). \tag{3.67}$$

An adaptation of the scheme in Figure 3.1 is given in Figure 3.34. This scheme illustrates the role of $\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})$ in the purple block, creating a linear system to control in the new coordinates. This variable change, in general, complicates the constraints (3.65) into a new set, restricting not only the new input $\boldsymbol{v}$ but also the state $\boldsymbol{\zeta}$. From (3.65) and (3.67), define such constraints as:

$$\mathcal{V} = \{(\boldsymbol{\zeta}, \boldsymbol{v}) : |\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{u}_{max}\}. \tag{3.68}$$

The aim now is to determine the control for the trivial system (3.66) with the complicated constraint set (3.68).

Remark 3.11. Generally, the mapping (3.67) does not linearize the system in closed-loop globally. It, in fact, comes with additional operating constraints (e.g., singularity or unstable region avoidance) on the state and input, for example, $\varphi_v(\boldsymbol{\zeta}, \boldsymbol{v}) \leq 0$. With the proposed approach, these constraints can be reformulated similarly by modifying $\mathcal{V}$ in (3.68) to $\mathcal{V}' = \left\{(\boldsymbol{\zeta}, \boldsymbol{v}) : \Phi'(\boldsymbol{\zeta}, \boldsymbol{v}) \triangleq \begin{bmatrix} |\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})| \\ \varphi_v(\boldsymbol{\zeta}, \boldsymbol{v}) \end{bmatrix} \leq \begin{bmatrix} \boldsymbol{u}_{max} \\ 0 \end{bmatrix}\right\}$.

To this aim, we will show that, with a ReLU-ANN-based approximation of $\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})$, denoted as $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$ the constraint $(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{V}$ in (3.68) can be inferred through MI linear constraints. This inference enables the construction of a tractable online optimization program. To proceed, let us revisit some standard setups of the ReLU-ANN as follows.

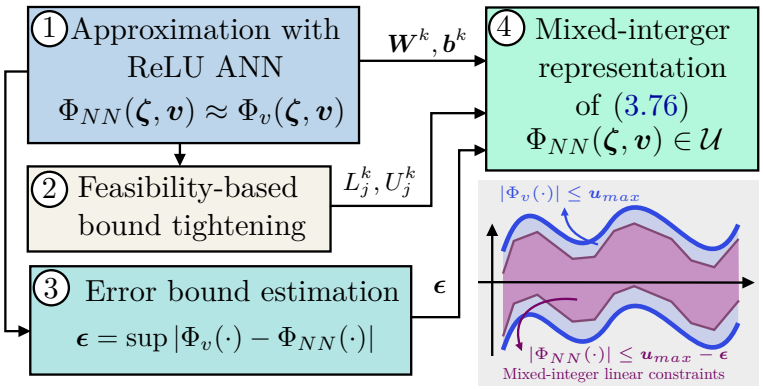


FIGURE 3.35: Constraint characterization: from ReLU-ANN to MILP.

Consider a ReLU-ANN of $K-1$ hidden layers and, essentially, a ReLU activation function for $\boldsymbol{y} \in \mathbb{R}^{n_y}$:

$$\sigma(\boldsymbol{y}) = \max\{0, \boldsymbol{y}\} = [\max\{0, y_1\} \dots \max\{0, y_{n_y}\}]^\top. \tag{3.69}$$

Denote n_k number of nodes in the kth layer, $k \in \mathcal{I}_{1,K}$ which is characterized by the weight $\boldsymbol{W}^k \in \mathbb{R}^{n_k \times n_{k-1}}$ and the bias $\boldsymbol{b}^k \in \mathbb{R}^{n_k}$. Then, the input-output relationship of the network can be calculated as follows, for $k \in \mathcal{I}_{1,K-1}$:

$$\boldsymbol{y}^k = \sigma(\boldsymbol{W}^k \boldsymbol{y}^{k-1} + \boldsymbol{b}^k), \boldsymbol{y}^K = \boldsymbol{W}^K \boldsymbol{y}^{K-1} + \boldsymbol{b}^K, \tag{3.70}$$

with $\boldsymbol{y}^0$, $\boldsymbol{y}^K$ as the network's input and output, respectively.

3.5.2 Constraint characterization with MIP and applications in control

In this part, we show that the constraint $(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{V}$ as in (3.68) can be guaranteed by ensuring a set of MI linear constraints. Then, their formulation will be addressed. The representation's application in control synthesis will be discussed subsequently.

3.5.2.1 Constraint characterization

The constraint $(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{V}$ as in (3.68) can be reformulated into MI linear constraints with the following proposition.

Proposition 3.3. *Consider an approximation function of $\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})$ based on ReLU-ANN, called $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$, with the network's parameters denoted as in (3.70). Assume that the approximation error is bounded, i.e., $\exists \boldsymbol{\epsilon} \in \mathbb{R}^m, \boldsymbol{\epsilon} > \boldsymbol{0}$:*

$$|\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v}) - \Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{\epsilon}, \tag{3.71}$$

in a compact region $(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{Z} \subset \mathbb{R}^{n_\zeta} \times \mathbb{R}^m$. *Then the constraint* $(\boldsymbol{\zeta}, \boldsymbol{v}) \in \mathcal{V}$ *can be imposed via MI linear constraints with respect to* $\boldsymbol{\zeta}$ *and* $\boldsymbol{v}$.

Proof. The proof is twofold. First, it can be shown that the constraint (3.68) can be guaranteed by imposing linear constraints on the output of the network $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$. Namely, $|\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{u}_{max}$ can be implied by:

$$|\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{u}^{\epsilon}_{max} \triangleq \boldsymbol{u}_{max} - \boldsymbol{\epsilon}. \tag{3.72}$$

This is because the ith row of $\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})$ is bounded as:

$$\begin{aligned} &|\Phi_{v,i}(\boldsymbol{\zeta}, \boldsymbol{v})| = |\Phi_{v,i}(\boldsymbol{\zeta}, \boldsymbol{v}) - \Phi_{NN,i}(\boldsymbol{\zeta}, \boldsymbol{v}) + \Phi_{NN,i}(\boldsymbol{\zeta}, \boldsymbol{v})| \\ &\leq |\Phi_{v,i}(\boldsymbol{\zeta}, \boldsymbol{v}) - \Phi_{NN,i}(\boldsymbol{\zeta}, \boldsymbol{v})| + |\Phi_{NN,i}(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \epsilon_i + |\Phi_{NN,i}(\boldsymbol{\zeta}, \boldsymbol{v})|. \end{aligned} \tag{3.73}$$

Thus, (3.72) implies: $|\Phi_v(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{\epsilon} + |\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})| \leq \boldsymbol{u}_{max}$.

Second, we show that the linear constraints (3.72) being imposed on the ReLU-ANN can be represented via MI linear constraints. More specifically, consider the case of a single node in the network, for some real scalar $y^\star$ and vector $\boldsymbol{y}^{in}$ with $\boldsymbol{w}, \boldsymbol{b}$ in the appropriate dimensions:

$$y^\star = \sigma(\boldsymbol{w}^\top \boldsymbol{y}^{in} + b). \tag{3.74}$$

Assume that the finite upper and lower bounds of $y^\star$ can be calculated, denoted respectively as U, L (i.e., $L \leq y^\star \leq U$), then (3.74) can be defined implicitly as [Grimstad and Andersson, 2019]:

$$\begin{cases} \boldsymbol{w}^\top \boldsymbol{y}^{in} + b = \overline{y}^\star - \underline{y}^\star, \overline{y}^\star \geq 0, \underline{y}^\star \geq 0, \\ \alpha \in \{0,1\}, \overline{y}^\star \leq U\alpha, \underline{y}^\star \leq L(\alpha - 1), y^\star = \overline{y}^\star, \end{cases} \tag{3.75}$$

where $\overline{y}^\star, \underline{y}^\star$ are real variables, and α is a binary variable reproducing the conditional activation (the max operation in (3.69)) in each node of the network. Then, by applying the same reformulation for the ReLU-ANN $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$ structured as in (3.70), the constraint (3.72) can be exactly rewritten as:

$$\begin{cases} \boldsymbol{y}^0 = [\boldsymbol{\zeta}, \boldsymbol{v}]^\top, k \in \mathcal{I}_{1,K-1}, j \in \mathcal{I}_{1,n_k}, & \text{(3.76a)} \\ \boldsymbol{W}^k \boldsymbol{y}^{k-1} + \boldsymbol{b}^k = \overline{\boldsymbol{y}}^k - \underline{\boldsymbol{y}}^k, \overline{\boldsymbol{y}}^k \geq \boldsymbol{0}, \underline{\boldsymbol{y}}^k \geq \boldsymbol{0}, & \text{(3.76b)} \\ \alpha^k_j \in \{0,1\}, \overline{y}^k_j \leq U^k_j \alpha^k_j, \underline{y}^k_j \leq -L^k_j(1 - \alpha^k_j), & \text{(3.76c)} \\ \boldsymbol{y}^K = \boldsymbol{W}^K \boldsymbol{y}^{K-1} + \boldsymbol{b}^K, |\boldsymbol{y}^K| \leq \boldsymbol{u}^{\epsilon}_{max} \text{ as in (3.72)}, & \text{(3.76d)} \end{cases}$$

where, for the ReLU-ANN $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$, $\boldsymbol{y}^k \triangleq \overline{\boldsymbol{y}}^k$, $\boldsymbol{W}^k, \boldsymbol{b}^k$ denote the output, weight matrix, and bias of its kth layer. L^k_j, U^k_j are the lower and upper bound of the jth node in the kth layer. The transformation from (3.74) to (3.75) is also referred to in the literature as the "big-M" technique. Then, the program (3.76) is a Mixed-Integer

Linear Program (MILP) with the real variables $\boldsymbol{y}^k, \overline{\boldsymbol{y}}^k, \underline{\boldsymbol{y}}^k$ and the binary variable α_j^k. An illustration of the formulation (3.76) is given in Figure 3.36. □

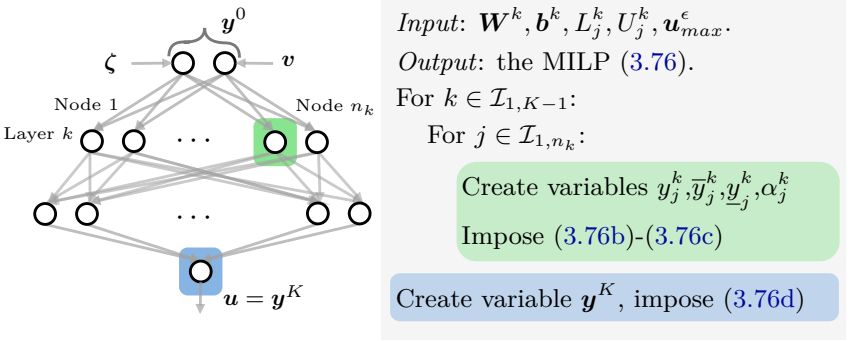


FIGURE 3.36: The formulation of the ReLU-ANN as MILP.

As noted in Figure 3.36, the three key ingredients of the formulation (3.76) include the network parameters $(\boldsymbol{W}^k, \boldsymbol{b}^k)$, the nodes' bounds (L_j^k, U_j^k) and the approximation error bound ($\boldsymbol{\epsilon}$ as in (3.71)). These components will be discussed subsequently.

1) The network parameters $\boldsymbol{W}^k, \boldsymbol{b}^k$ as in (3.76): In our setting, as the control problem is considered for a nominal model with perfect feedback (i.e., no disturbances or uncertainties), the training for $\Phi_{NN}(\boldsymbol{\zeta}, \boldsymbol{v})$ can be interpreted as a standard multivariate function fitting. For this reason, the selection of the training hyperparameters to optimize the approximation performance will not be discussed. In the simulation study, the ReLU-ANN will be generated by simply collecting data points from a dense grid of the space of interest and employing the toolbox from MATLAB [Matlab, 2021].

2) The nodes' bounds L_j^k, U_j^k as in (3.76c): As well-known in the MIP community, the "big-M" values L_j^k and U_j^k directly affect the solver efficiency if chosen too large, and their optimal selection, in general, still remains a topic of interest. Yet, with the focus on ReLU-ANN, several algorithms have been proposed to tighten these bounds with different levels of complexity. For a detailed survey and comparison, we send the readers to the work of Grimstad and Andersson [2019] and the references therein. In the next section, the *feasibility-based bound tightening* (FBBT) procedure is chosen due to its simplicity and solver-free nature. With the input layer $\boldsymbol{y}^0 \triangleq [\boldsymbol{\zeta}, \boldsymbol{v}]^\top \in \mathbb{R}^{n_z+m}$, the calculation will be adopted from Section 4.3 in [Grimstad and Andersson, 2019] and given in Appendix A.4.

3) Error bound $\boldsymbol{\epsilon}$ estimation in (3.71): As it is well-known that ReLU-ANNs are universal function approximators [Yarotsky, 2017], the error bound vector $\boldsymbol{\epsilon}$ defined in (3.71) can be optimized with a sufficiently large number of layers or nodes per layer. The sole challenge remaining is to determine and ensure this bound across a domain of interest

$\mathcal{Z} \subset \mathbb{R}^{n_\zeta} \times \mathbb{R}^m$, i.e.,

$$\epsilon_i = \max_{(\boldsymbol{\zeta},\boldsymbol{v})\in\mathcal{Z}} |\Phi_{v,i}(\boldsymbol{\zeta},\boldsymbol{v}) - \Phi_{NN,i}(\boldsymbol{\zeta},\boldsymbol{v})|, i \in \mathcal{I}_{1,m}. \tag{3.77}$$

This approximation bound, in general, is not straightforward to obtain. If $\Phi_v(\boldsymbol{\zeta},\boldsymbol{v})$ is continuous, the cost function in (3.77) is also continuous, since the ReLU-ANN $\Phi_{NN}(\boldsymbol{\zeta},\boldsymbol{v})$ is piece-wise linear continuous. The maximizer hence exists for a compact domain $\mathcal{Z}$. However, since the latter is not differentiable, gradient-based techniques appear inefficient to solve this maximization problem. In our small-scale case studies, to exploit the offline computational power, the estimation of $\boldsymbol{\epsilon}$ will be carried out using the conventional derivative-free particle swarm algorithm. The idea is to create numerous candidate solutions in the search space, associate them with stable virtual dynamics, and allow information exchange. In this way, the candidates can have exploratory behavior and convergence, with low computational complexity [Kennedy and Eberhart, 1995].

To elaborate, we will incorporate the constraint (3.72)[3] into two effective constraint-handling frameworks: CLF-CBF and MPC. This showcases the adaptability of the proposed method in dealing with both stability and safety constraints, along with constraints intricately linked to model predictions.

3.5.2.2 Control implementation

1) CLF-CBF framework: Given the MI linear constraint (3.72), one natural application is to combine it with other linear constraints, maintaining the structure of the optimization problem. Certainly, given a CLF and a CBF in the new coordinates, denoted as: $V(\boldsymbol{\zeta}), H(\boldsymbol{\zeta}) : \mathbb{R}^{n_\zeta} \to \mathbb{R}$, respectively, the control law can be adopted as [Li et al., 2023]:

$$\boldsymbol{v}^*(\boldsymbol{\zeta}) = \arg\min_{\boldsymbol{v},\delta} J(\boldsymbol{\zeta},\boldsymbol{v}) + \delta \tag{3.78a}$$
$$|\Phi_{NN}(\boldsymbol{\zeta},\boldsymbol{v})| \leq \boldsymbol{u}^{\epsilon}_{max} \text{ as in (3.72)}, \tag{3.78b}$$
$$\nabla V(\boldsymbol{\zeta})^\top(\boldsymbol{A}\boldsymbol{\zeta} + \boldsymbol{B}\boldsymbol{v}) \leq -\beta V(\boldsymbol{\zeta}) + \delta, \quad \delta \geq 0, \tag{3.78c}$$
$$\nabla H(\boldsymbol{\zeta})^\top(\boldsymbol{A}\boldsymbol{\zeta} + \boldsymbol{B}\boldsymbol{v}) \geq -\kappa H(\boldsymbol{\zeta}), \tag{3.78d}$$

where $\boldsymbol{A}, \boldsymbol{B}$ are from the linearized dynamics (3.66), $\kappa, \beta > 0$ and $\delta \in \mathbb{R}$ is the relaxation variable, prioritizing the safety certificate from (3.78d) over the tracking convergence from (3.78c). With regard to $J(\boldsymbol{\zeta},\boldsymbol{v})$, by minimizing $\dot{V}(\boldsymbol{\zeta})$, one can arrive to a mixed-integer linear program (MILP), i.e.,:

$$J(\boldsymbol{\zeta},\boldsymbol{v}) = \dot{V}(\boldsymbol{\zeta},\boldsymbol{v}) = \nabla V(\boldsymbol{\zeta})^\top(\boldsymbol{A}\boldsymbol{\zeta} + \boldsymbol{B}\boldsymbol{v}). \tag{3.79}$$

[3]Hereinafter, when constraint (3.72) is implemented, we mean the implementation of its MIP representation (3.76).

In another way, (3.78) can be a mixed-integer quadratic program (MIQP) if $J(\boldsymbol{\zeta}, \boldsymbol{v})$ is adapted as:

$$J(\boldsymbol{\zeta}, \boldsymbol{v}) = \|\boldsymbol{v} - \boldsymbol{k}(\boldsymbol{\zeta})\|^2, \tag{3.80}$$

where $\boldsymbol{k}(\boldsymbol{\zeta})$ is a desired control law for $\boldsymbol{v}$ to approach.

Remark 3.12. In the setting (3.78)–(3.80), the benefit of the FL scheme is prominent, especially from the offline design view point. This is because, with the obtained integrator dynamics (3.66) of which the poles have non-positive real parts, the construction of $V(\boldsymbol{\zeta})$ is more apparent. The simple choice can be a quadratic CLF from the linear quadratic regulator.

2) MPC framework: As the constraint (3.72) is proposed for a general set in (3.68), its usefulness may not stand in some particular scenarios. Specifically, one may argue that the function (3.67) often can be found in an affine form:

$$\boldsymbol{u} = \Phi_a(\boldsymbol{\zeta})\boldsymbol{v} + \Phi_b(\boldsymbol{\zeta}), \tag{3.81}$$

for some functions $\Phi_a(\boldsymbol{\zeta}), \Phi_b(\boldsymbol{\zeta})$. This form renders the setup in (3.78b) unnecessary, since, given a state feedback $\boldsymbol{\zeta}$, such MI constraints can be replaced by a linear constraint of $\boldsymbol{v}$:

$$|\Phi_a(\boldsymbol{\zeta})\boldsymbol{v} + \Phi_b(\boldsymbol{\zeta})| \leq \boldsymbol{u}_{max}, \tag{3.82}$$

and (3.78) become a QP. However, in specific applications that require model prediction, the simplicity of (3.82) cannot be preserved, even with the advantage of the form (3.81). For instance, in a discrete-time MPC setting with the sample rate t_s, with a state feedback $\boldsymbol{\zeta}(t)$, the constraints (3.82) imposed along the prediction horizon N_p, $k \in \mathcal{I}_{0,N_p-1}$, are:

$$|\Phi_a(\boldsymbol{\zeta}(t|kt_s))\boldsymbol{v}(t|kt_s) + \Phi_b(\boldsymbol{\zeta}(t|kt_s))| \leq \boldsymbol{u}_{max}, \tag{3.83a}$$

$$\boldsymbol{\zeta}(t|(k+1)t_s) = \boldsymbol{A}_d\boldsymbol{\zeta}(t|kt_s) + \boldsymbol{B}_d\boldsymbol{v}(t|kt_s), \tag{3.83b}$$

where $\boldsymbol{A}_d, \boldsymbol{B}_d$ denote the discretized model of (3.66). In this formulation, the constraint (3.83a) is no longer linear with respect to the control sequence $\boldsymbol{v}(t|kt_s)$ due to the dependence (3.83b) involved in the multiplication $\Phi_a(\boldsymbol{\zeta}(t|kt_s))\boldsymbol{v}(t|kt_s)$. Thus, even with the affine form (3.81), the constraints (3.83) are not trivial to handle [Kurtz and Henson, 1998]. Meanwhile, with the proposed setting, we will show that the MI constraints remain linear, even when enforced over a prediction horizon as in (3.83).

Indeed, the constraints (3.72) can be incorporated in an MPC, at time step t, as follows:

$$\boldsymbol{v}^* = \arg\min_{\boldsymbol{v}(t|kt_s)} \sum_{k=0}^{N_p-1} \ell(\boldsymbol{\zeta}(t|kt_s), \boldsymbol{v}(t|kt_s)) \tag{3.84a}$$

$$|\Phi_{NN}(\boldsymbol{\zeta}(t|kt_s), \boldsymbol{v}(t|kt_s))| \leq \boldsymbol{u}^{\epsilon}_{max}, \tag{3.84b}$$

$$\boldsymbol{\zeta}(t|(k+1)t_s) = \boldsymbol{A}_d\boldsymbol{\zeta}(t|kt_s) + \boldsymbol{B}_d\boldsymbol{v}(t|kt_s), \tag{3.84c}$$

$$\boldsymbol{\zeta}(t|kt_s) \in \mathcal{X}_z, k \in \mathcal{I}_{0,N_p-1}, \tag{3.84d}$$

where $\boldsymbol{A}_d, \boldsymbol{B}_d$ denote the discretized model of (3.66) with sampling time t_s, $\mathcal{X}_z$ is the polytopic state constraint, and $\ell(\boldsymbol{\zeta}, \boldsymbol{v})$ is a chosen cost function. While the constraints (3.84c)-(3.84d) are linear, from Proposition 3.3, the MI constraint (3.84b) is also linear for the variables $\boldsymbol{v}(t|kt_s), \boldsymbol{\zeta}(t|kt_s)$. Thus, with a standard quadratic cost $\ell(\boldsymbol{\zeta}, \boldsymbol{v})$, one can formulate a MIQP-based MPC with (3.84). To show the efficacy of the methods, we will examine numerical examples in the next section.

3.5.3 Simulation study

3.5.3.1 Simulation setup

For simplicity, hereinafter, for all ReLU-ANN approximations with $K-1$ hidden layers, each layer will have equally $n_k = \bar{n}$ nodes. The training data will be sampled from a hyperbox noted as $\mathcal{T}$ with n_s samples in each axis. The training is conducted with the method `fitrnet` from [Matlab, 2021] with default hyperparameters. Numerical specifications for the characterization are given in Table 3.11. The training time (TT) is also reported therein. All the MIPs were solved with the Yalmip toolbox [Lofberg, 2004] and CPLEX solver [CPLEX, 2009]. Next, the simulation studies will be presented, followed by the discussion.

Nonlinear mass-spring damper system (MSD)

Consider the dynamics [Romdlony and Jayawardhana, 2016]:

$$\dot{x}_1 = x_2, \dot{x}_2 = -s(x_2) - x_1 + u, \tag{3.85}$$

where x_1, x_2 represent the displacement and the velocity of the system, respectively, and u is the constrained input signal with $|u| \leq u_{max} = 5$. The damping is described by the Stribeck friction model $s(x_2) = (0.8 + 0.2e^{-100|x_2|})\tanh(10x_2) + x_2$. The model can be written in the form of (3.66) with $\boldsymbol{\zeta} = [\zeta_1, \zeta_2]^\top = [x_1, x_2]^\top$, $\boldsymbol{A} = \left[\begin{smallmatrix} 0 & 1 \\ 0 & 0 \end{smallmatrix}\right]$, $\boldsymbol{B} = [0, 1]^\top$, $\Phi_v(\boldsymbol{\zeta}, v) = v + s(\zeta_2) + \zeta_1$ and the input constraint now becomes $|\Phi_v(\boldsymbol{\zeta}, v)| \leq u_{max}$ which will be approximated and converted to MI constraints as in Section 3.5.2.1, with training points sampled from $\mathcal{T} = \{|\boldsymbol{\zeta}| \leq 5, |v| \leq 10\}$.

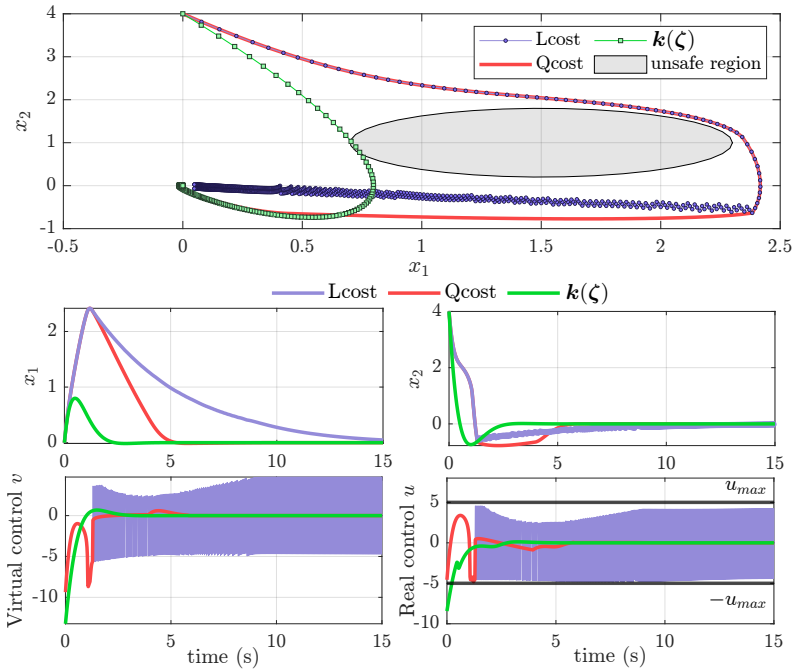


FIGURE 3.37: Stabilization with ANN-based constraint characterization.

TABLE 3.11: Numerical specifications

System	K	$\bar{n}$	$\boldsymbol{\epsilon}$ in (3.71)	κ	β	n_s	TT(s)
MSD (3.85)	4	20	0.2740	4.0	0.001	30	9.43
Quad-1D (3.86)	4	10	0.0322	×	×	35	2.19

To examine the setup in (3.78), consider an unsafe region $\mathcal{O} = \{\boldsymbol{\zeta} : H(\boldsymbol{\zeta}) \leq 0\}$ with $H(\boldsymbol{\zeta}) \triangleq \|\boldsymbol{\zeta} - \boldsymbol{\zeta}_o\|_2 - r_o$, $\boldsymbol{\zeta}_o = [1.5, 1]^\top, r_o = 0.8$. For control, the program (3.78) will be employed with both the linear cost (Lcost) as in (3.79) and quadratic cost (Qcost) as in (3.80). The CLF can be chosen as a standard quadratic function $V(\boldsymbol{\zeta}) = \|\boldsymbol{\zeta}\|_{\boldsymbol{P}}^2$, $\boldsymbol{P} = \left[\begin{smallmatrix} 4.58 & 10 \\ 10 & 45.83 \end{smallmatrix}\right]$ and $\boldsymbol{k}(\boldsymbol{\zeta}) = -[4.47, 3.37]\boldsymbol{\zeta}$. With $\boldsymbol{\zeta}(0) = [0, 4]^\top$, the simulation results were given in Figure 3.37 with the three controllers: Lcost, Qcost for (3.78) and $\boldsymbol{k}(\boldsymbol{\zeta})$ given previously.

Horizontal 1D quadrotor (Quad-1D) To validate the applicability of the MPC setup in (3.84), we examine the horizontal 1-D quadcopter model [Greeff and Schoellig, 2020]:

$$\ddot{x} = \Gamma \sin\theta - \gamma\dot{x},\ \dot{\theta} = \tau^{-1}(u - \theta), \tag{3.86}$$

where x is the horizontal displacement, θ is the pitch angle. u is the commanded pitch angle and constrained as $|u| \leq u_{max} = 0.1745$ (rad). The model parameters are $\Gamma = 10, \gamma = 0.3, \tau = 0.2$. With $\zeta_1 = x$, the nonlinear model (3.86) can be linearized to:

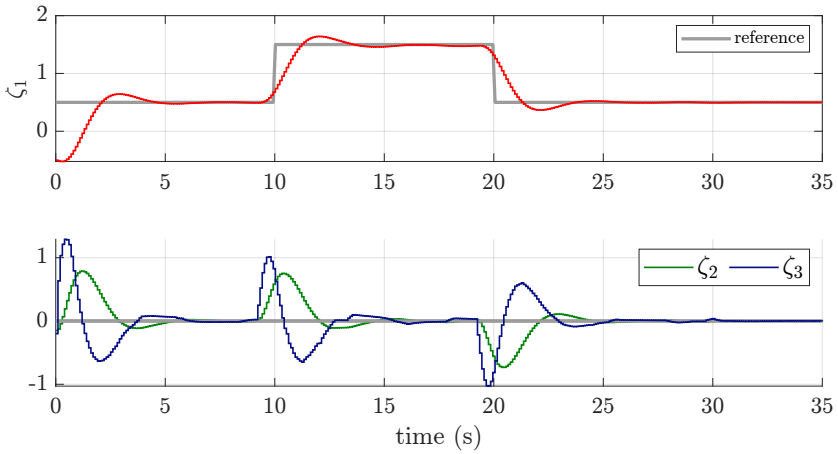


FIGURE 3.38: Trajectory tracking with the MPC (3.84) for (3.86).

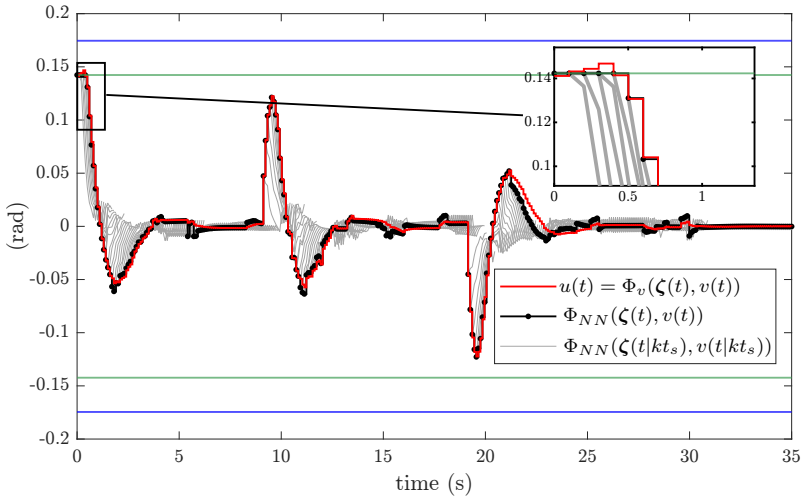


FIGURE 3.39: The input $u = \Phi_v(\cdot)$, the approximation $\Phi_{NN}(\cdot)$ and its predicted trajectories.

$\dddot{\zeta}_1 = v$, with the mapping:

$$\Phi_v(\boldsymbol{\zeta}, v) = \tau(v + \gamma\zeta_3)/\left(\Gamma\sqrt{1 - \mathfrak{h}^2(\boldsymbol{\zeta})}\right) + \sin^{-1}\mathfrak{h}(\boldsymbol{\zeta}), \tag{3.87}$$

where $\boldsymbol{\zeta} = [\zeta_1, \zeta_2, \zeta_3]^\top = [x, \dot{x}, \ddot{x}]^\top$, $\mathfrak{h}(\boldsymbol{\zeta}) = \Gamma^{-1}(\zeta_3 + \gamma\zeta_2)$. The new constraint hence yields $|\Phi_v(\boldsymbol{\zeta}, v)| \leq u_{max}$ which is approximated with samples from $\mathcal{T} = \{|\boldsymbol{\zeta}| \leq 5, |v| \leq 15\}$.

For the discrete implementation of the MPC (3.84), the linearized system is rewritten in the form of (3.66) and discretized with the forth-order Runge-Kutta method, sampled at $t_s = 100$ms. The prediction horizon is $N_p = 10$ steps, the time-varying reference $\boldsymbol{\zeta}^{\text{ref}}$

is depicted in gray on Figure 3.38 and the cost function $\ell(\cdot)$ as in (3.84a) is chosen as: $\ell(\boldsymbol{\zeta}, v) = \|\boldsymbol{\zeta} - \boldsymbol{\zeta}^{\text{ref}}\|^2_{\boldsymbol{Q}} + \|v\|^2_{\boldsymbol{R}}$, with $\boldsymbol{Q} = \left[\begin{smallmatrix} 10 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{smallmatrix}\right]$, $\boldsymbol{R} = 0.0175$, $\boldsymbol{\zeta}(0) = [\,-0.5 \;\; -0.15 \;\; -0.2\,]^\top$. The system's trajectory is given in Figure 3.38 and 3.39 with $u = \pm u_{max}$ and $u = \pm u^{\epsilon}_{max}$ depicted in blue and green, respectively.

while the online computation time in both examples is reported in Table 3.12 with its min, max and mean values.

TABLE 3.12: Computation time of the simulations (ms).

System	Controller	min	max	mean
MSD (3.85)	Lcost	20.96	34.52	25.15
	Qcost	20.62	30.81	23.83
	$\boldsymbol{k}(\boldsymbol{\zeta})$	8e−4	0.29	2.21e−3
Quad-1D (3.86)	MPC ($N_p = 10$)	42.76	5.96e+4	1.69e+3

3.5.3.2 Discussion

In general, in both investigated systems, the proposed setting is compatible with standard optimization-based control framework, achieving stability, safety (avoidance of the unsafe region), and most importantly, constraint satisfactions, even for forecasted trajectories.

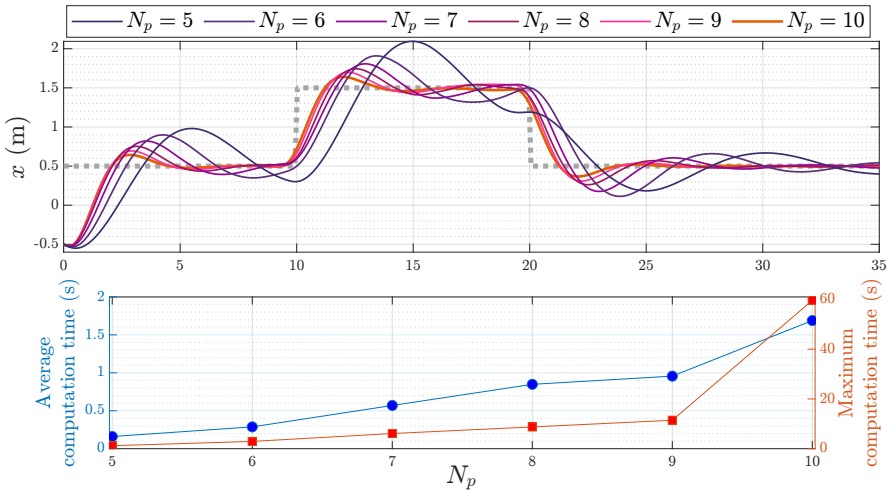


FIGURE 3.40: The displacement $x(t)$ and the computational cost w.r.t N_p.

For system (3.85), from Table 3.12, it is noteworthy that both Lcost and Qcost controllers can be implemented with relatively low computation time (from 20-40ms). The only shortcoming is the discontinuity of the Lcost controller, which is caused by the non-smooth characteristics of the constraints (3.78b)–(3.78d) in combination with the

linear cost. Yet, the problem is not present for the Qcost controller, and interestingly, the computation cost is even lower as compared to that of Lcost. Stable tracking can also be found in the example of the 1D quadrotor (3.86), where a time-varying trajectory can be tracked with the advantage of anticipation from MPC (see Figure 3.38). Given the error bound as in (3.71), the input constraint is guaranteed by imposing $|\Phi_{NN}(\cdot)| \leq u^{\epsilon}_{max}$. It is crucial to emphasize that in our approach, neither the predicted constraint nor the model underwent approximation through state feedback, as observed in [Kurtz and Henson, 1998, Nevistic and Del Re, 1994]. Hence, the constraints were rigorously enforced over the prediction horizon for the nominal dynamics. This advancement, from a theoretical standpoint, paves the way for constrained FL-based control design with rigorously guaranteed stability. One potential resolution involves adopting the standard MPC axioms [Mayne et al., 2000], incorporating a stabilizing selection of parameters $N_p, \mathcal{X}_z$ and $\ell(\cdot)$ in (3.84). This reiterates the necessity of delving deeper into understanding the proposed MI linear constraint (3.72) on a case-by-case basis.

Finally, although satisfactory results and general development were found, the real-time implementation with MPC is evidently still hindered by the solving of the MIQP (3.84) (see Figure 3.40), especially when the number of binary variables grows with the number of neurons and the prediction horizon[4] $\left(N_p \sum_{k=1}^{K-1} n_k\right)$. This implementational shortcoming becomes even more evident when multi-input systems are considered, since the linearizing mapping will certainly require a larger network. For the method to be real-time capable, future work concerns the simplification of the constraints and an optimal choice of the network parameters.

3.6 Concluding remarks

In this chapter, we first examined the constrained control problem of the quadcopter's outer loop. The model for this high-level control is a flat system and enjoys the property of feedback linearizability. The convoluted constrains introduced by the linearization were addressed through various convex inner approximations. These subsets of the feasible domain enabled the implementation of different advanced control techniques, including implicit MPC, explicit MPC, and explicit saturated control, ensuring both stability and constraint satisfaction in the original coordinates. Additionally, the simulation tests and experimental validations show the significant complexity reduction in real-time implementation of all three algorithms compared to related works in the literature. Although all the approaches exhibit relatively small computation times, differences are observed, which are related to their adaptability to new features and the conservatism of the inner approximations.

[4] Each time (3.84b) is imposed, $\sum_{k=1}^{K-1} n_k$ binary variables are used. Hence, over N_p steps, the number of binary variables is $N_p \sum_{k=1}^{K-1} n_k$.

Regarding the shortcomings, it is important to bring to light that, although successful validation under indoor environment was provided, the study with model (3.4) enjoyed several assumptions including small disturbances (e.g., drag), uncertainties or mismatch between the outer loop and the inner loop. These limitations will undeniably become more critical when larger models or outdoor applications are considered. Therefore, future completion of the methods will concentrate on the robustification of the scheme via additional countermeasures such as disturbance rejection with estimators or robust MPC [Sira-Ramírez, 2018, Mayne et al., 2011]. In addition, it is noteworthy that the proposed approaches resulted in a fully complete stabilized closed-loop with well-characterized stability. This immediately opens the door to extending performance or feasibility with "add-on" device such as reference governors and their successors [Convens et al., 2024, Skibik et al., 2022, 2021, Garone et al., 2017].

Secondly, we study a general approach to deal with the flatness-based feedback linearization control problem under the presence of convoluted constraints. The constraint characterization includes approximating the mapping by a rectified linear unit artificial neural network and reformulating it into the equivalent mixed-integer linear constraints. The applicability and validity of the proposed method were shown via simulation tests with the control Lyapunov function-control barrier function, and model predictive control strategies. Future theoretical directions focus on the investigation of the new constraints and their adaptation within the standard linear control design, showcasing the advantage with comparative study. Meanwhile, practical extensions will target optimization in the design parameters and the method's applications to multi-input systems. Furthermore, the geometric interpretation of the neural network is also a domain of interest [Samanipour and Poonawala, 2023, Ortiz et al., 2023, Jouret et al., 2023].

Chapter 4

Low complexity control synthesis with stability guaranteed

Previously, in the flat output space, we exploited the linearity of the dynamics to formulate different advanced controllers. In this chapter, under the same linearization of the quadcopter's outer loop, we examine the linearized dynamics from a design viewpoint for control affine systems. By doing so, we study a technique for exploiting open-loop-generated trajectories for a constrained control problem, using them to shape suitable non-quadratic control Lyapunov functions. These trajectories, generated off-line, allow detecting a suitable domain of attraction in which a candidate Lyapunov function has a negative derivative. Given suitably constructed basis functions, our working machinery is based on linear programming, hence, the technique can be applied to problems of non-trivial size in terms of the number of basis functions and points in the state space. For linearized systems, we seek convex control Lyapunov functions, which are homogeneous polynomials. With such an artificially generated stability certificate, we demonstrate again the complexity reduction with respect to model predictive control in exchange for a quantifiable suboptimality gap. Furthermore, the established results hold not only for the investigated system but also for the class of control affine systems in general. Briefly, accompanied by both comprehensive simulation tests and experimental results, the main contribution of the chapter is twofold:

- we propose an offline procedure to construct a CLF based on linear programming, exploiting the admissible control generated by an MPC law [J6];
- we address the concern of performance trade-off by formalizing the concept of Sub-Control Lyapunov Functions with both stability and performance analysis [J2];

4.1 Motivation . . . 113
4.2 Efficient control via LP-generated CLFs . . . 115
4.2.1 Main idea and machinery . . . 115
4.2.2 Admissible pair generator and control implementation . . . 122
4.2.3 Application on quadcopter position control . . . 124
4.2.4 Experimental validation . . . 129
4.3 Real-time complexity reduction with sub-control Lyapunov functions . . . 133
4.3.1 Sub-Control Lyapunov Functions . . . 134
4.3.2 Computation of a Sub-Control Lyapunov Function . . . 141
4.3.3 Numerical examples and experimental validation . . . 147
4.4 Concluding remarks . . . 155

4.1 Motivation

The previous chapter highlights the advantages and potentials of the system's linear description in the flat output space. However, as can be seen from the last sections, even when a controller of high performance is successfully synthesized, it is likely that the calculation is too complicated to implement online, especially when a sophisticated optimization problem such as MPC is involved. This is indeed a well-recognized challenge in the literature.

To address this issue, Bemporad et al. [2002] proposed using an explicit expression of the optimal control in the form of a look-up table for online use. However, their implementation becomes increasingly intractable with respect to the system's dimensions and the length of the prediction horizon [Kvasnica et al., 2013]. An alternative direction to attain online operability for complicated control policies is to learn the implicit law with machine learning using neural networks as a function approximator, such as in [Chen et al., 2018, Drgoňa et al., 2022, Mukherjee et al., 2022] or [Karg and Lucia, 2020], to name a few. This synthesis can be viewed as a supervised learning problem where the feedback variables are the features labeled with the input decisions made by such an "expert" control. Although the strategy significantly reduces the computational footprint, there is still room for improvement in terms of fitting performance, theoretical guarantee, and training data characterization, especially when large-scale systems come into play [Chen et al., 2022, Drgoňa et al., 2018].

From a control theoretic viewpoint, stability can be ensured if there exists a control Lyapunov function (CLF) for the closed-loop dynamics. If successfully synthesized, the function provides a stability certificate, which can be exploited to derive efficient control laws, for instance adopting Sontag's formula [Sontag, 1989, Yamashita et al., 2022] or a min-norm control [Freeman and Kokotovic, 1996] with a complexity of a standard QP.

With this direction, a considerable amount of effort has been devoted to computationally synthesizing CLF. In truth, this is not a trivial task and has to be handled on a case-by-case basis, with specific numerical tools and some restrictions on the number of states and inputs. For example, with sum-of-square methods, CLF for polynomial systems can be achieved with some relaxation via semidefinite programming (SDP) [Prajna et al., 2004, Chesi, 2010, Lavaei and Bridgeman, 2023] or linear programs [Steentjes et al., 2020, Julian et al., 1999, Fiacchini et al., 2015, Ravanbakhsh and Sankaranarayanan, 2019, Lazar and Jokic, 2009a]. In [Hu and Lin, 2003], a composite quadratic function was developed as the CLF for linear systems subject to saturation and state constraints, resulting in the domain of attraction described by a convex hull of ellipsoids. The study for linear piecewise affine systems is conducted in [Lazar and Jokić, 2010]. More generally for nonlinear systems, yet at the price of high computational burden with respect to the dimensionality problem, procedures to numerically compute a continuous piecewise affine CLF were developed in [Lavaei and Bridgeman, 2023, Steentjes et al., 2020] via state space triangulation. In [Min et al., 2023], under a known assumed approximation error bound, a neural network structure was proposed to learn the dynamics, the Lyapunov function, and the controller simultaneously. Online synthesis of state-dependent CLF as well as the corresponding control is proposed for discrete systems [Lazar and Jokic, 2009b, Lazar, 2009]. For certain low-dimensional systems, the Hamilton-Jacobi-Bellman partial differential equation can be solved to incorporate the notion of optimality [Beard et al., 1997, Aguilar and Krener, 2014]. For a more detailed survey of computational CLF synthesis methods, we refer to the review [Giesl and Hafstein, 2015].

To address such a computational difficulty and with a view to extending the existing literature, the contributions of this chapter are as follows.

Computational generation of a CLF with application to quadcopter: We first aim to apply a general procedure to computationally generate a CLF via a supposedly available optimal controller. The computation of the CLF, via a proper parameterization exhibits a complexity of a semi-infinite linear program (SILP). By sampling the state space, the problem is relaxed to a tractable standard LP. Eventually, only the resulting CLF is employed in the control, reducing the computational overhead for the online program. The practicability of the procedure will then be experimentally validated with the established quadcopter application in the flat output space. The result then lays the groundwork and open questions for a theoretical formalization subsequently [J6].

Real-time complexity reduction with sub-Control Lyapunov Function: We address theoretical concerns arise from the computational synthesis of the CLF with the

introduction of the concept of sub-control Lyapunov function (SCLF). With this formalization, we characterize the sub-optimality performance compromising the complexity reduction. At the same time, we propose a comprehensive computational procedure to generate an SCLF based on the cutting-plane technique for solving the SILP without any relaxation. The result is then applied for control affine system of non-trivial size and experimentally validated with the quadcopter application [J2].

With the presented structure, the chapter continues with the study on the computational generation of CLFs and the experimental validation over the quadcopter model.

4.2 Efficient control via LP-generated CLFs

This section presents a computationally efficient procedure to generate a CLF based on linear programming. The generated CLF is then validated with the quadcopter application characterized previously.

4.2.1 Main idea and machinery

In the light of addressing the generation of the CLF for a generic control affine system, let us specify the context by introducing a general setting as follows.

Consider the nonlinear control affine system:

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}, \tag{4.1}$$

with a convex and compact constraint set:

$$\boldsymbol{u} \in \mathcal{U}, \tag{4.2}$$

where $\boldsymbol{x} \in \mathbb{R}^n$ denotes the state, and $\boldsymbol{u} \in \mathbb{R}^m$ is the input vector constrained in $\mathcal{U}$.

Assumption 4.1. Assume the pair $\boldsymbol{x} = \mathbf{0}$ and $\boldsymbol{u} = \mathbf{0}$ is a steady state, i.e., $\mathcal{U}$ includes $\mathbf{0}$ in its interior and $f(\mathbf{0}) = \mathbf{0}$. □

Assume that we have an open-loop constrained trajectory software tool, capable of driving the system to $\mathbf{0}$. Then we could simply use the software in an MPC fashion for control. However, for systems with limited hardware, although a stabilizing controller can be theoretically generated, the *online* calculation of the MPC law might not be feasible in one sampling interval, due to the complexity of the optimization problem. In the sequel, it will be shown that this software can also be exploited in the *offline* design phase to synthesize a CLF and, consequently, a stabilizing controller.

The underlying idea is that of adopting this open-loop trajectory generator to simply determine candidate state control-pair optimal trajectories. These state control pairs are

then used to generate suitable constraints for finding a CLF, within a linearly parameterized family, through linear programming. These generated trajectories will eventually be disregarded, and only the CLF will be retained for actual control. To this aim, some standard definitions are recalled in the following.

For the sake of later use, we first adapt the definition of a CLF from Definition 2.7.

Definition 4.1 (Control Lyapunov Function)**.** *A positive definite smooth Lyapunov function, $V(\boldsymbol{x})$, is a CLF for (4.1) if there exists a function $\boldsymbol{u} = \boldsymbol{\Xi}(\boldsymbol{x})$ with values in $\mathcal{U}$ and*

$$\mathrm{d}V(\boldsymbol{x})/\mathrm{d}t = \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\Xi}(\boldsymbol{x})] \leq -\beta V(\boldsymbol{x}), \tag{4.3}$$

for some $\beta > 0$. □

Remark 4.1. Definition 4.1 is a specialized version of Definition 2.7, tailored to the proofs of Theorem 4.1 and Proposition 4.4 in the following parts.

Definition 4.2. *The state-control pair $(\boldsymbol{x}, \boldsymbol{u})$ for (4.1), $\boldsymbol{u} \in \mathcal{U}$ is admissible for $V(\boldsymbol{x})$ if it satisfies (4.3).* □

Definition 4.3 (Domain of Attraction [Blanchini and Miani, 2000])**.** *$\mathcal{D} \subseteq \mathbb{R}^n$ is a domain of attraction (DOA) of system (4.1) if there exist $\boldsymbol{u}(t) = \boldsymbol{\Xi}(\boldsymbol{x}(t))$, such that any trajectory with $\boldsymbol{x}(0) \in \mathcal{D}$, satisfies:*

$$\boldsymbol{x}(t) \in \mathcal{D}, \boldsymbol{u}(t) \in \mathcal{U}, \text{ and } \lim_{t\to\infty} \boldsymbol{x}(t) = \mathbf{0}. \tag{4.4}$$

□

Next, in the offline control synthesis, adopting the available software, we generate several admissible pairs and then, formulate the CLF based on these points. This is a simple idea to generate, under appropriate conditions, points that are inside the domain of attraction $\mathcal{D}$, eliminating all points outside, for which, no constrained control can bring the state to $\mathbf{0}$.

Let us assume that the open-loop trajectory generator ensures the existence of constrained trajectories $\boldsymbol{x}(t), \boldsymbol{u}(t)$, from which we can sample M pairs $(\boldsymbol{x}_j, \boldsymbol{u}_j)$, $\boldsymbol{u}_j \in \mathcal{U}, j \in \{1, ..., M\}$. Consider the following candidate CLF:

$$V(\boldsymbol{x}) = \sum_{q=0}^{N} \alpha_q V_q(\boldsymbol{x}), \quad \alpha_q \geq 0, \tag{4.5}$$

where $V_q(\boldsymbol{x})$ are smooth positive definite Lyapunov functions.

Next, consider the following linear programming problem with N variables, and M inequality constraints formulated from M state-input pairs collected previously:

$$\min_{\alpha_q} \sum_{q=1}^{N} \ell_q \alpha_q \tag{4.6a}$$

$$\sum_{q=1}^{N} \alpha_q \left[\nabla V_q(\boldsymbol{x}_j)\left[\mathbf{f}(\boldsymbol{x}_j) + \mathbf{g}(\boldsymbol{x}_j)\boldsymbol{u}_j\right] + \beta V_q(\boldsymbol{x}_j)\right] \leq 0, \tag{4.6b}$$

$$\textstyle\sum_{q=1}^{N} \alpha_q = 1, \alpha_q \geq 0, j \in \{1, ..., M\}, \tag{4.6c}$$

where ℓ_q are weights. These weights give freedom of choice in the computation. As an example, if $V_1(\boldsymbol{x})$ is a "privileged" function, for instance a quadratic function associated with a local control optimizing cost, then ℓ_1 can be adjusted to influence the function dominance within the parameterization (4.5). Then, as a result of (4.6), we achieve a parameterized CLF established as a linear combination of $V_q(\boldsymbol{x})$ with the coefficients α_q. An illustration of how the Lyapunov function $V(\boldsymbol{x})$ is shaped by the stabilizing constraint (4.6b) at each sampled data pairs is given in Figure 4.1.

Remark 4.2. With the intuition of parameterizing the function $V(\boldsymbol{x})$ via the set of $V_q(\boldsymbol{x})$, it is essential to provide a suitable choice for such candidates. This, indeed, can be done via an iterative process where, after one solution of (4.6), the function $V_q(\boldsymbol{x})$ with $\alpha_q \approx 0$ can be discarded and a new set of functions can be adapted. Note that, this can be accomplished thanks to the simplicity of the LP (4.6). A discussion on the choice of $V_q(\boldsymbol{x})$ will be given later in the illustrative example. □

Remark 4.3. For a large number of admissible pairs collected, there may be some redundant pairs, i.e., the pairs that render the constraint (4.6b) redundant with respect to those from the rest. This may unnecessarily expand the size of the optimization (4.6). To handle this problem, the redundant pairs can be detected and eliminated using the following proposition. □

Proposition 4.1. *For a finite set of pairs $\mathcal{A} = \{(\boldsymbol{x}_j, \boldsymbol{u}_j)\}$, the pair $(\boldsymbol{x}_c, \boldsymbol{u}_c)$ is redundant when being applied to (4.6b)-(4.6c) if $\gamma_c(\boldsymbol{x}_c, \boldsymbol{u}_c) \leq 0$ where $\gamma_c(\boldsymbol{x}_c, \boldsymbol{u}_c)$ is calculated with the following LP:*

$$\begin{aligned} \gamma_c(\boldsymbol{x}_c, \boldsymbol{u}_c) \triangleq & \max_{\boldsymbol{\alpha} \triangleq [\alpha_1, ..., \alpha_N]^\top} \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}_c, \boldsymbol{u}_c) \\ s.t. & \begin{cases} \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) \leq 0, (\boldsymbol{x}, \boldsymbol{u}) \in \mathcal{A}, \\ \sum_{q=1}^{N} \alpha_q = 1, \alpha_q \geq 0. \end{cases} \end{aligned} \tag{4.7}$$

and $\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) \triangleq \sum_{q=1}^{N} \alpha_q [\nabla V_q(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \beta V_q(\boldsymbol{x})]$. □

Proof. Consider a scalar function $\Gamma(\boldsymbol{y}) : \mathbb{R}^p \to \mathbb{R}$ and the set $\mathcal{H} \subset \mathbb{R}^p$, the constraint $\Gamma(\boldsymbol{y}) \leq 0$ is redundant with respect to $\mathcal{H}$ if $\forall \boldsymbol{y} \in \mathcal{H}, \Gamma(\boldsymbol{y}) \leq 0$, or equivalently:

$$\max_{\boldsymbol{y} \in \mathcal{H}} \Gamma(\boldsymbol{y}) \leq 0. \tag{4.8}$$

Then, by applying the same reasoning for the constraint $\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}_c, \boldsymbol{u}_c) \leq 0$ with respect to (4.6b)-(4.6c) defined with the pairs in $\mathcal{A}$, we arrive at (4.7). □

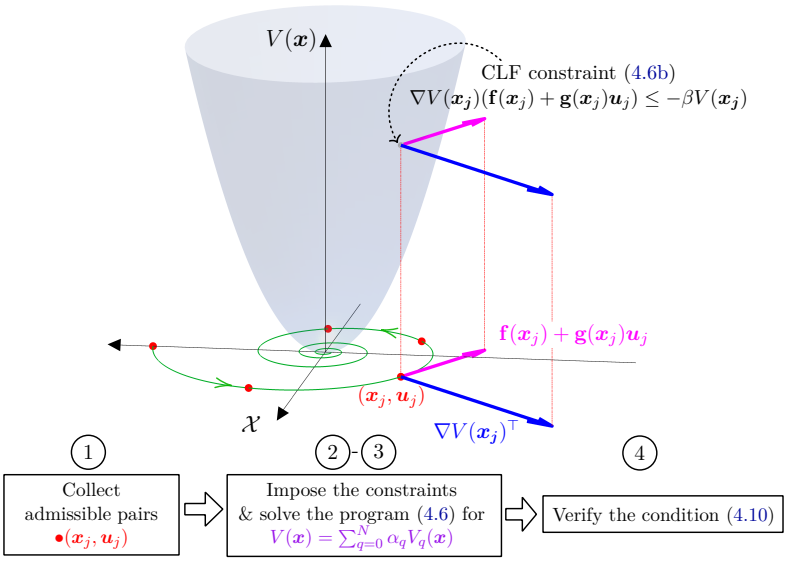


FIGURE 4.1: Geometric interpretation of the stabilizing condition (4.6b) and the procedure to compute the CLF.

Forming the CLF with (4.6) has some pros and cons:

- by employing an LP, it is possible to use a considerably large number of points and basis functions $V_q(\boldsymbol{x})$, ensuring the existence of α_q for some β in (4.6), even in high-dimensional systems;
- although the number of generating data pairs $(\boldsymbol{x}_j, \boldsymbol{u}_j)$ can be huge, we are still working on a finite set.

To address a preliminary answer to the second point, let us consider the following propositions. For the set $\mathcal{U}$, denote by $\Psi_{\mathcal{U}}(\boldsymbol{w})$ its support functional:

$$\Psi_{\mathcal{U}}(\boldsymbol{w}) \triangleq \max_{\boldsymbol{u} \in \mathcal{U}} \boldsymbol{w}^\top \boldsymbol{u}. \tag{4.9}$$

Note that $\Psi_{\mathcal{U}}(\mathbf{0}) = \mathbf{0}$ and that, in view of Assumption 4.1, $\Psi_{\mathcal{U}}(\boldsymbol{w})$ is positive definite and positively homogeneous of degree one: $\Psi_{\mathcal{U}}(\lambda \boldsymbol{w}) = \lambda \Psi_{\mathcal{U}}(\boldsymbol{w}), \forall\, \lambda > 0$.

Proposition 4.2. *The function $V(\boldsymbol{x})$ is a CLF on a given domain $\mathcal{X}$ containing $\mathbf{0}$ in its interior iff it satisfies the following inequality on $\mathcal{X}$:*

$$\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \Psi_{\mathcal{U}}(-\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top) + \beta V(\boldsymbol{x}) \leq 0, \tag{4.10}$$

for some $\beta > 0$. □

Proof. By Definition 4.1, $V(\boldsymbol{x})$ is a CLF iff $\forall \boldsymbol{x} \in \mathcal{X}$:

$$\exists \boldsymbol{u} \in \mathcal{U}, \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \beta V(\boldsymbol{x}) \leq 0, \tag{4.11}$$

or, equivalently[1], for $\boldsymbol{x} \in \mathcal{X}$:

$$\min_{\boldsymbol{u}\in\mathcal{U}} \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \beta V(\boldsymbol{x}) \leq 0. \tag{4.12}$$

Condition (4.12) can then be rewritten as:

$$\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \max_{\boldsymbol{u}\in\mathcal{U}} [-\nabla V(\boldsymbol{x})\mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \beta V(\boldsymbol{x}) \leq 0. \tag{4.13}$$

Then, by the support function definition in (4.9), one gets to Proposition 4.2. □

Remark 4.4. It is worth noticing that condition (4.10) does not refer to a specific control action $\boldsymbol{u} = \boldsymbol{\Xi}(\boldsymbol{x})$ but depends only on the shape of $\mathcal{U}$. For instance, if, as often assumed in the literature, $\mathcal{U}$ is the unit ball $\|\boldsymbol{u}\| \leq 1$, then equation (4.10) becomes:

$$\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \|\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\| + \beta V(\boldsymbol{x}) \leq 0. \tag{4.14}$$

Note that the condition $\dot{V}(\boldsymbol{x}) \leq -\beta V(\boldsymbol{x})$ ensures asymptotic stability along with the performance condition $V(t) \leq V(0)e^{-\beta t}$ in terms of convergence. □

For a domain of interest $\mathcal{X}$, the proposition is useful since it allows testing the generated function $V(\boldsymbol{x})$ in (4.6), by dense gridding and verifying the condition (4.10) in $\mathcal{X}$. To recap, we propose the following algorithm to determine $V(\boldsymbol{x})$.

Aligning with the common CLF setup, the aforementioned considerations are formalized in the following propositions.

Proposition 4.3. *Suppose that Algorithm 4.1 is successful and that there exists $\kappa > 0$ for which inequality (4.10) is verified in the sublevel set $\mathcal{D}_\kappa = \{\boldsymbol{x} : V(\boldsymbol{x}) \leq \kappa\}$, then $V(\boldsymbol{x})$ is a CLF for the problem (4.1) with the DOA $\mathcal{D}_\kappa$.* □

Although Proposition 4.3 follows directly the definitions in Section 4.2.1, it poses a question on how the condition (4.10) can be verified for *all* points in a level set of $V(\boldsymbol{x})$,

[1] If (4.11) is true for some $\boldsymbol{u}$, for sure the minimum value of the expression (4.12) is less than 0. Conversely, if such minimum is non-positive as in (4.12), and achieved at some $\boldsymbol{u}^*$ then such $\boldsymbol{u}$ is the control required in (4.11).

Algorithm 4.1: Offline CLF generation

1 Collect M admissible pairs $(\boldsymbol{x}_j, \boldsymbol{u}_j)$ from a stabilizing controller for system (4.1). This can be sampled pairs from the system's trajectories with several random initial conditions (see also Figure 4.1);
2 Fix a basis function $V_q(\boldsymbol{x})$ as in (4.5), $q = 1, ..., N$ of positive definite functions, and $\beta > 0$ as in (4.3);
3 Formulate and solve the linear program (4.6) for α_q;
4 If problem (4.6) is feasible, grid the state space within a region of interest and check (4.10) for all points in the grid;

rather than only for a finite grid of points. For this reason, in the sequel, we show that, under certain assumptions, the CLF condition (4.10) can be continuously verified.

Proposition 4.4. *Assume condition (4.10) is satisfied for $V(\boldsymbol{x})$ on the grid points $\boldsymbol{x}_j$ and let $d = \max_{\boldsymbol{x}\in\mathcal{X}} \min_j \|\boldsymbol{x} - \boldsymbol{x}_j\|$. Let c_1 and c_2 be the Lipschitz constants of the function $\Theta(\boldsymbol{x}) = \nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \Psi_{\mathcal{U}}\left(-\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\right)$ and $V(\boldsymbol{x})$, respectively, say such that:*

$$|\Theta(\boldsymbol{x}) - \Theta(\boldsymbol{y})| \leq c_1\|\boldsymbol{x} - \boldsymbol{y}\|, |V(\boldsymbol{x}) - V(\boldsymbol{y})| \leq c_2\|\boldsymbol{x} - \boldsymbol{y}\|.$$

Then, for every $\beta_1 < \beta$, $\exists \epsilon > 0 : \forall \boldsymbol{x} \in \{\boldsymbol{x} : V(\boldsymbol{x}) \geq \epsilon\}$,

$$\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \Psi_{\mathcal{U}}\left(-\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\right) \leq -\beta_1 V(\boldsymbol{x}), \tag{4.15}$$

and $\epsilon = (\beta c_2 + c_1)d/(\beta - \beta_1)$. □

Proof. By assumption, we have $\Theta(\boldsymbol{x}_j) \leq -\beta V(\boldsymbol{x}_j)$, then,

$$\begin{aligned}
&\Theta(\boldsymbol{x}) = \Theta(\boldsymbol{x}_j) + \Theta(\boldsymbol{x}) - \Theta(\boldsymbol{x}_j) \\
&\leq \Theta(\boldsymbol{x}_j) + c_1\|\boldsymbol{x} - \boldsymbol{x}_j\| \leq -\beta V(\boldsymbol{x}_j) + c_1 d \\
&= -\beta(V(\boldsymbol{x}_i) - V(\boldsymbol{x})) - \beta V(\boldsymbol{x}) + c_1 d \leq -\beta V(\boldsymbol{x}) + \beta c_2\|\boldsymbol{x}_j - \boldsymbol{x}\| + c_1 d \\
&= -\beta V(\boldsymbol{x}) + (\beta c_2 + c_1)d \leq -\beta_1 V(\boldsymbol{x}) + (\beta_1 - \beta)V(\boldsymbol{x}) + (\beta c_2 + c_1)d \\
&\leq -\beta_1 V(\boldsymbol{x}).
\end{aligned}$$

□

Besides intensely gridding the space inside $\mathcal{D}_\kappa$ to verify (4.10), one can also achieve an inner estimation as follows.

Proposition 4.5. *Consider an ellipsoid inscribed in the input constraint set $\mathcal{U}$ in (4.2), which has the form:*

$$\mathcal{E} = \{\boldsymbol{u} = \boldsymbol{E}\boldsymbol{w}, \|\boldsymbol{w}\| \leq 1\} \text{ and } \mathcal{E} \subset \mathcal{U}. \tag{4.16}$$

Let $\vartheta(\kappa_e)$ be defined as:

$$\vartheta(\kappa_e) \triangleq \max_{V(\boldsymbol{x})\leq\kappa_e} \Big(\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \|\boldsymbol{E}\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\| + \beta V(\boldsymbol{x})\Big) \tag{4.17}$$

then, for $\kappa_e > 0$, if $\vartheta(\kappa_e) \leq 0$, $\mathcal{D}_{\kappa_e}$ is a DOA. □

Proof. From the convexity and compactness of $\mathcal{E}$ and $\mathcal{U}$, one can state [Vitale, 1985]:

$$\mathcal{E} \subset \mathcal{U} \Rightarrow \Psi_{\mathcal{E}}(\boldsymbol{v}) \leq \Psi_{\mathcal{U}}(\boldsymbol{v}), \forall \boldsymbol{v} \in \mathbb{R}^m. \tag{4.18}$$

Thus,

$$\begin{aligned} &\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \Psi_{\mathcal{U}}(-\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top) + \beta V(\boldsymbol{x}) \\ &\leq \nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \Psi_{\mathcal{E}}(-\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top) + \beta V(\boldsymbol{x}) \\ &= \nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x}) - \|\boldsymbol{E}\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\| + \beta V(\boldsymbol{x}). \end{aligned} \tag{4.19}$$

Consequently, if for $\kappa_e > 0$, $\vartheta(\kappa_e) \leq 0$, then, by Proposition 4.2, $\mathcal{D}_{\kappa_e}$ is a DOA. With this method, we iteratively seek also for the largest κ_e that satisfies $\vartheta(\kappa_e) \leq 0$. The trade-off for this is the conservative employment of an ellipsoidal subset $\mathcal{E}$ instead of the original constraint set $\mathcal{U}$. □

Remark 4.5. It is noteworthy that the LP structure can still be preserved in the presence of bounded additive disturbances. More specifically, consider the disturbed case of (4.1):

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u} + \boldsymbol{d}, \|\boldsymbol{d}\| \leq \mu. \tag{4.20}$$

Then the condition:

$$\dot{V}(\boldsymbol{x}) = \nabla V(\boldsymbol{x})(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u} + \boldsymbol{d}) \leq -\beta V(\boldsymbol{x}) \tag{4.21}$$

is implied by

$$\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \sqrt{\sum_{j=1}^{n}\sum_{q=1}^{N} \left(\frac{\partial V_i}{\partial x_j}(\boldsymbol{x})\right)^2} \leq 0, \tag{4.22}$$

with $\vartheta(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u})$ as in (4.7). This condition is linear with respect to the decision variables α_q in the optimization problem (4.6). The proof for this implication is given in Appendix A.3.

To complete the synthesis, the remaining discussion resides on how to formulate the generator of admissible pairs $(\boldsymbol{x}_j, \boldsymbol{u}_j)$ for (4.6). Moreover, with the established CLF, the choice of the associated control law is an important question as well. These two issues will be addressed in the next section.

4.2.2 Admissible pair generator and control implementation

In this part, the collection of admissible pairs at step 1 in Algorithm 4.1 is first discussed. Next, the online control policy employed with the generated CLF will be introduced.

4.2.2.1 Admissible pair generation

As mentioned before, we aim to employ an MPC solver to achieve stabilizing trajectories from which admissible pairs are sampled. However, the MPC strategy commonly needs to be implemented in a discretized manner. With such a discretization, the generated pairs do not necessarily imply the stability property or the accurate evolution of the continuous system (4.1). For this reason, we employ the Euler Auxiliary System (EAS) [Blanchini et al., 2003]:

$$\boldsymbol{x}(k+1) = \boldsymbol{x}(k) + \tau \left[\mathbf{f}(\boldsymbol{x}(k)) + \mathbf{g}(\boldsymbol{x}(k))\boldsymbol{u}(k)\right], \tag{4.23}$$

where $\tau > 0$ is the discretization constant parameter. Then, with this setup and a CLF generated with (4.23), we have the following result.

Theorem 4.1. *Any convex function $V(\boldsymbol{x})$ which is a CLF for the EAS in (4.23) is also a CLF for the associated continuous time system in (4.1).* □

Proof. The convexity of $V(\boldsymbol{x})$ implies that the difference quotient function is non-decreasing with regard to $h \in \mathbb{R}^+$ and the Dini derivative yields [Rockafellar, 1997]:

$$\begin{aligned} \frac{\mathrm{d}}{\mathrm{d}t}V(\boldsymbol{x}) &= \lim_{h\to 0^+} \sup \frac{V(\boldsymbol{x} + h[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}]) - V(\boldsymbol{x})}{h} \\ &\leq \frac{V(\boldsymbol{x} + \tau[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}]) - V(\boldsymbol{x})}{\tau}, \text{ for } \tau \geq h. \end{aligned} \tag{4.24}$$

Meanwhile, being a CLF of the EAS implies, $\exists \beta_d > 0$:

$$V(\boldsymbol{x} + \tau[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}]) - V(\boldsymbol{x}) \leq -\beta_d V(\boldsymbol{x}). \tag{4.25}$$

Thus, from (4.24), (4.25) and Definition 4.1, one can state that :

$$\frac{\mathrm{d}}{\mathrm{d}t}V(\boldsymbol{x}) \leq -\frac{\beta_d}{\tau}V(\boldsymbol{x}), \tag{4.26}$$

which completes the proof. □

Therefore, for the offline admissible pair generator in Algorithm 4.1, step 1, we employ the following MPC strategy:

$$\arg\min_{\boldsymbol{u}(\cdot)} \sum_{j=0}^{N_p-1} \left(\|\boldsymbol{x}(k|k+j)\|_{\boldsymbol{Q}}^2 + \|\boldsymbol{u}(k|k+j)\|_{\boldsymbol{R}}^2 \right) + \|\boldsymbol{x}(k|k+N_p)\|_{\boldsymbol{P}}^2 \tag{4.27a}$$

$$\text{s.t} \begin{cases} \boldsymbol{x}(k|k+j+1) = \boldsymbol{x}(k|k+j) + \tau \left[\mathbf{f}(\boldsymbol{x}(k|k+j)) + \mathbf{g}(\boldsymbol{x}(k|k+j))\boldsymbol{u}(k|k+j) \right], \\ \boldsymbol{u}(k|k+j) \in \mathcal{U}, \boldsymbol{x}(k|k+j) \in \mathcal{X}, \boldsymbol{x}(k|k+N_p) \in \mathcal{X}_f, \end{cases} \tag{4.27b}$$

where $\boldsymbol{Q}, \boldsymbol{R}$ are the user-defined weightings penalizing the tracking and input, respectively. $\boldsymbol{P}$ and $\mathcal{X}_f$ denote the weighting for the terminal cost and its terminal set, which are associated with a locally optimal control $\boldsymbol{u}_{opt}(\boldsymbol{x})$. This standard penalization and constraints guarantee the recursive feasibility and stability of the system [Mayne et al., 2000]. Then, for a sampled state vector, $\boldsymbol{x}_j$, the admissible control $\boldsymbol{u}_j$ is taken as the first component of the minimizing sequence of (4.27b) with $\boldsymbol{x}(k=0) = \boldsymbol{x}_j$. This formulation hence completes the proposed procedure to collect admissible pairs.

4.2.2.2 Associated nonlinear control law

Prior to this point, the ingredients for the offline generation of the CLF have been presented. Yet, a further question is how one can impose a desired performance over such a CLF during online implementation. Following the continuous stabilizer in [Grammatico et al., 2013], consider the controller:

$$\boldsymbol{u} = \arg\min_{\boldsymbol{u}} \|\boldsymbol{u} - \boldsymbol{u}_d(\boldsymbol{x})\|^2 \tag{4.28a}$$

$$\text{s.t } \boldsymbol{u} \in \mathcal{U} \text{ and } \nabla V(\boldsymbol{x}) \left[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u} \right] \leq -\beta V(\boldsymbol{x}). \tag{4.28b}$$

Namely, the constrained control ensures that the time derivative of $V(\boldsymbol{x})$ is negative, while the objective function minimizes the deviation of $\boldsymbol{u}$ from the desired one, $\boldsymbol{u}_d(\boldsymbol{x})$. The above optimization is convex due to the convexity of $\mathcal{U}$ in (4.2) and the linear constraint in (4.28b). From a practical viewpoint, the role of $V(\boldsymbol{x})$ is to ensure stability. Meanwhile, choosing $\boldsymbol{u}_d(\boldsymbol{x})$ is also important for performance and it is typically selected as the locally optimal one or some stabilizing solutions of the unconstrained control of (4.1).

Remark 4.6. Suppose that there exists a DOA $\mathcal{D}_\kappa = \{\boldsymbol{x} : V(\boldsymbol{x}) \leq \kappa\}$ deduced from the generated CLF $V(\boldsymbol{x})$. It follows directly from the differential inequality (4.3) that the control (4.28) ensures the exponential convergence with the decay constant β inside $\mathcal{D}_\kappa$. Moreover, note that we can always ensure that such a κ exists if we insert among the basis function a locally optimal quadratic function with the standard LQ control.

Next, to demonstrate and highlight the applicability of the procedure, let us proceed with a case study of the quadcopter position control problem (a.k.a. outer loop or high level control) in the next section.

4.2.3 Application on quadcopter position control

In this part, the procedure presented in Section 4.2.2 will be applied to the quadcopter's linearized dynamics. For illustration and better choice of the basis function, the CLFs will be first generated for each axis of (3.11) in 2D. Then, based on the obtained result, a 6D CLF will be computed for (3.12).

4.2.3.1 Problem formulation for quadcopter position control

For the quadcopter model in (3.4), it was shown that the linearized system accepts two equivalent descriptions: a concatenation of three double integrators as in (3.11) and a 6D linear system as in (3.12). In what follows, we first generate three CLFs for each double integrator in (3.11). Then, these successfully validated functions will pave the way for the generation of the full 6D dynamics (3.12). More specifically, we consider the two following setups.

Case 1: Three double integrators with the constraint set $\mathcal{B}_v$ as in (3.26):

In this case, we examine the linearized dynamics of the quadcopter under the form of three double integrators as in (3.11) with independent constraints for each axis described by $\mathcal{B}_v$ in (3.26). In this manner, the CLF construction can be applied for each constrained subsystem in (3.11), i.e.,:

$$\dot{\mathbf{p}}_i = \boldsymbol{A}_{2D}\mathbf{p}_i + \boldsymbol{B}_{2D}v_i, \text{ s.t } |v_i| \leq \bar{v}_i \text{ as in (3.26)}, i \in \{1,2,3\}. \tag{4.29}$$

Case 2: Six-dimensional dynamics and a polytopic input constraint:

Less conservative than $\mathcal{B}_v$ in (3.26), the tight inner approximation $\mathcal{V}_c^{\text{in}}$ as in (3.22) will be considered with the 6D dynamics in (3.12), i.e.,:

$$\dot{\boldsymbol{\zeta}} = \boldsymbol{A}_{6D}\boldsymbol{\zeta} + \boldsymbol{B}_{6D}\boldsymbol{v}, \text{ s.t } \boldsymbol{v} \in \mathcal{V}_c^{\text{in}} \text{ as in (3.22)}. \tag{4.30}$$

Remark 4.7. It is undeniable that Algorithm 4.1 can be immediately applied for the system (3.12) in Case 2. However, we believe that exploiting the decoupled representation in Case 1, i.e. examining each double integrator at a time, provides not only illustrative understanding but also fundamental insights for the choice of the basis functions $V_i(\cdot)$ as in (4.6). Thus, in the sequel, we proceed by investigating first the decoupled representation in Case 1 (or (3.11)), then the procedure will be extended to Case 2 with the concatenated dynamics (3.12). □

4.2.3.2 CLF computation

In this part, the formulation of the CLFs for (3.11) and (3.12) subject to (3.26) and (3.22), respectively, will be discussed. With these functions come two ways of controlling the drone, as the following. In the former, three CLFs will be generated for each axis $i \in \{1,2,3\}$. These functions then will be separately combined with the setup (4.28) to compute the controller v_i. In the latter, one CLF will be generated for the concatenated dynamics (3.12) with the polytopic constraint (3.22). Then, the controller $\boldsymbol{v} = [v_1, v_2, v_3]^\top$ will be computed similarly with (4.28) and the generated function. In both cases, the controller $\boldsymbol{v}$ will be transformed back to the inputs T, ϕ, θ by means of (3.7), then applied to the vehicle.

For Case 1: The idea is to apply Algorithm 4.1 to three subsystems in (3.11) with $|v_i| \leq \bar{v}_i, i \in \{1,2,3\}$ and to create one CLF for each double integrator. For simplicity, we use $V^i(\cdot)$ to denote the generated CLF for the i axis (e.g., $V^1(\cdot) : \mathbb{R}^2 \to \mathbb{R}$ is the CLF for the first double integrator in (3.11), or $i = 1$). Then, the same procedure is carried out for all three subsystems as follows.

First, the general admissible pair generator (4.27) will be adapted with the weighting as in TABLE 4.1. The corresponding terminal cost adopts the weighting $\boldsymbol{P}_{2D}$ from the discrete Riccati equation for the EAS of (3.11) ($\tau = 0.1$s) with the penalty weighting $\boldsymbol{Q}_{2D}, \boldsymbol{R}_{2D}$, while the local controller uses the standard infinite-horizon unconstrained control. The terminal region $\mathcal{X}_f$ is the maximal positive invariant (MPI) set computed with such a local control and input constraint $|v_i| \leq \bar{v}_i$ for each subsystem. Numerical details are given as $\boldsymbol{Q}_{2D} = \left[\begin{smallmatrix} 50 & 0 \\ 0 & 5 \end{smallmatrix}\right], \boldsymbol{R}_{2D} = 5$, $\boldsymbol{P}_{2D} = \left[\begin{smallmatrix} 479.6118 & 181.0469 \\ 181.0469 & 155.5598 \end{smallmatrix}\right]$, $\boldsymbol{K}_{2D} = -[2.7617\ 2.6491]$. The domain of interest for each double integrator in (3.11) is chosen as $\mathcal{X}_i = \{|\mathbf{p}_i| \leq 1.5\}, i \in \{1,2,3\}$.

TABLE 4.1: Parameters for point generator (4.27) with $\tau = 0.1$ (s), and the LP (4.6), $q \in \{x, y, z\}$, $N_p = 50$.

	$\boldsymbol{x}, \boldsymbol{u}$ in (4.1)	$\boldsymbol{Q}$	$\boldsymbol{R}$	$\boldsymbol{P}$	$\mathcal{X}$,$\mathcal{U}$ in (4.27)	β	ℓ_i
Case 1	$\mathbf{p}_1, v_1$	$\boldsymbol{Q}_{2D}$	$\boldsymbol{R}_{2D}$	$\boldsymbol{P}_{2D}$	$\lvert\mathbf{p}_i\rvert \leq 1.5$, $\lvert v_i\rvert \leq \bar{v}_i$ in (3.26)	0.1275	$\ell_1 = 0.05$ $\ell_{q\neq 1} = 0.1$
	$\mathbf{p}_2, v_2$					0.1275	
	$\mathbf{p}_3, v_3$					2.65	
Case 2	$\boldsymbol{\zeta}, \boldsymbol{v}$	$\boldsymbol{Q}_{6D}$	$\boldsymbol{R}_{6D}$	$\boldsymbol{P}_{6D}$	$\lvert\boldsymbol{\zeta}\rvert \leq 1.5$, $\boldsymbol{v} \in \mathcal{V}_c^{\text{in}}$ in (3.22)	0.15	$\ell_1 = 0.2$ $\ell_{q\neq 1} = 1$

The next step resides on how to choose a collection basis functions $V_q(\cdot)$ as in (4.5). On one hand, encoding the optimality, at least locally near the origin, is important. For this reason, we select the first basis as the quadratic function:

$$V_1(\boldsymbol{p}) = \|\boldsymbol{p}\|^2_{\boldsymbol{P}^{lqr}_{2D}}, \boldsymbol{p} \in \mathbb{R}^2, \tag{4.31}$$

where $\boldsymbol{P}_{2D}^{lqr}$ denotes the result of the continuous Riccati equation, with the state and input penalty abusively taken from $\boldsymbol{Q}_{2D}$ and $\boldsymbol{R}_{2D}$. On the other hand, it is necessary that the basis functions are sufficiently rich so that (4.6) is feasible. At the same time, it was proposed that a suitable Lyapunov function for constrained stabilization is the $2p$-norm $\|\boldsymbol{Gp}\|_{2p}^{2p}$ where for $\boldsymbol{p} \in \mathbb{R}^n$, $\boldsymbol{G} \in \mathbb{R}^{n \times r}$ is a full row rank matrix [Blanchini and Miani, 1999a]. Thus, the remaining members of the collection of $V_q(\boldsymbol{p})$ are particularly populated with the following polynomials:

$$V_{2 \leq q \leq 100}(\boldsymbol{p}) \in \left\{ \|\eta \boldsymbol{F} \boldsymbol{p}\|_4^4, \eta \in \mathcal{N}(33, 0.4, 2.0), \boldsymbol{F} \in \left\{ \begin{bmatrix} 1 & 0 \end{bmatrix}, \begin{bmatrix} 0 & 1 \end{bmatrix}, \begin{bmatrix} 1 & 1 \end{bmatrix} \right\} \right\}. \quad (4.32)$$

Remark 4.8. In (4.32), η plays a role in increasingly enriching the basis with different value scales, while $\boldsymbol{F}$ is responsible for the "orientation" of the surfaces $V_q(\cdot)$. The 2D case of the first double integrator is depicted in Figure 4.2. The feasibility of (4.6) can be managed by meticulously collecting more value of η and vector $\boldsymbol{F}$. Note that the first two rows of $\boldsymbol{F}$ are associated with the constraints on the position and the velocity, while the third row is associated with a mixed position speed constraint. □

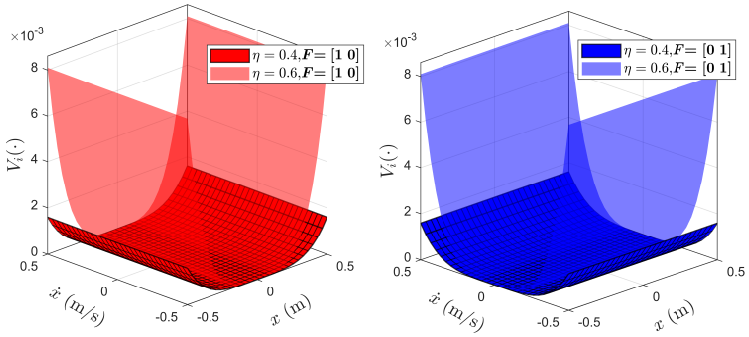


FIGURE 4.2: The surface of $V_i(\cdot)$ with different η and $\boldsymbol{F}$ (Case 1).

Then, by partitioning the domain $\mathcal{X}_i$ by a 25×25 grid and collecting the control v_i given by (4.27) at those points (See Figure 4.5), we achieved the three following CLFs from (4.6) with the basis (4.31) and (4.32) after applying our procedure and solving the LP problem:

- For the x, y axis or $i \in \{1, 2\}$ in (3.11):

$$V^i(\mathbf{p}_i) = 0.016\|\mathbf{p}_i\|_{\boldsymbol{P}_{2D}^{lqr}}^2 + 0.2293(2[1\ 0]\mathbf{p}_i)^4 + 0.5406(2[0\ 1]\mathbf{p}_i)^4 + 0.2141(2[1\ 1]\mathbf{p}_i)^4. \quad (4.33)$$

- For the z axis or $i = 3$ in (3.11):

$$V^3(\mathbf{p}_3) = 0.0119\|\mathbf{p}_3\|_{\boldsymbol{P}_{2D}^{lqr}}^2 + 0.3862(2[1\ 0]\mathbf{p}_3)^4 + 0.6019(2[1\ 1]\mathbf{p}_3)^4. \quad (4.34)$$

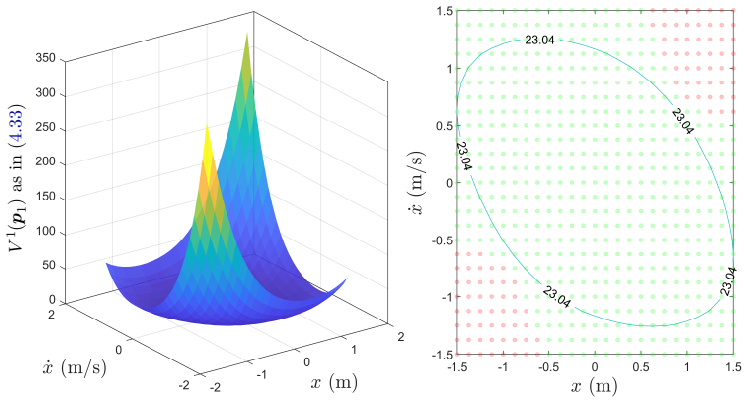


FIGURE 4.3: Generated CLF and its DOA for the first subsystem of (3.11) ($i = 1$).

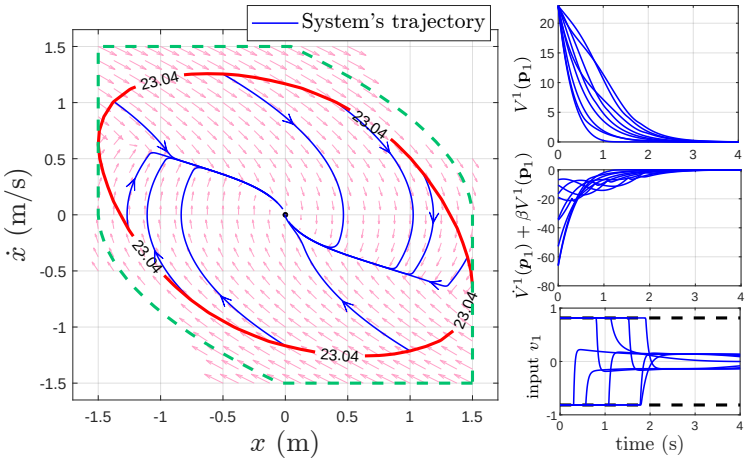


FIGURE 4.4: Set-point tracking simulation with the first subsystem of (3.11) ($q = x$).

The computation of α_q in (4.6) is carried out with MATLAB 2021b and its *linprog* solver, while the pair generator (4.27) and the online controller (4.28) is computed with *quadprog* solver for the simulation. The illustration for (4.33) is in Figure 4.3. The DOA can be estimated by verifying the state-space with the condition (4.10), which, in this case, is given as the sublevel set $V^1(\mathbf{p}_1) \leq 23.04$. The positive invariance of the DOA can also be confirmed via the simulated trajectories and vector field depicted in Figure 4.4. Therein, the green-dashed line represents the DOA provided by the MPC point generator (4.27) while the red solid curve depicts the sublevel set DOA given by the generated CLF. It can also be seen again that there always exists $|v_1| \leq \bar{v}_1$ rendering $V^1(\mathbf{p}_1)$ decreasing and $\dot{V}^1(\mathbf{p}_1) + \beta V^1(\mathbf{p}_1) \leq 0$. A comparison between the control policy of the MPC (4.27) and (4.28) is given in Figure 4.5 .

For Case 2: Similarly, we use the setting (4.27) as the point generator with the concatenated state $\boldsymbol{\zeta}$ as in (3.12) and the parameters computed as $\boldsymbol{Q}_{6D} = \text{diag}(\boldsymbol{Q}_{2D}, \boldsymbol{Q}_{2D}, \boldsymbol{Q}_{2D})$,

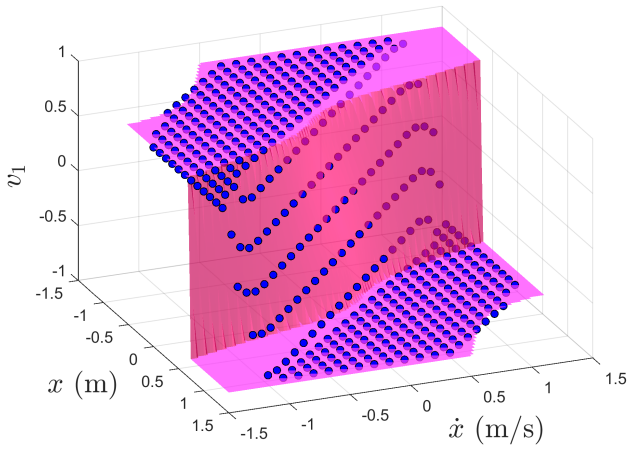


FIGURE 4.5: Comparison between the control sampled from (4.27) (blue points) and the control (4.28) (pink surface) for Case 1, $i = 1$ or the x axis.

$\boldsymbol{R}_{6D} = \text{diag}(\boldsymbol{R}_{2D}, \boldsymbol{R}_{2D}, \boldsymbol{R}_{2D})$, $\boldsymbol{P}_{6D} = \text{diag}(\boldsymbol{P}_{2D}, \boldsymbol{P}_{2D}, \boldsymbol{P}_{2D})$ and consequently, $\boldsymbol{K}_{6D} = \text{diag}(\boldsymbol{K}_{2D}, \boldsymbol{K}_{2D}, \boldsymbol{K}_{2D})$. Parameters' details are given in TABLE 4.1.

Different from the previous case, for the admissible point collection, random initial conditions in $\{\boldsymbol{\zeta} \in \mathbb{R}^6 : |\boldsymbol{\zeta}| \leq 1.5\}$. The toolbox given in [Benham, 2011] is employed to provide a randomly distributed initial conditions. Then, for each feasible initial condition, we sample along its trajectory to collect state-control pairs since the recursive feasibility of (4.27) ensures the existence of admissible pairs propagated from that initial state. The projections of those states are provided in Figure 4.6.

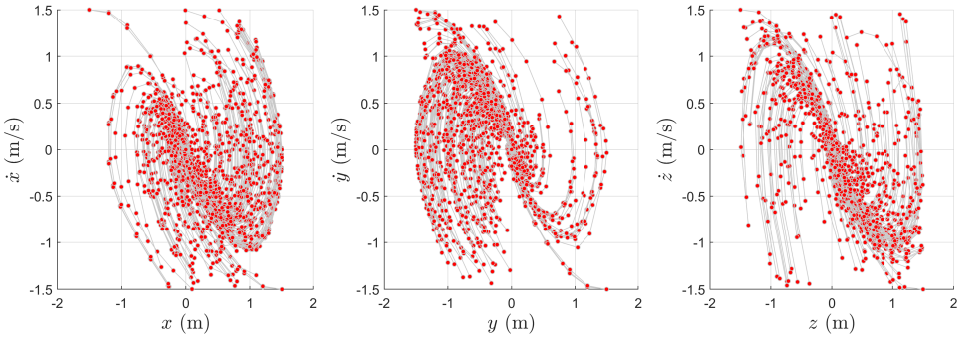


FIGURE 4.6: Admissible pairs (red) collected for Case 2.

Then the new set of basis functions for this setup is reselected as follows: The first basis function remains as the LQR-based quadratic function in the same choice from (4.31):

$$V_1(\boldsymbol{\zeta}) = \|\boldsymbol{\zeta}\|^2_{\boldsymbol{P}^{lqr}_{6D}}, \boldsymbol{\zeta} \in \mathbb{R}^6. \tag{4.35}$$

TABLE 4.2: Specifications of the LP (4.6) for the two cases

	Case 1			Case 2
	$V^1(\mathbf{p}_1)$	$V^2(\mathbf{p}_2)$	$V^3(\mathbf{p}_3)$	$V(\boldsymbol{\zeta})$
No. basis functions N	100	100	100	10
No. admissible pairs M	625	625	625	6330
Computation time of (4.6)	0.03 (s)	0.03 (s)	0.05 (s)	0.11 (s)

Moreover, taking advantage of the successfully generated CLFs for each subspace in (4.33) and (4.34), we adopt their basis by augmenting the vector $\boldsymbol{F}$ in (4.32), for $i \in \{1, 2, 3\}$, with the extended value assigned as 0. For example, for $i = 1$ or the x axis, [1 1] becomes [1 1 0 0 0 0].

For the tuning of ℓ_q, as suggested before, it is essential to include the LQR-based quadratic Lyapunov function in the parameterized CLF in order to attain local optimality. Therefore, in both cases, the weighting ℓ_1 associated with the function is progressively decreased so that its corresponding coefficient computed from (4.6) is non-zero.

With the pairs sampled and the choice of basis functions given, the LP (4.6) provides the CLF as:

$$\begin{aligned} V(\boldsymbol{\zeta}) &= 0.0244\|\boldsymbol{\zeta}\|^2_{\boldsymbol{P}_{6D}^{lqr}} + 0.4508([0\,1\,0\,0\,0\,0]\boldsymbol{\zeta})^4 + 0.0674([1\,1\,0\,0\,0\,0]\boldsymbol{\zeta})^4 \\ &+ 0.1631([0\,0\,1\,0\,0\,0]\boldsymbol{\zeta})^4 + 0.1841([0\,0\,1\,1\,0\,0]\boldsymbol{\zeta})^4 + 0.1103([0\,0\,0\,0\,0\,1]\boldsymbol{\zeta})^4, \end{aligned} \tag{4.36}$$

with $V(\boldsymbol{\zeta})$ denoting the generated Lyapunov function for the six-dimensional system (3.12). The computational details of the procedure are provided in TABLE 4.2. Therein, as expected, it is noticeable that the computation time for the LP (4.6) is relatively small in both cases, confirming the low complexity of the technique.

To this end, the CLFs were generated for the control problem and examined via simulation tests. In the next section, we verify the constructed CLFs via experimental validation, showing their applicability in practice.

4.2.4 Experimental validation

Herein, the results from the previous section are validated via experimental tests over the nano-drone platform presented in Section 3.2.3.

4.2.4.1 Experiment setup

With the platform presented in Section 3.2.3, the experimental setup is given as the following. The sampling time is fixed at $t_s = 100$ ms. For all the optimization problems employed in the experiments, the IPOPT solver is used within the Casadi optimization framework in Python 3.8.8, Intel Core, i5-10300H CPU @ 2.50GHz and 16GB RAM. The experiment video can be found at https://youtu.be/XPyZAJoN2Ps. The description of the test cases is given as follows.

On one hand, we use the controller (4.28) with the two setups *Case 1* and *Case 2* in the previous section with the same tuning given in TABLE 4.1. The desired control $\boldsymbol{u}_d(\cdot)$ is chosen as $\boldsymbol{u}_d(\mathbf{p}_i) = -[k_p,\, k_d]\mathbf{p}_i$ and $\boldsymbol{u}_d(\boldsymbol{\zeta}) = -\text{diag}([k_p,\, k_d], [k_p,\, k_d], [k_p,\, k_d])\boldsymbol{\zeta}$ for Case 1 and 2, respectively, and $k_p = 1, k_d = 1.45$. Then, for comparison, we stabilize the tracking error dynamics by employing the following optimization problem in a receding horizon fashion.

$$\min \sum_{k=0}^{N_p^*} \left(\|\boldsymbol{\zeta}(k) - \boldsymbol{\zeta}^{\text{ref}}(k)\|^2_{\boldsymbol{Q}_{6D}} + \|\boldsymbol{v}(k)\|^2_{\boldsymbol{R}_{6D}} \right) \|\boldsymbol{\zeta}(N_p^*) - \boldsymbol{\zeta}^{\text{ref}}(N_p^*)\|^2_{\boldsymbol{P}_{6D}} \tag{4.37a}$$

$$\text{s.t} \begin{cases} \boldsymbol{\zeta}(k+1) = (\boldsymbol{A}_{6D} + t_s \boldsymbol{I})\boldsymbol{\zeta}(k) + t_s \boldsymbol{B}_{6D}\boldsymbol{v}(k), \\ \boldsymbol{v}(k) \in \mathcal{V}_c^{\text{in}} \text{ as in (3.22)}, \\ \boldsymbol{\zeta}(k) - \boldsymbol{\zeta}^{\text{ref}}(k) \in \mathcal{Z}, \boldsymbol{\zeta}(N_p^*) - \boldsymbol{\zeta}^{\text{ref}}(N_p^*) \in \mathcal{Z}_f, \end{cases} \tag{4.37b}$$

where $\boldsymbol{A}_{6D}, \boldsymbol{B}_{6D}, \boldsymbol{P}_{6D}, \boldsymbol{Q}_{6D}, \boldsymbol{R}_{6D}$ are given in Section 4.2.3.1, $\boldsymbol{\zeta}^{\text{ref}}(k)$ denotes the reference for $\boldsymbol{\zeta}(k)$ to follow. $\mathcal{Z} = \{\boldsymbol{\zeta} : \|\boldsymbol{\zeta}\|_\infty \leq 1.5\}$ is the state constraint corresponding to that of the admissible pair generator in Case 1 and 2. $\mathcal{Z}_f$ is the MPI set (as the terminal constraint) associated with the full-state gain $\boldsymbol{K}_{6D}$ given therein. For our setup, the largest prediction horizon N_p^* for (4.37) is 20, so that the online computation time does not exceed the sampling time t_s.

On the other hand, for the position reference, the examined controllers will first track a set-point $x^{\text{ref}} = 0.6, y^{\text{ref}} = 0.6, z^{\text{ref}} = 0.8$ (m), noted as Scenario 1 (Sce. 1). Then, to further emphasize the computational advantage, we apply the same control laws for three Crazyflies in a centralized manner. The control then will be sent to each drone via the `Swarm` Python interface [Bitcraze, 2023]. With this Scenario 2 (Sce. 2), a position swapping reference was given to the three drones, which is depicted Fig. 4.9.

4.2.4.2 Experimental results and discussions

The result of the first scenario (Sce. 1) is shown in Fig. 4.7. It can be seen that, the proposed schemes' tracking performances (Case 1 and 2) are commensurate with the MPC while respecting all the input constraints in (3.5). However, it is noted from TABLE 4.3 and Fig. 4.11 that the computation time (CT) of the proposed laws is 5

to 7 times lower than that of the MPC (4.37). This advantage directly comes from the simple QP formulation of (4.28) compared to (4.37) with regard to the complexity of both the cost and constraints.

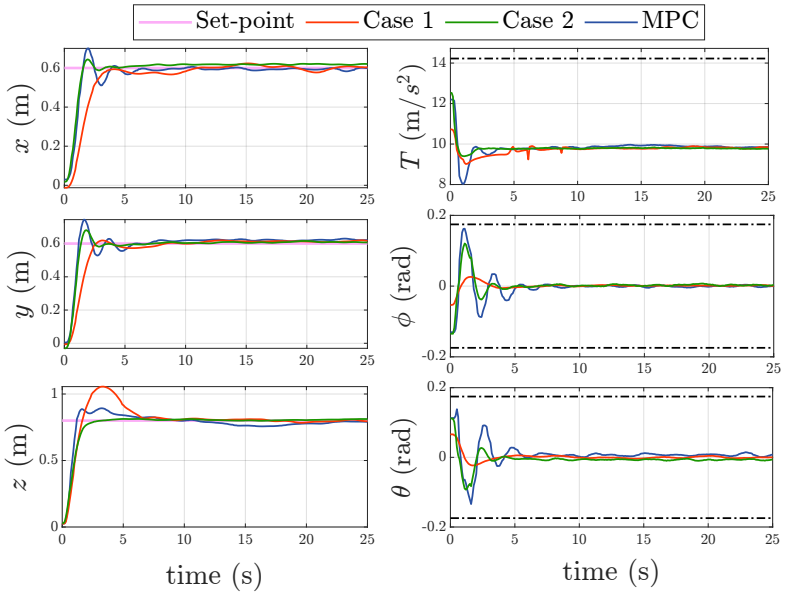


FIGURE 4.7: Left: Set-point reference tracking experimented with the three scenarios. Right: The real input signals and their amplitude bound (dashed-dotted line).

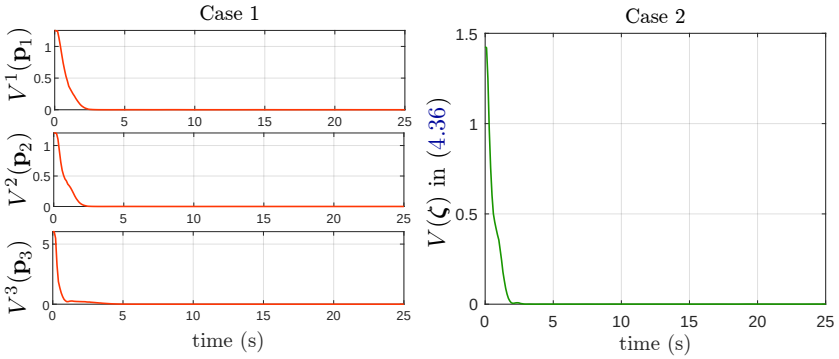


FIGURE 4.8: Lyapunov function for Case 1 and 2 controllers in Scenario 1.

In Sce. 2, the proposed schemes are also successfully validated. It is noticeable that in this scenario, a centralized MPC for three quadcopters could not be performed since the required calculation time exceeds the sampling time t_s. Meanwhile, even with three quadcopters, the tracking was possible for both Case 1 and 2 with the Avg. CT around 20ms. Fig. 4.8 depicts the value of the CLF in both cases and their monotonicity. This shows that the discussed technique generates effective CLF and is applicable in real-life implementation.

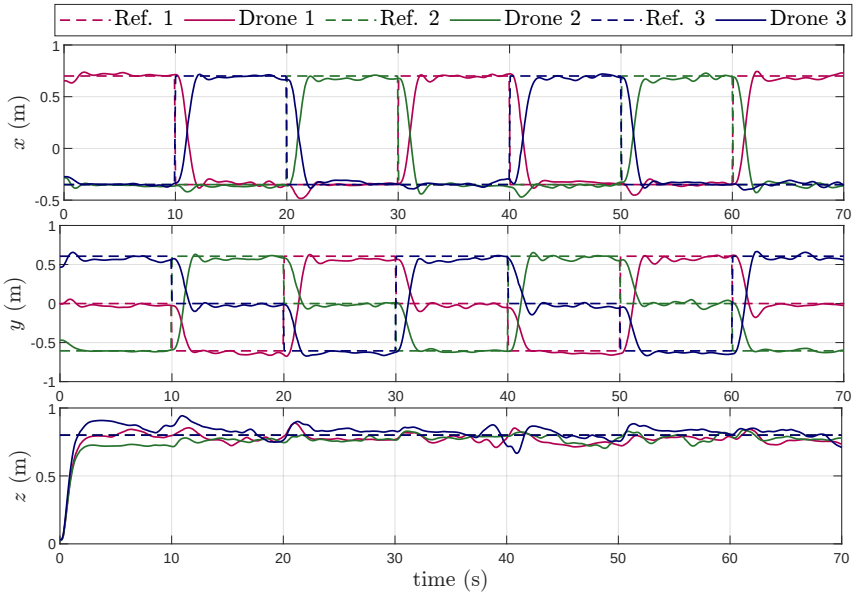


FIGURE 4.9: Position swapping with three Crazyflies using Case 2 controller.

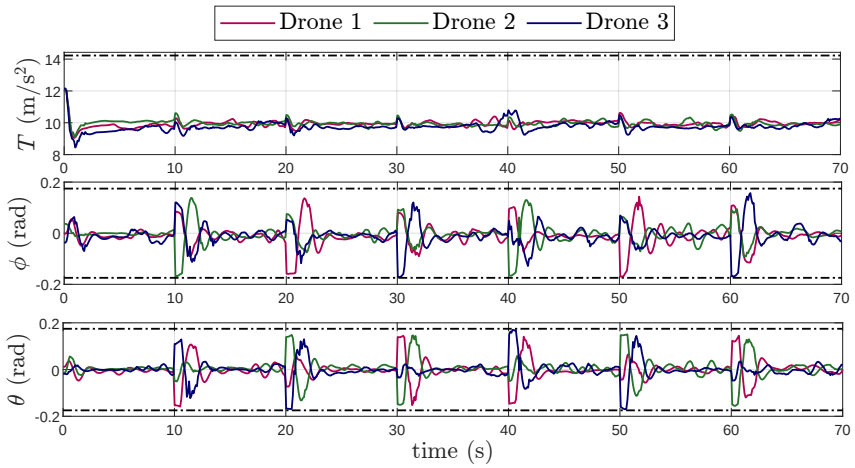


FIGURE 4.10: The outer loop's input signals of the three quadcopters in Sce. 2 with the Case 2 controller.

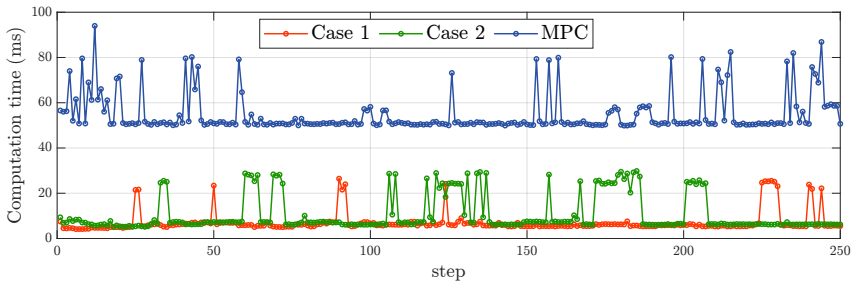


FIGURE 4.11: Computation time in Scenario 1.

TABLE 4.3: Experiment results

	Controller	RMS tracking error (cm)			Avg. CT
Sce. 1	Case 1	14.11			7.02 ms
	Case 2	11.39			10.74 ms
	MPC	11.48			54.70 ms
Sce. 2		Drone 1	Drone 2	Drone 3	
	Case 1	24.09	22.20	21.33	24.09 ms
	Case 2	19.14	18.57	18.71	20.51 ms
	MPC	×	×	×	×

With the experimental results, the generated non-quadratic CLF can be employed to reduce the online computational demand while maintaining stability. Yet, the next question is to analyze the performance trade-off to obtain such a complexity reduction. Furthermore, although experimentally validated, the theoretical guarantee of Proposition 4.2 is only valid discretely at the sampled admissible pair. Thus, the analysis of the continuous state space is left open. With these questions in mind, let us extend the presented results by formalizing a theoretical framework to further investigate the idea of reducing complexity via CLF, at the expense of suboptimal performance.

4.3 Real-time complexity reduction with sub-control Lyapunov functions

Herein, the framework of generating a CLF via linear programming is extended to formalize the theoretical contributions presented previously. Herein, the generated CLF will be called the sub-control Lyapunov function (SCLF), marking the performance suboptimality gap between the computationally heavy baseline controller and the SCLF-based one.

4.3.1 Sub-Control Lyapunov Functions

The context to encrypt the stability and optimality of the baseline controller into the SCLF will be specified in this part. Then, the definition and properties of SCLF will follow.

4.3.1.1 Definition and basic properties

Consider again the constrained control problem (4.1). We call a CLF, associated with the control $\boldsymbol{\omega}(\boldsymbol{x}) : \mathbb{R}^n \to \mathcal{U}$, a locally Lipschitz positive definite function $\Psi(\boldsymbol{x}) : \mathbb{R}^n \to \mathbb{R}$ with the property:

$$D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = 0 \tag{4.38}$$

where $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ locally Lipschitz and positive for non-zero $(\boldsymbol{x}, \boldsymbol{u})$, and where we have denoted by

$$D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = \lim_{h \to 0^+} \sup \frac{\Psi(\boldsymbol{x} + h(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x}))) - \Psi(\boldsymbol{x})}{h}, \tag{4.39}$$

the generalized directional derivative. In the smooth case of $\Psi(\boldsymbol{x})$, one clearly has:

$$D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = \nabla\Psi(\boldsymbol{x}) \left(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x})\right). \tag{4.40}$$

We consider the generalized derivative, since $\Psi(\boldsymbol{x})$ could be non-differentiable.

Note that function $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ having the property (4.38) can be associated with the performance index:

$$J(\boldsymbol{x}(0), \boldsymbol{u}(\cdot)) = \int_0^\infty \mathfrak{g}(\boldsymbol{x}(t), \boldsymbol{u}(t))\mathrm{d}t, \tag{4.41}$$

while $\Psi(\boldsymbol{x})$ is the cost–to–go function, solution of the Hamilton-Jacobi-Bellman equation:

$$\min_{u \in \mathcal{U}} \left[D^+\Psi(\boldsymbol{x}, \boldsymbol{u}) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})\right] = D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = 0, \tag{4.42}$$

where we have denoted by $\boldsymbol{\omega}(\boldsymbol{x})$ the minimizer control.

Finding $\Psi(\boldsymbol{x})$ and a an optimal control law $\boldsymbol{\omega}(\boldsymbol{x})$, often requires brute force computation via numerical approaches, moreover they are often unsuitable for online implementation. To arrive at an approximation scheme, we introduce the following definition.

Definition 4.4. *Given a polynomially bounded nominal control Lyapunov function (CLF) $\Psi(\boldsymbol{x})$ equipped with the control $\boldsymbol{\omega}(\boldsymbol{x})$ we say that the polynomially bounded positive definite function $V(\boldsymbol{x})$ is a global sub-control Lyapunov function (SCLF) of $\Psi(\boldsymbol{x})$, if*

$$D^+V(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})), \ \forall \boldsymbol{x}. \tag{4.43}$$

It is a local SCLF if the inequality holds for $V(\boldsymbol{x}) \leq \kappa$, for some $\kappa > 0$.

There is an important relation between $V(\boldsymbol{x})$ and $\Psi(\boldsymbol{x})$.

Proposition 4.6. *If $V(\boldsymbol{x})$ is a SCLF of $\Psi(\boldsymbol{x})$, then $V(\boldsymbol{x}) \geq \Psi(\boldsymbol{x})$.*

Proof. Let $\boldsymbol{x}_0$ be an initial condition and consider the converging trajectory $\boldsymbol{\chi}(t)$, such that $\boldsymbol{\chi}(0) = \boldsymbol{x}_0$, associated with the control $\boldsymbol{\omega}(\boldsymbol{x})$. By integrating, we get

$$\begin{cases} V(\boldsymbol{x}_0) - V(\boldsymbol{\chi}(\infty)) = -\int_0^\infty \; D^+V(\chi(t), \boldsymbol{\omega}(\boldsymbol{\chi}(t)))\mathrm{d}t \\ \Psi(\boldsymbol{x}_0) - \Psi(\boldsymbol{\chi}(\infty)) = -\int_0^\infty \; D^+\Psi(\chi(t), \boldsymbol{\omega}(\boldsymbol{\chi}(t)))\mathrm{d}t. \end{cases}$$

As the control is stabilizing, $\boldsymbol{\chi}(\infty) = 0$. Hence, $V(\boldsymbol{x}_0) \geq \Psi(\boldsymbol{x}_0)$. Since $\boldsymbol{x}_0$ is arbitrary, the claim follows. □

In the sequel, for practical implementation, we consider smooth candidate SCLF.

4.3.1.2 Suboptimal control laws from SCLFs

Now we show how low-cost controllers can be associated with a smooth SCLF (with a performance loss). As can be seen from (4.38), by integrating with $\boldsymbol{x}(0) = \boldsymbol{x}_0$, one can write the optimal performance as:

$$J(\boldsymbol{x}_0, \boldsymbol{\omega}(\boldsymbol{x})) = -\int_0^\infty D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = \Psi(\boldsymbol{x}_0), \tag{4.44}$$

If the control $\boldsymbol{u} = \boldsymbol{\omega}(\boldsymbol{x})$ is unsuitable for real time implementation, we could replace $\Psi(\boldsymbol{x})$ by a smooth SCLF $V(\boldsymbol{x})$ defined as in (4.43), and adopt the following control:

$$\bar{\boldsymbol{u}}(\boldsymbol{x}) = \arg\min_{\boldsymbol{u}\in\mathcal{U}} \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{u}). \tag{4.45}$$

It is fundamental to note that problem (4.45) is feasible because $\boldsymbol{u} = \boldsymbol{\omega}(\boldsymbol{x})$, the optimal control law value, is a feasible solution. The next theorem explains how this control performs.

Theorem 4.2. *Assume that the nominal $\Psi(\boldsymbol{x})$ satisfies (4.38). Consider a SCLF $V(\boldsymbol{x})$. Then control $\bar{\boldsymbol{u}}(\boldsymbol{x})$ as in (4.45) is stabilizing. Moreover, given the cost function $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ and the initial condition $\boldsymbol{x}(0) = \boldsymbol{x}_0$, the performance is upper bounded by $V(\boldsymbol{x}_0)$.*

Proof. We have that

$$\begin{aligned} &\dot{V}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) = \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\bar{\boldsymbol{u}}(\boldsymbol{x})] + \mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) \\ &\leq \underbrace{\nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x})]}_{D^+V(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))} + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = 0. \end{aligned} \tag{4.46}$$

The first inequality is due to the fact that $\bar{\boldsymbol{u}}(\boldsymbol{x})$ is the minimizer of the optimization problem (4.45), the second equality follows from the definition of a SCLF (4.43). Adopting the control $\bar{\boldsymbol{u}}(\boldsymbol{x})$, the time derivative of $V(\boldsymbol{x})$ is

$$\dot{V}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) = \nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\bar{\boldsymbol{u}}(\boldsymbol{x})] \leq -\mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})), \tag{4.47}$$

which implies asymptotic stability because $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ is positive definite, hence,

$$V(\infty) = \lim_{t \to \infty} V(\boldsymbol{x}(t)) = 0.$$

Then, integrating (4.47) leads to:

$$\int_0^\infty \mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x}))\mathrm{d}t \leq V(\boldsymbol{x}_0) - V(\infty) = V(\boldsymbol{x}_0). \tag{4.48}$$

□

The computational complexity of the optimization problem (4.45) (to be solved in real-time), is very low and we can even provide a simple explicit solution in the case of a quadratic cost $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ and a box-type set $\mathcal{U}$ in (4.1).

Proposition 4.7. *Consider a quadratic cost:*

$$\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u}) = \boldsymbol{x}^\top \boldsymbol{Q}\boldsymbol{x} + \boldsymbol{u}^\top \boldsymbol{R}\boldsymbol{u}, \tag{4.49}$$

with a positive definite diagonal matrix $\boldsymbol{R} = diag(r_1, r_2, ..., r_m)$ *and a constraint set:*

$$\mathcal{U} = \{\boldsymbol{u} \in \mathbb{R}^m : |\boldsymbol{u}| \leq \boldsymbol{u}_{max}\}. \tag{4.50}$$

Then, the minimizer of (4.45) *can be explicitly computed as:*

$$\bar{\boldsymbol{u}}(\boldsymbol{x}) = \underset{|\boldsymbol{u}| \leq \boldsymbol{u}_{max}}{sat} \left(-\tfrac{1}{2}\boldsymbol{R}^{-1}\mathbf{g}(\boldsymbol{x})^\top \nabla V(\boldsymbol{x})^\top\right) \tag{4.51}$$

Proof. Let $\mathfrak{z} = \left(\frac{1}{2}\nabla V(\boldsymbol{x})\mathbf{g}(\boldsymbol{x})\right)^\top$ and $\mathfrak{y} = \boldsymbol{R}^{-1}\mathfrak{z}$, then the problem (4.45) can be rewritten as:

$$\begin{aligned} \bar{\boldsymbol{u}}(\boldsymbol{x}) &= \arg\min_{u \in \mathcal{U}} \; 2\mathfrak{z}^\top \boldsymbol{u} + \boldsymbol{u}^\top \boldsymbol{R}\boldsymbol{u}, \\ &= \arg\min_{\boldsymbol{u} \in \mathcal{U}} \; (\boldsymbol{u} + \mathfrak{y})^\top \boldsymbol{R}(\boldsymbol{u} + \mathfrak{y}) - \mathfrak{y}^\top \boldsymbol{R}\mathfrak{y}. \end{aligned} \tag{4.52}$$

With the set $\mathcal{U}$ in (4.50), (4.52) is decoupled

$$\bar{\boldsymbol{u}}(\boldsymbol{x}) = \underset{|\boldsymbol{u}| \leq \boldsymbol{u}_{max}}{\arg\min} \sum_{i=1}^{m} r_i(u_i + \mathfrak{y}_i)^2. \tag{4.53}$$

For each row $\bar{u}_i(\boldsymbol{x})$ of $\bar{\boldsymbol{u}}(\boldsymbol{x})$, the minimizer of (4.53) is: $\bar{u}_i(\boldsymbol{x}) = \underset{|u_i|\leq u_{i,max}}{\text{sat}}(-\mathfrak{y}_i)$, or equivalently, (4.51). □

For non-diagonal positive definite $\boldsymbol{R}$, Proposition 4.7 does not hold. Still, one obtains a solution for the optimization problem as follows.

Proposition 4.8. *Let $\boldsymbol{u} \in \mathcal{U}$ a polytope. Then problem (9) with cost (12), reduces to a minimum distance problem.*

Proof. Denoting by $\mathfrak{z} = \frac{1}{2}\left(\nabla V(\boldsymbol{x})\mathbf{g}(\boldsymbol{x})\right)^\top$, $\bar{\boldsymbol{u}}(\boldsymbol{x})$ is the minimizer of

$$\bar{\boldsymbol{u}}(\boldsymbol{x}) = \arg\min_{\boldsymbol{u}\in\mathcal{U}} 2\mathfrak{z}^\top \boldsymbol{u} + \boldsymbol{u}^\top \boldsymbol{R}\boldsymbol{u}$$

Let $\mathfrak{v} = \boldsymbol{R}^{\frac{1}{2}}\boldsymbol{u}$ with $\mathfrak{v} \in \mathcal{V}_\mathfrak{v} \triangleq \boldsymbol{R}^{\frac{1}{2}}\mathcal{U}$, the problem is equivalent, after completing the squares, to:

$$\begin{aligned}
&\min_{\mathfrak{v}\in\mathcal{V}_\mathfrak{v}} \left\{-\mathfrak{z}^\top \boldsymbol{R}^{-1}\mathfrak{z} + \mathfrak{z}^\top \boldsymbol{R}^{-1}\mathfrak{z} + 2\mathfrak{z}^\top \boldsymbol{R}^{-\frac{1}{2}}\mathfrak{v} + \mathfrak{v}^\top\mathfrak{v}\right\} \\
&= \min_{\mathfrak{v}\in\mathcal{V}_\mathfrak{v}} \left\{-\mathfrak{z}^\top \boldsymbol{R}^{-1}\mathfrak{z} + \left(\boldsymbol{R}^{-\frac{1}{2}}\mathfrak{z} + \mathfrak{v}\right)^\top \left(\boldsymbol{R}^{-\frac{1}{2}}\mathfrak{z} + \mathfrak{v})^\top\right)\right\} \\
&= \min_{\mathfrak{v}\in\mathcal{V}_\mathfrak{v}} \left\{-\mathfrak{z}^\top \boldsymbol{R}^{-1}\mathfrak{z} + \|\boldsymbol{R}^{-\frac{1}{2}}\mathfrak{z} + \mathfrak{v}\|^2\right\}.
\end{aligned}$$

So $\mathfrak{v} = \boldsymbol{R}^{\frac{1}{2}}\boldsymbol{u}$ is the point in $\mathcal{V}_\mathfrak{v}$ closest to $-\boldsymbol{R}^{-\frac{1}{2}}\mathfrak{z}$. □

Remark 4.9. Solving the minimum distance problem is straightforward. Moreover, if $\mathfrak{z}(\boldsymbol{x}) = \frac{1}{2}\left(\nabla V(\boldsymbol{x})\mathbf{g}(\boldsymbol{x})\right)^\top$ is continuous, then the minimizer function $\bar{\boldsymbol{u}}(\boldsymbol{x})$ is continuous as well being the so called minimal selection [Aubin and Cellina, 1984].

An essential property of a SCLF is that it admits, among other controllers the true optimal $\boldsymbol{\omega}(\boldsymbol{x})$ beside the simpler $\bar{\boldsymbol{u}}(\boldsymbol{x})$ discussed above. We expect that $\bar{\boldsymbol{u}}(\boldsymbol{x})$ is close to $\boldsymbol{\omega}(\boldsymbol{x})$. We argue that we can achieve optimality close to the origin. Indeed, assume a cost of the form (4.49) and the set of all control actions that ensure (4.47), or:

$$\nabla V(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] \leq -\boldsymbol{x}^\top Q\boldsymbol{x} - \boldsymbol{u}^\top \boldsymbol{R}\boldsymbol{u}, \boldsymbol{u} \in \mathcal{U}. \tag{4.54}$$

Assume that, by means of linearization, we have determined an optimal desired gain $\boldsymbol{u} = \boldsymbol{K}\boldsymbol{x}$, the unconstrained locally optimal, so that $\boldsymbol{\omega}(\boldsymbol{x}) \approx \boldsymbol{K}\boldsymbol{x}$ close to $\mathbf{0}$. To see this, we parameterize the control as $\boldsymbol{u} = \boldsymbol{K}\boldsymbol{x} + \mathfrak{v} \in \mathcal{U}$, complete the square and rewrite (4.54) as:

$$\boldsymbol{K}\boldsymbol{x}+\mathfrak{v}\in\mathcal{U}:\left\|\boldsymbol{R}^{\frac{1}{2}}\left[\mathfrak{v}+\boldsymbol{K}\boldsymbol{x}+\frac{1}{2}\boldsymbol{R}^{-1}\mathbf{g}(\boldsymbol{x})^{\top}\nabla V(\boldsymbol{x})^{\top}\right]\right\|^{2}$$
$$\leq-\boldsymbol{x}^{\top}\boldsymbol{Q}\boldsymbol{x}-\nabla V(\boldsymbol{x})\mathbf{f}(\boldsymbol{x})+\left\|\frac{1}{2}\boldsymbol{R}^{-\frac{1}{2}}\mathbf{g}(\boldsymbol{x})^{\top}\nabla V(\boldsymbol{x})^{\top}\right\|^{2}. \tag{4.55}$$

By construction, (4.54) and (4.55) are always feasible. Hence, we can compute the control as

$$\min\|\mathfrak{v}\|\ \text{s.t.}\quad \boldsymbol{K}\boldsymbol{x}+\mathfrak{v}\in\mathcal{U},\ \text{and (4.55)} \tag{4.56}$$

which is, again, a straightforward minimum norm problem with quadratic and linear constraints. Since we assume that $\boldsymbol{\omega}(\boldsymbol{x})\approx\boldsymbol{K}\boldsymbol{x}$, and that $\boldsymbol{\omega}(\boldsymbol{x})$ is a feasible control, in a neighborhood of 0 we have the minimizer $\mathfrak{v}=0$ so that this new control will be virtually identical to $\boldsymbol{\omega}(\boldsymbol{x})$.

4.3.1.3 Performance loss evaluation

Replacing $\Psi(\boldsymbol{x})$ by a SCLF $V(\boldsymbol{x})\geq\Psi(\boldsymbol{x})$ (by Proposition 4.6) entails a performance loss which can be quantified by the index:

$$\text{loss}=\frac{V(\boldsymbol{x}_0)}{\Psi(\boldsymbol{x}_0)}\geq 1. \tag{4.57}$$

Computing this index requires determining $\Psi(\boldsymbol{x})$ and $\boldsymbol{\omega}(\boldsymbol{x})$, to impose the condition $D^{+}V(\boldsymbol{x},\boldsymbol{\omega}(\boldsymbol{x}))\leq D^{+}\Psi(\boldsymbol{x},\boldsymbol{\omega}(\boldsymbol{x}))=-\mathfrak{g}(\boldsymbol{x},\boldsymbol{\omega}(\boldsymbol{x}))$ to find $V(\boldsymbol{x})$ as we will see later. In practice, computing $\Psi(\boldsymbol{x})$, $D^{+}\Psi(\boldsymbol{x},\boldsymbol{\omega}(\boldsymbol{x}))$ explicitly and solving the HJB optimality equation are affected by numerical and even non–differentiability issues [Bardi and Capuzzo Dolcetta, 1997]. Instead of solving the HJB equations, we can evaluate $\Psi(\boldsymbol{x})$ and $\boldsymbol{\omega}(\boldsymbol{x})$ in (4.38) implicitly, by considering the problem:

$$\Psi(\boldsymbol{x}_0)=\Psi_{\infty}(\boldsymbol{x}_0)=\inf_{\boldsymbol{u}(\cdot)\in\mathcal{U}}\int_0^{\infty}\mathfrak{g}(\boldsymbol{x}(\tau),\boldsymbol{u}(\tau))\mathtt{d}\tau \tag{4.58a}$$
$$\dot{\boldsymbol{x}}=\mathbf{f}(\boldsymbol{x})+\mathbf{g}(\boldsymbol{x})\boldsymbol{u}, \tag{4.58b}$$
$$\boldsymbol{x}(0)=\boldsymbol{x}_0. \tag{4.58c}$$

The control $\boldsymbol{\omega}(\boldsymbol{x}_0)$ is the initial control value of the open-loop problem (4.58a)–(4.58c). Solving the infinite horizon problem (4.58a)–(4.58c) exactly may be difficult and, in

practice, one has to approximate it by a finite (large) horizon problem [Alamir, 2006]:

$$\Psi_{\mathtt{T}}(\boldsymbol{x}_0) = \min_{\boldsymbol{u}(\cdot)} \int_0^{\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\tau), \boldsymbol{u}(\tau))\mathrm{d}\tau + \Phi_{\mathtt{T}}(\boldsymbol{x}(\mathtt{T})), \tag{4.59a}$$

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}, \tag{4.59b}$$

$$\boldsymbol{u}(\tau) \in \mathcal{U}, \tau \in [0, \mathtt{T}], \tag{4.59c}$$

$$\boldsymbol{x}(0) = \boldsymbol{x}_0, \tag{4.59d}$$

$$\boldsymbol{x}(\mathtt{T}) \in \mathcal{P}, \tag{4.59e}$$

where $\mathtt{T}$ is the prediction horizon. $\mathcal{P}$ and $\Phi_{\mathtt{T}}(\cdot)$ denote the controlled invariant terminal region and terminal cost function, respectively.

We now borrow ideas from Chen and Allgöwer [1998], to evaluate the infinite-time performance. Let us consider the following assumption.

Assumption 4.2. The terminal cost $\Phi_{\mathtt{T}}(\boldsymbol{x})$ is smooth, positive definite and satisfies the HJB-like inequality

$$\min_{u\in\mathcal{U}}\{\dot{\Phi}_{\mathtt{T}}(\boldsymbol{x}, \boldsymbol{u}) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})\} = \min_{u\in\mathcal{U}}\{\nabla\Phi_{\mathtt{T}}(\boldsymbol{x})[\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{u}] + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})\} \leq 0,$$

in the terminal set $\mathcal{P} = \{x : \Phi_{\mathtt{T}}(\boldsymbol{x}) \leq \kappa\}$.

Remark 4.10. For locally linearizable systems with the quadratic cost (4.49), we can take $\Phi_{\mathtt{T}}(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{S}\boldsymbol{x}$. Then $\Phi_{\mathtt{T}}(\boldsymbol{x})$ is a solution to (4.60) if $\boldsymbol{S}$ satisfies the Riccati inequality

$$\bar{\boldsymbol{A}}^\top \boldsymbol{S} + \boldsymbol{S}\bar{\boldsymbol{A}} - \boldsymbol{S}\bar{\boldsymbol{B}}\boldsymbol{R}^{-1}\bar{\boldsymbol{B}}^\top \boldsymbol{S} + \boldsymbol{Q} \leq 0$$

where $\bar{\boldsymbol{A}}, \bar{\boldsymbol{B}}$ are from the Jacobian linearization of (4.1) at $\boldsymbol{x} = \mathbf{0}, \boldsymbol{u} = \mathbf{0}$. The value $\kappa > 0$ should be small enough so that (4.60) remains valid in $\mathcal{P} = \{\boldsymbol{x} : \boldsymbol{x}^\top \boldsymbol{S}\boldsymbol{x} \leq \kappa\}$ [Chen and Allgöwer, 1998].

Then we have the following result.

Theorem 4.3. *Under Assumption 4.2 consider the finite horizon model predictive scheme* (4.59)*. This problem is recursively feasible. Assume that the optimal control* $\boldsymbol{\omega}(\boldsymbol{x})$ *is continuous and denote by* $\Psi_{\mathtt{T}}(\boldsymbol{x})$ *its value function. We have*

$$D^+\Psi_{\mathtt{T}}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})), \tag{4.60}$$

hence $\Psi_{\mathtt{T}}(\boldsymbol{x}_0)$ *is a bound for the* infinite time *performance with* $x(0) = \boldsymbol{x}_0$*, i.e.,*

$$\int_0^\infty \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))\mathrm{d}t \leq \Psi_{\mathtt{T}}(\boldsymbol{x}_0). \tag{4.61}$$

Proof. We have that the region $\mathcal{P}$ is controlled invariant and that if we consider the minimizing control $\boldsymbol{u}^*$ in (4.60), for $\boldsymbol{x}_0 \in \mathcal{P}$, by integrating, we get:

$$\int_0^\infty \mathfrak{g}(\boldsymbol{x}, \boldsymbol{u}^*)\mathrm{d}t \leq \Phi_{\mathtt{T}}(\boldsymbol{x}_0).$$

This means that the infinite-horizon open loop optimal cost with initial condition $\boldsymbol{x}_0 \in \mathcal{P}$ is upper bounded by $\Phi_{\mathtt{T}}(\boldsymbol{x}_0)$. Recursive feasibility then follows from [Chen and Allgöwer, 1998].

Take any initial $\boldsymbol{x}(t) = \boldsymbol{x}_t$ and denote by $\bar{\boldsymbol{u}}_t$ and $\bar{\boldsymbol{x}}_t$ the optimal solution on interval $[t, t+\mathtt{T}]$. Consider a shifted interval $[\tau, \tau+\mathtt{T}]$, with $t < \tau$ and take the difference

$$\begin{aligned}
\Delta &= \Psi_{\mathtt{T}}(\boldsymbol{x}(\tau)) - \Psi_{\mathtt{T}}(\boldsymbol{x}(t)) \\
&= \min_{\boldsymbol{u}} \left\{ \int_\tau^{\tau+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(\tau+\mathtt{T})) \right\} \\
&\qquad - \min_{\boldsymbol{u}} \left\{ \int_t^{t+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(t+\mathtt{T})) \right\} \\
&= \min_{\boldsymbol{u}} \left\{ \int_\tau^{\tau+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(\tau+\mathtt{T})) \right\} \\
&\qquad - \left\{ \int_t^{t+\mathtt{T}} \mathfrak{g}(\bar{x}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\bar{x}_t(t+\mathtt{T})) \right\}.
\end{aligned} \tag{4.62}$$

Now we consider the solution $\bar{\boldsymbol{u}}_t$ and $\bar{\boldsymbol{x}}_t$ in the intersection interval $[\tau, t+\mathtt{T}]$ in the first term, so achieving an inequality, and eliminate the common term $\int_\tau^{t+\mathtt{T}} \mathfrak{g}(\bar{\boldsymbol{x}}_t, \bar{\boldsymbol{u}}_t)\mathrm{d}\sigma$

$$\begin{aligned}
\Delta &\leq \min_{\boldsymbol{u}} \left\{ \int_\tau^{t+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \int_{t+\mathtt{T}}^{\tau+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(\tau+\mathtt{T})) \right\} \\
&\qquad - \left\{ \int_t^{t+\mathtt{T}} \mathfrak{g}(\bar{\boldsymbol{x}}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\bar{\boldsymbol{x}}_t(t+\mathtt{T})) \right\} \\
&= \min_{\boldsymbol{u}} \left\{ \int_{t+\mathtt{T}}^{\tau+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(\tau+\mathtt{T})) \right\} \\
&\qquad - \int_t^{\tau} \mathfrak{g}(\bar{\boldsymbol{x}}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma))\mathrm{d}\sigma - \Phi_{\mathtt{T}}(\bar{\boldsymbol{x}}_t(t+\mathtt{T})) \\
&= \underbrace{\min_{\boldsymbol{u}} \int_{t+\mathtt{T}}^{\tau+\mathtt{T}} \mathfrak{g}(\boldsymbol{x}(\sigma), \boldsymbol{u}(\sigma))\mathrm{d}\sigma + \Phi_{\mathtt{T}}(\boldsymbol{x}(\tau+\mathtt{T})) - \Phi_{\mathtt{T}}(\bar{\boldsymbol{x}}_t(t+\mathtt{T}))}_{\leq 0} \\
&\qquad - \int_t^{\tau} \mathfrak{g}(\bar{\boldsymbol{x}}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma))\mathrm{d}\sigma \leq - \int_t^{\tau} \mathfrak{g}(\bar{\boldsymbol{x}}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma))\mathrm{d}\sigma.
\end{aligned} \tag{4.63}$$

The non–positivity of the term, follows by considering the initial state $\bar{\boldsymbol{x}}_t(t+\mathtt{T}) \in \mathcal{P}$ and integrating (4.60) in Assumption 4.2, bearing in mind that $\boldsymbol{x}(\sigma) \in \mathcal{P}$ if $\sigma \in [t+\mathtt{T}, \tau+\mathtt{T}]$.

Now we need only to reconsider Δ divide by $\tau - t$ and take the limit $\tau - t \to 0$

$$D^+\Psi_{\mathtt{T}}(\boldsymbol{x}(t), \boldsymbol{\omega}(\boldsymbol{x}(t))) = \limsup_{\tau - t \to 0} \frac{\Psi_{\mathtt{T}}(\boldsymbol{x}(\tau)) - \Psi_{\mathtt{T}}(\boldsymbol{x}(t))}{\tau - t}$$
$$= \limsup_{\tau - t \to 0} \frac{-1}{\tau - t} \int_t^\tau \mathfrak{g}(\bar{\boldsymbol{x}}_t(\sigma), \bar{\boldsymbol{u}}_t(\sigma)) \mathrm{d}\sigma \leq -\mathfrak{g}(\boldsymbol{x}(t), \boldsymbol{\omega}(\boldsymbol{x}(t))).$$

□

Remark 4.11. We have that (4.61) is not an equality as (4.42). However, for $\mathtt{T}$ large enough and $\mathcal{P}$ small, there is no essential difference between the infinite and finite horizon performance, namely $\Psi(\boldsymbol{x}) \approx \Psi_{\mathtt{T}}(\boldsymbol{x})$, and (4.61) becomes close to an equality and roughly, $D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \approx -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$. Hence, to search for an SCLF in practice, we will impose the condition $D^+V(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$ rather than $D^+V(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$ to avoid computing $D^+\Psi(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$.

4.3.2 Computation of a Sub-Control Lyapunov Function

In this part, given a suitable setup, the computation of a SCLF will be cast into semi-infinite linear programming (SILP), i.e., a linear program with infinitely many constraints [Steentjes et al., 2020, Julian et al., 1999, Giesl and Hafstein, 2014, Ravanbakhsh and Sankaranarayanan, 2019]. Two procedures to solve such a problem will be discussed with different levels of complexity and efficiency.

One key point of the procedure is that we do not need to have the function $\Psi(\boldsymbol{x})$ to derive a sub-control Lyapunov function. We need just an algorithm to evaluate the optimal control $\boldsymbol{\omega}(\boldsymbol{x})$ function and the given performance function $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$ for off–line synthesis.

4.3.2.1 Finding an SCLF with SILP

Herein, under the standing assumption that the baseline control $\boldsymbol{\omega}(\boldsymbol{x})$ is off-line computable, the construction of the SCLF can be carried out as follows. First, consider an SCLF candidate $V(\boldsymbol{x})$ of the form (4.5), i.e.,:

$$V(\boldsymbol{x}) = \sum_{q=1}^{N} \alpha_q V_q(\boldsymbol{x}), \tag{4.64}$$

where $V_q(\boldsymbol{x}), \alpha_q \geq 0$ are smooth positive definite functions and the corresponding coefficients. Then, consider the following program:

$$\alpha = \arg\min_{\alpha_q} \sum_{k=1}^{N} \alpha_q, \tag{4.65a}$$

$$\alpha_q \geq 0, \tag{4.65b}$$

$$\Omega(\boldsymbol{\alpha}, \boldsymbol{x}) \leq 0, \tag{4.65c}$$

$$\boldsymbol{x} \in \mathcal{X} \subset \mathbb{R}^n. \tag{4.65d}$$

where $\boldsymbol{\alpha} = [\alpha_1, ..., \alpha_N]^\top$ collects the coefficients α_q in (4.64), $\mathcal{X}$ is a compact region of interest containing the origin in its interior. $\Omega(\boldsymbol{\alpha}, \boldsymbol{x}) : \mathbb{R}^N \times \mathbb{R}^n \to \mathbb{R}$ denotes the condition function:

$$\begin{aligned} \Omega(\boldsymbol{\alpha}, \boldsymbol{x}) &\triangleq \dot{V}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \\ &= \nabla V(\boldsymbol{x})(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \\ &= \sum_{q=1}^{N} \alpha_q \nabla V_q(\boldsymbol{x})(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})). \end{aligned} \tag{4.66}$$

The program (4.65) is a linear program with N variables α_q and infinitely many linear constraints of the form (4.65c) with $\boldsymbol{x}$ taken from $\mathcal{X}$. This leads to a so-called SILP which is a convex optimization problem and has been deeply investigated in the literature [Anderson and Nash, 1987, Tanaka et al., 1988, Hettich and Kortanek, 1993, Polak, 2012]. By solving (4.65), one can obtain the following result.

Proposition 4.9. *If the semi-infinite linear program* (4.65) *has a feasible solution* $\bar{\boldsymbol{\alpha}} = [\bar{\alpha}_1, ..., \bar{\alpha}_N]^\top$, *then* $V(\boldsymbol{x}) = \sum_{q=1}^{N} \bar{\alpha}_q V_q(\boldsymbol{x})$ *is an SCLF.*

Proof. The proof follows by the satisfaction of (4.65c)–(4.65d) and Definition 4.4. □

Remark 4.12. The choice of $\sum_q \alpha_q$ as cost in the linear program aims at finding the smallest value of $V(\boldsymbol{x})$ which is a CLF, to ensure the best performance (Theorem 4.2). Assume that $\boldsymbol{x}_0$ is a nominal initial condition, representing a nominal perturbation. Then, as an alternative possibility, one can replace the cost function by $\sum_q \alpha_q V_q(\boldsymbol{x}_0)$, to specifically improve the performance from this state.

4.3.2.2 Discretization-based solution

To compute a feasible solution for (4.65), one direct approach is to sample or discretize the set $\mathcal{X}$ into a sufficiently dense grid of points $\mathcal{X}_g \subset \mathcal{X}$. Then, a candidate coefficient $\boldsymbol{\alpha}$ can be found by solving (4.65a)–(4.65c) for $\boldsymbol{x} \in \mathcal{X}_g$, making the optimization a standard

Linear Program (LP):

$$\begin{aligned} \alpha &= \arg\min_{\alpha_q \geq 0} \sum_{q=1}^{N} \alpha_q \\ \text{s.t. } \boldsymbol{\alpha} &\in \bigcap_{\boldsymbol{x} \in \mathcal{X}_g} \{\boldsymbol{\alpha} : \Omega(\boldsymbol{\alpha}, \boldsymbol{x}) \leq 0\}, \end{aligned} \tag{4.67}$$

with $\Omega(\boldsymbol{\alpha}, \boldsymbol{x})$ defined in (4.66). An immediate follow-up question would be how dense the grid $\mathcal{X}_g$ should be to obtain an acceptable solution within a certain range of tolerance for the constraints in (4.65). To address this concern, let us proceed with the following setup. Given a finite set $\mathcal{X}_g \subset \mathcal{X}$, define a granularity function:

$$\rho(\boldsymbol{x}) = \inf_{\boldsymbol{x}_g \in \mathcal{X}_g} \|\boldsymbol{x} - \boldsymbol{x}_g\|. \tag{4.68}$$

It is further assumed that the time derivative of $V(\boldsymbol{x})$ driven by the control $\boldsymbol{\omega}(\boldsymbol{x})$ and the corresponding penalty $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x}))$ is Lipschitz continuous. Namely,

$$\begin{aligned} &|\Gamma_V(\boldsymbol{x}_a) - \Gamma_V(\boldsymbol{x}_b)| \leq \nu \|\boldsymbol{x}_a - \boldsymbol{x}_b\| \\ &|\mathfrak{g}(\boldsymbol{x}_a, \boldsymbol{\omega}(\boldsymbol{x}_a)) - \mathfrak{g}(\boldsymbol{x}_b, \boldsymbol{\omega}(\boldsymbol{x}_b))| \leq \eta \|\boldsymbol{x}_a - \boldsymbol{x}_b\|, \end{aligned} \tag{4.69}$$

with $\boldsymbol{x}_a, \boldsymbol{x}_b \in \mathcal{X}$ and for brevity, we use the notation:

$$\Gamma_V(\boldsymbol{x}) \triangleq \dot{V}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) = \nabla V(\boldsymbol{x})(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\boldsymbol{\omega}(\boldsymbol{x})). \tag{4.70}$$

Then we have the following proposition.

Proposition 4.10. *Suppose that we have a candidate $V(\boldsymbol{x})$ defined as in (4.64) by solving (4.67) for a finite set $\mathcal{X}_g \subset \mathcal{X}$. Namely, $\Omega(\boldsymbol{\alpha}, \boldsymbol{x}_g) \leq 0 \ \forall \boldsymbol{x}_g \in \mathcal{X}_g$, or,*

$$\Gamma_V(\boldsymbol{x}_g) \leq -\mathfrak{g}(\boldsymbol{x}_g, \boldsymbol{\omega}(\boldsymbol{x}_g)) \ \forall \boldsymbol{x}_g \in \mathcal{X}_g. \tag{4.71}$$

Then, given $0 < \varphi < 1, \exists \zeta(\varphi) > 0$:

$$\Gamma_V(\boldsymbol{x}) \leq -\varphi \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})), \tag{4.72}$$

$\forall \boldsymbol{x} \in \mathcal{X} \cap \{\boldsymbol{x} : \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \geq \zeta(\varphi)\}$, and $\zeta(\varphi) \triangleq (\eta + \nu)\bar{\rho}/(1 - \varphi)$, $\bar{\rho} = \sup_{\boldsymbol{x} \in \mathcal{X}} \rho(\boldsymbol{x})$.

Proof. Consider $\boldsymbol{x} \in \mathcal{X}$, $\boldsymbol{x}_g \in \mathcal{X}_g \cap \mathcal{B}_{\bar{\rho}}(\boldsymbol{x})$, we have:

$$\begin{aligned} &\Gamma_V(\boldsymbol{x}) = \Gamma_V(\boldsymbol{x}_g) + \Gamma_V(\boldsymbol{x}) - \Gamma_V(\boldsymbol{x}_g) \leq -\mathfrak{g}(\boldsymbol{x}_g, \boldsymbol{\omega}(\boldsymbol{x}_g)) + \nu \|\boldsymbol{x} - \boldsymbol{x}_g\| \\ &= -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) - \mathfrak{g}(\boldsymbol{x}_g, \boldsymbol{\omega}(\boldsymbol{x}_g)) + \nu \|\boldsymbol{x} - \boldsymbol{x}_g\| \\ &\leq -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + \eta \|\boldsymbol{x} - \boldsymbol{x}_g\| + \nu \|\boldsymbol{x} - \boldsymbol{x}_g\| \leq -\mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) + (\eta + \nu)\bar{\rho} \leq -\varphi \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})), \end{aligned} \tag{4.73}$$

where the last inequality follows from (4.72). □

This is to say, if an accuracy $0 < \varphi < 1$ is selected (possibly close to 1), the inequality (4.72) is guaranteed continuously outside the set $\{\boldsymbol{x} : \mathfrak{g}(\boldsymbol{x}, \boldsymbol{\omega}(\boldsymbol{x})) \leq \zeta(\varphi)\}$. The volume of this level set can be shrunken by squeezing $\bar{\rho}$, or increasing the density of the grid $\mathcal{X}_g$. Proposition 4.10 hence provides a theoretical analysis on the relation between the discretization of the continuous set $\mathcal{X}$ and the the characteristic parameters. However, although programmatically simple for low-dimensional systems, for high-dimensional systems, uniformly gridding the set $\mathcal{X}$ and solving (4.67) are not practical since the required number of points in $\mathcal{X}_g$ may grow aggressively with respect to the state dimension n. For this reason, in the following, as opposed to a fixed grid as in [Johansen, 2000][J6], we establish an iterative procedure for suitably refining the set $\mathcal{X}_g$ based on the cutting-plane technique.

4.3.2.3 Cutting-plane-based solution

In this part, the cutting-plane (CP) method is employed to provide an iterative procedure for grid adaptation. The intuition is that of iteratively enforcing the constraint (4.65c) at the state where it is most violated [Ravanbakhsh and Sankaranarayanan, 2019]. Namely, in the variable space of $\boldsymbol{\alpha}$, linear constraints (a.k.a., cuts) are added correspondingly. In this work, the CP technique is chosen, since in the problem (4.65), there is only one continuous constraint described by (4.65c)–(4.65d). Hence, the violation can be evaluated by checking the non-positivity of the scalar function $\Omega(\boldsymbol{\alpha}, \boldsymbol{x})$ in (4.66) over the domain $\mathcal{X}$ for a given candidate $\boldsymbol{\alpha}$ resulted from (4.67), i.e., verifying:

$$\Omega^* = \Omega(\boldsymbol{\alpha}, \boldsymbol{x}^*) \leq 0, \tag{4.74a}$$

$$\text{with } \boldsymbol{x}^* = \arg\max_{\boldsymbol{x} \in \mathcal{X}} \Omega(\boldsymbol{\alpha}, \boldsymbol{x}). \tag{4.74b}$$

Algorithm 4.2: SCLF construction with CP method

Input: A compact set $\mathcal{X}$, a set of basis functions $V_q(\boldsymbol{x})$, and the control law $\boldsymbol{\omega}(\boldsymbol{x})$.
Output: the SCLF $V(\boldsymbol{x})$.
Select an initial finite grid $\mathcal{X}_g \subset \mathcal{X}$;
Solve the LP (4.67) with $\mathcal{X}_g$ to get $\boldsymbol{\alpha}$;
Define the function $V(\boldsymbol{x})$ as in (4.64);
Define $\Omega(\boldsymbol{\alpha}, \boldsymbol{x})$ as in (4.66);
Compute $\boldsymbol{x}^*$ and Ω^* as in (4.74);
if $\Omega^* > 0$
    Append $\{\boldsymbol{x}^*\}$ to the the grid points $\mathcal{X}_g$;
    go to step 2;
else
    return $V(\boldsymbol{x})$;

If the certificate (4.74a) does not hold, $\boldsymbol{x}^*$ in found (4.74b) is the point where the constraint $\Omega(\boldsymbol{\alpha}, \boldsymbol{x}) \leq 0$ (4.65c) is violated the most. This point will then be appended to the current grid $\mathcal{X}_g$ and the procedure will be reinitalized with such an updated grid. Concisely, Algorithm 4.2 summarizes the main steps of the SCLF construction based on CP technique, an illustrative flow chart is also provided in Figure 4.12.

In the following part, to proceed with the construction of the SCLF for the numerical examples, we particularize the choice of the $\boldsymbol{\omega}(\boldsymbol{x})$ and the basis functions $V_q(\boldsymbol{x})$.

4.3.2.4 Discussion

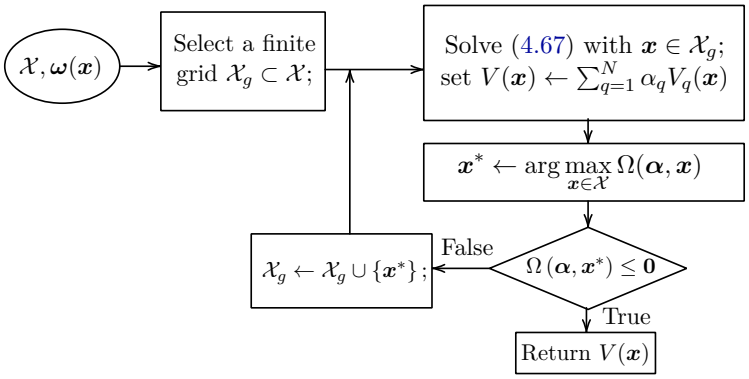


FIGURE 4.12: Pipeline for the SCLF construction based on cutting-plane technique.

i) The ideal control $\boldsymbol{\omega}(\boldsymbol{x})$:

To obtain a control law of optimal performance, among practical control strategies, quasi-infinite-horizon MPC [Chen and Allgöwer, 1998] will be chosen as the software for generating stable closed-loop trajectories due to its standardized synthesis procedure.

For numerical implementation, the constraint (4.59b) is approximated via the Euler discretization as in Section 4.2.2 with a sampling parameter $\tau_s > 0$ [Blanchini et al., 2003].

The next fundamental component will be the set of basis functions parameterizing the SCLF. In order to provide a rationale for our selection of such a set for the numerical examples, a brief review will be presented subsequently.

ii) The basis functions $V_q(\boldsymbol{x})$ choice

In the literature, the parameterization of the form (4.64) to find a CLF has been intensively studied. Hence, there are several options for the choice of the basis functions $V_q(\boldsymbol{x})$. Generally, this is often a trade-off between their flexibility and computational complexity in both offline synthesis and online deployment stages. For example, in the context of

artificially generated CLF, one customary choice is a set of piecewise-affine (PWA) functions, making the synthesis abridge to an LP, accompanied by generalized analytical and practical guarantees [Rubagotti et al., 2016, Giesl and Hafstein, 2014, Steentjes et al., 2020]. Yet, if the processing power is limited, one encounters the problem of dimensionality again while dividing the state space into partitions (e.g., triangulations) during the offline phase. Meanwhile, the online phase faces the so-called *point location problem* when PWA functions are employed in the form of look-up tables. In the context of stability analysis for autonomous systems, a more complete set was used in [Johansen, 2000] in the form of a Gaussian basis. This can be thought of as an interpolation of smooth quadratic functions, with the weight matrices being the decision variables, resulting in, again, an SILP. In this manner, the problem of imposing continuity on the function was also sidestepped. However, with this interpolation, the basis is again defined over a set of discrete knots, complicating the online applications with respect to the system's size. Hence, depending on the computational power at hand, different choices of basis can be combined to acquire a satisfactory characterization. With these analyses, in view of maintaining smoothness, avoiding discretization-based interpolation, and taking advantage of the results for linear systems in [Blanchini and Miani, 1999a][J6], we elaborate the set of basis functions composed as follows.

Let us first include in the basis a quadratic function:

$$V_1(\boldsymbol{x}) = \boldsymbol{x}^\top \boldsymbol{P} \boldsymbol{x}, \tag{4.75}$$

with the weight $\boldsymbol{P} \succ 0$ computed from the algebraic Riccati equation for the linearly approximated dynamics of (4.1) around the origin. In this manner, we ensure that, around a neighborhood of the origin, we obtain some local optimality with the resulting function.

Next, the set of basis functions will be populated with $2p$-norm-like polynomial [Blanchini and Miani, 1999a]:

$$V_q(\boldsymbol{x}) = (\boldsymbol{a}_q^\top \boldsymbol{x})^{2p}, \tag{4.76}$$

where the function is characterized by the choice of the vector $\boldsymbol{a}_q \in \mathbb{R}^n$. With this setting, by simply collecting $\boldsymbol{a}_q$ with different directions and lengths, we can effectively expand the set of basis functions to achieve more representational capacity. An illustration in $\mathbb{R}^2$ is given in Figure 4.13.

With the characterization of $V_q(\boldsymbol{x})$ in (4.75)–(4.76), the idea is to increase the completeness of the basis with a possibly large number of basis (i.e., N in (4.64)). This is the expense of fixing a priori the structure (shape and value) of the basis, as in (4.76). Yet, it will be shown later in the examples that, eventually, after the generation of the SCLF, there exist many basis functions associated with insignificantly small coefficients. Hence, in practice, those components of the parameterization can be discarded. In the numerical examples, a threshold $\varepsilon_N \ll 1$ will be defined so that only the coefficients

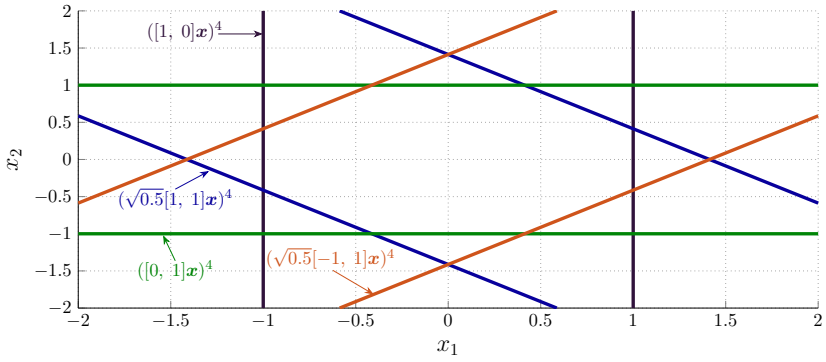


FIGURE 4.13: Unit level set of some basis functions of the form (4.76) and a linear combination of them (red), with $\boldsymbol{x} \in \mathbb{R}^2, p = 2$.

$\alpha_q \geq \varepsilon_N$ are kept. The number of those remaining functions with will be denoted as $\underline{N}$. These functions will then be retained for the online control via (4.45).

So far, all the ingredients for the control synthesis with an SCLF are ready. Next, numerical examples will be provided with both simulation and experimental results, showing that the approach can be applied to non-trivial cases.

4.3.3 Numerical examples and experimental validation

Herein, three systems will be examined to illustrate and showcase the effectiveness of the proposed machinery. First, the dynamics investigated will be recapitulated, and then the numerical results will be reported. Note that in the next part, without further explanation, the notation $\boldsymbol{x}, \boldsymbol{u}$ will be abused to denote the state and input vector of the system under discussion.

4.3.3.1 System description

Let us proceed by briefly summarizing the three control problems employed, as follows.

i) Bioreactor process (Bioreactor)

For the sake of illustration, we first examine a two-dimensional system of a bioreactor process governed by the following nonlinear dynamics [Brengel and Seider, 1989, Ramaswamy et al., 2005]:

$$\begin{aligned}\dot{x}_1 &= -(C_{1S} + x_1)(1 + u) + D_{aS}(1 - x_2 - C_{2S})e^{(C_{2S}+x_2)/\gamma_s}(C_{1S} + x_1)\\ \dot{x}_2 &= -(C_{2S} + x_2)(1 + u) + D_{aS}(1 - x_2 - C_{2S})e^{(C_{2S}+x_2)/\gamma_s}\frac{(1 + \beta_s)(C_{1S} + x_1)}{1 + \beta_s - x_2 - C_{2S}}.\end{aligned} \tag{4.77}$$

with the parameters $\gamma_s = 0.4, \beta_s = 0.02, D_{aS} = 1.2,\ C_{1S} = 0.0938, C_{2S} = 0.9155$. x_1 and x_2 denote the deviation from the equilibrium dedimensionalized states, while the dimensionless flow rate u is the system's input and its amplitude is constrained as:

$$|u| \leq u_{max} = 1. \tag{4.78}$$

The control objective for system (4.77) is to stabilize $\boldsymbol{x} = [x_1, x_2]^\top$ towards the origin. The domain of interest is set as $\mathcal{X} = \{\boldsymbol{x} : |\boldsymbol{x}| \leq 0.1\}$. With this model, it is important to note that the state, the input and even the time clock have been normalized. However, for consistency, the same notations for the variables and their derivatives will be used. For the technical analysis of the model construction, we would like to send the readers to [Brengel and Seider, 1989, Ramaswamy et al., 2005] and the references therein.

ii) Planar vertical take-off and landing aircraft (VTOL)

Next, to showcase the effectiveness of the technique over a non-trivial size system, we examine the normalized model of a vertical take-off and landing (VTOL) aircraft driven by the following dynamics [Hauser et al., 1992, Martin et al., 1996]:

$$\ddot{\mathsf{x}}_1 = -f_1 \sin \mathsf{x}_3 + f_2 \varepsilon_{vtol} \cos \mathsf{x}_3,\ \ddot{\mathsf{x}}_2 = f_1 \cos \mathsf{x}_3 + f_2 \varepsilon_{vtol} \sin \mathsf{x}_3 - 1,\ \ddot{\mathsf{x}}_3 = f_2, \tag{4.79}$$

where in (4.79), $\mathsf{x}_1, \mathsf{x}_2$ and x_3 denote the 2-D positions of the aircraft's center of mass and its roll angle, respectively. $\varepsilon_{vtol} = 0.2$ is the system's parameter, and "-1" represents the normalized gravitational acceleration. Finally, $\mathfrak{f} = [f_1,\ f_2]^\top$ collects the thrust and rolling moment (see Figure 4.14). The control objective here is to stabilize the system at the equilibrium point where:

$$\mathsf{x}_i = 0, \dot{\mathsf{x}}_i = 0, i \in \{1, 2, 3\},\ \text{and}\ \mathfrak{f} = \mathfrak{f}_e = [1,\ 0]^\top. \tag{4.80}$$

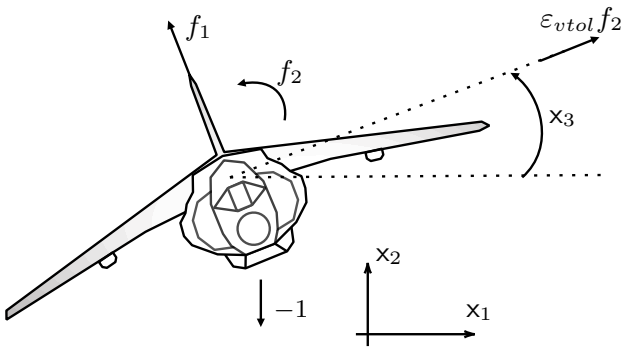


FIGURE 4.14: The planar VTOL model

To verify the SCLF found with the explicit control (4.51), the constraint on the input $\mathfrak{f}$ is intentionally chosen as:

$$[-2,\ -2]^\top \leq \mathfrak{f} \leq [4,\ 2]^\top. \tag{4.81}$$

In this way, the error dynamics can be rewritten in the control affine form as:

$$\begin{aligned}\dot{x}_1 &= x_2, \dot{x}_2 = -(u_1+1)\sin x_5 + u_2\varepsilon_{vtol}\cos x_5,\\ \dot{x}_3 &= x_4, \dot{x}_4 = (u_1+1)\cos x_5 + u_2\varepsilon_{vtol}\sin x_5 - 1,\\ \dot{x}_5 &= x_6, \dot{x}_6 = u_2.\end{aligned} \tag{4.82}$$

with $\boldsymbol{x}_{2i-1} = \mathsf{x}_i, i \in \{1,2,3\}$ as in (4.79), while the input $\boldsymbol{u} = [u_1, u_2]^\top = \mathfrak{f} - \mathfrak{f}_e$ is restricted in the form of (4.50):

$$|\boldsymbol{u}| \leq \boldsymbol{u}_{max} = [3,\ 2]^\top. \tag{4.83}$$

The investigated state space for this system is set as:

$$\mathcal{X} = \Big\{\boldsymbol{x} \in \mathbb{R}^6 : |x_1|, |x_3| \leq 1, |x_2|, |x_4| \leq 0.5, |x_5| \leq \pi/3, x_6 \leq \pi/6\Big\}. \tag{4.84}$$

TABLE 4.4: Numerical parameters used in the SCLF synthesis and online control

	Bioreactor	VTOL	Drone
System & $\mathcal{U}$	(4.77) and (4.78)	(4.82) and (4.83)	(3.10) and (3.22)
$\boldsymbol{Q}$	diag(5, 5)	diag([5, 1, 5, 1, 10, 1])	diag([50, 50, 50, 5, 5, 5])
$\boldsymbol{R}$	1	diag(4, 4)	diag(10, 10)
τ_s (s)	005	0.05	01
n_g	20	1500	1500
$\mathtt{T}^{\text{off}}$ (s)	17.5	15	12
$\mathtt{T}^{\text{on}}$ (s)	0.5	3.25	3.5

iii) Quadcopter position control (Drone)

The final example will be the experimental validation for the position control (a.k.a., outer-loop control) of a nano-drone as characterized in (3.2). with the objective being to stabilize the dynamics[2] (3.10) at $\boldsymbol{\xi} = \mathbf{0}, \boldsymbol{v} = \mathbf{0}$. In brief, the control problem involves the stabilization of system (3.10) subject to $\boldsymbol{v} = [v_1, v_2, v_3]^\top \in \mathcal{V}_c^{\text{in}}$ as in (3.22). The studied domain is chosen as:

$$\mathcal{X} = \{\boldsymbol{\xi} \in \mathbb{R}^6 \text{ as in (3.10)} : |\boldsymbol{\xi}| \leq 1.5\}. \tag{4.85}$$

In summary, the three systems and their constraints are summarized in Table 4.4.

[2]for the sake of coherence, we keep the same notation defined for the drone as in (3.10) for the state and input.

4.3.3.2 Numerical results and discussion

In this part, the numerical results for the offline synthesis of the SCLF will be reported first. Subsequently, the applicability of the constructed SCLF will be demonstrated via simulations and experiments in comparison with the quasi-infinite-horizon MPC.

i) Offline synthesis for SCLFs

The offline construction will be carried out with the control software $\boldsymbol{\omega}(\boldsymbol{x})$ calculated with the routine (4.59). For this setup, the cost function $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$ will be chosen in the quadratic form as in (4.49). The corresponding terminal cost and constraints ($\Phi_{\mathtt{T}}(\boldsymbol{x})$ and $\mathcal{P}$) therein will be uniquely determined as the quadratic function from the linear quadratic regulator and its ellipsoidal level set computed for the approximated dynamics at the origin [Chen and Allgöwer, 1998]. Furthermore, as this is an offline effort, the prediction horizon in (4.59) is preferably chosen to be relatively large and denoted as $\mathtt{T} = \mathtt{T}^{\text{off}}$. Note that the superscript "off" is to distinguish the value from the prediction horizon used later for online comparison. To start Algorithm 4.2, the initial grid was chosen as a set of n_g random points in $\mathcal{X}$. All the aforementioned values are reported in Table 4.4. All the receding horizon routines and quadratic programs will be solved with IPOPT software [Wächter and Biegler, 2006] via CasADi interface [Andersson et al., 2019]. To evaluate the certificate (4.74), `fmincon` solver of MATLAB 2021b will be used with the sequential quadratic programming (SQP) approach.

TABLE 4.5: Specifications and total build time (BT) for the three systems

Model	n	m	N in (4.64)	ε_N	$\underline{N}$	Tota BT (h)
Bioreactor	2	1	1303	5e−7	6	3.144
VTOL	6	2	1627	1e−5	29	22.59
Drone	6	3	2561	6e−6	38	80.93

Regarding the results in Table 4.5, we recall that N denotes the number of basis functions employed in the optimization (4.65). ε_N is the chosen threshold below which all the coefficients α_q found will be discarded, leaving $\underline{N}$ functions parameterizing $V(\boldsymbol{x})$ as in (4.64). The total build time (BT) used for running Algorithm 4.2 is also reported therein. Figure 4.15 the values of Ω^* in (4.74a) found (blue) and the corresponding build time (orange) to find them along the iteration steps of the algorithm.

Expectedly, the offline construction time grows with respect to the number of states and inputs. This can be explained by the search for x^* in (4.74b) which requires a continuous evaluation of $\boldsymbol{\omega}(\boldsymbol{x})$ within the SQP solver employed. It is also noticeable in Table 4.5 that, although using a large number of basis functions to find $V(\boldsymbol{x})$, the number of meaningful coefficients remaining is relatively small. This is, indeed, a positive sign, since later on, during the online implementation of (4.45), the calculation of $\nabla V(\boldsymbol{x})$ will require less computational power. Regarding the effectiveness of Algorithm 4.2, it can be

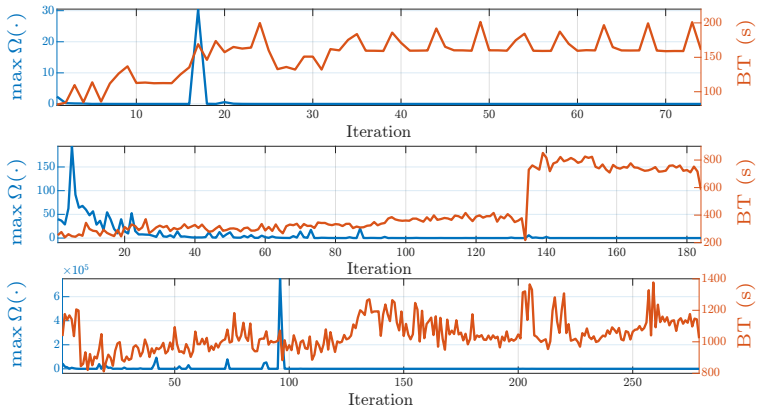


FIGURE 4.15: Largest violation found and build time (BT) for Bioreactor, VTOL and Drone model (top to bottom, respectively).

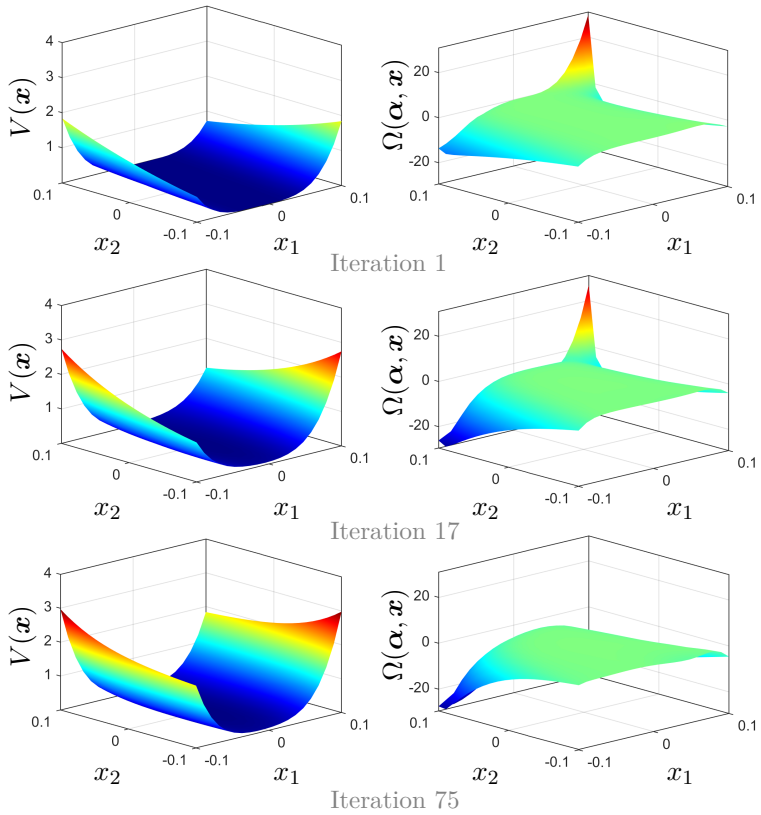


FIGURE 4.16: Snapshots of the SCLF candidates (left) and the verifier $\boldsymbol{\Omega}(\boldsymbol{\alpha}, \boldsymbol{x})$ as in (4.66) (right) during the execution of Algorithm 4.2 for the bioreactor model (4.77).

seen from Figure 4.15 and Table 4.4 that the total number of points[3] in state space used to enforce the constraint (4.65c) is relatively small, in comparison with, for instance, a loose grid of 5 samples (5^n points) in each axis.

Particularly for the Bioreactor model (4.77), the numerical result from Algorithm 4.2 is:

$$V(\boldsymbol{x}) = 1.957\boldsymbol{x}^\top \begin{bmatrix} 16.973 & -0.147 \\ -0.147 & 1.747 \end{bmatrix} \boldsymbol{x} + 19.003([5,\ 0]\boldsymbol{x})^4 + 0.229([-2.823,\ 4.127]\boldsymbol{x})^4 \\ +1.074([-3.141,\ 3.890]\boldsymbol{x})^4 + 0.833([-3.687,\ 3.377]\boldsymbol{x})^4 + 9.003([-4.950,\ 0.706]\boldsymbol{x})^4. \tag{4.86}$$

With this function, Figure 4.15 and 4.16 also illustrate that the non-positivity of the surface $\Omega(\boldsymbol{\alpha}, \boldsymbol{x})$ (i.e., the satisfaction of (4.65c)) is gradually enforced as Algorithm 4.2 goes through its steps, especially after the 17th iteration.

ii) Online control with the generated SCLFs

After achieving the function $V(\boldsymbol{x})$ from the offline stage, let us proceed by verifying the proposed control via simulations and experiments. The first performance indicator we would like to use is Theorem 4.2, or particularly the property in (4.47). With this inequality, we can show that the closed-loop system is asymptotically stable and verify the performance analysis in the theorem. For this reason, let us denote:

$$\Theta(\boldsymbol{x}) \triangleq \dot{V}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})) = \nabla V(\boldsymbol{x})(\mathbf{f}(\boldsymbol{x}) + \mathbf{g}(\boldsymbol{x})\bar{\boldsymbol{u}}(\boldsymbol{x})) + \mathfrak{g}(\boldsymbol{x}, \bar{\boldsymbol{u}}(\boldsymbol{x})), \tag{4.87}$$

the sum of the time derivative of $V(\boldsymbol{x})$ along the closed-loop trajectory driven by $u = \bar{\boldsymbol{u}}(\boldsymbol{x})$ as in (4.45) and the corresponding cost function $\mathfrak{g}(\boldsymbol{x}, \boldsymbol{u})$. Then, the system's convergence can be shown via the non-positivity of $\Theta(\boldsymbol{x})$.

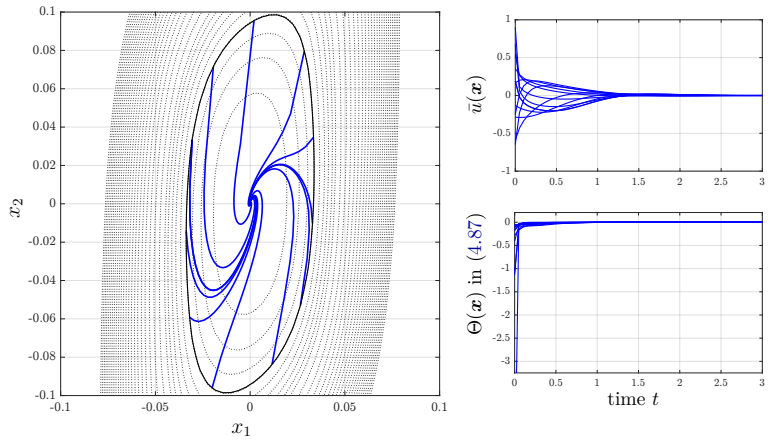


FIGURE 4.17: Closed-loop trajectories (blue) and the level set of the SCLF (dotted black) of the bioreactor model.

[3]the total of the number of points in the initial grid n_g and the number of iterations.

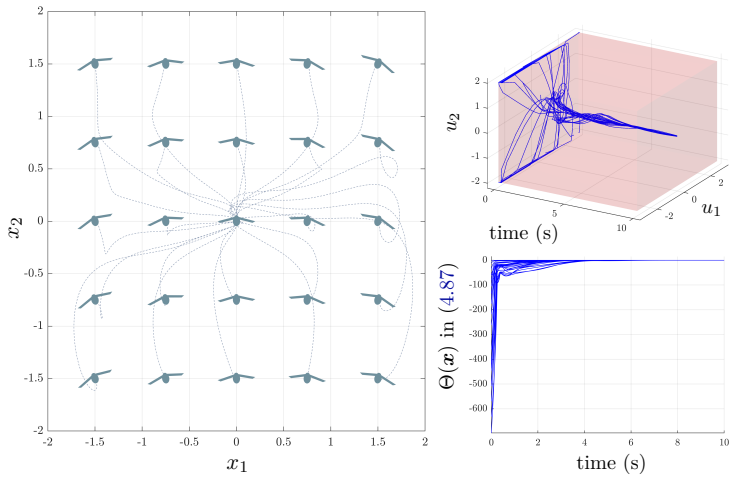


FIGURE 4.18: Planar VTOL stabilization with input constraints (the red box).

For the digital implementation, taking advantage of the input constraint's shape, we employed the explicit control law in Proposition 4.7 for the Bioreactor and VTOL models. With a non-standard input constraint as in (3.22) for the Drone, during the experiments, the QP (4.45) was solved directly via IPOPT solver.

To highlight the computational advantage of the proposed control, for the same initial states, the routine (4.59) is considered for online control with the same choice of cost and constraints as in the offline synthesis part. The difference is only the horizon size, which is chosen as the minimum value of $\mathtt{T}$ offering a feasible solution for the tested initial states. This value is denoted as $\mathtt{T}^{\text{on}}$ and given in Table 4.4. Then, the root-mean-square error (RMSE) will be computed as in (4.88) to evaluate how the controls perform:

$$\text{RMSE} = \sqrt{\frac{1}{N_s}\sum_{k=1}^{N_s} \|\boldsymbol{x}(kt_s)\|^2}. \tag{4.88}$$

Figure 4.17, 4.18 and 4.19 present the closed-loop trajectories with the stability indicator $\Theta(\boldsymbol{x})$ as in (4.87). Therein, it can be seen that with the SCLFs artificially generated before, the constrained systems are successfully stabilized, and the guarantee of $\Theta(\boldsymbol{x}) \leq 0$ is also confirmed.

TABLE 4.6: Comparison between SCLF-induced controller and MPC

	SCLF		MPC	
Model	CT (ms)	Avg. RMSE	CT (ms)	Avg. RMSE
Bioreactor	0.031	0.106	4.662	0.088
VTOL	0.207	0.706	22.174	0.6303
Drone	6.51	0.393	32.66	0.396

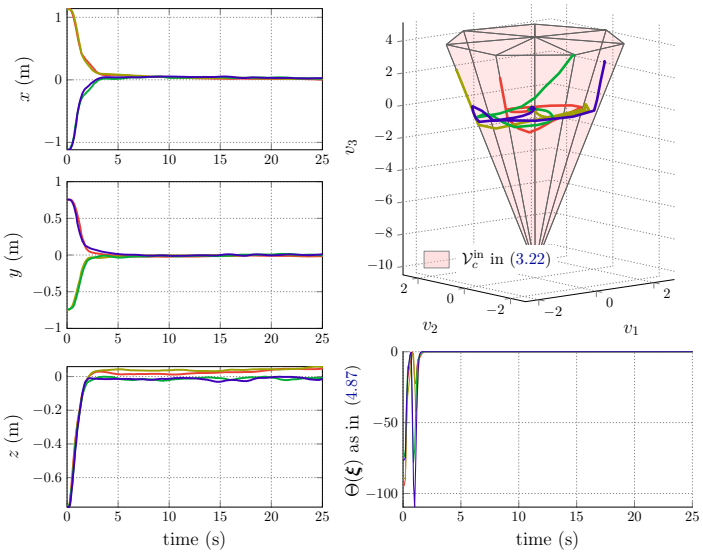


FIGURE 4.19: Experiments for the quadcopter position tracking problem (3.10) with the generated SCLF.

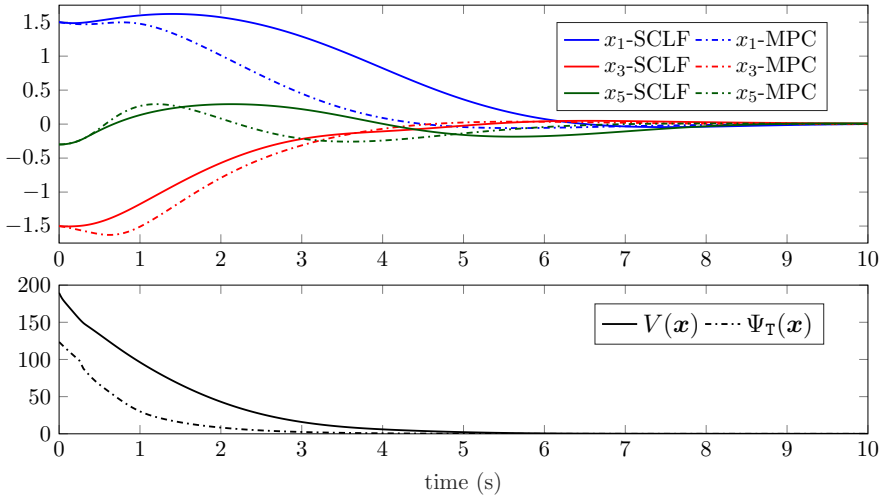


FIGURE 4.20: Suboptimal performance of the SCLF controller w.r.t. MPC in (4.59) for the VTOL model.

A trajectory of the VTOL is reported in Figure 4.20 for both the SCLF control (4.45) and MPC (4.59) for $\boldsymbol{x}_0 = [1.50, -0.19, -1.50, -0.17, -0.30, -0.02]^\top$. Regarding the performance, the suboptimal transient behavior of the SCLF controller is evident through a larger rise time for the states. Additionally, interpreting the cost-to-go function's upper bound, the SCLF $V(\boldsymbol{x})$ consistently exhibits a larger value compared to $\Psi_{\text{T}}(\boldsymbol{x})$ (4.59).

To quantify the performance loss, we approximate the achievable cost of the infinite horizon formula (4.58) with Euler discretization of 1ms and 50s prediction horizon, yielding $\Psi_\infty(\boldsymbol{x}_0) \approx 62.96$. Meanwhile, its upper bounds are given by the finite-horizon cost and the SCLF with $\Psi_{\mathrm{T}}(\boldsymbol{x}_0) = 123.55$ and $V(\boldsymbol{x}_0) = 189.62$, gauging the trade-off to reduce implementation complexity. Indeed, although the proposed control deduced from the SCLFs showed slightly larger tracking error compared to the MPC law (4.59), its computational simplicity is a significant advantage. This simplicity is not only apparent with the explicit solution for the Bioreactor and VTOL, but also with the implicit law (4.45) for the Drone system, which requires significantly less computation time (CT) (See Table 4.6).

Finally, with the experiments on the Drone model, although satisfactory tracking is shown, one practical shortcoming of the proposed control (4.45) is revealed with the steady state error observed in Figure 4.19. More specifically, there has not been any robustification measure taken into account in both offline and online phases. One possible direction is to incorporate the SCLF with an observer, compensating for the disturbance and uncertainty online.

4.4 Concluding remarks

In this chapter, taking advantage of the linearized dynamics of the quadcopter model presented in Chapter 3, we study a technique to artificially synthesize a CLF for control affine systems in the first part. The procedure includes imposing stabilizing constraints continuously over a subset of the state-space and parameterizing the CLF by a family of basis functions. As a result, the calculation requires solving an LP with infinitely many constraints, or SILP. The problem is then relaxed to a standard LP by uniformly sampling the state space with an MPC routine that generates admissible state-control pairs. The computed CLF is experimentally validated via the Crazyflie 2.1 nano-drone platform, showing a significant reduction in online computation time.

The findings are then extended in the second part with a theoretical formalization. The contribution includes the introduction of the concept called sub-control Lyapunov functions (SCLFs), a subclass of CLFs that allows analyzing the trade-off between real-time complexity and optimality in performance. An efficient procedure to solve the SILP and synthesize SCLFs based on the cutting-plane technique is also provided, showing the applicability for different examples of control affine systems. The study is then complete with the comprehensive simulations and the experimental validation of drone control.

As future work, it is practical to account for uncertainty and external disturbance in the offline synthesis. This can be done by introducing additional constraints to the linear program. Furthermore, it is also important to employ different objectives in the construction of the (S)CLFs, such as the size of the domain of attraction, the convergence speed or the simplicity of the generated functions [Johansen, 2000]. In addition, we

envision to extend the online implementation for the output feedback case as opposed to the developed state feedback setting, incorporating efficient state estimators and disturbance observers [Surana, 2016]. Last but not least, it interesting to point out that the coordinate change from flatness not only linearizes the model in closed-loop but also connects the dynamics to the class of control-affine systems.

> The linear representation of the quadcopter in the flat output space allows us to apply CLF-based techniques for the non-control-affine model (3.4).

Therefore, another future direction involves computing a CLF in the flat output space and then using it as the stability certificate also for the original systems [Fujii and Nakamura, 2018, Kubo et al., 2020].

Chapter 5

Insights on robustness and different flat outputs analysis

This chapter is devoted to studying the supporting questions for the presented results, including the robustness analysis, parameter tuning of the proposed settings and the implication of different flat outputs. Specifically, it explores various aspects of controlling constrained flat systems in the linearizing coordinates, moving from a specific case of drone control to a broader formulation for an online tractable program. Although the theory was successfully validated, there are practical concerns about how to robustify the proposed MPC designs or how to obtain satisfactory parameter tuning for the saturated control. Furthermore, given that all the established results are formulated for a single flat output, an important question arises: if different flat outputs are chosen, how would the formulations and results be affected?

Considering those points, the contributions of the chapter can be summarized as:

- we adapt the MPC framework in Section 3.3 with the robust tube MPC for the control of the quadcopter in the flat output space [C5];
- we complete the saturated design in Section 3.3.3 with an adaptive scheme to deal with the high gain tuning problem and provide the robustness analysis on the established scheme [C4];
- we analyze the implications of different flat outputs with the trajectory generation and tracking of the UAV model [C1];
- we formulate a generic setting to combine extended state observer framework with the linearized dynamics and apply it to the UAV with different flat outputs [C2];

5.1 Motivation . . . 159
5.2 Flatness-based tube MPC for quadcopter trajectory tracking . . . 160
5.2.1 Robust Tube MPC formulation for a quadcopter . . . 160
5.2.2 Experimental validation . . . 163
5.3 Flatness-based adaptive saturated control for quadcopter outer loop . . . 168
5.3.1 Saturated control with adaptive gain . . . 168
5.3.2 Robustness analysis . . . 171
5.3.3 Experimental validation . . . 174
5.4 Analysis of different flat outputs for the fixed-wing UAV model . . . 178
5.4.1 Various flat representations of the fixed-wing UAV model . . . 178
5.4.2 Trajectory generation using flatness . . . 180
5.4.3 Tracking control with feedback linearization . . . 183
5.4.4 Simulation result . . . 184
5.5 Robust flatness-based control for a fixed-wing UAV . . . 190
5.5.1 Output feedback for flat systems with extended state observers . . . 190
5.5.2 Application for a fixed-wing UAV . . . 194
5.5.3 Simulation results . . . 198
5.6 Concluding remarks . . . 202

5.1 Motivation

Emerged from a model-based linearization law, it is inevitable that the controllers proposed in the previous chapters will expose a sensitivity to model uncertainty or external disturbance. This weakness, if not treated properly, leads to detrimental effects such as undesirable tracking or even instability. For this reason, in this part we extend the previous designs with more robust control laws. The readjustment includes modifying the optimization problem of the MPC setting and adopting an adaptive law for the saturated control. In addition, the question of robustness once again raises the concern about the relation between the choice of flat output and the closed-loop performance. Thus, in what follows, we analyze the problem in the context of optimal trajectory generation and control of a fixed-wing UAV model, identifying and laying the groundwork for the future directions.

Firstly in Section 5.2, to adapt the MPC formulations in Section 3.3.1 and 3.3.2, since the model and the constraint are both linear, robust linear MPC techniques are the appropriate choice to handle model imperfection or disturbance. Among the existing strategies, tube-based MPC will be selected. This is due to the lack of structural information on the disturbances affecting the quadcopter, while tube-based MPC requires only the assumption on the boundedness of the disturbance. This mild assumption evidently leads to the conservativeness in the feasible domain, however, the formulation will be rather simple compared to that of min-max MPC [Limón et al., 2006] or stochastic MPC [Mesbah, 2016], for example.

Secondly, in Section 5.3, along the same line of addressing practical implementation concerns, the saturated control in Section 3.3.3 will be adjusted to tackle the gain selection problem. Theoretically, with the design in Proposition 3.1, the scalar gain $\gamma \geq 1$ can have an arbitrarily high value without changing the stability and constraint satisfaction guarantees. However, practically, excessively high-gain feedback may cause either the excitement of the non-modeled neglected dynamics (e.g., rotational dynamics, propeller actuation) or control over-actuation. Therefore, to better select γ, we enhance the proposed controller through a so-called "λ-tracking adaptation" [Blanchini and Giannattasio, 2002, Ilchmann and Ryan, 1994]. Then, to complete the scheme, we also accommodate the synthesis when external disturbances are taken into account and when the control objective is changed from stabilization to trajectory tracking.

Thirdly, Section 5.4 and 5.5 revisit the pertinent question of how the choice or change of flat output affects the performance of planning and control synthesis. To build on the framework presented throughout the manuscript, we conduct the performance analysis using the B-spline parameterization and set theory. This approach ensures that the evaluation remains consistent with the previously presented findings.

Finally, Section 5.6 concludes and suggests promising future extensions.

5.2 Flatness-based tube MPC for quadcopter trajectory tracking

The first section presents the adaptation of the tube MPC framework to the quadcopter linearized model (3.12) under the constraints (3.22).

5.2.1 Robust Tube MPC formulation for a quadcopter

In this part, we establish and implement the robust MPC strategy for the quadcopter's tracking error dynamics by stabilizing the tracking error dynamics with robust MPC.

5.2.1.1 Trajectory tracking error dynamics

First, recall the disturbed quadcopter model (3.4) in discrete time:

$$\boldsymbol{\zeta}(k+1) = \boldsymbol{A}_{6D}^{d}\boldsymbol{\zeta}(k) + \boldsymbol{B}_{6D}^{d}\boldsymbol{h}_{\psi}(\boldsymbol{u}(k)) + \boldsymbol{w}, \tag{5.1}$$

where $\boldsymbol{\zeta}, \boldsymbol{A}_{6D}^{d}, \boldsymbol{B}_{6D}^{d}$ are defined as in (3.60) and $\boldsymbol{w} \in \mathbb{R}^6$ denotes the system's disturbance which is assumed to be bounded, i.e.: $\boldsymbol{w} \in \mathcal{W}$. Then, by using the flat mapping (3.7), the system (5.1) becomes:

$$\boldsymbol{\zeta}(k+1) = \boldsymbol{A}_{6D}^{d}\boldsymbol{\zeta}(k) + \boldsymbol{B}_{6D}^{d}\boldsymbol{v}(k) + \boldsymbol{w}, \tag{5.2}$$

where the input $\boldsymbol{v}(k)$ is subject to the constraint $\boldsymbol{v}(k) \in \mathcal{V}_c^{\text{in}}$ as in (3.22). Next, we assume to have a well-defined reference trajectory of the quadcopter with all constraints satisfied, namely:

$$\begin{cases} \boldsymbol{\zeta}_{k+1}^{\text{ref}} = \boldsymbol{A}_{6D}^{d}\boldsymbol{\zeta}^{\text{ref}}(k) + \boldsymbol{B}_{6D}^{d}\boldsymbol{v}^{\text{ref}}(k), & \text{(5.3a)} \\ \boldsymbol{v}^{\text{ref}}(k) \in \mathcal{V}_c^{\text{ref}} \subset \mathcal{V}_c^{\text{in}} \text{ as in (3.22)}, & \text{(5.3b)} \end{cases}$$

where $\mathcal{V}_c^{\text{ref}}$ represents the polytope containing the input reference sequence $\boldsymbol{v}^{\text{ref}}(k)$. This set $\mathcal{V}_c^{\text{ref}}$ is assumed to be included by the constraint set $\mathcal{V}_c^{\text{in}}$ of $\boldsymbol{v}(k)$. Note that the dynamics expressed in (5.3a) simply represent the trajectory of three independent double integrators. Hence, effective time parameterization tools can be used to generate the reference signal satisfying the constraints (5.3b) [Nguyen et al., 2020a]. Subsequently, from (5.3) and (5.2), we achieve the tracking error dynamics:

$$\begin{cases} \tilde{\boldsymbol{\zeta}}_{k+1} = \boldsymbol{A}_{6D}^{d}\tilde{\boldsymbol{\zeta}}(k) + \boldsymbol{B}_{6D}^{d}\tilde{\boldsymbol{v}}(k) + \boldsymbol{w} & \text{(5.4a)} \\ \text{s.t } \tilde{\boldsymbol{v}}(k) \in \tilde{\mathcal{V}}, & \text{(5.4b)} \end{cases}$$

where $\tilde{\boldsymbol{\zeta}}(k) \triangleq \boldsymbol{\zeta}(k) - \boldsymbol{\zeta}^{\text{ref}}(k)$ denotes the deviation of the state $\boldsymbol{\zeta}(k)$ with respect to the reference $\boldsymbol{\zeta}^{\text{ref}}(k)$; likewise, $\tilde{\boldsymbol{v}}(k) \triangleq \boldsymbol{v}(k) - \boldsymbol{v}^{\text{ref}}(k)$ denotes the control action employed to

compensate such deviation and disturbances while $\tilde{\mathcal{V}} \triangleq \mathcal{V}_c^{\text{in}} \ominus \mathcal{V}_c^{\text{ref}}$ is the corresponding constraint set for the input $\tilde{\boldsymbol{v}}(k)$ of system (5.4a).

Prior to this point, the trajectory tracking problem has been reduced to the stabilization of the linear disturbed system (5.4) towards the origin $\tilde{\boldsymbol{\zeta}}_e = \mathbf{0}, \tilde{\boldsymbol{v}}_e = \mathbf{0}$. Therefore, in the next part, we synthesize the robust MPC [Mayne et al., 2005] subject to the bounded disturbance $\boldsymbol{w}$.

5.2.1.2 Robust model predictive control

As introduced above, let us formulate the robust MPC presented in [Mayne et al., 2005] for the constrained and disturbed system (5.4). The online optimization at step k is defined as follows:

$$\min \sum_{i=0}^{N_p-1} V_s(\tilde{\boldsymbol{\zeta}}^*(k|k+i), \tilde{\boldsymbol{v}}^*(k|k+i)) + V_f(\tilde{\boldsymbol{\zeta}}^*(k|k+N_p)) \tag{5.5}$$

$$\text{s.t} \begin{cases} \tilde{\boldsymbol{\zeta}}^*(k|k+i+1) = \boldsymbol{A}_{6D}^d \tilde{\boldsymbol{\zeta}}^*(k|k+1) + \boldsymbol{B}_{6D}^d \tilde{\boldsymbol{v}}^*(k|k+1); & \text{(5.6a)} \\ \tilde{\boldsymbol{\zeta}}^*(k) \in \tilde{\boldsymbol{\zeta}}(k) \oplus \mathcal{Z};\ \tilde{\boldsymbol{\zeta}}^*(k|k+N_p) \in \mathcal{E}_f; & \text{(5.6b)} \\ \tilde{\boldsymbol{v}}^*(k|k+1) \in \mathcal{V}^\star \triangleq \tilde{\mathcal{V}} \ominus \boldsymbol{K}\mathcal{Z},\ i \in \{0, 1, ..., N_p - 1\}, & \text{(5.6c)} \end{cases}$$

where $\boldsymbol{A}_{6D}^d, \boldsymbol{B}_{6D}^d$ is defined as in (3.60); $\tilde{\mathcal{V}}$ is defined in (5.4); $V_s(\cdot), V_f(\cdot)$ are the quadratic cost given in (2.81) with given weighting $\boldsymbol{Q} \succ 0, \boldsymbol{R} \succeq 0$ of choice; N_p is the prediction horizon; $\tilde{\boldsymbol{\zeta}}^*(k|k+1)$ and $\tilde{\boldsymbol{v}}^*(k|k+1)$ denote the predicted state and control action at time step $i+k$, respectively, for the nominal version of system (5.2) (i.e., without the effect of the disturbance $\boldsymbol{w}$). The remaining ingredients in the constraints (5.6) are defined as follows. Firstly, given a state feedback gain $\boldsymbol{K}$ that renders the closed-loop matrix $\boldsymbol{A}_{cl} = \boldsymbol{A}_{6D}^d + \boldsymbol{B}_{6D}^d \boldsymbol{K}$ a Schur matrix, then $\mathcal{Z}$ is a robust positively invariant (RPI) set [Rakovic et al., 2005, Kolmanovsky and Gilbert, 1998] for the autonomous system: $\tilde{\boldsymbol{\zeta}}_{k+1} = \boldsymbol{A}_{cl}\tilde{\boldsymbol{\zeta}}(k) + \boldsymbol{w}$. In this setting, we employ the outer-approximation of the minimal RPI proposed in [Rakovic et al., 2005]. Then, the set $\mathcal{Z}$ plays a role as the "origin" for our stabilizing problem [Mayne et al., 2005]. Next, the terminal ingredients ($\boldsymbol{P}$ and the set $\mathcal{E}_f$) for the nominal MPC settings are found exactly with the Proposition 2.6 with the gain $\boldsymbol{K}$ chosen as aforementioned, the dynamics (5.6a) and the constraint set $\mathcal{V}^\star \triangleq \tilde{\mathcal{V}} \ominus \boldsymbol{K}\mathcal{Z}$. It is also assumed that the disturbance's bound $\mathcal{W}$ is sufficiently manageable and hence results in non-empty feasible sets for the constraints in (5.6).

In short, the offline synthesis can be summarized as in Algorithm 5.1.

Remark 5.1. To avoid computational issues caused by the system's large dimensionality, we do not employ the computation of the maximal positively invariant set [Gilbert and Tan, 1991] for the closed-loop $\tilde{\boldsymbol{\zeta}}_{k+1} = \boldsymbol{A}_{cl}\tilde{\boldsymbol{\zeta}}(k)$ to play the role of the terminal region $\mathcal{E}_f$. More specifically, although proven finite, the number of equalities representing such a set for this 6 dimensional problem may exceed the memory capacity provided for

Algorithm 5.1: Tube MPC synthesis for quadcopter trajectory tracking

1 Choose $\boldsymbol{Q} \succ 0$ and $\boldsymbol{R} \succeq 0$ to formulate the stage and terminal cost value function of the form (2.81); ;
2 Choose the gain $\boldsymbol{K}$ stabilizing the pair $(\boldsymbol{A}_{6D}^{d}, \boldsymbol{B}_{6D}^{d})$; ;
3 Compute the mRPI set $\mathcal{Z}$ as proposed in [Rakovic et al., 2005]; ;
4 Compute the terminal ingredients subject to the constraint $\mathcal{V}^{\star} \triangleq \tilde{\mathcal{V}} \ominus \boldsymbol{K}\mathcal{Z}$ as in Proposition 2.6.

the computation software. Yet, if the polyhedral set can be computed, the terminal constraints $\mathcal{E}_f$ will become linear, as opposed to being quadratic as in (2.79), and reduce the complexity of the online problem (5.5)–(5.6).

Nevertheless, the control input applied to the system (5.4a) is defined as:

$$\tilde{\boldsymbol{v}}(k) = \tilde{\boldsymbol{v}}_{N_p}^{\mathrm{mpc}}(\tilde{\boldsymbol{\zeta}}(k)) \triangleq \tilde{\boldsymbol{v}}^{*}(k|k) + \boldsymbol{K}(\tilde{\boldsymbol{\zeta}}(k) - \tilde{\boldsymbol{\zeta}}^{*}(k)). \tag{5.7}$$

Consequently, with this scheme, as proven in [Mayne et al., 2005], the set $\mathcal{Z}$ is robustly exponentially stable for the closed-loop:

$$\tilde{\boldsymbol{\zeta}}_{k+1} = \boldsymbol{A}_{6D}^{d}\tilde{\boldsymbol{\zeta}}(k) + \boldsymbol{B}_{6D}^{d}\tilde{\boldsymbol{v}}_{N_p}^{\mathrm{mpc}}(\tilde{\boldsymbol{\zeta}}(k)) + \boldsymbol{w}. \tag{5.8}$$

Namely, $\tilde{\boldsymbol{\zeta}}(k)$ is maintained inside the "tube" $\tilde{\boldsymbol{\zeta}}^{*}(k) \oplus \mathcal{Z}$ which converges to $\mathcal{Z}$ when $k \to \infty$. In other words, one can state:

$$\boldsymbol{\zeta}(k) - \boldsymbol{\zeta}^{\mathrm{ref}}(k) = \tilde{\boldsymbol{\zeta}}(k) \in \tilde{\boldsymbol{\zeta}}^{*}(k) \oplus \mathcal{Z} \Leftrightarrow \boldsymbol{\zeta}(k) \in \boldsymbol{\zeta}^{\mathrm{ref}}(k) \oplus \tilde{\boldsymbol{\zeta}}^{*}(k) \oplus \mathcal{Z} \tag{5.9}$$

with $\tilde{\boldsymbol{\zeta}}^{*}(k)$ exponentially converges to $\boldsymbol{0}_{6\times 1}$. With the sequence of the reference for $\boldsymbol{\zeta}(k)$ and $\boldsymbol{v}(k)$ denoted as $\boldsymbol{\zeta}^{\mathrm{ref}}, \boldsymbol{v}^{\mathrm{ref}}$, respectively, the control implementation is given by Algorithm 5.2.

Algorithm 5.2: Robust MPC for trajectory tracking

Input: Reference sequence $\boldsymbol{\zeta}^{\mathrm{ref}}, \boldsymbol{v}^{\mathrm{ref}}$; the optimization setup as in (5.5)-(5.6).
1 **for** $(\boldsymbol{\zeta}^{\mathrm{ref}}(k), \boldsymbol{v}^{\mathrm{ref}}(k))$ **in** $\boldsymbol{\zeta}^{\mathrm{ref}} \times \boldsymbol{v}^{\mathrm{ref}}$ **do**
2 Measure $\boldsymbol{\zeta}(k)$ and ψ at time step k;
3 Compute $\tilde{\boldsymbol{\zeta}}^{*}(k|k+i)$ and $\tilde{\boldsymbol{v}}^{*}(k|k+1), i \in \{0, ..., N_p - 1\}$ by solving (5.5)-(5.6);
4 $\tilde{\boldsymbol{v}}_{N_p}^{\mathrm{mpc}} \leftarrow \tilde{\boldsymbol{v}}^{*}(k|k) + \boldsymbol{K}(\tilde{\boldsymbol{\zeta}}(k) - \tilde{\boldsymbol{\zeta}}^{*}(k))$ as in (5.7);
5 $\boldsymbol{u}(k) \leftarrow \boldsymbol{\beta}_{\psi}(\boldsymbol{v}^{\mathrm{ref}}(k) + \tilde{\boldsymbol{v}}_{N_p}^{\mathrm{mpc}})$ as in (3.7);
6 Apply $\boldsymbol{u}(k)$ to the vehicle;
7 **end**

The next part presents the experimental validation of the proposed framework, analyzing its applicability and the effectiveness.

5.2.2 Experimental validation

With the experimental platform presented in Section 3.2, we consider the following validation tests.

5.2.2.1 Parameter setup

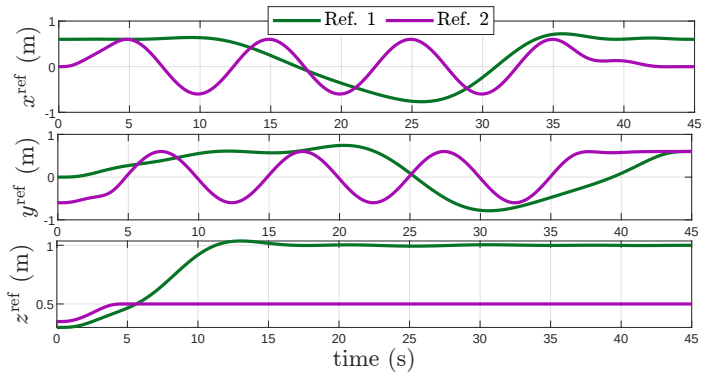


FIGURE 5.1: Two references employed for the experimental validation.

Firstly, the reference is required to be at least two time differentiable to guarantee the continuity and boundedness of $\boldsymbol{v}^{\text{ref}}$ as in (5.3b). Herein,we use two references as:

- *Ref. 1*: The position of the drone will be parameterized in time with the B-spline method [Nguyen et al., 2020a]. More specifically, at timestamp $t_r\,(s)$, we imposed the position of the drone to pass through a waypoint $\boldsymbol{p}_r$ as:

$$\begin{bmatrix} x^{\text{ref}} \\ y^{\text{ref}} \\ z^{\text{ref}} \end{bmatrix} \in \left\{ \begin{matrix} [6,0,3];[6,3,5];[6,6,10]; \\ [0,6,10];[-6,6,10];[-6,-6,10]; \\ [6,-6,10];[6,0,10];[6,6,10] \end{matrix} \right\} \times 10\,(\text{cm}), \tag{5.10}$$

  with $t_r \in \mathcal{N}(9,0,45)$ (s). Then, by minimizing the curve's length and restraining their second derivatives (which is the mathematical interpretation of $\boldsymbol{v}^{\text{ref}}(k)$) inside $\mathcal{V}_c^{\text{in}}$, we obtain the green solid curve depicted in Figure 5.1.

- *Ref. 2*: The circular motion is parameterized as:

$$x^{\text{ref}}(t) = 0.6\cos\omega t, y^{\text{ref}}(t) = \sin 0.6\omega t, z^{\text{ref}}(t) = 0.5\ (\text{m}), \omega = 0.2\pi\ \text{rad/s}, \tag{5.11}$$

and illustrated as the purple curve in Figure 5.1. Note that at the beginning and the end of the reference, we introduce smooth curves to reduce the overshoot effect caused by introducing a sudden reference.

Consequently, $\mathcal{V}_c^{\text{ref}}$ as in (5.3b) can be obtained by bounding the $\boldsymbol{v}^{\text{ref}}$ with a box-type constraint (See Figure 5.2).

The parameter of constraint set $\mathcal{U}$ as in (3.5) is provided in Table 5.1 while the corresponding input constraint is the set $\mathcal{V}_c^{\text{in}}$ as in (3.22). Finally, the disturbance set $\mathcal{W}$ as

TABLE 5.1: Controller's parameters and system's specification

Parameters	Values
Sampling time t_s (s)	0.2
Prediction horizon N_p	10 steps
T_{max}, ϵ_{max} in (3.5)	$1.45g \approx 14.22\, m/s^2$; $0.1745\, rad$
$(\mu_K, \mu_p, \mu_v, \mu_r)$ in (5.13)-(5.14)	(0.5, 50, 15, 25)

in (5.1) is considered to be bounded as:

$$\mathcal{W} = \{\boldsymbol{w} : |\boldsymbol{w}| \leq 0.02\}, \tag{5.12}$$

which is estimated from our data of the difference between the system predicted and actual behavior after the same control action [Michel et al., 2019]. The set was intentionally chosen to be symmetric so that the heavy computation of the mRPI set $\mathcal{Z}$ in 6 dimensional space is reduced to that of the three double integrators as in (5.2) and their Cartesian product.

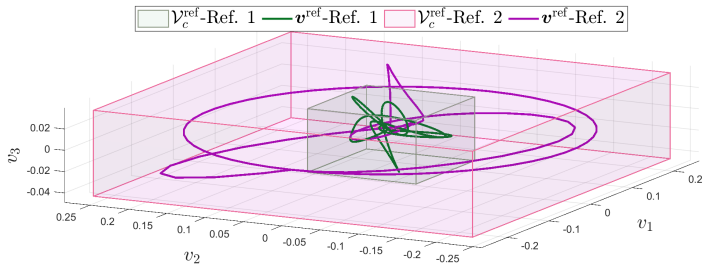


FIGURE 5.2: Two input $\boldsymbol{v}^{\text{ref}}$ of the two references and their bounding boxes.

5.2.2.2 Controller tuning

First, for simplicity, $\boldsymbol{Q}, \boldsymbol{R}$ in (5.5) are chosen in the following form:

$$\begin{cases} \boldsymbol{Q} = \text{diag}(\boldsymbol{Q}^*, \boldsymbol{Q}^*, \boldsymbol{Q}^*); \boldsymbol{Q}^* \triangleq \text{diag}(\mu_p, \mu_v); \\ \boldsymbol{R} = \text{diag}(\mu_r, \mu_r, \mu_r), \end{cases} \tag{5.13}$$

where μ_p, μ_v, μ_r can be interpreted as the penalties for the deviation in position, velocity and input from the reference. The gain $\boldsymbol{K}$ in (5.6c) is found by placing the poles $p_i, i \in \{1, ..., 6\}$ in continuous time and computing the corresponding poles via the Z-transformation (i.e., $z_i = e^{p_i t_s}$) for the closed-loop $\boldsymbol{A}_{cl} = \boldsymbol{A}_{6D}^d + \boldsymbol{B}_{6D}^d \boldsymbol{K}$. We proceed to choose the poles in continuous time as:

$$p_1 = p_2 = p_3 = -4\mu_K \text{ and } p_4 = p_5 = p_6 = -5\mu_K, \tag{5.14}$$

with the intention of analyzing the effect of choosing poles with different rise times or convergence rates. Then we provide some criteria for choosing the gain and matrices appropriately. Illustrative examples for the tracking problem of Ref. 1 are given to analyze the closed-loop characteristics.

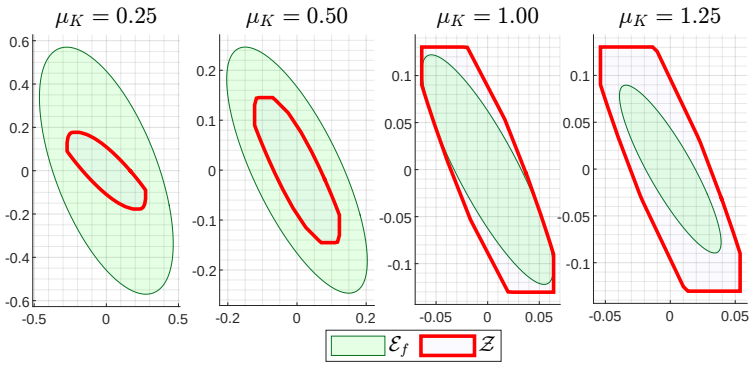


FIGURE 5.3: Effect of changing the gain K towards fast closed-loop convergence, projected into the first two axes of $\tilde{\boldsymbol{\zeta}}(k)$.

On one hand, as mentioned in Remark 2.5 and Proposition 2.6, for the nominal system, the stabilizing gain $\boldsymbol{K}$ also needs to be chosen so that $\mathbf{0}_{6\times 1} \in \tilde{\mathcal{X}} \triangleq \{\tilde{\boldsymbol{\zeta}} : \boldsymbol{K}\tilde{\boldsymbol{\zeta}} \in \mathcal{V}^\star\}$. Meanwhile, under the presence of $\mathcal{W}$, $\boldsymbol{K}$ needs to stabilize the closed-loop robustly or fast enough so that the corresponding mRPI set $\mathcal{Z}$ in (5.6b) becomes satisfactory in terms of size. More specifically, as $\mathcal{Z}$ represents the bound of the tracking error, it reflects the controller's performance guarantee. With these observations, firstly, we would like to choose the gain $\boldsymbol{K}$ as "aggressive" as possible, or in our settings, μ_K as large as possible. Secondly, the choice of gain $\boldsymbol{K}$ is also tangled with the size of $\mathcal{E}_f$ which necessarily requires including $\mathcal{Z}$ (i.e., $\mathcal{Z} \subset \mathcal{E}_f$) so that the input constraints $\boldsymbol{K}\tilde{\boldsymbol{\zeta}} \in \mathcal{V}^\star = \tilde{\mathcal{V}} \ominus \boldsymbol{K}\mathcal{Z}$ are also satisfied inside $\mathcal{Z}$, and so is the closed-loop performance. Therefore, as illustrated in Figure 5.3, the two last choices of $\mu_K = 1.0$ and 1.25 violate the latter, while the first choice $\mu_K = 0.25$ results in a poor guarantee for the closed-loop (with the weighting fixed as $\mu_q = 50, \mu_v = 15$ and $\mu_r = 25$) as stated in the former principle.

On the other hand, after the suitable choice of $\boldsymbol{K}$ fixing the set $\mathcal{Z}$, it is also possible to modify the set $\mathcal{E}_f$ with $\boldsymbol{Q}$ and $\boldsymbol{R}$. The motivation for this adjustment comes from the fact that, for the stabilization of system (5.4), $\mathcal{E}_f \oplus \mathcal{Z}$ plays the role of the new terminal region

for the system's tube trajectory[Mayne et al., 2005]. Therefore, for a given initial tracking error point, the set can be modified to be "closer" or even to include the point, hence, resulting in a shorter prediction interval N_p. Depending on how the penalty weights are chosen, the set $\mathcal{E}_f$ can be characterized to some extent, while the condition $\mathcal{Z} \subset \mathcal{E}_f$ also needs to be ensured. For the experiments, the weighting matrices and gain are given in Table 5.1. The experiment video can be found at `https://youtu.be/aJHZyouXtwo`.

5.2.2.3 Experimental results and discussion

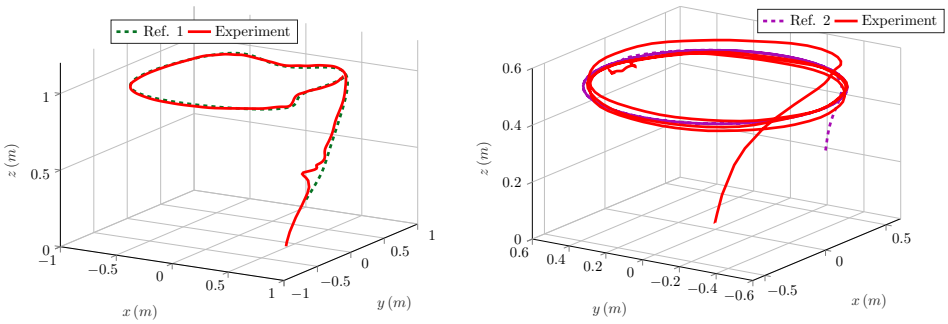


FIGURE 5.4: Position tracking with the two references.

Figure 5.4 and Figure 5.5 show the position tracking performance and how the tracking error of the quadcopter is restrained inside the tube $\mathcal{Z}$, respectively. Numerical details are also given in Table 5.2 where, the matrix $\boldsymbol{P}$ is represented as:

$$\boldsymbol{P} \triangleq \text{diag}(\boldsymbol{P}^*, \boldsymbol{P}^*, \boldsymbol{P}^*). \tag{5.15}$$

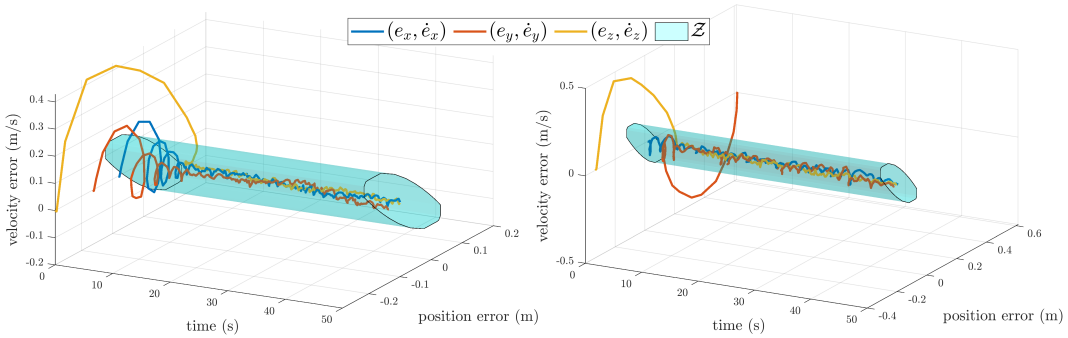


FIGURE 5.5: Tracking error $e_q = q - q^{\text{ref}}, q \in \{x, y, z\}$ for Ref. 1 (left) and Ref. 2 (right).

Overall, the controller is successfully implemented and verified within the flight tests with around 3-5 cm in root-mean-square (RMS) of tracking error and under 40ms for the average computation time at each control loop (see Figure 5.6). These results show

that not only is the performance guaranteed, but also that the method is computationally efficient thanks to the linear model prediction obtained via the coordinate change (3.7).

TABLE 5.2: Experimental results and control parameters

	Ref. 1	Ref. 2
RMS of tracking error	3.25 cm	5.46 cm
Average computation time	39.40 ms	38.11 ms
$\boldsymbol{P}^*$ as in (5.15)	$\begin{bmatrix} 513.54, 328.59 \\ 328.59, 356.89 \end{bmatrix}$	$\begin{bmatrix} 542.18, 329.80 \\ 329.80, 368.92 \end{bmatrix}$
ε_f as in (2.79)	8.7104	6.0724

It is also noticeable that, although later adequately concealed in the tube, there are significant overshoots of errors at the beginning of both flights. This problem comes from the fact that the drone starts from a static state with zero angular velocity of the propellers, which later entails a sudden acceleration of the vehicle. However, all three inputs are not saturated and remain constrained in their bounds (see Figure 5.7).

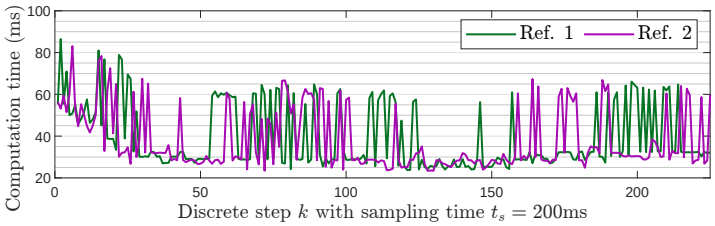


FIGURE 5.6: Computation time for the proposed MPC scheme.

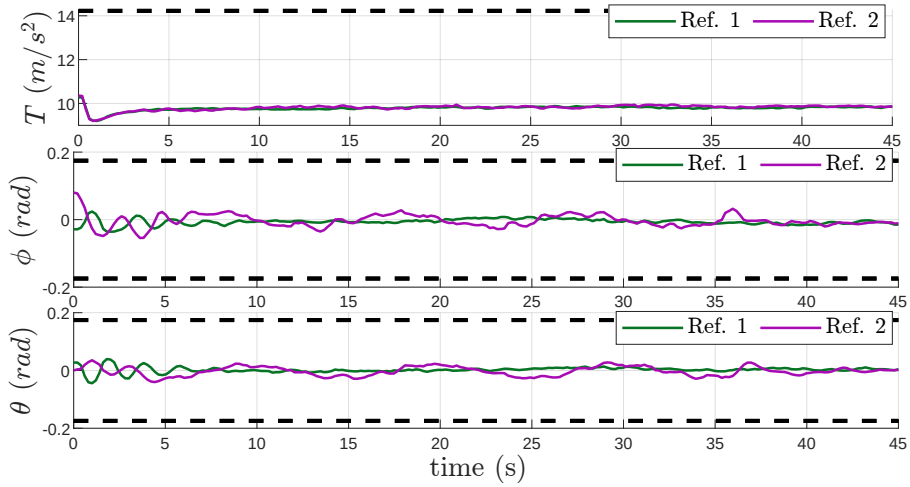


FIGURE 5.7: Quadcopter's inputs for the tracking of Ref.1 and 2.

The above validation confirms again the benefits of flatness-based linearization in the application of linear tube MPC to the nonlinear control problem of the quadcopter.

Next, the explicit saturated control designed in Section 3.3.3 will be completed with an adaptive scheme, solving the parameter tuning problem.

5.3 Flatness-based adaptive saturated control for quadcopter outer loop

As previously proven with the controller (3.32), although the stability can be maintained for all $\gamma \geq 1$, the gain should not be chosen excessively high. In the following, let us discuss an adaptive law assigned to γ to remedy the problem of gain selection.

5.3.1 Saturated control with adaptive gain

As noted in Section 3.3.3, exploiting the controller's stabilizing effect for all $\gamma \geq 1$, we assign to $\gamma(t)$ a non-decreasing dynamics and a stopping condition associated with the convergence of the Lyapunov function (3.41). In such manner, $\gamma(t)$ can be proven to converge to a finite bound, solving the problem of manually tuning the parameter. More specifically, consider the following control mechanism:

$$\boldsymbol{v} = \mathrm{sat}_{\lambda}(-\gamma(t)\boldsymbol{B}^{\top}\boldsymbol{P}\boldsymbol{\xi}), \tag{5.16a}$$

$$\dot{\gamma} = \mu\sigma_t(V(\boldsymbol{\xi})),\ \gamma(0) = \gamma_0 \geq 1, \tag{5.16b}$$

where, $\sigma_t(\mathrm{s})$ defines the threshold function [Ilchmann and Ryan, 1994, Blanchini and Giannattasio, 2002]:

$$\sigma_t(\mathrm{s}) = \begin{cases} 0 & \text{if } 0 \leq \mathrm{s} \leq V_{\infty}, \\ \mathrm{s} - V_{\infty} & \text{if } \mathrm{s} \geq V_{\infty}, \end{cases} \tag{5.17}$$

while $\mu > 0$ is the scalar factor adjusting the adaptation speed and V_{∞} is the stopping threshold for the adaptation of $\gamma(t)$. The matrix $\boldsymbol{P}$ is designed as in Proposition 3.1 with the invariant set $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35).

Remark 5.2. As discussed in [Blanchini and Giannattasio, 2002], with the adaptive law (5.16), $\gamma(t)$ describes a non-decreasing value function starting from $\gamma(0) = \gamma_0 \geq 1$, Thus, by the synthesis in Proposition 3.1, the stability and invariance of the system remain valid inside the ellipsoid $\mathcal{E}(\boldsymbol{P}, \varepsilon)$ as in (3.35). Meanwhile, it can be proven that $\gamma(t)$ when $t \to \infty$ is bounded by a finite scalar. More specifically, from a given initial point $\boldsymbol{\xi}(0) \in \mathcal{E}(\boldsymbol{P}, \varepsilon)$, when $V(\boldsymbol{\xi}) \geq V_{\infty}$, owing to (3.49), we have:

$$\begin{aligned} \frac{\mathrm{d}\gamma(t)}{\mathrm{d}t} &= \mu(V(\boldsymbol{\xi}(t)) - V_{\infty}) \leq \mu(V(\boldsymbol{\xi}(0))e^{-\alpha t} - V_{\infty}) \\ \Rightarrow \gamma(t_{\infty}) &\leq \gamma(0) + \mu(V_{\infty}t_{\infty} + \alpha^{-1}V(\boldsymbol{\xi}(0))(1 - e^{-\alpha t_{\infty}})) \end{aligned} \tag{5.18}$$

where t_∞ denotes the required time for $V(\boldsymbol{\xi}(t))$ to reach V_∞ starting from $V(\boldsymbol{\xi}(0))$ which can be bounded from the exponential convergence of $V(\boldsymbol{\xi})$ (given in (3.49)) as:

$$\frac{\mathrm{d}V(\boldsymbol{\xi})}{\mathrm{d}t} \leq -\alpha V(\boldsymbol{\xi}) \Leftrightarrow \ln \frac{V_\infty}{V(\boldsymbol{\xi}(0))} \leq -\alpha t_\infty \Leftrightarrow t_\infty \leq \frac{1}{\alpha} \ln \frac{V(\boldsymbol{\xi}(0))}{V_\infty}. \tag{5.19}$$

Hence, from (5.18) and (5.19), we can compute the finite upper bound of $\gamma(t)$.

Simulation study: For the sake of illustration and parameter study, let us simulate the proposed controller in the following setup. We study the behavior of the system (3.4) under the controller (5.16) from $\boldsymbol{\xi}(0) = [1.05, 1.04, 0.85, -0.02, -1.39, -0.42,]^\top$ where the adaptive gain μ is examined with different values. Detailed numerical parameters are provided in TABLE 5.3.

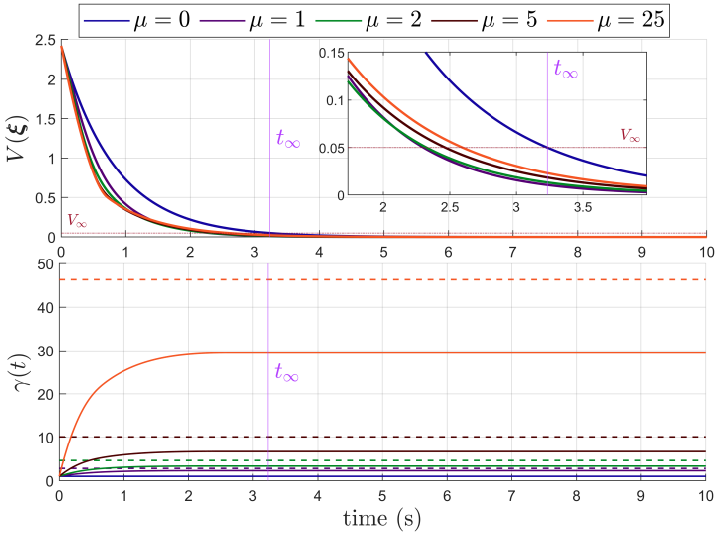


FIGURE 5.8: Lyapunov function $V(\boldsymbol{\xi})$ (top), the evolution of $\gamma(t)$ and its upper bound, which is color-coded with μ in dashed-line (bottom).

In Figure 5.8, with μ varying from 1 to 5, the convergence time is improved compared to that of the case $\mu = 0$ (i.e., without any adaptation). However, in the same figure, the redundancy of excessively high adaptation speed is also highlighted. More specifically, although the performance in the transition state (from 0 to 2 seconds) is ameliorated accordingly to the increase of μ, the convergence times near the steady state (from 2 to 3.5 seconds) become higher. This can be explained by the "drifting" saturation effect of the input $\boldsymbol{v}$ on the surface of $\mathcal{V}_c$ (see Figure 5.9), leading to unnecessary control effort to bring the system to the equilibrium.

Moreover, although the input's constraint satisfaction, its continuity and the system's stability are always guaranteed, the saturation effect with high gain feedback result in the input signal $\boldsymbol{u}$ with relatively high rate of change. This might exceed the physical limit of

TABLE 5.3: Controller's parameters for the quadcopter system

Parameters	Values
T_{max} as in (3.5)	$1.45g \approx 14.22m/s^2$
ϵ_{max} as in (3.5)	0.1745 rad (10^o)
(γ_0, V_∞) as in (5.16b),(5.17)	$(1, 0.05)$
$\boldsymbol{Q}$ and α as in (3.56)	$\begin{bmatrix} 2.3148\boldsymbol{I}_3 & -1.3889\boldsymbol{I}_3 \\ -1.3889\boldsymbol{I}_3 & 1.6667\boldsymbol{I}_3 \end{bmatrix}, 1.2$
ρ as in (3.31), ε as in (3.35)	$2.9019, 2.4182$
The bound of t_∞ as in (5.19)	3.2323 (s)

the system's actuators. Therefore, an overly big adaptation gain μ is not recommended both for theoretical simulation and practical implementation. Besides, our upper-bound given in (5.18) for the non-decreasing evolution of $\gamma(t)$ is also validated via the simulation tests, as depicted in Figure 5.8. It can be observed that, due to the increasingly fast transitional behavior with respect to μ, the first inequality of (5.18) become loosen, hence creating a correspondingly larger gap between $\gamma(t_\infty)$ and the precomputed upper bound as given in (5.18).

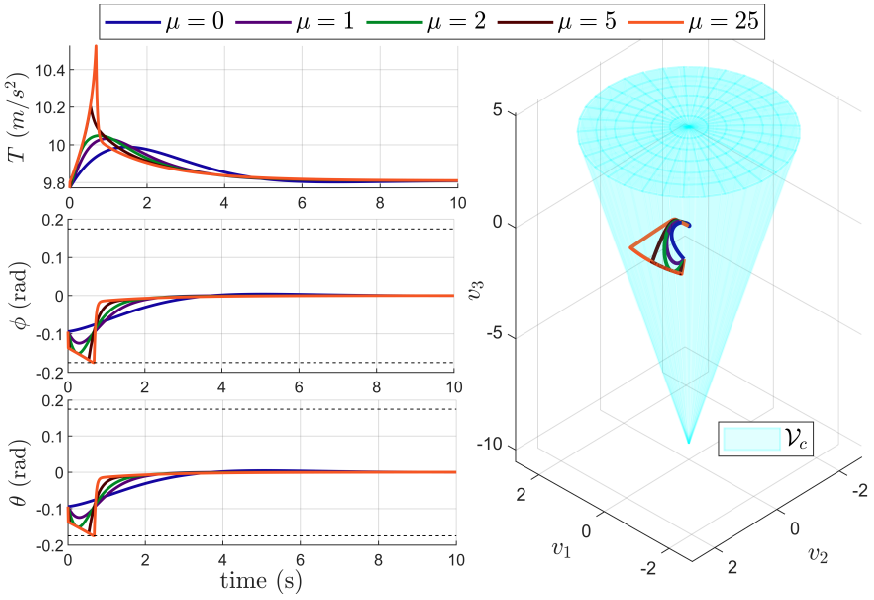


FIGURE 5.9: Real input $\boldsymbol{u}$ and the input $\boldsymbol{v}$ in the new coordinates.

Next, the control strategy is modified to characterize the robustness of the method in the presence of disturbances.

5.3.2 Robustness analysis

Consider the disturbed model of the linearized quadcopter (3.10) as follows:

$$\dot{\boldsymbol{\xi}} = \boldsymbol{A}\boldsymbol{\xi} + \boldsymbol{B}\boldsymbol{v} + \boldsymbol{E}\boldsymbol{w}. \tag{5.20}$$

Then, our saturated control is adjusted as follows.

Proposition 5.1. *Take a box $\mathcal{B}_v \subset \mathcal{V}_c$ in (3.14) described as:*

$$\mathcal{B}_v \triangleq \{\boldsymbol{v} = [v_1, v_2, v_3]^\top : |v_i| \leq \bar{v}_i, i = 1, 2, 3\}. \tag{5.21}$$

Then, the ellipsoid

$$\mathcal{E}_w = \{\boldsymbol{\xi} \in \mathbb{R}^6 : \boldsymbol{\xi}^\top \boldsymbol{P}_w \boldsymbol{\xi} \leq 1\}, \tag{5.22}$$

is robust positively invariant, under the controller:

$$\boldsymbol{v} = sat_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi}), \forall \gamma \geq 1, \tag{5.23}$$

where, for some scalar $\beta > 0$, $\boldsymbol{P}_w \triangleq \boldsymbol{Q}_w^{-1}$ is found by solving the following LMIs:

$$\begin{cases} \begin{bmatrix} \bar{v}_i^2 & r_i(\boldsymbol{B}^\top) \\ r_i(\boldsymbol{B}) & \boldsymbol{Q}_w \end{bmatrix} \succeq 0, i \in \{1, 2, 3\}, & \text{(5.24a)} \\ \boldsymbol{Q}_w \boldsymbol{A}^\top + \boldsymbol{A}\boldsymbol{Q}_w - 2\boldsymbol{B}\boldsymbol{B}^\top + \beta \boldsymbol{Q}_w + \beta^{-1}\boldsymbol{E}\boldsymbol{E}^\top, & \text{(5.24b)} \end{cases}$$

where $r_i(\boldsymbol{B})$ denotes the ith row of the matrix $\boldsymbol{B}$.

Proof. The proof for the invariance of $\mathcal{E}_w$ is threefold. First, we show that with (5.24a), the nominal control yields:

$$\boldsymbol{v}_{nom} \triangleq -\boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi} \in \mathcal{B}_v, \forall \boldsymbol{\xi} \in \mathcal{E}_w. \tag{5.25}$$

Indeed, inside $\mathcal{E}_w$, each row of $\boldsymbol{v}_{nom}$ is bounded as:

$$|\mathrm{r}_i(-\boldsymbol{B}^\top \boldsymbol{P}_w)\boldsymbol{\xi}| \leq \sqrt{\mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w)\boldsymbol{Q}_w \mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w)^\top}, \tag{5.26}$$

for $i \in \{1, 2, 3\}$. Then, bounding the right-hand side of (5.26) with $\bar{v}_i$ as in (5.21), performing the Schur complement and pre-post multiplying the result with $\begin{bmatrix} 1 & \boldsymbol{0}_{1\times 6} \\ \boldsymbol{0}_{6\times 1} & \boldsymbol{Q}_w \end{bmatrix}$ respectively yield:

$$\begin{aligned} &\mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w)\boldsymbol{Q}_w \mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w)^\top \leq \bar{v}_i^2 \\ &\Leftrightarrow \begin{bmatrix} \bar{v}_i^2 & \mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi}) \\ \mathrm{r}_i(\boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi})^\top & \boldsymbol{Q}_w^{-1} \end{bmatrix} \succeq 0 \Leftrightarrow \begin{bmatrix} \bar{v}_i^2 & \mathrm{r}_i(\boldsymbol{B}^\top) \\ \mathrm{r}_i(\boldsymbol{B}) & \boldsymbol{Q}_w \end{bmatrix} \succeq 0. \end{aligned} \tag{5.27}$$

Hence, (5.25) holds under the condition (5.24a). Second, by the definition (3.33), $\forall \boldsymbol{v} \in \mathcal{B}_v \subset \mathcal{V}_c, \gamma \geq 1$, we always have:

$$\|\text{sat}_\lambda(\gamma \boldsymbol{v})\|_2^2 = \|\gamma \lambda^*(\gamma \boldsymbol{v})\boldsymbol{v}\|_2^2 \geq \|\eta^*(\boldsymbol{v})\boldsymbol{v}\|_2^2 \geq \|\boldsymbol{v}\|_2^2, \tag{5.28}$$

where $\eta^*(\boldsymbol{v}) \triangleq \max_{\eta \boldsymbol{v} \in \mathcal{B}_v} \eta$, which leads to:

$$\gamma \lambda^*(\gamma \boldsymbol{v}) \geq 1, \forall \boldsymbol{v} \in \mathcal{B}_v \text{ in } (5.21), \gamma \geq 1. \tag{5.29}$$

Then, finally, by using (5.29) for $\boldsymbol{v}_{nom} \in \mathcal{B}_v, \forall \boldsymbol{\xi} \in \mathcal{E}_w$ as in (5.25), we can show that:

$$\begin{aligned}
&\boldsymbol{Q}_w \boldsymbol{A}_{cl}(\boldsymbol{\xi})^\top + \boldsymbol{A}_{cl}(\boldsymbol{\xi})\boldsymbol{Q}_w + \beta \boldsymbol{Q}_w + \beta^{-1}\boldsymbol{E}\boldsymbol{E}^\top \\
&\preceq \boldsymbol{Q}_w \boldsymbol{A}^\top + \boldsymbol{A}\boldsymbol{Q}_w - 2\boldsymbol{B}\boldsymbol{B}^\top + \beta \boldsymbol{Q}_w + \beta^{-1}\boldsymbol{E}\boldsymbol{E}^\top - (\gamma \lambda^*(-\gamma \boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi}) - 1)\boldsymbol{P}\boldsymbol{B}\boldsymbol{B}^\top \boldsymbol{P} \\
&\preceq \boldsymbol{Q}_w A^\top + \boldsymbol{A}\boldsymbol{Q}_w - 2\boldsymbol{B}\boldsymbol{B}^\top + \beta \boldsymbol{Q}_w + \beta^{-1}\boldsymbol{E}\boldsymbol{E}^\top \preceq 0
\end{aligned}$$

with $\boldsymbol{A}_{cl}(\boldsymbol{\xi}) \triangleq \left(\boldsymbol{A} - \boldsymbol{B}\gamma \lambda^*(-\gamma \boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi})\boldsymbol{B}^\top \boldsymbol{P}\right)$. Namely, with (5.24), and the controller (5.23), the closed-loop dynamics:

$$\dot{\boldsymbol{\xi}} = \boldsymbol{A}\boldsymbol{\xi} + \boldsymbol{B}\text{sat}_\lambda(-\gamma \boldsymbol{B}^\top \boldsymbol{P}_w \boldsymbol{\xi}) + \boldsymbol{E}\boldsymbol{w} \tag{5.30}$$

satisfy the invariance condition given in Proposition 2.5. □

Simulation study: Let us analyze the procedure with two different choices of γ for the stabilization starting from the boundary points of $\mathcal{E}_w$. The disturbance will be simulated by the bounded wind on x, y axis $\boldsymbol{w} \triangleq [w_x, w_y, 0, 0, 0, 0]^\top$. Simulation specifications are given in TABLE 5.4.

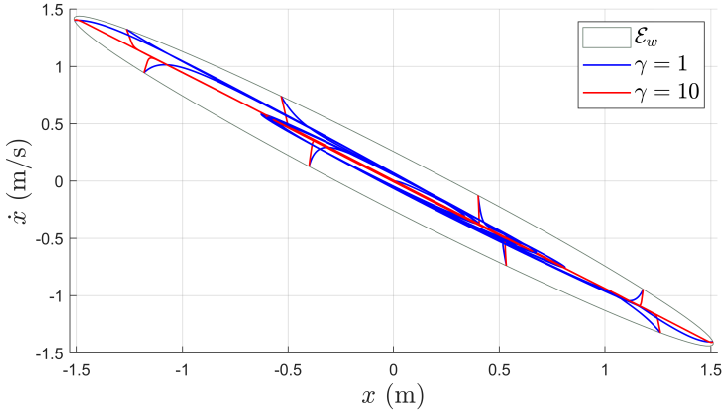


FIGURE 5.10: The system trajectories from various initial points, subject to disturbances (projected in $(x, \dot{x})$ subspace)

Via the simulation, the invariance is validated for both gains $\gamma = 1$ and $\gamma = 10$. Besides, one noticeable difference between the two choices of gains is that the system's trajectory appears to be more compressed with the latter gain towards the surface of $\boldsymbol{B}^\top \boldsymbol{P}\boldsymbol{\xi} = 0$ (See Figure 5.10). This behavior can be explained by the more aggressive control action generated locally near the surface.

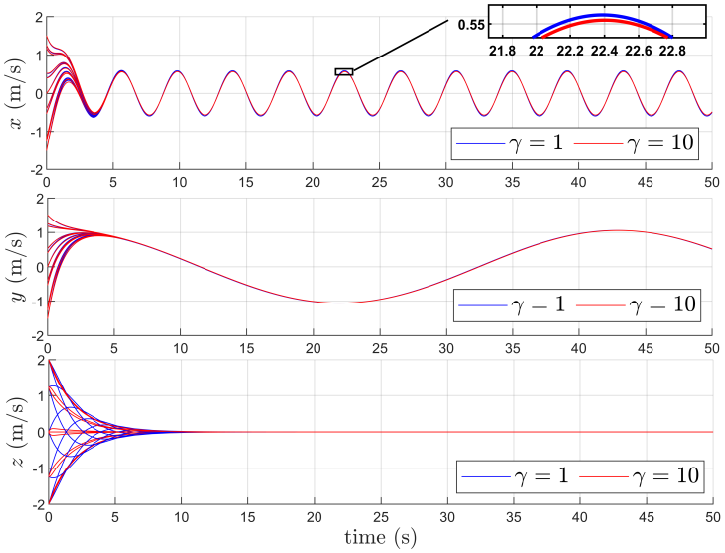


FIGURE 5.11: Real input $\boldsymbol{u}$ and the new input $\boldsymbol{v}$ simulated with disturbances

Furthermore, since no disturbance counter-measures (e.g., disturbance estimators) were taken into account by the controller, although the errors are proven to be bounded, few improvements can be observed for both the nominal gain $\gamma = 1$ and the high gain $\gamma = 10$ (See Figure 5.11).

Remark 5.3. The design procedure can be easily adjusted for the *trajectory tracking* problem as follows. First, we assume that the reference trajectory for the linearized system satisfies the dynamics constraints:

$$\dot{\boldsymbol{\xi}}^{\text{ref}} = \boldsymbol{A}\boldsymbol{\xi}^{\text{ref}} + \boldsymbol{B}\boldsymbol{v}^{\text{ref}};\ \boldsymbol{v}^{\text{ref}} \in \mathcal{V}^{\text{ref}} \subset \mathcal{V}_c \tag{5.31}$$

where $\boldsymbol{\xi}^{\text{ref}}, \boldsymbol{v}^{\text{ref}}$ denote, respectively, the reference for the state $\boldsymbol{\xi}$ and input $\boldsymbol{v}$; $\mathcal{V}^{\text{ref}}$ is a polytope subset of $\mathcal{V}_c$ entirely wrapping $\boldsymbol{v}^{\text{ref}}$. Then, the error dynamics yields:

$$\dot{\tilde{\boldsymbol{\xi}}} = \boldsymbol{A}\tilde{\boldsymbol{\xi}} + \boldsymbol{B}\tilde{\boldsymbol{v}} + \boldsymbol{E}\boldsymbol{w},\ \text{subject to: } \tilde{\boldsymbol{v}} \in \tilde{\mathcal{V}} \tag{5.32}$$

with $\tilde{\boldsymbol{\xi}} \triangleq \boldsymbol{\xi} - \boldsymbol{\xi}^{\text{ref}}, \tilde{\boldsymbol{v}} \triangleq \boldsymbol{v} - \boldsymbol{v}^{\text{ref}}$ and $\tilde{\mathcal{V}} = \mathcal{V}_c \ominus \mathcal{V}^{\text{ref}}$. In this fashion, the trajectory tracking can be achieved by applying the proposed stabilizing controller for system (5.32). To simplify the computation of $\tilde{\mathcal{V}}$, $\mathcal{V}_c$ will be approximated with the polytope $\mathcal{V}_c^{\text{in}}$ in (3.22).

TABLE 5.4: Simulation parameters for robustness analysis

Parameters	Values
T_{max}, ϵ_{max}	$2g, 0.698$ (rad)
Q_w and β in (5.24)	$\begin{bmatrix} \text{diag}(2.29, 2.29, 4.42) & -2.14\boldsymbol{I}_3 \\ -2.14\boldsymbol{I}_3 & 2.07\boldsymbol{I}_3 \end{bmatrix}, 0.9668$
w_x, w_y	$\sin(1.5t + \pi/8), \cos(0.15)$ (m/s)
$\bar{v}_1, \bar{v}_2, \bar{v}_3$ in (5.21)	$3.8804, 3.8804, 3.2700$

Moreover, assume $\tilde{\mathcal{V}}$ is given in half-space form:

$$\tilde{\mathcal{V}} = \{\boldsymbol{v} : \boldsymbol{a}_{\tilde{v},i}^\top \boldsymbol{v} \leq b_{\tilde{v},i}, b_{\tilde{v},i} > 0, i \in \{1, ..., N_{\tilde{\mathcal{V}}}\}\}, \tag{5.33}$$

Then, the saturation function over the set $\tilde{\mathcal{V}}$ is described as:

$$\text{sat}_{\tilde{\lambda}}(\tilde{\boldsymbol{v}}) = \tilde{\lambda}^*(\tilde{\boldsymbol{v}})\tilde{\boldsymbol{v}}, \text{with } \tilde{\lambda}^*(\tilde{\boldsymbol{v}}) = \begin{cases} 1 & \text{if } \tilde{\boldsymbol{v}} \in \tilde{\mathcal{V}}, \\ \max\limits_{\tilde{\lambda}\boldsymbol{v} \in \tilde{\mathcal{V}}} \tilde{\lambda} & \text{if } \tilde{\boldsymbol{v}} \notin \tilde{\mathcal{V}}, \end{cases} \tag{5.34}$$

and can be explicitly computed, with $i \in \{1, ..., N_{\tilde{\mathcal{V}}}\}$, as:

$$\tilde{\text{sat}}_{\lambda}(\tilde{\boldsymbol{v}}) = \begin{cases} \tilde{\boldsymbol{v}} & \text{if } \tilde{\boldsymbol{v}} \in \tilde{\mathcal{V}}, \\ \tilde{\boldsymbol{v}}\left(\min\limits_{\boldsymbol{a}_{\tilde{v},i}^\top \tilde{\boldsymbol{v}} > 0} b_{\tilde{v},i}/(\boldsymbol{a}_{\tilde{v},i}^\top \tilde{\boldsymbol{v}})\right) & \text{if } \tilde{\boldsymbol{v}} \notin \tilde{\mathcal{V}}. \end{cases} \tag{5.35}$$

Namely, the formula (5.35) is the solution of the LP employed in (5.34).

5.3.3 Experimental validation

For the experiment, the continuous dynamics of $\gamma(t)$ as in (5.16b) will be discretized via the forward Euler method together with the sampling time of 0.1 (s). Next, to highlight the controller's applicability, we first present the experiment tests applied, then discussions will follow.

5.3.3.1 Experiment setups

In this section, we carry out the following scenarios.

- *Test 1*: the real system behavior under different choices of adaptive gain varying from $\mu = 1$ to $\mu = 5$ is studied to provide insights on the proper choice of such parameter. The control objective is to track a static point located at $\boldsymbol{\xi} = [0.6, 0.6, 0.8, 0, 0, 0]^\top$ starting from the initial point $\boldsymbol{\xi}(0) = [-0.6, -0.6, 0, 0, 0, 0]^\top$.

TABLE 5.5: Numerical parameters & results for test 2

	Drone 1	Drone 2	Drone 3
RMS tracking errors (m)	0.0703	0.0826	0.0602
Average computation time	1.4809 (ms)		
$(\gamma_0, \mu, V_\infty)$	$(1.0, 2.0, 0.05)$		

- *Test 2*: To underline the computational advantage, three nanodrones are utilized to track their assigned trajectories in a centralized manner with the proposed scheme. The scenario describes the swapping position of the drones with three anchor waypoints: $\{[0,-6,1],[6,6,1],[-6,6,1]\} \times 10$ (cm) while the references are generated with B-spline trajectory generation from Section 2.3.1.3. The three drones and their corresponding references are labeled as Drone r and Ref. r with $r \in \{1,2,3\}$.

The matrix $\boldsymbol{P}$ and ε forming the invariant set $\mathcal{E}(\boldsymbol{P},\varepsilon)$ are fixed as: $\boldsymbol{P} = \begin{bmatrix} 0.98\boldsymbol{I}_3 & 0.78\boldsymbol{I}_3 \\ 0.78\boldsymbol{I}_3 & 1.25\boldsymbol{I}_3 \end{bmatrix}$ and 2.3215, respectively, with Proposition 3.1 and the parameters in TABLE 5.3. The experiment video can be found at https://youtu.be/kwfXiJ6odnc.

5.3.3.2 Experimental results and discussion

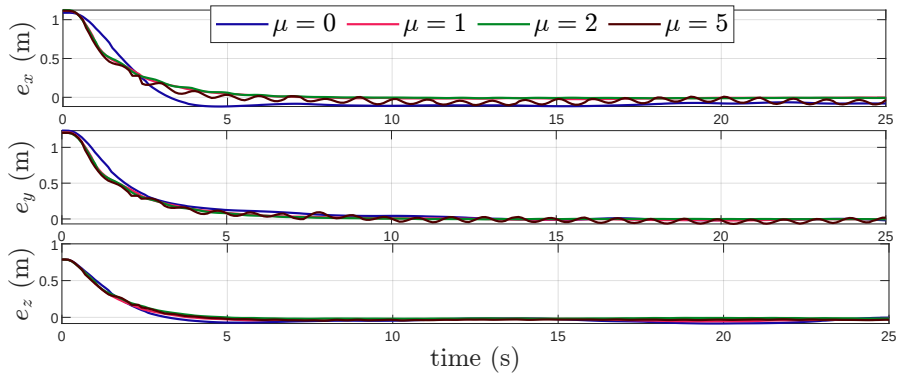


FIGURE 5.12: Set-point tracking errors ($e_q = q - q^{\text{ref}}, q \in \{x,y,z\}$) via different choices of adaptive gain μ (Test 1).

In general, the stabilization effect and constraint satisfaction guarantee are validated when the quadcopter arrives to the reference point with the input $\boldsymbol{u}$ remaining correctly bounded in $\mathcal{U}$ (See Figure 5.14). However, as in Figure 5.13, although the stability is secured for all cases, the Lyapunov function of $\mu = 5$ is not confined under the threshold $V = V_\infty$ as in (5.17). This oscillation can also be observed in the tracking error (Figure 5.12). The behavior was caused by the high adaptation speed μ abruptly increasing

the control gain $\gamma(t)$ to a large value, which excites the neglected high frequency dynamics (e.g., rotation dynamics). Furthermore, this high gain of μ also prevents $\gamma(t)$ from converging (Figure 5.13). Hence, although in the examined scenarios, the stability is maintained, an unreasonably high adaptation speed μ is not recommended.

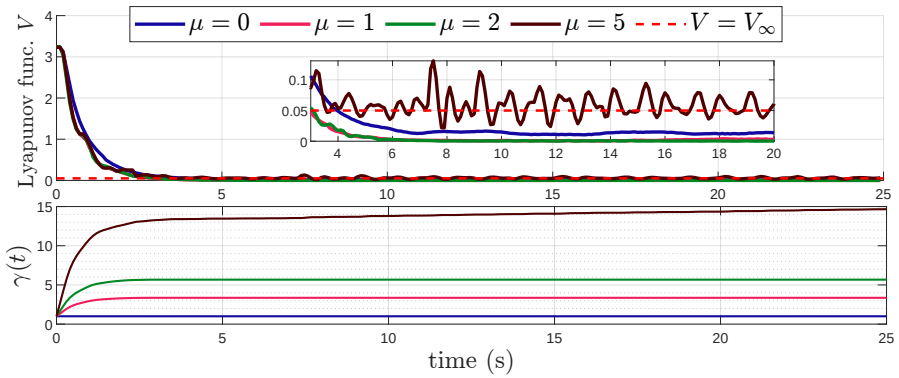


FIGURE 5.13: Values of the Lyapunov function (top) and the evolution of $\gamma(t)$ (bottom) with different adaptive rate μ.

On the contrary, with a moderate choice of μ ($\mu = 1$ or 2), the steady state tracking error is removed (See Figure 5.12). Furthermore, the appropriate value of $\gamma(t)$ also converges to a fixed value (Figure 5.13), solving the problem of choosing the gain γ for the saturated control described in (3.32).

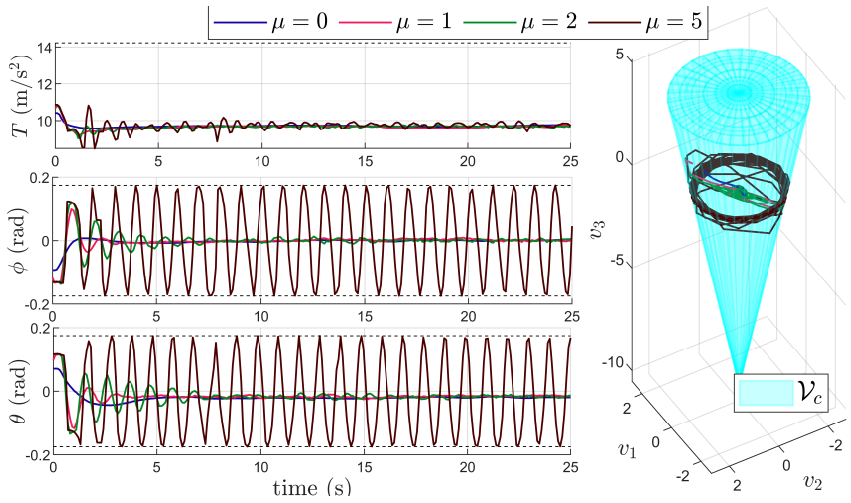


FIGURE 5.14: Input $\boldsymbol{u}$ and $\boldsymbol{v}$ via different choices of adaptive gain μ.

Besides, thanks to the explicit solution of the saturated function as given in (3.58), the computation time remains under 3 ms which is relatively small compared to the existing results of nonlinear, optimization-based controller providing the same guarantees

[Nguyen et al., 2020b, Mueller and D'Andrea, 2013, Nguyen et al., 2019]. This advantage is further highlighted with the reliable control of three quadcopters in a centralized manner in Test 2 with less than 2 ms in the average of computation time (TABLE 5.5) with under 10 cm of root-mean-square (RMS) tracking errors.

However, from Figure 5.15, although the $\gamma(t)$ value of Drone 3 has the tendency to converge, that of Drone 1 and Drone 2 increases correspondingly to the vehicle's deviation from the reference due to disturbances. This phenomenon, as a future concern, complicates both the problem of disturbance rejection for the strategy and the adjustment for the non-monotonicity of the gain $\gamma(t)$ in the presence of disturbances so that input over-exploitation can be avoided.

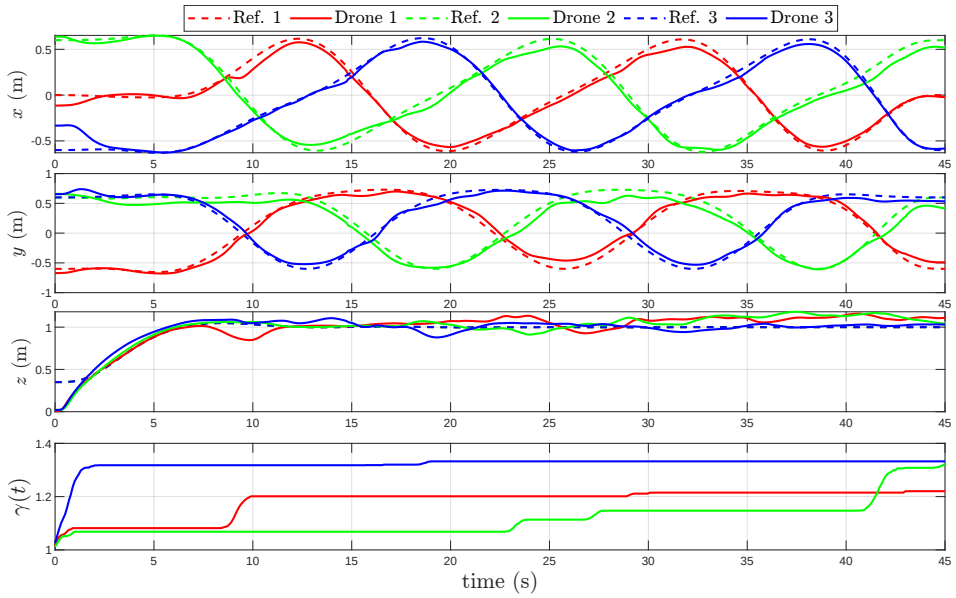


FIGURE 5.15: Position swapping with three Crazyflies (Test 2)

In closing, the validated results of the tube MPC in Section 5.2 and of the adaptive scheme in Section 5.3 conclude our effort to robustify the controllers presented in Chapter 3 for the quadcopter in the flat output space. In the next sections, we revisit a more general concern about how the benefits of flat systems change with respect to different choices of flat outputs.

5.4 Analysis of different flat outputs for the fixed-wing UAV model

With different efficient controllers derived from flatness as in the preceding sections, we would like to come back to addressing the fundamental question of what if the choice of flat output is not unique. In other words, will the properties extracted from flatness remain beneficial and practical? More specifically, as stated in Section 2.2.2, most results from the literature rapidly move on from the selection of a flat output and proceed to the trajectory generation or control design steps. Herein, with a case study of the fixed-wing UAV model in (2.4), we investigate several flat representations and observe how they affect (for better or worse) the procedures of guidance and navigation.

5.4.1 Various flat representations of the fixed-wing UAV model

In this part, we propose different flat representations computed from the chosen flat outputs of the model (2.4).

5.4.1.1 Flat output 1

Consider the following flat output in terms of the system's position:

$$\boldsymbol{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \end{bmatrix}. \tag{5.36}$$

Next, the remaining states and inputs are expressed as in the equation (5.37) which is detailed in Table 5.6:

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{u} \end{bmatrix} = \Phi_1 \left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}} \right). \tag{5.37}$$

5.4.1.2 Flat output 2

The same process is applied for the second flat representation. In this selection, cylindrical coordinate is used to describe the flat output:

$$\boldsymbol{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} \sqrt{x_1^2 + x_2^2} \\ \arctan \dfrac{x_2}{x_1} \end{bmatrix}. \tag{5.38}$$

	x_1	x_2	$\tan x_3$	u_1	$\tan u_2$
$\Phi_1(\cdot)$	z_1	z_2	$\frac{\dot{z}_2}{\dot{z}_1}$	$\|\dot{\boldsymbol{z}}\|_2$	$\frac{\ddot{z}_2\dot{z}_1 - \ddot{z}_1\dot{z}_2}{g\|\dot{\boldsymbol{z}}\|_2}$
$\Phi_2(\cdot)$	$z_1 \cos z_2$	$z_1 \sin z_2$	$\Phi_{2,x_3}(\boldsymbol{z}, \dot{\boldsymbol{z}})$	$\sqrt{\dot{z}_1^2 + z_1^2\dot{z}_2^2}$	$\Phi_{2,u_2}(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}})$
$\Phi_3(\cdot)$	z_1	$z_2 - \dot{z}_1$	$\frac{\dot{z}_2 - \ddot{z}_1}{\dot{z}_1}$	$\sqrt{\dot{z}_1^2 + (\dot{z}_2 - \ddot{z}_1)^2}$	$\Phi_{3,u_2}(\boldsymbol{z}, ..., \dddot{\boldsymbol{z}})$
$\Phi_4(\cdot)$	$z_2 - \dot{z}_1$	$z_1 + \ddot{z}_1 - \dot{z}_2$	$\Phi_{4,x_3}(\cdot)$	$\Phi_{4,u_1}(\dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}})$	$\Phi_{4,u_2}(\boldsymbol{z}, ..., \boldsymbol{z}^{(4)})$

TABLE 5.6: Flat representations corresponding to different flat outputs.

The states and inputs are then expressed as in the equation (5.39) which is detailed in the third row of Table 5.6:

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{u} \end{bmatrix} = \Phi_2\left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}\right), \tag{5.39}$$

and

$$\Phi_{2,x_3}(\boldsymbol{z}, \dot{\boldsymbol{z}}) = \frac{\dot{z}_1 \sin z_2 + \dot{z}_2 z_1 \cos z_2}{\dot{z}_1 \cos z_2 - \dot{z}_2 z_1 \sin z_2}, \Phi_{2,u_2}(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}) = \frac{\ddot{z}_2\dot{z}_1 z_1 - \ddot{z}_1\dot{z}_2 z_1 + 2\dot{z}_1^2\dot{z}_2 + \dot{z}_2^3 z_1^2}{g\sqrt{\dot{z}_1^2 + z_1^2\dot{z}_2^2}}$$

5.4.1.3 Flat output 3

Herein, we take into account the fact that when $[z_1,\ z_2]^\top$ is a flat output, $[z_1,\ z_2 + \dot{z}_1]^\top$ is also a flat output [Levine, 2009]. Hence, we have:

$$z = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 + \dot{x}_1 \end{bmatrix}. \tag{5.40}$$

The remaining states and inputs are expressed as in equation (5.41) which is detailed in the forth row of Table 5.6:

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{u} \end{bmatrix} = \Phi_3\left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}}\right), \tag{5.41}$$

where

$$\Phi_{3,u_2}\left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}}\right) = \frac{(\dot{z}_2 - \ddot{z}_1)\ddot{z}_1 - (\ddot{z}_2 - \dddot{z}_1)\dot{z}_1}{g\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}.$$

5.4.1.4 Flat output 4

The following flat representation is a mix among the system's position, velocity and the acceleration:

$$\boldsymbol{z} = \begin{bmatrix} z_1 \\ z_2 \end{bmatrix} = \begin{bmatrix} \dot{x}_1 + x_2 \\ x_1 + \ddot{x}_1 + \dot{x}_2 \end{bmatrix}. \tag{5.42}$$

The remaining states and inputs are expressed as in equation (5.43) which is detailed in the last column of Table 5.6:

$$\begin{bmatrix} \boldsymbol{x} \\ \boldsymbol{u} \end{bmatrix} = \Phi_4 \left(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}}, \boldsymbol{z}^{(4)} \right). \tag{5.43}$$

where

$$\Phi_{4,x_3}(\dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}}) = \frac{\dddot{z}_1 + \dot{z}_1 - \ddot{z}_2}{\dot{z}_2 - \ddot{z}_1},$$
$$\Phi_{4,u_1}(\dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}, \dddot{\boldsymbol{z}}) = \sqrt{(\dddot{z}_1 + \dot{z}_1 - \ddot{z}_2)^2 + (\dot{z}_2 - \ddot{z}_1)^2},$$
$$\Phi_{4,u_2}(\dot{\boldsymbol{z}}, \cdots, \boldsymbol{z}^{(4)}) = \frac{(\dot{z}_2 - \ddot{z}_1)(z_1^{(4)} + \ddot{z}_1 - \dddot{z}_2) + (\ddot{z}_2 - \dddot{z}_1)(\dddot{z}_1 + \dot{z}_1 - \ddot{z}_2)}{g\sqrt{(\dot{z}_2 - \ddot{z}_1)^2 + (\dddot{z}_1 + \dot{z}_1 - \ddot{z}_2)^2}}.$$

Remark 5.4. These flat outputs are inspired from various sources: the flat output 1 in (5.36) is the one most employed in the literature while the second flat output in (5.38) exploits a change of coordinates and the two last flat outputs are derive from the fact that if $[z_1 \ z_2]^\top$ is a flat output, then $[z_1 \ z_2 + \dot{z}_1]^\top$ is also a flat output.

Remark 5.5. All the flat representations above are found knowing the complete description of the system (inputs, outputs, actuators and the like). There is, however, some works related to "flat input" representation [Nicolau et al., 2018]. In this study, proper actuators or inputs are chosen such that given output becomes a flat output.

In the next parts, we integrate the different flat representations into the trajectory planning and feedback linearization control described in Chapter 2.

5.4.2 Trajectory generation using flatness

In equations (2.38), we presented a generic motion planning problem and in Section 5.4.1 we provided several flat representations. In what follows, we show how the construction of both constraints and costs change depending on the particular flat representation employed.

More specifically, we propose the constraints $\boldsymbol{w}_k$ concerning the position of the UAV which associate with the same amount of time-stamps t_k:

$$\mathbb{W} = \{\boldsymbol{w}_1, \boldsymbol{w}_2, ..., \boldsymbol{w}_N\}, \quad \mathbb{T}_W = \{t_0, t_1, ..., t_N\}. \tag{5.44}$$

In other words, at the time instant $t_k \in \mathbb{T}_W$, the vector $[x_1(t_k)\ x_2(t_k)]^\top$ must satisfy:

$$\begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \boldsymbol{w}_k \in \mathbb{W}. \tag{5.45}$$

Furthermore, under the B-spline parameterization (2.27), the general cost mentioned in (2.21) will be replaced by the integral:

$$\mathbf{P} = \arg\min_{\mathbf{P}} \int_{t_0}^{t_N} \Xi_i(\mathbf{B}_d(t), \mathbf{P}) \mathrm{d}t = \arg\min_{\mathbf{P}} \int_{t_0}^{t_N} \left(\dot{x}_1^2 + \dot{x}_2^2\right) \mathrm{d}t, \tag{5.46}$$

where $\mathbf{P}$ is the matrix collecting the control points of the B-spline parameterization and $\mathbf{B}_d(t)$ gathers the basis functions defined as in (2.28). $\Xi_i(\mathbf{B}_d(t), \mathbf{P})$ is the cost function defined with the control points in $\boldsymbol{P}$ as decision variables characterized by the flat output i. So far, the constraints (5.45) and the cost (5.46) have been given in the position components (x_1, x_2) and now, for each flat representation, we rewrite them in terms of the flat output.

5.4.2.1 Cost function with flat output 1

With the representation of flat output 1 in (5.37), we can rewrite the problem (5.46) with the energy function as:

$$\Xi_1(\mathbf{B}_d(t), \mathbf{P}) = \dot{z}_1^2 + \dot{z}_2^2 = \dot{\boldsymbol{z}}^\top \dot{\boldsymbol{z}}. \tag{5.47}$$

Similarly, the constraints in (5.45) are rewritten as:

$$\begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \begin{bmatrix} z_1(t_k) \\ z_2(t_k) \end{bmatrix} = \boldsymbol{w}_k. \tag{5.48}$$

5.4.2.2 Cost function with flat output 2

Similarly, the cost function (5.46) can be written with the second flat output as:

$$\Xi_2(\mathbf{B}_d(t), \mathbf{P}) = \dot{z}_1^2 + z_1^2 \dot{z}_2^2. \tag{5.49}$$

The corresponding form of (5.45) with the flat output (5.38) is characterized by:

$$\begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \begin{bmatrix} z_1(t_k)\cos z_2(t_k) \\ z_1(t_k)\sin z_2(t_k) \end{bmatrix} = \boldsymbol{w}_k. \tag{5.50}$$

However, it is trivial to show that equation (5.50) does not give unique solution for $[z_1(t_k)\ z_2(t_k)]^\top$ due to the periodicity of the trigonometric functions. Hence, another way to specify the desired value for the flat output is introduced as:

$$\begin{cases} \begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \boldsymbol{w}_k, \\ z_1(t_k) = \sqrt{x_1(t_k)^2 + x_2(t_k)^2}, \\ z_2(t_k) = \arctan \dfrac{x_2(t_k)}{x_1(t_k)} + 2k\pi. \end{cases} \tag{5.51}$$

where k can be chosen so that the continuity of $z_2(t)$ will not break the desired direction of the trajectory.

5.4.2.3 Cost function with flat output 3

With the representation of the flat output 3 in (5.40), we rewrite the problem (5.46) as:

$$\Xi_3(\mathbf{B}_d(t), \mathbf{P}) = \dot{z}_1^2 + (\dot{z}_2 - \ddot{z}_1)^2. \tag{5.52}$$

Similarly, the constraints in (5.45) are indicated as:

$$\begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \begin{bmatrix} z_1(t_k) \\ z_2(t_k) - \dot{z}_1(t_k) \end{bmatrix} = \boldsymbol{w}_k. \tag{5.53}$$

5.4.2.4 Cost function with flat output 4

Equivalently, with flat output 4, we can rewrite the problem (5.46) as:

$$\Xi_4(\mathbf{B}_d(t), \mathbf{P}) = (\dddot{z}_1 + \dot{z}_1 - \ddot{z}_2)^2 + (\dot{z}_2 - \ddot{z}_1)^2. \tag{5.54}$$

Similarly, the constraints in (5.45) are indicated as:

$$\begin{bmatrix} x_1(t_k) \\ x_2(t_k) \end{bmatrix} = \begin{bmatrix} z_2(t_k) - \dot{z}_1(t_k) \\ z_1(t_k) + \ddot{z}_1(t_k) - \dot{z}_2(t_k) \end{bmatrix} = \boldsymbol{w}_k. \tag{5.55}$$

Next, to see also the effect of different flat output in control, we investigate the two first flat representations in the setting of feedback linearization described in Section 2.2.1.

5.4.3 Tracking control with feedback linearization

Let us recall the process mentioned in Section 2.2.1, we consider the new input of the system as:

$$\boldsymbol{u} = \begin{bmatrix} u_1 \\ u_2 \end{bmatrix} = \boldsymbol{u}(\boldsymbol{z}, \dot{\boldsymbol{z}}, \ddot{\boldsymbol{z}}). \tag{5.56}$$

Then, the virtual input is chosen as:

$$\boldsymbol{v} = \ddot{\boldsymbol{z}}. \tag{5.57}$$

With such transformation, virtual input (5.57) will be computed with respect to the linear error dynamics:

$$\boldsymbol{v} = \ddot{\boldsymbol{z}}^{\text{ref}} + K_1 \dot{\boldsymbol{e}}_z + K_2 \int \boldsymbol{e}_z \mathrm{d}t + K_3 \boldsymbol{e}_z, \tag{5.58}$$

with $\boldsymbol{e}_z = \boldsymbol{z}^{\text{ref}}(t) - \boldsymbol{z}(t)$ where $\boldsymbol{z}^{\text{ref}}(t)$ is the reference signal and $z(t)$ is the feedback flat output.

However, with the flat representations from (5.41) and (5.43), there will exist the derivative of the flat output with the order greater than 2 in the representation of $\dot{u}_1$ and $\tan u_2$. Hence, those two flat outputs do not meet the condition in (5.56).

Therefore, using only the representation in (5.37) and (5.39), we have the new input:

- for flat output 1 given in (5.37) :

$$\begin{aligned} \dot{u}_1 &= \frac{\ddot{z}_1 \dot{z}_1 + \ddot{z}_2 \dot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}, \\ \tan u_2 &= \frac{\ddot{z}_2 \dot{z}_1 - \ddot{z}_1 \dot{z}_2}{g\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}. \end{aligned} \tag{5.59}$$

- for flat output 2 given in (5.39):

$$\begin{aligned} \dot{u}_1 &= \frac{\ddot{z}_2 \dot{z}_2 z_1^2 + \ddot{z}_1 \dot{z}_1 + \dot{z}_2^2 \dot{z}_1 z_1}{\sqrt{\dot{z}_1^2 + z_1^2 \dot{z}_2^2}}, \\ \tan u_2 &= \frac{\ddot{z}_2 \dot{z}_1 z_1 - \ddot{z}_1 \dot{z}_2 z_1 + 2\dot{z}_1^2 \dot{z}_2 + \dot{z}_2^3 z_1^2}{g\sqrt{\dot{z}_1^2 + z_1^2 \dot{z}_2^2}}. \end{aligned} \tag{5.60}$$

Hence, the virtual input $\boldsymbol{v}$ can be formulated as in (5.61) and computed with the linear dynamics proposed in (5.58).

$$\boldsymbol{v} = \begin{bmatrix} v_1 \\ v_2 \end{bmatrix} = \ddot{\boldsymbol{z}} = \begin{bmatrix} \ddot{z}_1 \\ \ddot{z}_2, \end{bmatrix} \tag{5.61}$$

Ultimately, with the flat output (5.36) and (5.38), we obtain the two different control laws (5.62) and (5.63), respectively:

$$\dot{u}_1 = \frac{v_1\dot{z}_1 + v_2\dot{z}_2}{\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}, \tan u_2 = \frac{v_2\dot{z}_1 - v_1\dot{z}_2}{g\sqrt{\dot{z}_1^2 + \dot{z}_2^2}}, \tag{5.62}$$

$$\dot{u}_1 = \frac{v_2\dot{z}_2 z_1^2 + v_1\dot{z}_1 + \dot{z}_2^2\dot{z}_1 z_1}{\sqrt{\dot{z}_1^2 + z_1^2\dot{z}_2^2}}, \tan u_2 = \frac{v_2\dot{z}_1 z_1 - v_1\dot{z}_2 z_1 + 2\dot{z}_1^2\dot{z}_2 + \dot{z}_2^3 z_1^2}{g\sqrt{\dot{z}_1^2 + z_1^2\dot{z}_2^2}}, \tag{5.63}$$

where v_1 and v_2 are computed from feedback as in (5.58).

Next, with the presented setups, we analyze the results for different flat outputs with the following numerical tests.

5.4.4 Simulation result

5.4.4.1 Trajectory generation

For illustration, we use the list of 6 way-points and associated time-stamps taken equidistantly between t_0 and $t_N = 45s$ for the trajectory generation problem:

$$\begin{aligned} \mathbb{W} &= 10^2 \times \left\{ \begin{bmatrix} -1.5 \\ 0.1 \end{bmatrix}, \begin{bmatrix} -1.3 \\ 1 \end{bmatrix}, \begin{bmatrix} -0.1 \\ 1.2 \end{bmatrix}, \begin{bmatrix} 0.8 \\ 0.1 \end{bmatrix}, \begin{bmatrix} 1.6 \\ -0.9 \end{bmatrix}, \begin{bmatrix} 0.2 \\ -1.2 \end{bmatrix} \right\} \\ \mathbb{T}_W &= \{0, 9, 18, 27, 36, 45\}. \end{aligned} \tag{5.64}$$

Next, we require that all generated states and inputs are continuous. Since the flat output is a weighted sum of spline functions, this means in fact that we have to choose the splines to be of sufficient degree, as detailed in Table 5.7.

Note that each further increase in continuity (e.g., velocity or acceleration are continuous themselves) translates into an incrementation of the B-spline functions' degree.

Fig. 5.16 depicts the profiles of the state $\boldsymbol{x}(t)$ which are generated with four proposed flat representations (5.37), (5.39), (5.41) and (5.43) respectively. Hereinafter, for simplicity, the 4 aforementioned representations with their corresponding trajectories will be labeled

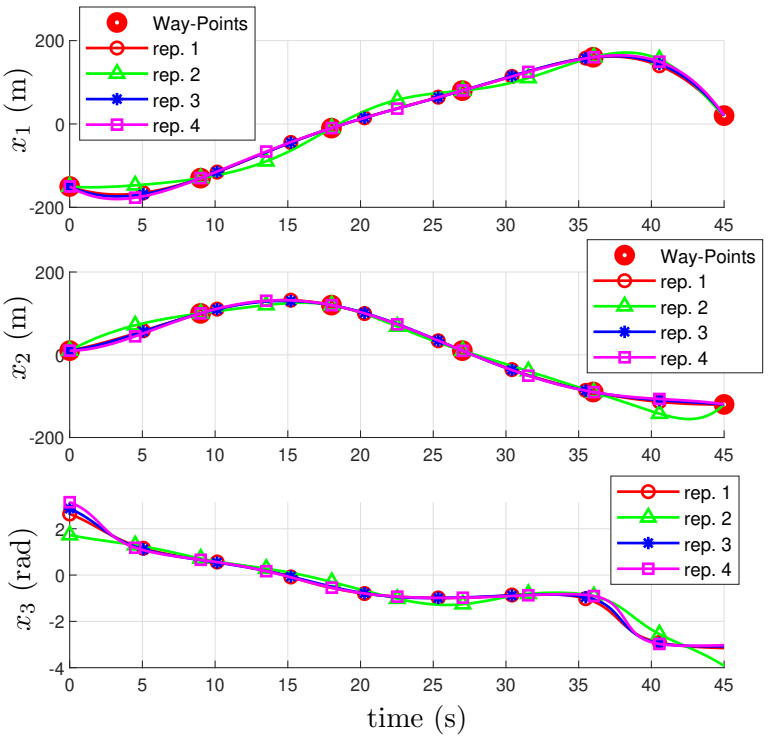


FIGURE 5.16: $x_1(t)$, $x_2(t)$ and $x_3(t)$ with the 4 flat outputs.

TABLE 5.7: Data specification and computation results

	rep. 1	rep. 2	rep. 3	rep. 4
Number of control points	6	6	6	6
Polynomial order of B-spline	3	3	4	5
Cost's value ($\times 10^4 m^2/s$)	1.1332	1.3540	1.1772	1.2580
Curve's length (m)	670.75	709.09	675.46	689.46

representation as r (rep. r) with $r \in \{1, 2, 3, 4\}$. Similarly, the reference for the inputs is shown in Fig. 5.17 and the corresponding position is in Fig. 5.18.

All the reference signals are computed such that they satisfy the optimization problem (5.46), the constraints in (5.64) and the continuity constraints, as given in Table 5.7.

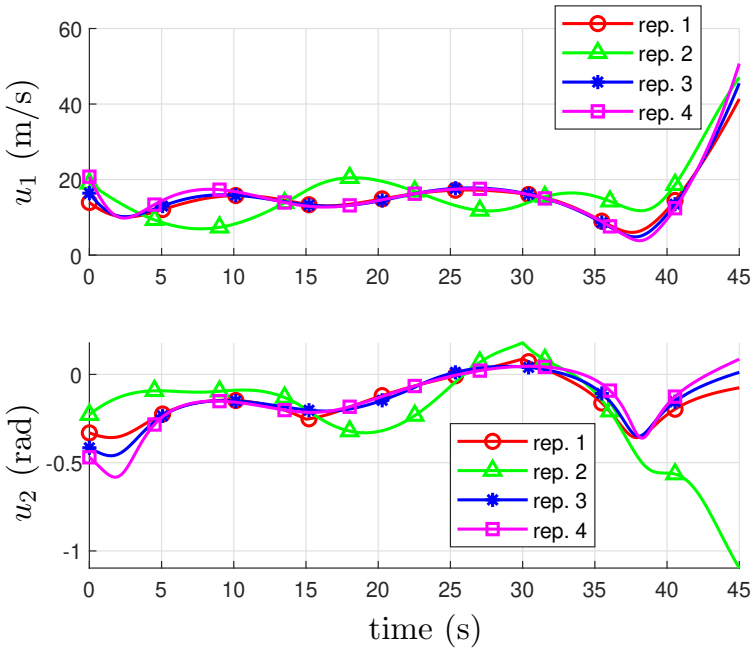


FIGURE 5.17: Input $u_1(t)$ and $u_2(t)$ with 4 flat outputs

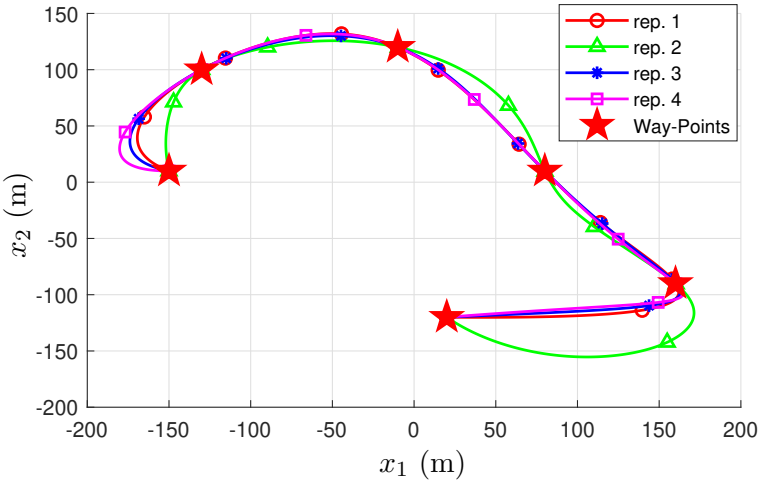


FIGURE 5.18: Position (x_1, x_2) planned with 4 flat outputs

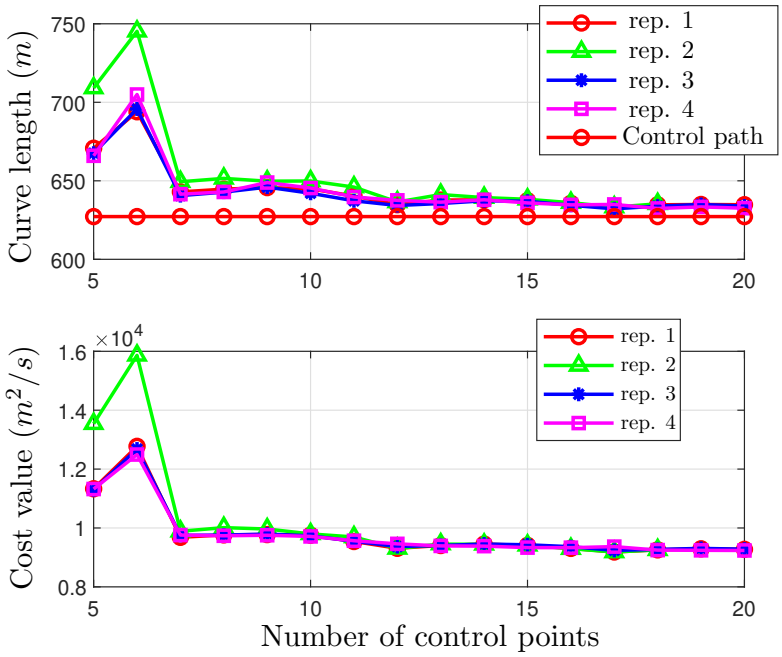


FIGURE 5.19: The the curves' length and objective function values in terms of the number of control points.

5.4.4.2 Feedback linearization control

On the other hand, in the control simulation process, we reuse the model (2.4) but disturb it with constant wind perturbation as follows:

$$\dot{x}_1 = u_1 \cos x_3 + W_x, \quad \dot{x}_2 = u_1 \sin x_3 + W_y, \quad \dot{x}_3 = g(\tan u_2)/u_1. \tag{5.65}$$

The reference signal is the profile generated with flat output 1 (the curve denoted as rep. 1 in Fig. 5.18) and simulation data specification is detailed in Table 5.8 where the parameters K_1, K_2 and K_3 are chosen from the characteristic equation and its poles p_1, p_2 and p_3 such that:

$$(s - p_1)(s - p_2)(s - p_3) = s^3 - K_3 s^2 + K_1 s + K_2. \tag{5.66}$$

For the sake of illustration, the poles are chosen identically as $p_1 = p_2 = p_3 = -2$.

In this simulation, the tracking errors will be labeled correspondingly with the control laws proposed in the transformation (5.59) and (5.60) as control law 1 (ctrl. law 1) and

TABLE 5.8: Simulation specifications

Control parameters	$K_1 = 12, K_2 = 8, K_3 = 6$
Wind [m/s]	$W = [W_x, W_y] = [5, 5]$
Simulation time [s]	45s
Sample time [s]	0.01s

control law 2 (ctrl. law 2). Details for the errors are given in Fig. 5.20 for the position x_1, x_2 and the heading angle x_3.

Besides, in this simulation, we provide 4 scenarios of wind perturbation vector ($\boldsymbol{W}$) to experiment on the dependence of the tracking error on the wind direction. Details are provided in Table 5.9 where $RMS(e_{ij})$ is the root mean square value of the tracking error in the i axis ($i \in \{1, 2\}$) implemented with the control law j ($j \in \{1, 2\}$).

TABLE 5.9: Tracking error with different wind scenarios

RMS (m) / $\boldsymbol{W}$(m/s)	e_{x1}	e_{x2}	e_{y1}	e_{y2}
[+5;+5]	0.0869	0.1317	0.0588	0.0744
[–5;+5]	0.0675	0.1301	0.1079	0.1146
[+5;–5]	2×10^{21}	1×10^{20}	8×10^{20}	3×10^{20}
[–5;–5]	5×10^{18}	0.3008	3×10^{18}	0.1947

5.4.4.3 Discussions

Let us provide some discussions and insights related to the UAV trajectory generation and tracking when considering various flat representations.

i) Trajectory generation.

As expected, from Table 5.7, all the representations can be used to satisfy the way-points constraints. It is also notable that the number of control points needed is not affected by the variation of flat outputs, confirming the unique dependence of the control points on the number of way-points.

From the parameterization point of view, it can also be seen in the table that the polynomial order of the trajectories are dissimilar, which can be explained with the degree of derivatives existing within each flat output. More specifically, in representation 1 in (5.37) and 2 in (5.39), the highest derivative order is 2 while that of representation 3 (5.41) and 4 (5.43) are 3 and 4, respectively, resulting in different requirement on the B-spline functions' degree to satisfy the continuity constraints.

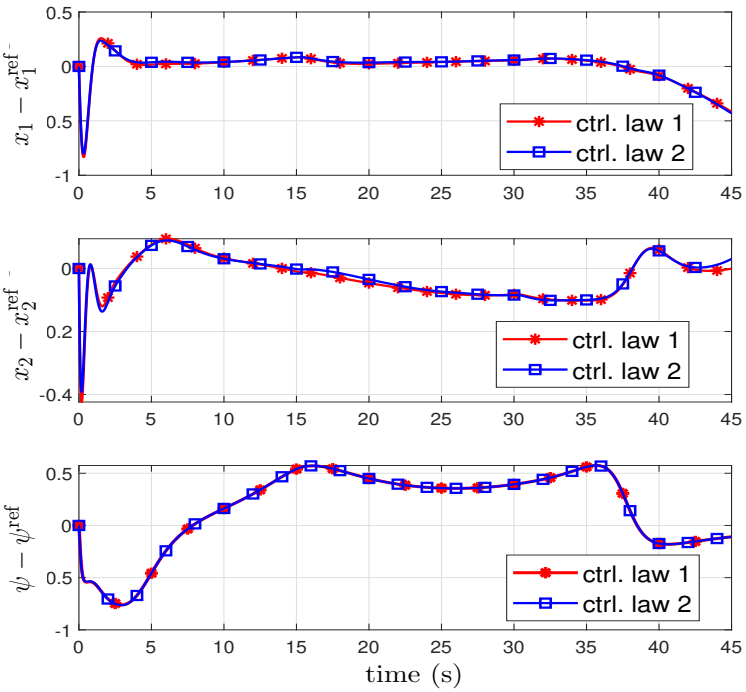


FIGURE 5.20: Tracking error of $x_1(t), x_2(t)$ and $x_3(t)$

Moreover, when the number of control points is considered, the variation of flat representations brings about the differences in not only the curves' length but also the value of the cost function (Fig. 5.19). Nonetheless, these differences fades when this number gets larger. This can be explained by the convergence of the trajectory to the control path (the straight lines connecting all the way-points), suggesting that after a certain number of control points, there is no use in increasing the number.

Finally, of interest is also that the cost and path length do not greatly depend on which flat representation is chosen. Still, the complexity of the representation plays a role, by making the problem less (or more so) complex. More precisely, if we look at the representation 1, 3 and 4, the two values increase along with the number of derivative involving in the representations. On the other hand, the second flat output proves more complex, since it contains trigonometric functions, whose periodicity affects the optimization procedure.

ii) Trajectory tracking: While the trajectories are distinguishable, the tracking problems seem to have similar behaviors:

As depicted in Fig. 5.20, there are differences between the error dynamics of the two control system. These dissimilarities come from the variable where the error dynamic is

applied. In detail, the first control law implement the linear dynamic in the first states, which are $x_1(t)$ and $x_2(t)$. Meanwhile, with the second law, linearity is applied to the flat output, which is a combination of the two first states. However, this property may not have a strong enough impact on the tracking error, thus, the results present little meaningful information.

However, the results given in Table 5.9 suggest the reliance of the system's performance on both the direction of the wind and the control laws. More particularly, the tracking of the UAV shows better results for the first two scenarios ($\boldsymbol{W} = [+5; +5]$ and $\boldsymbol{W} = [-5; +5]m/s$). On the contrary, when $\boldsymbol{W} = [-5; -5]$, only the control law 2 can stabilize the system and when $\boldsymbol{W} = [+5; -5]$ both of the controllers cannot handle the perturbation.

With these observations, it can be seen in the disturbed case that the tracking of the flat outputs with (5.58) somewhat imply the that of the states. But via different linearizing mappings from the different flat outputs, the disturbances propagates in varying ways, creating dissimilar tracking performances. To see this more clearly, let us proceed to the next part, focusing on the tracking performance of different flat outputs under disturbances, incorporating the setting of extended state observers (ESO).

5.5 Robust flatness-based control for a fixed-wing UAV

Previously, it has been shown that the flat output and its derivatives are required to linearize the system and close the loop for trajectory tracking. However, high derivatives of the flat output are typically not available via sensor measurements. Therefore, a state estimator, or somewhat in this case, a differentiator is needed to obtain the derivatives from the feedback value of the output. With this idea in mind, in this section, we employ the notion of extended state observers (ESO) to estimate the values of the flat output's derivatives. Furthermore, thanks to the representation of chains of integrators, this estimation allows also the evaluation over the disturbances, which is later characterized by the notion of robust positive invariant set. The procedure can be generally described as the following.

5.5.1 Output feedback for flat systems with extended state observers

In this part, the general linearization of flat systems in Section 2.2.1 is reconsidered in the presences of external disturbances. Under the linearizing diffiomorphism, the linear dynamics in the flat output space is also disturbed under the form of matched disturbances. The ESO framework is then applied to estimate the new states and such disturbances, enabling the output feedback design for trajectory tracking.

5.5.1.1 Tracking controller design with disturbance rejection

Consider a disturbed version of the general flat system in (2.1):

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}, \boldsymbol{u}) + \boldsymbol{d_x} \tag{5.67}$$

By applying the flatness-based linearization law as in (2.8), the dynamics (5.67) is transformed into a system of differential equations:

$$z_i^{(r_i)} = v_i + d_i, i \in \{1, ..., m\} \tag{5.68}$$

for some integer r_i as in (2.12) and d_i denotes the propagated disturbances through the linearizing mapping. The objective is to ensure that the output z_i tracks its a priori given reference signal $z_i^{\text{ref}}(t)$.

Next, we proceed by rewriting each equation of (5.68) in a classical state-space representation as follows:

$$\begin{cases} \dot{\boldsymbol{\sigma}}_i &= \mathbf{A}_i\boldsymbol{\sigma}_i + \mathbf{B}_i v_i + \mathbf{E}_i\tau_i \ \ (1 \leq i \leq m), \\ y_i &= \mathbf{C}_i\boldsymbol{\sigma}_i \triangleq \begin{bmatrix}\mathbf{0}_{1\times r_i}\end{bmatrix}\boldsymbol{\sigma}_i = z_i, \end{cases} \tag{5.69}$$

$$\begin{cases} \boldsymbol{\sigma}_i &= \begin{bmatrix}\sigma_{i,1} & \sigma_{i,2} & ... & \sigma_{i,r_i} & \sigma_{i,(r_i+1)}\end{bmatrix}^\top \\ &\triangleq \begin{bmatrix} z_i & \dot{z}_i & ... & z_i^{(r_i-1)} & d_i\end{bmatrix}^\top, \\ \tau_i &= \dot{d}_i, \end{cases} \tag{5.70}$$

$$\mathbf{A}_i = \begin{bmatrix}\mathbf{0}_{r_i\times 1} & \boldsymbol{I}_{r_i} \\ 0 & \mathbf{0}_{1\times r_i}\end{bmatrix}, \mathbf{B}_i = \begin{bmatrix}\mathbf{0}_{(r_i-1)\times 1} \\ 1 \\ 0\end{bmatrix}, \mathbf{E}_i = \begin{bmatrix}\mathbf{0}_{r_i\times 1} \\ 1\end{bmatrix}. \tag{5.71}$$

Considering $y_i = z_i$ as the output of (5.69) and that the pair of matrices $(\mathbf{A}_i, \mathbf{C}_i)$ is observable, we construct for the linear model (5.69) an observer as follow:

$$\dot{\hat{\boldsymbol{\sigma}}}_i = \mathbf{A}_i\hat{\boldsymbol{\sigma}}_i + \mathbf{B}_i v_i + \mathbf{L}_i(y_i - \hat{y}_i), \tag{5.72}$$

where $\hat{\boldsymbol{\sigma}}_i \triangleq [\hat{\sigma}_{i,1} \ ... \ \hat{\sigma}_{i,(r_i+1)}]^\top$ is the estimated value of $\boldsymbol{\sigma}_i$ and $\hat{y}_i$ is that of y_i. The estimation error between the current and the estimated value $\hat{\boldsymbol{e}}_i = \boldsymbol{\sigma}_i - \hat{\boldsymbol{\sigma}}_i$ yields:

$$\dot{\hat{\boldsymbol{e}}}_i = \dot{\boldsymbol{\sigma}}_i - \dot{\hat{\boldsymbol{\sigma}}}_i = (\mathbf{A}_i - \mathbf{L}_i\mathbf{C}_i)\hat{\boldsymbol{e}}_i + \mathbf{E}_i\tau_i. \tag{5.73}$$

An illustration of the observer (5.72) for (5.69) is presented in Figure 5.21.

With this formulation, providing that d_i or τ_i is bounded, the error $\lim_{t\to\infty}\hat{\boldsymbol{e}}_i(t)$ will also be bounded with a suitable choice of L_i stabilizing dynamics (5.73), see [Yang and

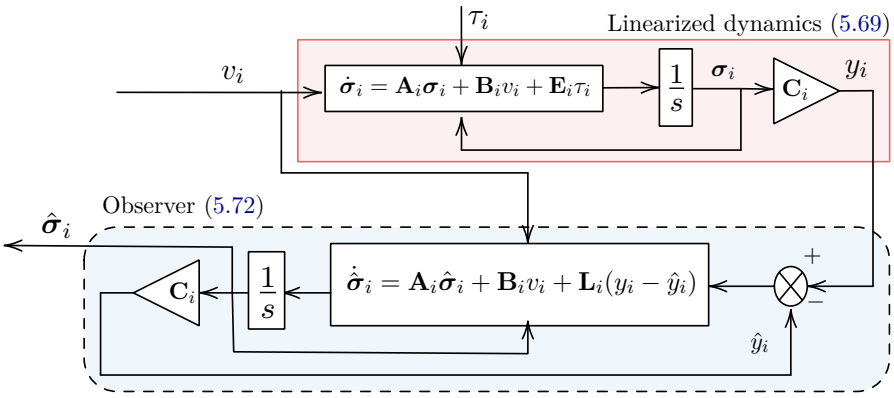


FIGURE 5.21: Extended state observer design for flat systems.

Huang, 2009, Abadi et al., 2020]. From the estimation achieved in (5.72), let us define the control law for virtual input v_i as follow:

$$v_i = \dot{\sigma}_{i,r_i}^{\text{ref}} + \boldsymbol{K}_i \begin{bmatrix} \sigma_{i,1\,ref} - \hat{\sigma}_{i,1} \\ \dots \\ \sigma_{i,r_i\,ref} - \hat{\sigma}_{i,r_i} \end{bmatrix} - \hat{\sigma}_{i,(r_i+1)}, \tag{5.74}$$

where $\boldsymbol{K}_i = [k_{i,1}\ k_{i,2}\ \dots\ k_{i,r_i}]$ is the control matrix of tuning parameters and $\sigma_{i,s\,ref} \triangleq (z_i^{\text{ref}})^{(s-1)}\ \ (1 \leq s \leq r_i)$.

Proposition 5.2. *With the virtual input proposed in (5.74), and a suitable choice of* $\mathbf{L}_i$ *and* $\mathbf{K}_i$*, the tracking and estimation error can be stabilized in closed-loop.*

Proof. First, we rewrite (5.74) as:

$$v_i = \dot{\sigma}_{i,r_i\,ref} + \mathbf{K}_i\boldsymbol{e}_i + \tilde{\mathbf{K}}_i\hat{\boldsymbol{e}}_i - \sigma_{i,(r_i+1)}\ , \tag{5.75}$$

where $\tilde{\mathbf{K}}_i = [\mathbf{K}_i\ 1]$, $e_{i,s} = \sigma_{i,s\,ref} - \sigma_{i,s}$, $(1 \leq s \leq r_i)$ and $\boldsymbol{e}_i = [e_{i,1}\ \dots\ e_{i,r_i}]^\top$. For further use, consider the notations:

$$\mathbf{A}_i^* = \begin{bmatrix} \mathbf{0}_{(r_i-1)\times 1} & \boldsymbol{I}_{(r_i-1)} \\ 0 & \mathbf{0}_{1\times(r_i-1)} \end{bmatrix}, \mathbf{B}_i^* = \begin{bmatrix} \mathbf{0}_{(r_i-1)\times 1}^\top & 1 \end{bmatrix}^\top. \tag{5.76}$$

Hence, on one hand, we may rewrite the tracking error as:

$$\dot{\boldsymbol{e}}_i = \frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix} \sigma_{i,1\,ref} - \sigma_{i,1} \\ \dots \\ \sigma_{i,r_i\,ref} - \sigma_{i,r_i} \end{bmatrix} = \mathbf{A}_i^*\boldsymbol{e}_i + \mathbf{B}_i^*\dot{\sigma}_{i,r_i\,ref} - \mathbf{B}_i^*\dot{\sigma}_{i,r_i}. \tag{5.77}$$

On the other hand, applying the virtual input (5.75) to the dynamics in (5.67) and then replacing the value of $\dot{\sigma}_{i,r_i} = z_i^{(r_i)}$ into equation (5.77), we obtain the closed loop for the

tracking error dynamics as:

$$\begin{aligned}\dot{\boldsymbol{e}}_i &= \mathbf{A}_i^* \boldsymbol{e}_i + \mathbf{B}_i^* \dot{\sigma}_{i,r_i\,ref} - \mathbf{B}_i^* (\dot{\sigma}_{i,r_i\,ref} + \mathbf{K}_i \boldsymbol{e}_i + \tilde{\mathbf{K}}_i \hat{\boldsymbol{e}}_i) \\ &= (\mathbf{A}_i^* - \mathbf{B}_i^* \mathbf{K}_i)\, \boldsymbol{e}_i - \mathbf{B}_i^* \tilde{\mathbf{K}}_i \hat{\boldsymbol{e}}_i. \end{aligned} \tag{5.78}$$

Rearranging (5.73) and (5.78), we have:

$$\frac{\mathrm{d}}{\mathrm{d}t} \begin{bmatrix} \boldsymbol{e}_i \\ \hat{\boldsymbol{e}}_i \end{bmatrix} = \begin{bmatrix} (\mathbf{A}_i^* - \mathbf{B}_i^* \mathbf{K}_i) & -\mathbf{B}_i^* \tilde{\mathbf{K}}_i \\ \mathbf{0}_{(r_i+1)\times r_i} & (\mathbf{A}_i - \mathbf{L}_i \mathbf{C}_i) \end{bmatrix} \begin{bmatrix} \boldsymbol{e}_i \\ \hat{\boldsymbol{e}}_i \end{bmatrix} + \begin{bmatrix} \mathbf{0}_{r_i \times 1} \\ \mathbf{E}_i \end{bmatrix} \tau_i. \tag{5.79}$$

From here we clearly ensure the stability of the closed-loop with a right choice of the controller's and observer's poles (respectively in (5.78) and (5.73)) knowing that the observability for (5.69) and controllability for (5.76) are satisfied. □

The evaluation of the system's performance is now also facilitated via the linearization. In the next subsection, we recall the construction of an approximation of the minimal robust positive invariant (mRPI) set, [Stoican et al., 2015a]. This is useful for confining the system's trajectory and quantifying its performance based on the chosen poles for the linear controllers.

5.5.1.2 Tight approximation for the mRPI set

While there exist various methods to construct RPI sets, the issue is still open [Blanchini and Miani, 2008]. The computation time, the complexity of the result and even the fact that often the procedures are designed for the discrete-time case are significant hurdles. In this section we propose an extension of the result for the discrete-case from [Stoican et al., 2015a] which in turn is based on the null-controllable set construction from [Hu et al., 2002b]. We consider the autonomous LTI system:

$$\dot{\boldsymbol{x}}(t) = \bar{\mathbf{A}} \boldsymbol{x}(t) + \boldsymbol{d}\delta(t), \tag{5.80}$$

with $\boldsymbol{x}(t) \in \mathbb{R}^n$ the state vector, $\delta(t) \in \begin{bmatrix} -1; & 1 \end{bmatrix}$ a scalar disturbance, $\bar{\mathbf{A}} \in \mathbb{R}^{n\times n}$ a stable state-matrix and $\boldsymbol{d} \in \mathbb{R}^n$. The key factors are: i) the stability of $\bar{\mathbf{A}}$ (a reasonable assumption as long as (5.80) denotes a closed-loop dynamics); and, ii) the disturbance affecting the dynamics is given by a scalar (again reasonable, as this is the case in many applications where the disturbance is bounded elementwise). Without detailing the apparatus, we note that the construction is based on three interlocking elements:

i) a monotonously increasing sequence of $n-1$ moments

$$\mathbf{t} = \{t_1 \leq t_2 \leq \cdots \leq t_{n-1}\}; \tag{5.81}$$

ii) a vector $\mathbf{c_t} \in \mathbb{R}^d$ which verifies

$$\mathbf{c_t}^\top e^{\bar{\mathbf{A}} t_i} \boldsymbol{d} = 0, \; \forall 1 \leq i \leq n-1; \tag{5.82}$$

iii) a sequence of extremal points

$$\boldsymbol{x}_\mathbf{t}^\star = \left(2 \sum_{i=1}^{n-1} (-1)^i e^{\bar{\mathbf{A}} t_i} + (-1)^n \boldsymbol{I}_n \right) \bar{\mathbf{A}}^{-1} \boldsymbol{d}. \tag{5.83}$$

Let Ω_∞ be the mRPI associated with (5.80). As detailed elsewhere ([Stoican et al., 2015a]), extremal points (5.83) sit on the boundary of Ω_∞ and hyperplanes $\{\boldsymbol{x} \in \mathbb{R}^d : \mathbf{c_t}^\top \boldsymbol{x} = \pm \mathbf{c_t}^\top \boldsymbol{x}_\mathbf{t}^\star\}$ are support hyperplanes to Ω_∞, tangent to it at $\boldsymbol{x}_\mathbf{t}^\star$. Thus any finite sequence of points (5.83) provides an inner approximation (in generator form) while the corresponding sequence of normal vectors (5.82) provides an outer approximation (in half-space form) of Ω_∞:

$$\begin{aligned} \underline{\Omega}_\mathbb{T} = \{\boldsymbol{x} \in \mathbb{R}^n : \; \boldsymbol{x} = \sum_{\mathbf{t} \in \mathbb{T}} \alpha_\mathbf{t}^\pm \left(\pm \boldsymbol{x}_\mathbf{t}^\star \right), \; \forall \alpha_\mathbf{t}^\pm \geq 0, \; \sum_{\mathbf{t} \in \mathbb{T}} \alpha_\mathbf{t}^\pm = 1 \} \\ \subset \Omega_\infty \subset \overline{\Omega}_\mathbb{T} = \{\boldsymbol{x} \in \mathbb{R}^n : \; |\mathbf{c_t}^\top \boldsymbol{x}| \leq |\mathbf{c_t}^\top \boldsymbol{x}_\mathbf{t}^\star|, \; \forall \mathbf{t} \in \mathbb{T}\}, \end{aligned} \tag{5.84}$$

where $\mathbb{T}$ denotes a list of sequences as in (5.81).

Having both inner and outer approximations as in (5.84) allows to bracket Ω_∞ arbitrarily tight (without any iterative procedure) and, equally important, to have an upper bound of the approximation error (e.g., the Hausdorff distance between $\underline{\Omega}_\mathbb{T}$ and $\overline{\Omega}_\mathbb{T}$). It is simpler, from an implementation viewpoint, to start with a collection of time-sequences $\mathbb{T} = \{\mathbf{t}\}$ from which to construct both $\mathbf{c_t}$ as in (5.82) and $\boldsymbol{x}_\mathbf{t}^\star$ as in (5.83). Two details are worthwhile of mentioning. First, note that (5.82) provides $n-1$ equalities for an $\mathbb{R}^n$ vector. To uniquely identify it, we add an n-th regularization equality, $\mathbf{c_t}^\top \mathbf{1} = 1$. Second, among the time sequences $\mathbf{t}$ we consider $\{0, \ldots, 0\}$ and $\{\infty, \ldots, \infty\}$ as those correspond to the fix points of the dynamic for $\delta(t) = \pm 1$, i.e., the steady state values $\boldsymbol{x}_\pm^\star = \mp \bar{\mathbf{A}}^{-1} \boldsymbol{d}$.

The above mRPI construction, (5.84) will be instrumental in the next sections for evaluating the controllers' and observers' performance.

5.5.2 Application for a fixed-wing UAV

In this section, we implement the two design procedures for a 3-DOF UAV model. We will first use the optimal trajectory generation solution as in Section 2.3.1.3 to set up the reference for the system. Then the flat output 1 and 2 in the previous section will be further used for analyzing the performance of the tracking controller design. Simulations, comparisons and discussions are addressed in Section 5.5.3.

5.5.2.1 3-DOF fixed-wing UAV model

Consider the fixed-wing model (2.4) with disturbed dynamics:

$$\dot{x}_1 = u_1 \cos x_3 + d_{x_1},\ \dot{x}_2 = u_1 \sin x_3 + d_{x_2},\ \dot{x}_3 = \frac{g \tan u_2}{u_1} + d_{x_3}, \tag{5.85}$$

where x_1, x_2, x_3 are the states and $\boldsymbol{u} = [u_1,\ u_2]^\top$ denotes the two inputs as in (2.4). d_{x_1}, d_{x_2} and d_{x_3} represent the bounded disturbances acting on the three states, their values will be considered zero when the nominal model is taken into account. The equations in (5.85) are rewritten in a form of (5.67) as:

$$\dot{\boldsymbol{x}} = \mathbf{f}(\boldsymbol{x}, \boldsymbol{u}) + \boldsymbol{d}_{\boldsymbol{x}}, \tag{5.86}$$

where $\boldsymbol{x} = [x_1, x_2, x_3]^\top$ and $\boldsymbol{d}_{\boldsymbol{x}} = [d_{x_1},\ d_{x_2},\ d_{x_3}]^\top$.

5.5.2.2 Flat characterization of the UAV model

For the nominal model, there exist several flat outputs as shown in the last section, with the help of which, one may choose to compute the linearization law. In this application, we recall the following two flat output vectors for which we analyse the differences in performance with respect to disturbance rejection (see Sections 5.5.1.1). The first flat output (flat output I) we study is:

$$\boldsymbol{z}^I = \begin{bmatrix} z_1^I & z_2^I \end{bmatrix}^\top \triangleq \begin{bmatrix} x_1 & x_2 \end{bmatrix}^\top, \tag{5.87}$$

while the second (flat output II) is:

$$\boldsymbol{z}^{II} = \begin{bmatrix} z_1^{II}, & z_2^{II} \end{bmatrix}^\top \triangleq \begin{bmatrix} \sqrt{x_1^2 + x_2^2}, & \arctan\left(x_2/x_1\right) \end{bmatrix}^\top. \tag{5.88}$$

where the superscript denotes the currently considered flat output (flat output I and II), its corresponding representation and parameters.

i) First flat representation:

With the flat output given in (5.87), we express the inputs in the form of (2.3b) as follows:

$$u_1 = \sqrt{(\dot{z}_1^I)^2 + (\dot{z}_2^I)^2}, \tan u_2 = \frac{\ddot{z}_2^I \dot{z}_1^I - \ddot{z}_1^I \dot{z}_2^I}{g\sqrt{(\dot{z}_1^I)^2 + (\dot{z}_2^I)^2}}. \tag{5.89}$$

To linearize the model, evidently, u_1 requires a dynamical extension so that the relation between high order derivatives of the flat output and the extension is invertible. Specifically, introduce the augmented state $\eta = u_1$ and $u_\eta = \dot{\eta}$ into the nominal model

of (2.4):

$$\begin{aligned} \dot{x}_1 &= \eta \cos x_3, \quad \dot{x}_3 = \frac{g \tan u_2}{\eta}, \\ \dot{x}_2 &= \eta \sin x_3, \qquad \dot{\eta} = u_\eta, \end{aligned} \tag{5.90}$$

then the inputs $[u_\eta, \tan u_2]$ of the new system yields:

$$u_\eta = \frac{\ddot{z}_1^I \dot{z}_1^I + \ddot{z}_2^I \dot{z}_2^I}{\sqrt{(\dot{z}_1^I)^2 + (\dot{z}_2^I)^2}}, \quad \tan u_2 = \frac{\ddot{z}_2^I \dot{z}_1^I - \ddot{z}_1^I \dot{z}_2^I}{g\sqrt{(\dot{z}_1^I)^2 + (\dot{z}_2^I)^2}}. \tag{5.91}$$

Subsequently, the dynamics are linearized by replacing $\ddot{z}_1^I, \ddot{z}_2^I$ respectively by the virtual input v_1^I, v_2^I into (5.91) which , when introduced in (5.85), leads to:

$$\ddot{z}_i^I = v_i^I + d_i^I, \ \ (i \in \{1, 2\}) \tag{5.92}$$

where d_1^I and d_2^I are the propagated disturbances affecting the flat output which are assumed to be bounded and, through linearization (5.91), are shown to be:

$$\begin{aligned} d_1^I &= \dot{d}_{x_1} - u_1 d_{x_3} \sin x_3, \\ d_2^I &= \dot{d}_{x_2} + u_1 d_{x_3} \cos x_3, \end{aligned} \tag{5.93}$$

where all states $\boldsymbol{x}$ and control $\boldsymbol{u}$ have to be expressed in terms of the flat output like in (2.3) (see also Section 5.4).

ii) Second flat representation:

The similar design procedure is applied for the second flat output, given in (5.88). With the same extension $\eta = u_1$, replacing $\ddot{z}_1^{II}, \ddot{z}_2^{II}$ respectively by the virtual input v_1^{II}, v_2^{II} into the representation of the input $[u_\eta, \tan u_2]$ we define the linearizing control law as:

$$\begin{aligned} u_\eta &= \frac{v_2^{II} \dot{z}_2^{II} (z_1^{II})^2 + v_1^{II} \dot{z}_1^{II} + (\dot{z}_2^{II})^2 \dot{z}_1^{II} z_1^{II}}{\sqrt{(\dot{z}_1^{II})^2 + (z_1^{II} \dot{z}_2^{II})^2}}, \\ \tan u_2 &= \frac{v_2^{II} \dot{z}_1^{II} z_1^{II} - v_1^{II} \dot{z}_2^{II} z_1^{II} + 2(\dot{z}_1^{II})^2 \dot{z}_2^{II} + (\dot{z}_2^{II})^3 (z_1^{II})^2}{g\sqrt{(\dot{z}_1^{II})^2 + (z_1^{II} \dot{z}_2^{II})^2}}. \end{aligned} \tag{5.94}$$

Equation (5.94) then turns dynamics (5.85) into:

$$\ddot{z}_i^{II} = v_i^{II} + d_i^{II}, \ \ (i \in \{1, 2\}) \tag{5.95}$$

where the propagation of the disturbance is expressed as:

$$\begin{aligned} d_1^{II} &= \frac{x_1(\dot{d}_{x_1} - u_1 d_{x_3}\sin x_3) + x_2(\dot{d}_{x_2} + u_1 d_{x_3}\cos x_3)}{\sqrt{x_1^2 + x_2^2}} \\ &+ \frac{2u_1(d_{x_1}\cos x_3 + d_{x_2}\sin x_3) + d_{x_1}^2 + d_{x_2}^2}{\sqrt{x^2 + y^2}} \\ &+ \frac{(x_1 d_{x_1} + x_2 d_{x_2})(x_1 d_{x_1} + x_2 d_{x_2} + 2u_1(x_1\cos x_3 + x_2\sin x_3))}{\left(\sqrt{x_1^2 + x_2^2}\right)^{3/2}}, \\ d_2^{II} &= \frac{x_1(\dot{d}_{x_2} - u_1 d_{x_3}\cos x_3) + x_2(-\dot{d}_{x_1} + u_1 d_{x_3}\sin x_3)}{x_1^2 + x_2^2}. \end{aligned} \tag{5.96}$$

Remark 5.6. The objective of deducing (5.93) and (5.96) is not to compute the exact value of the disturbance propagated through the linearization process. Rather, we would like to highlight the fact that different flat outputs lead to different linearization laws, and, hence, different representations of the disturbances in such new coordinate systems. Therefore, although we control the systems (5.92) and (5.95) with identical control laws for v_i^I and v_i^{II}, their closed-loop performance will definitely differ from each other. This will again, raise the question of which flat output is the most convenient to use while linearizing the system.

5.5.2.3 Tracking control design

In this subsection, the application of the procedure introduced in Section 5.5.1.1 is implemented for the UAV using the two linearized dynamics from (5.92) and (5.95). Since both of them share a common double-integrator form:

$$\ddot{z}_i^j = v_i^j + d_i^j \quad (\text{with } i \in \{1,2\}, j \in \{I, II\}) \tag{5.97}$$

following Section 5.5.1.1, (5.97) is rewritten as:

$$\dot{\boldsymbol{\sigma}}_i^j = \mathbf{A}_i^j\,\boldsymbol{\sigma}_i^j + \mathbf{B}_i^j\,v_i^j + \mathbf{E}_i^j\,\tau_i^j, \tag{5.98}$$

where the matrices are defined as:

$$\boldsymbol{\sigma}_i^j = [\sigma_{i,1}^j \quad \sigma_{i,2}^j \quad \sigma_{i,3}^j]^\top \triangleq [z_i^j \quad \dot{z}_i^j \quad d_i^j]^\top, \tag{5.99}$$

$$\mathbf{A}_i^j = \begin{bmatrix} 0 & 1 & 0 \\ 0 & 0 & 1 \\ 0 & 0 & 0 \end{bmatrix}; \mathbf{B}_i^j = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}; \mathbf{E}_i^j = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}; \tau_i^j = \dot{d}_i^j. \tag{5.100}$$

Next, the observer is formulated similarly to (5.72) as:

$$\dot{\hat{\boldsymbol{\sigma}}}_i^j = \mathbf{A}_i^j \hat{\boldsymbol{\sigma}}_i^j + \mathbf{B}_i^j v_i^j + \mathbf{L}_i^j (y_i^j - \hat{y}_i^j) \tag{5.101}$$

where $\mathbf{L}_i^j \in \mathbb{R}^{3\times 1}$ is the tuning parameter for the estimator and $y_i^j = \sigma_{i,1}^j = z_i^j$ is the measured output. Equivalently, y_i^j may be rewritten as: $y_i^j = \mathbf{C}_i^j \boldsymbol{\sigma}_i^j$ with $\mathbf{C}_i^j = [1\ 0\ 0]$. The estimation error yields:

$$\dot{\hat{\boldsymbol{e}}}_i^j = \dot{\boldsymbol{\sigma}}_i^j - \dot{\hat{\boldsymbol{\sigma}}}_i^j = (\mathbf{A}_i^j - \mathbf{L}_i^j \mathbf{C}_i^j)\, \hat{\boldsymbol{e}}_i^j + \mathbf{E}_i^j \tau_i^j. \tag{5.102}$$

Using form (5.74), the virtual inputs v_i^j are constructed as:

$$v_i^j = \dot{\sigma}_{i,2\,ref}^j + \mathbf{K}_i^j \boldsymbol{e}_i^j - \hat{\sigma}_{i,3}^j \tag{5.103}$$

where $\boldsymbol{e}_i^j = [e_{i,1}^j\ e_{i,2}^j]^\top \triangleq \left[\sigma_{i,1\,ref}^j - \hat{\sigma}_{i,1}^j,\ \ \sigma_{i,2\,ref}^j - \hat{\sigma}_{i,2}^j\right]^\top$ and $\mathbf{K}_i^j \triangleq [k_{i,1}^j\ \ k_{i,2}^j] \in \mathbb{R}^2$ denote the tracking error and the controller's gain respectively; the subscript "ref" implies the corresponding reference signal that the system needs to track i.e, $\sigma_{i,1\,ref}^j = z_{i\,ref}^j$ and $\sigma_{i2\,ref}^j = \dot{z}_{i\,ref}^j$. Finally, in accordance with Proposition 5.2, the closed-loop dynamics are written as:

$$\frac{\mathrm{d}}{\mathrm{d}t}\begin{bmatrix}\boldsymbol{e}_i^j \\ \hat{\boldsymbol{e}}_i^j\end{bmatrix} = \begin{bmatrix}\mathbf{A}_i^{j*} - \mathbf{B}_i^{j*}\mathbf{K}_i^j & -\mathbf{B}_i^{j*}\,\tilde{\mathbf{K}}_i^j \\ \mathbf{0}_{3\times 2} & \mathbf{A}_i^j - \mathbf{L}_i^j\mathbf{C}_i^j\end{bmatrix}\begin{bmatrix}\boldsymbol{e}_i^j \\ \hat{\boldsymbol{e}}_i^j\end{bmatrix} + \begin{bmatrix}\mathbf{0}_{2\times 1} \\ \mathbf{E}_i^j\end{bmatrix}\tau_i^j, \tag{5.104}$$

with $\mathbf{A}_i^{j*} = \left[\begin{smallmatrix}0 & 1\\ 0 & 0\end{smallmatrix}\right]$; $\mathbf{B}_i^{j*} = \left[\begin{smallmatrix}0\\ 1\end{smallmatrix}\right]$; $\tilde{\mathbf{K}}_i^j = [\mathbf{K}_i^j\ \ 1]$. Then by a suitable choice of poles for dynamics (5.104), with the help of $\mathbf{K}_i^j$ and $\mathbf{L}_i^j$, we can ensure its stability under the effect of the disturbance τ_i^j. Finally, since system (5.104) is under form (5.80), we can compute the approximation of the mRPI set given in (5.84), for each pair of $i \in \{1, 2\}$ and $j \in \{I, II\}$, denoted respectively by Ω_i^j.

5.5.3 Simulation results

5.5.3.1 Numerical simulation

For illustration purposes, numerical simulation will be carried out in this section. For all $i, j \in \{1, 2\}$ the poles of the closed-loop system, given by (5.104) will be placed identically. Furthermore, it is also assumed that the bounds of τ_i^j are numerically computed via their representations in (5.93), (5.96) and simulations. Simulation specifications are given in Table 5.10.

TABLE 5.10: Simulation specifications

Variables	Values
$[x(0),\, y(0),\, x_3(0)]^\top$	$[54 \quad 160 \quad 1.8726]^\top$
w_{x_1}	$0.5\sin(0.2t)$
w_{x_2}	$-0.2\cos(0.5t)$
w_{x_3}	$-0.05\sin(0.5t)$
u_1	$10 \leq u_1 \leq 30$
$\tan u_2$	$-0.6 \leq \tan u_2 \leq 0.6$
$\mathbf{K}_i^j$	$[12 \quad 7]$
$\mathbf{L}_i^j$	$[15.5 \quad 59.5 \quad 45]^\top$

For defining the reference signal, let us briefly recall the optimal trajectory framework in [Stoican et al., 2015b] with the way-point passing constraints described by:

$$\mathbb{W} = 10^2 \times \left\{ \begin{bmatrix} 0.5 \\ 1.6 \end{bmatrix}, \begin{bmatrix} 0.7 \\ 2.5 \end{bmatrix}, \begin{bmatrix} 1.9 \\ 2.7 \end{bmatrix}, \begin{bmatrix} 2.8 \\ 1.6 \end{bmatrix}, \begin{bmatrix} 3.6 \\ 0.6 \end{bmatrix}, \begin{bmatrix} 5 \\ 1.75 \end{bmatrix} \right\} \quad [m]$$

$$\mathbb{T}_W = \{0, 8, 16, 24, 32, 40\} \quad [s], \tag{5.105}$$

where $\mathbb{W}$ is the set of way-points that the system needs to pass through at the associated time stamps in $\mathbb{T}_W$. The trajectory here is parametrized as a weighted sum of 7 B-spline functions or order 5 (which ensures passing through the way-points of (5.105) and guarantees curve smoothness for up to its 4th order derivative).

TABLE 5.11: Tracking performance with different controller and observer poles

p_c	p_o	RMS($e_{x_1 I}$)	RMS($e_{x_2 I}$)	RMS($e_{x_1 II}$)	RMS($e_{x_2 II}$)
$[-2; -1.5]$	$[-10; -4.5; -1]$	0.67	0.41	3.98	4.11
$[-4; -3]$	$[-10; -4.5; -1]$	0.47	0.14	1.10	1.03
$[-4; -3]$	$[-10; -6; -3]$	0.49	0.16	1.11	1.04

With the reference represented by the dashed red curve, Figure 5.22 depicts the tracking performances of the two controllers deduced from the two flat outputs (respectively denoted by Ctrl. I and Ctrl. II). Simultaneously, by calculating the outer approximation of the mRPI set, given in (5.84), of (5.104) for each pair of $i \in \{1, 2\}, j \in \{I, II\}$, we obtain two pairs of polytopic sets corresponding to the two flat representations. The projections of each pair into Cartesian coordinates are also depicted in Figure 5.22 with the notation of $\overline{\Omega}_j$ which corresponds to the controller Ctrl. j.

We provide also different scenarios in Table 5.11 for which the poles of the closed loop are chosen differently. Let p_c and p_o denote the poles for the controllers and observers respectively. $RMS(e_{lj})$, with $l \in \{x_1, x_2\}$ and $j \in \{I, II\}$, represents the

root mean square (RMS) of the tracking error on the l axis implemented with the Ctrl. j. Finally, let us consider the perturbation in the form of $[d_{x_1}, d_{x_2}, d_{x_3}]^\top = [0.5\sin\omega t, -0.2\cos\omega t, -0.05\sin\omega t]^\top$ to analyse the system response under different frequencies ω of the perturbation. Illustrations of the RMS of the tracking error are provided in Figure 5.23.

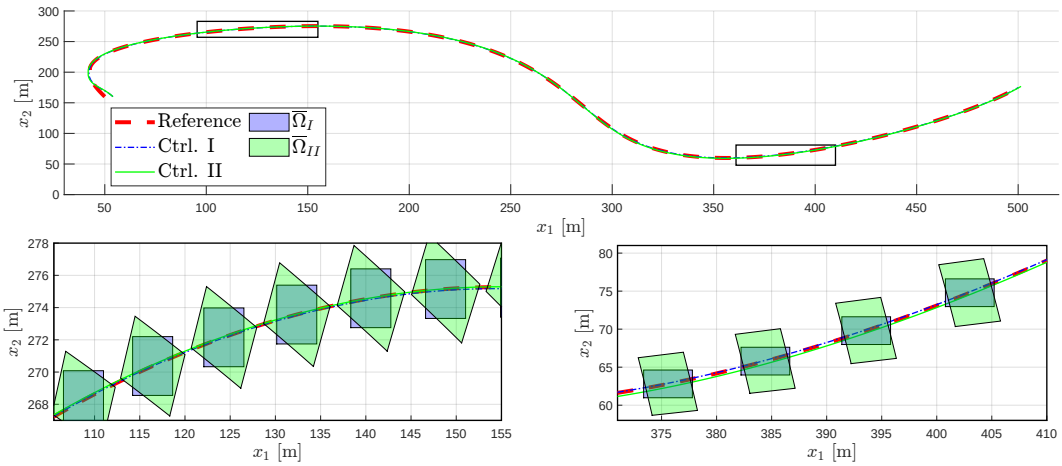


FIGURE 5.22: Ultimate bounds tube projected in the Cartesian space.

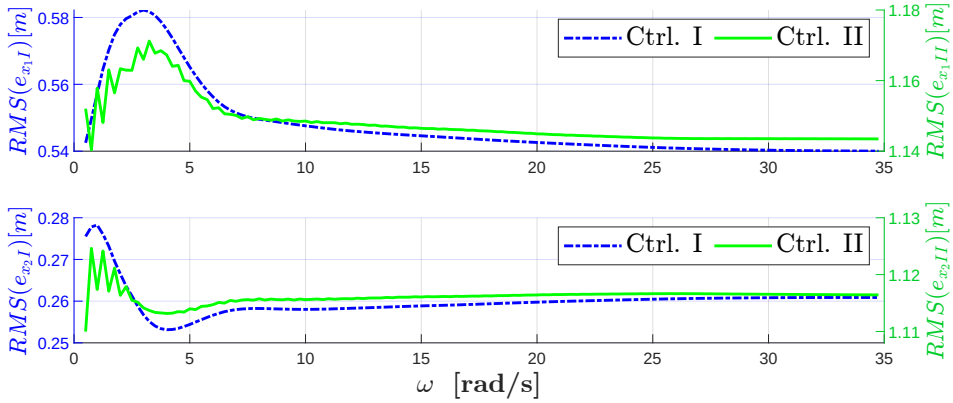


FIGURE 5.23: RMS of tracking errors under disturbances with different frequencies.

5.5.3.2 Discussions

In the following we address some remarks on the approach implemented over the UAV model and the analysis of different effects on the system's performance led by the change in the flat outputs.

i) Control design:

As a powerful combination, flatness and ESO provide a framework for not only estimating the states' value but also rejecting the disturbances in nonlinear control design. However, although the system's error may be proven to be bounded, this confirmation can only be drawn after the choice of the poles in the observer and the controller designs. This yields the question if there exist more efficient choices for the gains $\mathbf{K}_i^j$ and $\mathbf{L}_i^j$ in the sense that there will be less energy spent on the input while the tolerance for the error is still satisfied. Besides, from the frequency domain standpoint, in order to filter out the perturbation, the poles must be chosen differently due to the variation (in amplitude, in frequency) of the original disturbance via the linearization procedure. Likewise, it is noted in Figure 5.23 that for the same choice of poles, the errors' RMS behave differently in the frequency domain. Hence, this issue needs to be studied at the parameters' tuning stage. Moreover, (5.93) and (5.96) have different dependence on the derivative degree of the original disturbance, posing the need to consider the smoothness of disturbances while choosing among the representations.

ii) Analysis for different flat outputs:

As aforementioned, the bounded or stable tracking error of flat outputs provides insights on the system's tracking errors. Particularly, as seen in the simulations for the UAV model, the two trajectories resulting from the two different controllers lie within the tubes computed from two different flat representations (Figure 5.22). However, it is notable that the geometrical size and orientation of the tubes' cross-section vary along its trajectory. This comes from the fact that these two flat outputs do not have the same physical interpretation. Specifically, from a geometrical viewpoint, the first flat output represents the Cartesian position of the UAV while the second one corresponds to the UAV's distance from the origin and the angular coordinate in cylindrical coordinates. Therefore, when plotted in Cartesian coordinates, the tube from the second flat representation changes its volume depending on the UAV's position.

Moreover, as discussed previously, the choice of flat output directly affects the propagated disturbances acting on the virtual input, which may suggest that, the storage and processing requirements of a particular flat representation should not be neglected. Moreover, as stated in Table 5.11, the first control law performs better compared to the second one in all scenarios, implying that in this context, the best choice with respect to tracking performance may be the first controller. However, analytical proofs are still definitely required to certify this observation.

Furthermore, it should be also noted that these two flat representations allow us to represent the sets in the original coordinates, facilitating the comparison between their performance. However, with more complicated flat outputs, e.g., as the ones mentioned in Section 5.4 ([C1]C1), the comparison is no longer evident (or applicable at all).

Finally, the specific choice of the flat output influences the disturbance and thus, the shape and size of the invariant sets associated to the tracking and estimation errors. This becomes especially relevant in the case of complex dynamics and/or strongly nonlinear flat representations.

5.6 Concluding remarks

In this chapter, we gather and explore the follow-up questions which are raised throughout the manuscript regarding robustness and the implications of different flat outputs in flatness-based control.

In the first two sections, for the quadcopter dynamics in the flat output space, we enhanced both the MPC formulation and explicit saturated control in Chapter 3 with a tube MPC trajectory tracking design and an adaptive gain regulation, respectively. The results show that for the tube MPC implementation, disturbances were accounted for during the synthesis. However, the online computation time increased notably. This is because the with the nature of making a control decision in the worst-case scenario, tube MPC restricts the feasible domain significantly, hence, complicating the online solving for the optimal control. In the adaptive scheme, the tuning problem was solved by introducing virtual dynamics into the gain, allowing the value to increase as the system approaches the set point. Future work in this direction will concentrate on adapting the gain for each channel of the linearized dynamics rather than using a single scalar parameter for all three inputs.

In the remaining parts, the issue of using different flat outputs is addressed via the example of the fixed-wing UAV example. The results showed that, with different flat outputs, the trajectory generation problem will have different computational complexity and continuity requirements for the parameterized curves. This, consequently, provides different optimal trajectory for the same physical cost and constraints. For the online tracking, it is shown that the disturbances propagate differently via different flatness-based linearizing mapping, resulting in distinct closed-loop behaviors in the both time and frequency domain. This analysis opens the direction for studying the optimal choice of flat output with the objective of minimizing some performance indicators such as tracking error or the size of positive robust invariant sets.

Chapter 6

Conclusions and future developments

6.1 Conclusion

This manuscript has presented our endeavors in exploiting differential flatness for complexity reduction in nonlinear control systems. The contributions emerged from the application of quadcopter outer loop control, leading to the development of various control designs with different levels of simplicity. Additionally, the theoretical findings incorporate control Lyapunov function-based design, exploiting the control affinity in the linearizing space of the system's dynamics.

In the context of quadcopter control, flatness theory transforms the nonlinear dynamics into equivalent linear dynamics in a new coordinate system, allowing the use of well-established linear system techniques such as Model Predictive Control (MPC) and saturated control.

For MPC we developed a general zonotope-based approximation procedure to approximate the nonlinear convex constraint set with a polytope in the flat output space. This approximation enabled the implementation of implicit, explicit, and, finally, robust tube MPC, taking into account the constraints explicitly with stability robustly guaranteed. The main advantage of the proposed setting is the significant reduction in computational complexity, since the online control problem requires only the solution of tractable problems such as quadratic programs. In addition, the synthesis procedure is also further simplified with the standard axioms in linear MPC design.

For saturated control, the complexity decreases even more by using a specialized saturation function for the new set of constraints, giving rise to the implementation of

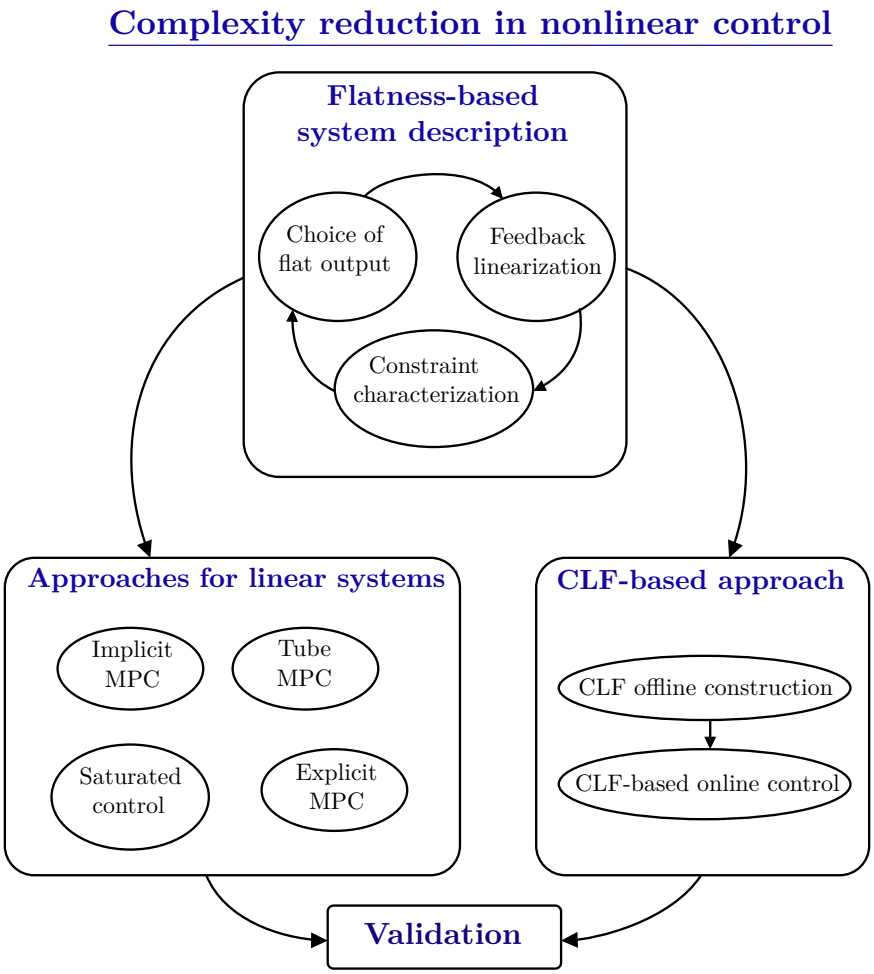


FIGURE 6.1: Complexity reduction for nonlinear control: a flatness-based perspective.

the well-known gradient-based saturated design for linear dynamics with quadratic Lyapunov function. The result is then extended with a gain adaptation to solve the tuning problem. All these original contributions are experimentally validated and compared with the literature via our localization and nano-drones platform.

With the successful validation of the quadcopter results, we have demonstrated that by using a convex inner approximation of the constraint set in the flat output space, it becomes more straightforward to design a stabilizing control and implement it online.

This analysis then motivates, on one hand, our first attempt to generalize the control design framework via an inner approximation characterized by neural networks. The general approximation is shown to be applicable via mixed-integer linear programming (MILP) or mixed-integer quadratic programming (MIQP), although these programs still face challenges related to dimensionality in real-time implementation.

On the other hand, our efforts to generalize control synthesis in the flat output space included the artificial synthesis of control Lyapunov functions (CLF). While this is not a revolutionary theoretical development, it is an area with limited experimental validation. Our procedure to synthesize a CLF requires solving a semi-infinite linear program (SILP). Then, by sampling the state space with a baseline MPC software of high performance, the computation was relaxed into a tractable linear program. To improve the efficiency in the offline computation and quantify the performance loss, we extended the setting with the a cutting-plane technique to solve the SILP and with the introduction of sub-control Lyapunov functions (SCLF), respectively. The results were then validated via both comprehensive simulations of control affine systems and experiments of the quadcopter.

Last but not least, the contribution also included the investigation of how changing the flat output affects the developed framework, from trajectory generation to tracking control. The analysis was demonstrated via the fixed-wing unmanned aerial vehicle example, highlighting the need to select an optimal flat output to simplify the planning and enhance the closed-loop performance.

To sum up, using differential flatness to cancel the system's nonlinearity inevitably complicates the constraints description. Throughout this thesis, we demonstrated for the quadcopter application that this challenge can be addressed through appropriate inner approximations of the feasible domain. By doing so, low-cost constraint compliant controllers can be synthesized, offering varying degrees of optimality, flexibility and generality. Over the course of the work, we outlined a comprehensive procedure to formulate the control problem in the flat output space, from choosing the flat output and characterizing the transformed constraints to constructing the control strategy. Interestingly, the linearization via flatness also gives rise to the use of CLF-based strategies with a quadratic-program complexity, opening a promising connection between flat systems and Lyapunov analysis. Although a universal generalization of the presented methods was not fully achieved within this three-year project, we believe that the manuscript has laid important groundwork to bridge the gap between nonlinear and linear control paradigms. To extend the results, future directions will be discussed in the next section.

6.2 Future developments

Throughout the manuscript, although several supporting questions have been addressed in Chapter 5, the majority remain open and can be categorized as follows.

1) Characterize the performance of the hierarchical scheme: In the hierarchical architecture of the quadcopter presented in Section 3.2, only the outer loop design was considered throughout the thesis. Thus, future studies should examine the impact of the inner loop on the overall scheme's performance. Specifically, it is important to determine the conditions under which the interdependence between the outer and inner loops guarantees asymptotic stability. We propose as future direction the consideration

of the Singular Perturbation theory [Kokotović et al., 1999, Bertrand et al., 2011] or the Small Gain theory [Jiang et al., 1996, Nicotra, 2016], to quantify how fast the inner loop should be to ensure the stability of the cascade architecture. Notably, this task of interconnection characterization may be simplified via flatness. The rotational dynamics, which govern the inner loop, also form a differentially flat system and, hence, enjoy the benefits of dynamic feedback linearization [Nguyen, 2019, Greiff, 2021]. This approach reduces the problem to the interconnection of two linear systems in the new coordinates [Nicolau and Iovine, 2022]. Finally, it would be valuable to revisit the implications of different flat outputs for the model as reviewed in Chapter 2.

2) Explore other strategies of low computational cost: With the linearized dynamics, future research could explore the low-complexity strategies of Reference Management Device [Blanchini and Miani, 2008], such as reference or command governors [Garone et al., 2017, Castroviejo-Fernandez et al., 2024] to maintain computational efficiency. By modifying the reference signal to avoid constraint violation, this "reference-filtering" software can be combined with a pre-stabilized flat system to handle state and input constraints efficiently. The approach is highly relevant since the pre-stabilization of the linearized flat systems can be locally characterized via standard static feedback (e.g., as in Section 3.3.3). At the same time, for linear systems, the governors can be implemented via linear, quadratic programs [Garone et al., 2017] or explicit formulations [Garone and Nicotra, 2015]. Additionally, in the similar context of stalling the reference to avoid constraint violation, set-based methods of computing a virtual set-point offer promising avenues [Alvarado et al., 2007, Blanchini and Miani, 2000].

3) Simplify and formalize the neural network-based control: With the mixed-integer linear constraint (3.76), a pertinent area for improvement lies in computational techniques that simplify the online optimization process. We believe Boolean algebra techniques such as the hyperplane arrangement idea [Prodan et al., 2015b] [J3] will be useful in reducing the number of integer variables. In parallel, theoretical extensions are needed to provide practical guarantees, such as stability and robustness within the mixed-integer optimization framework [Li et al., 2024]. Furthermore, it has been shown that the feasible domain structured by the neural networks can be represented by the so-called hybrid zonotopes, a recently developed class of of non-convex shapes employing integer variables [Zhang and Xu, 2023, Ortiz et al., 2023, Koeln et al., 2024]. This extension could facilitate the stability and reachability analysis of the linearized dynamics via such a geometrical setting.

4) Extend to robust control Lyapunov functions: The final improvement we propose is to incorporate uncertainties and disturbances into both the offline synthesis and online implementation of the control Lyapunov function (CLF)-based stabilization. In Chapter 4, only the stability and optimality constraints were applied during the synthesis of the CLF and the sub-control Lyapunov function (SCLF), respectively. Taking a step further, we can introduce robustness constraints to the synthesis, generating a so-called robust CLF [Blanchini, 1995, Ledyaev and Sontag, 1999]. The main challenge to

overcome is how to maintain the simplicity of the synthesis program, i.e., a semi-infinite linear program, even with the introduction of the robustness requirement. Furthermore, another significant performance improvement is to employ more advanced techniques in CLF-based control rather than the standard min-norm strategy as in (4.28). A possible candidate can be the model predictive control (MPC) formulations based on CLFs, e.g., CLF-based MPC [Grimm et al., 2005, Christofides et al., 2011, Wu et al., 2019]. This formulation involves using the constructed CLF locally around the set-point, combining the powerful qualities of MPC strategy, such as the enlargement of the domain of attraction.

Appendices

Appendix A

Appendix

A.1 Largest box inscribed in $\mathcal{V}_c$

With the zonotopic setup in [Do and Prodan, 2023], the maximum volume set $\mathcal{B}_v$ as in (3.26) inscribed in $\mathcal{V}_c$ in (3.14) is found by solving:

$$(\bar{v}_1, \bar{v}_2, \bar{v}_3)^* = \arg\max \bar{v}_1, \bar{v}_2, \bar{v}_3, \tag{A.1a}$$

$$\text{s.t: } \text{Vert}(\mathcal{B}_v) \in \mathcal{V}_c \text{ as in (3.14)}. \tag{A.1b}$$

where $\text{Vert}(\mathcal{B}_v)$ denotes set of vertices of $\mathcal{B}_v$. The finite condition (3.16b) implies and is implied by the containment $\mathcal{B}_v \subset \mathcal{V}_c$ thanks to the convexity of $\mathcal{V}_c$, while the cost function (3.16a) maximizes the volume of $\mathcal{B}_v$. With the continuity of the cost function and the boundedness of the constraints in (3.16), the existence of an optimal solution is certain. By analyzing the KKT necessary conditions, the description of the set can be briefly expressed as follows:

- if $2c^\star(g + c^\star) - \tan(\epsilon_{max})^2(c^\star - g)^2 < 0$, then:

$$\bar{v}_3 = c^\star; \bar{v}_1 = \bar{v}_2 = \sqrt{c^\star(c^\star + g)}. \tag{A.2}$$

- Otherwise,

$$\bar{v}_3 = g/3; \bar{v}_1 = \bar{v}_2 = \tan(\epsilon_{max})\sqrt{\bar{v}_3(g - \bar{v}_3)}, \tag{A.3}$$

where $c^\star \triangleq \left(-2g + \sqrt{g^2 + 3T_{max}^2}\right)/3$.

A.2 PWA-MPC formulation

We adapt the implementation of PWA-MPC from [Prodan et al., 2015a] as follows. Along the system's reference trajectory, we choose N_l points around which the dynamics is approximated by using Taylor expansion as:

$$\boldsymbol{\xi}(k+1) \approx \boldsymbol{A}_l \boldsymbol{\xi}(k) + \boldsymbol{B}_l \boldsymbol{u}(k) + \boldsymbol{r}_l \tag{A.4}$$

with $\boldsymbol{f}_d$ being the discretized dynamics of (3.4), i.e., $\boldsymbol{\xi}(k+1) = \boldsymbol{f}_d(\boldsymbol{\xi}(k), \boldsymbol{u}(k))$, $\boldsymbol{A}_l = \partial \boldsymbol{f}_d / \partial \boldsymbol{\xi}|_{\boldsymbol{\xi}_l, \boldsymbol{u}_l}$, $\boldsymbol{B}_l = \partial \boldsymbol{f}_d / \partial \boldsymbol{\xi}|_{\boldsymbol{\xi}_l, \boldsymbol{u}_l}$ and $\boldsymbol{r}_l = \boldsymbol{f}_d(\boldsymbol{\xi}_l, \boldsymbol{u}_l) - \boldsymbol{A}_l \boldsymbol{\xi}_l - \boldsymbol{B}_l \boldsymbol{u}_l$ while $\boldsymbol{\xi}_l, \boldsymbol{u}_l$ respectively denote the l-th state and input value in the collection of N_l points equidistantly chronologically sampled from the nominal trajectory. During the implementation, $\boldsymbol{A}_l, \boldsymbol{B}_l$ and $\boldsymbol{R}_l$ are flexibly chosen according to the drone's closest point. Hence, the online optimization problem is expressed as:

$$\underset{\boldsymbol{u}(k|k+j)}{\arg\min} \sum_{j=0}^{N_p-1} \|\boldsymbol{\xi}(k|k+j) - \boldsymbol{\xi}^{\text{ref}}(k|k+j)\|_{\boldsymbol{Q}}^2 + \|\boldsymbol{u}(k|k+j) - \boldsymbol{u}^{\text{ref}}(k|k+j)\|_{\boldsymbol{R}}^2 \tag{A.5}$$

$$\text{s.t} : \begin{cases} \boldsymbol{\xi}(k|j+k+1) = \boldsymbol{A}_l \boldsymbol{\xi}(k|k+j) + \boldsymbol{B}_l \boldsymbol{u}(k|k+j) + \boldsymbol{r}_l \\ \boldsymbol{u}(k|k+j) \in \mathcal{U},\ j \in \{0, 1, ..., N_p - 1\} \end{cases}$$

with $\boldsymbol{\xi}^{\text{ref}}(k), \boldsymbol{v}^{\text{ref}}(k)$ denoting the reference for $\boldsymbol{\xi}(k)$ and $\boldsymbol{u}(k)$.

A.3 Proof of Remark 4.5

Condition (4.21) can be rewritten as:

$$\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \nabla V(\boldsymbol{x}) \boldsymbol{d} \leq 0. \tag{A.6}$$

The left-hand side of (A.6) yields:

$$\begin{aligned} &\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \nabla V(\boldsymbol{x}) \boldsymbol{d} \\ &\leq \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \|\nabla V(\boldsymbol{x})^\top\| = \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \|\sum_{q=1}^{N} \alpha_q \nabla V_q(\boldsymbol{x})^\top\| \\ &= \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \sqrt{\sum_{j=1}^{n} \left(\sum_{q=1}^{N} \alpha_q \frac{\partial V_q}{\partial x_j}(\boldsymbol{x}) \right)^2} \\ &\leq \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \sqrt{\sum_{j=1}^{n} \left(\sum_{q=1}^{N} \alpha_q^2 \sum_{q=1}^{N} \left(\frac{\partial V_q}{\partial x_j}(\boldsymbol{x}) \right)^2 \right)}. \end{aligned} \tag{A.7}$$

Meanwhile, the conditions $\sum_{q=1}^{N} \alpha_q = 1$ and $\alpha_q \geq 0$ lead to:

$$\sum_{q=1}^{N} \alpha_q^2 = \left(\sum_{q=1}^{N} \alpha_q\right)^2 - 2\sum_{u<v} \alpha_u \alpha_v < \left(\sum_{q=1}^{N} \alpha_q\right)^2 = 1. \tag{A.8}$$

From (A.7) and (A.8), one has:

$$\Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \nabla V(\boldsymbol{x})\boldsymbol{d} \leq \Upsilon(\boldsymbol{\alpha}, \boldsymbol{x}, \boldsymbol{u}) + \mu \sqrt{\sum_{j=1}^{n} \sum_{q=1}^{N} \left(\frac{\partial V_q}{\partial x_j}(\boldsymbol{x})\right)^2}. \tag{A.9}$$

Therefore, (A.6), or equivalently (4.21), is implied by (4.22).

A.4 Feasibility-based bound tightening algorithm

The feasibility-based bound tightening procedure employed in Section 3.5 is given as in Algorithm A.1.

Algorithm A.1: Calculate L_j^k, U_j^k in (3.76) with FBBT.

Input: The input layer's bound $L_i^0, U_i^0, i \in \mathcal{I}_{1,n_z+m}$.
Output: L_j^k, U_j^k, $k \in \mathcal{I}_{1,K-1}$, $j \in \mathcal{I}_{1,n_k}$.

for $j = 1$ **to** n_1 **do**
  $U_j^1 = b_j^1 + \sum_{i=1}^{n_k-1} \max\{W_{ji}^1 U_i^0, W_{ji}^1 L_i^0\}$;
  $L_j^1 = b_j^1 + \sum_{i=1}^{n_k-1} \min\{W_{ji}^1 U_i^0, W_{ji}^1 L_i^0\}$;
end
for $k = 2$ **to** $K-1$ **do**
  for $j = 1$ **to** n_k **do**
    $U_j^k = \sum_{i=1}^{n_{k-1}} \big(\max \{W_{ji}^k \max(0, U_i^k),$
      $W_{ji}^k \max(0, L_i^k)\}\big) + b_j^k$;
    $L_j^k = \sum_{i=1}^{n_{k-1}} \big(\min \{W_{ji}^k \max(0, U_i^k),$
      $W_{ji}^k \max(0, L_i^k)\}\big) + b_j^k$;
  end
end
//W_{ji}^k is the jth row, ith column entry of $\boldsymbol{W}^k$.

Bibliography

Amine Abadi, Adnen El Amraoui, Hassen Mekki, and Nacim Ramdani. Robust tracking control of quadrotor based on flatness and active disturbance rejection control. *IET Control Theory & Applications*, 14(8):1057–1068, 2020.

Cesar O Aguilar and Arthur J Krener. Numerical solutions to the Bellman equation of optimal control. *Journal of optimization theory and applications*, 160:527–552, 2014.

Babak Akbari and Melissa Greeff. A computationally efficient learning-based model predictive control for multirotors under aerodynamic disturbances. In *2024 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 185–192, 2024. doi: 10.1109/ICUAS60882.2024.10557089.

Fahad Saleh Al-Ismail. Dc microgrid planning, operation, and control: A comprehensive review. *IEEE Access*, 9:36154–36172, 2021.

M. Alamir. *Stabilization of Nonlinear Systems Using Receding Horizon Technique*. Lecture Notes in Control and Information Sciences, 339. Springer, Berlin, 2006.

Mohammad Alsalti, Julian Berberich, Victor G Lopez, Frank Allgöwer, and Matthias A Müller. Data-based system analysis and control of flat nonlinear systems. In *2021 60th IEEE Conference on Decision and Control (CDC)*, pages 1484–1489. IEEE, 2021.

Ignacio Alvarado, Daniel Limón, Teodoro Alamo, Mirko Fiacchini, and Eduardo F Camacho. Robust tube based mpc for tracking of piece-wise constant references. In *2007 46th IEEE Conference on Decision and Control*, pages 1820–1825. IEEE, 2007.

Edward J Anderson and Peter Nash. *Linear Programming in Infinite-dimensional Spaces: Theory and Applications*. A Wiley-Interscience publication. Wiley, 1987. ISBN 9780471912507.

Joel AE Andersson, Joris Gillis, Greg Horn, James B Rawlings, and Moritz Diehl. Casadi: a software framework for nonlinear optimization and optimal control. *Mathematical Programming Computation*, 11:1–36, 2019.

Julian Andrej and Thomas Meurer. Flatness-based constrained optimal control of reaction-diffusion systems. In *2018 Annual American Control Conference (ACC)*, pages 2539–2544. IEEE, 2018.

Felix Antritter and Gregor Günther Verhoeven. On symbolic computation of flat outputs for differentially flat systems. *IFAC Proceedings Volumes*, 43(14):677–682, 2010.

Zvi Artstein. Stabilization with relaxed controls. *Nonlinear Analysis: Theory, Methods & Applications*, 7(11):1163–1173, 1983.

J. P. Aubin and A. Cellina. *Differential inclusions*, volume 264 of *Grundlehren der Mathematischen Wissenschaften [Fundamental Principles of Mathematical Sciences]*. Springer-Verlag, Berlin, 1984. ISBN 3-540-13105-1. Set-valued maps and viability theory.

M. Bardi and I. Capuzzo Dolcetta. *Optimal Control and Viscosity Solutions of Hamilton-Jacobi-Bellman Equations*. Birkhäuser, Boston, 1997.

Randal W Beard and Timothy W McLain. *Small unmanned aircraft: Theory and practice*. Princeton university press, 2012.

Randal W Beard, George N Saridis, and John T Wen. Galerkin approximations of the generalized Hamilton-Jacobi-Bellman equation. *Automatica*, 33(12):2159–2177, 1997.

Logan E Beaver and Andreas A Malikopoulos. Optimal control of differentially flat systems is surprisingly easy. *Automatica*, 159:111404, 2024.

Maria Bekcheva. *Flatness-based constrained control and model-free control applications to quadrotors and cloud computing*. PhD thesis, Université Paris Saclay (COmUE), 2019.

Alberto Bemporad, Manfred Morari, Vivek Dua, and Efstratios N Pistikopoulos. The explicit linear quadratic regulator for constrained systems. *Automatica*, 2002.

Tim Benham. Uniform distribution over a convex polytope. *MATLAB Central File Exchange*, 2011. URL https://www.mathworks.com/matlabcentral/fileexchange/34208-uniform-distribution-over-a-convex-polytope.

Julian Berberich, Johannes Köhler, Matthias A Müller, and Frank Allgöwer. Linear tracking mpc for nonlinear systems—part ii: The data-driven case. *IEEE Transactions on Automatic Control*, 67(9):4406–4421, 2022.

S Bernštein. Démonstration du théoreme de weierstrass fondée sur le calcul des probabilities. *Comm. Soc. Math. Kharkov*, 13:1–2, 1912.

Sylvain Bertrand, Nicolas Guénard, Tarek Hamel, Hélène Piet-Lahanier, and Laurent Eck. A hierarchical controller for miniature vtol uavs: Design and stability analysis using singular perturbation theory. *Control Engineering Practice*, 19(10):1099–1108, 2011.

Luigi Biagiotti and Claudio Melchiorri. *Trajectory planning for automatic machines and robots*. Springer Science & Business Media, 2008.

AB Bitcraze. Step-by-step: Swarm interface, 2023. URL https://www.bitcraze.io/documentation/repository/crazyflie-lib-python/master/user-guides/sbs_swarm_interface/.

F Blanchini. Feedback control for linear systems with state and control bounds in the presence of disturbance. *IEEE Trans. Aut. Control*, 35:1131–1135, 1990.

Franco Blanchini. Nonquadratic lyapunov functions for robust control. *Automatica*, 31 (3):451–461, 1995.

Franco Blanchini and Pietro Giannattasio. Adaptive control of compressor surge instability. *Automatica*, 38(8):1373–1380, 2002.

Franco Blanchini and Stefano Miani. Constrained stabilization of continuous-time linear systems. *Systems & control letters*, 28(2):95–102, 1996.

Franco Blanchini and Stefano Miani. A new class of universal Lyapunov functions for the control of uncertain linear systems. *IEEE Transactions on Automatic Control*, 44 (3):641–647, 1999a.

Franco Blanchini and Stefano Miani. Numerical computation of polyhedral lyapunov functions for robust synthesis. *IFAC Proceedings Volumes*, 32(2):2065–2070, 1999b.

Franco Blanchini and Stefano Miani. Any domain of attraction for a linear constrained system is a tracking domain of attraction. *SIAM Journal on Control and Optimization*, 38(3):971–994, 2000.

Franco Blanchini and Stefano Miani. *Set-theoretic methods in control*. Springer, 2008.

Franco Blanchini, Stefano Miani, and Felice Andrea Pellegrino. Suboptimal receding horizon control for continuous-time systems. *IEEE Transactions on Automatic Control*, 48(6):1081–1086, 2003.

Stephen Boyd, Laurent El Ghaoui, Eric Feron, and Venkataramanan Balakrishnan. *Linear matrix inequalities in system and control theory*. SIAM, 1994.

Stephen Boyd, Stephen P Boyd, and Lieven Vandenberghe. *Convex optimization*. Cambridge university press, 2004.

Brahim Brahmi, Tanvir Ahmed, Ibrahim El Bojairami, Asif Al Zubayer Swapnil, Md Assad-Uz-Zaman, Katie Schultz, Erin McGonigle, and Mohammad Habibur Rahman. Flatness based control of a novel smart exoskeleton robot. *IEEE/ASME Transactions on Mechatronics*, 27(2):974–984, 2021.

David D Brengel and Warren D Seider. Multistep nonlinear predictive controller. *Industrial & engineering chemistry research*, 28(12):1812–1822, 1989.

Florian D Brunner, Mircea Lazar, and Frank Allgöwer. Stabilizing model predictive control: On the enlargement of the terminal set. *International Journal of Robust and Nonlinear Control*, 25(15):2646–2670, 2015.

Ning Cao and Alan F. Lynch. Inner–outer loop control for quadrotor uavs with input and state constraints. *IEEE Transactions on Control Systems Technology*, 24(5):1797–1804, 2016. doi: 10.1109/TCST.2015.2505642.

Miguel Castroviejo-Fernandez, Jordan Leung, and Ilya Kolmanovsky. Robust reference governor for input-constrained model predictive control to enforce state constraints at low computational cost. *International Journal of Control*, pages 1–14, 2024.

Dong Eui Chang and Yongsoon Eun. Construction of an atlas for global flatness-based parameterization and dynamic feedback linearization of quadcopter dynamics. In *53rd IEEE Conference on Decision and Control*, pages 686–691. IEEE, 2014.

Dong Eui Chang, Jean Lévine, Jeongdai Jo, and Kyung-Hyun Choi. Control of roll-to-roll web systems via differential flatness and dynamic feedback linearization. *IEEE Transactions on Control Systems Technology*, 21(4):1309–1317, 2012.

Benoît Charlet, Jean Lévine, and Riccardo Marino. Sufficient conditions for dynamic state feedback linearization. *SIAM journal on control and optimization*, 29(1):38–57, 1991.

Hong Chen and Frank Allgöwer. A quasi-infinite horizon nonlinear model predictive control scheme with guaranteed stability. *Automatica*, 34(10):1205–1217, 1998.

Jing Chen, Tianbo Liu, and Shaojie Shen. Online generation of collision-free trajectories for quadrotor flight in unknown cluttered environments. In *2016 IEEE international conference on robotics and automation (ICRA)*, pages 1476–1483. IEEE, 2016.

Steven Chen, Kelsey Saulnier, Nikolay Atanasov, Daniel D Lee, Vijay Kumar, George J Pappas, and Manfred Morari. Approximating explicit model predictive control using constrained neural networks. In *2018 Annual American control conference (ACC)*, pages 1520–1527. IEEE, 2018.

Steven W Chen, Tianyu Wang, Nikolay Atanasov, Vijay Kumar, and Manfred Morari. Large scale model predictive control with neural networks and primal active sets. *Automatica*, 135:109947, 2022.

Graziano Chesi. LMI techniques for optimization over polynomials in control: a survey. *IEEE Trans. on Automatic Control*, 55(11), 2010.

Panagiotis D Christofides, Jinfeng Liu, David Muñoz de la Peña, Panagiotis D Christofides, Jinfeng Liu, and David Muñoz de la Peña. Lyapunov-based model predictive control. *Networked and Distributed Predictive Control: Methods and Nonlinear Process Network Applications*, pages 13–45, 2011.

Alessandro Colombo and Antoine Girard. An approximate abstraction approach to safety control of differentially flat systems. In *2013 European Control Conference (ECC)*, pages 4226–4231. IEEE, 2013.

Bryan Convens, Dominic Liao-McPherson, Kelly Merckaert, Bram Vanderborght, and Marco M Nicotra. A terminal state feasibility governor for real-time nonlinear model predictive control over arbitrary horizons. *IEEE Transactions on Control Systems Technology*, 2024.

Ian D Cowling, Oleg A Yakimenko, James F Whidborne, and Alastair K Cooke. A prototype of an autonomous controller for a quadrotor uav. In *2007 European Control Conference (ECC)*, pages 4001–4008. IEEE, 2007.

IBM ILOG CPLEX. V12. 1: User's manual for CPLEX. *International Business Machines Corporation*, 46(53):157, 2009.

John J. Craig. *Introduction to Robotics: Mechanics and Control*. Pearson, 2004. ISBN 0201543613.

Claus Danielson and Francesco Borrelli. Symmetric linear model predictive control. *IEEE Transactions on Automatic Control*, 60(5):1244–1259, 2014.

Claudio De Persis, Darshan Gadginmath, Fabio Pasqualetti, and Pietro Tesi. Data-driven feedback linearization with complete dictionaries. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pages 3037–3042. IEEE, 2023.

Santosh Devasia. Should model-based inverse inputs be used as feedforward under plant uncertainty? *IEEE Transactions on Automatic Control*, 47(11):1865–1871, 2002.

Cong Khanh Dinh, Ionela Prodan, and Florin Stoican. Online b-spline based trajectory planning for swarm of agents using distributed model predictive control. In *2024 International Conference on Unmanned Aircraft Systems, ICUAS'24*, 2024.

Huu-Thinh Do and Ionela Prodan. Indoor experimental validation of mpc-based trajectory tracking for a quadcopter via a flat mapping approach. In *2023 European Control Conference (ECC)*, pages 1–6, 2023. doi: 10.23919/ECC57647.2023.10178260.

Huu-Thinh Do, Ionela Prodan, and Florin Stoican. Analysis of alternative flat representations of a uav for trajectory generation and tracking. In *2021 25th International Conference on System Theory, Control and Computing (ICSTCC)*, pages 58–63. IEEE, 2021.

Ján Drgoňa, Damien Picard, Michal Kvasnica, and Lieve Helsen. Approximate model predictive building control via machine learning. *Applied energy*, 218:199–216, 2018.

Ján Drgoňa, Karol Kiš, Aaron Tuor, Draguna Vrabie, and Martin Klaučo. Differentiable predictive control: Deep learning alternative to explicit model predictive control for unknown nonlinear systems. *Journal of Process Control*, 116:80–92, 2022.

Nadeem Faiz, Sunil K Agrawal, and Richard M Murray. Trajectory planning of differentially flat systems with dynamics and inequalities. *Journal of Guidance, Control, and Dynamics*, 24(2):219–227, 2001.

Samira S Farahani, Ivan Papusha, Catharine McGhan, and Richard M Murray. Constrained autonomous satellite docking via differential flatness and model predictive control. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 3306–3311. IEEE, 2016.

Gerald Farin. *Curves and surfaces for computer-aided geometric design: a practical guide*. Elsevier, 2014.

Timm Faulwasser, Veit Hagenmeyer, and Rolf Findeisen. Constrained reachability and trajectory generation for flat systems. *Automatica*, 50(4):1151–1159, 2014.

Mirko Fiacchini, Christophe Prieur, and Sophie Tarbouriech. On the computation of set-induced control lyapunov functions for continuous-time systems. *SIAM Journal on Control and Optimization*, 53(3):1305–1327, 2015.

Ferdinand Fischer and Joachim Deutscher. Flatness-based algebraic fault diagnosis for distributed-parameter systems. *Automatica*, 117:108987, 2020.

Matteo Fischetti and Jason Jo. Deep neural networks and mixed integer linear optimization. *Constraints*, 23(3):296–309, 2018.

Michel Fliess, Jean Lévine, Philippe Martin, and Pierre Rouchon. Sur lessystèmesnon linéaires différentiellement plats. *CR Acad. Sci. Paris*, page 619, 1992.

Michel Fliess, Jean Lévine, Philippe Martin, and Pierre Rouchon. On differentially flat nonlinear systems. In *Nonlinear Control Systems Design 1992*, pages 159–163. Elsevier, 1993.

Michel Fliess, Jean Lévine, Philippe Martin, and Pierre Rouchon. Flatness and defect of non-linear systems: introductory theory and examples. *International Journal of Control*, 61(6):1327–1361, 1995.

Michel Fliess, Jean Lévine, Philippe Martin, and Pierre Rouchon. A lie-backlund approach to equivalence and flatness of nonlinear systems. *IEEE Transactions on automatic control*, 44(5):922–937, 1999a.

Michel Fliess, Jean Lévine, Philippe Martin, and Pierre Rouchon. Some open questions related to flat nonlinear systems. *Open problems in mathematical systems and control theory*, pages 99–103, 1999b.

Philipp Foehn, Elia Kaufmann, Angel Romero, Robert Penicka, Sihao Sun, Leonard Bauersfeld, Thomas Laengle, Giovanni Cioffi, Yunlong Song, Antonio Loquercio, et al. Agilicious: Open-source and open-hardware agile quadrotor for vision-based flight. *Science robotics*, 7(67):eabl6259, 2022.

Simone Formentin and Marco Lovera. Flatness-based control of a quadrotor helicopter via feedforward linearization. In *2011 50th IEEE Conference on Decision and Control and European Control Conference*, pages 6171–6176. IEEE, 2011.

Julian Förster. System identification of the crazyflie 2.0 nano quadrocopter. B.S. thesis, ETH Zurich, 2015.

Matthias Franke and Klaus Röbenack. On the computation of flat outputs for nonlinear control systems. In *2013 European Control Conference (ECC)*, pages 167–172. IEEE, 2013.

Randy A Freeman and Petar V Kokotovic. Inverse optimality in robust stabilization. *SIAM journal on control and optimization*, 34(4):1365–1391, 1996.

Victor Freire and Xiangru Xu. Optimal control for kinematic bicycle model with continuous-time safety guarantees: A sequential second-order cone programming approach. *IEEE Robotics and Automation Letters*, 7(4):11681–11688, 2022.

Victor Freire and Xiangru Xu. Flatness-based quadcopter trajectory planning and tracking with continuous-time safety guarantees. *IEEE Transactions on Control Systems Technology*, 2023.

Yasuhiro Fujii and Hisakazu Nakamura. Input-to-state stabilization of differentially flat systems. *IFAC-PapersOnLine*, 51(29):256–261, 2018.

Quentin Galvane, Marc Christie, Chrsitophe Lino, and Rémi Ronfard. Camera-on-rails: automated computation of constrained camera paths. In *Proceedings of the 8th ACM SIGGRAPH Conference on Motion in Games*, pages 151–157, 2015.

Gonzalo A Garcia, A Ram Kim, Ethan Jackson, Shawn S Keshmiri, and Daksh Shukla. Modeling and flight control of a commercial nano quadrotor. In *2017 International Conference on Unmanned Aircraft Systems (ICUAS)*, pages 524–532. IEEE, 2017.

Emanuele Garone and Marco M Nicotra. Explicit reference governor for constrained nonlinear systems. *IEEE Transactions on Automatic Control*, 61(5):1379–1384, 2015.

Emanuele Garone, Stefano Di Cairano, and Ilya Kolmanovsky. Reference and command governors for systems with constraints: A survey on theory and applications. *Automatica*, 75:306–328, 2017.

Clément Gazzino, Denis Arzelier, Luca Cerri, Damiana Losa, Christophe Louembet, and Christelle Pittet. Solving the minimum-fuel low-thrust geostationary station keeping problem via the switching systems theory. In *European Conf. Aeronaut. Space Sci*, 2017.

Wojciech Giernacki, Mateusz Skwierczyński, Wojciech Witwicki, Paweł Wroński, and Piotr Kozierski. Crazyflie 2.0 quadrotor as a platform for research and education in robotics and control engineering. In *2017 22nd International Conference on Methods and Models in Automation and Robotics*, pages 37–42. IEEE, 2017.

Peter Giesl and Sigurdur Hafstein. Review on computational methods for lyapunov functions. *Discrete and Continuous Dynamical Systems-B*, 20(8):2291–2331, 2015.

Peter A Giesl and Sigurdur F Hafstein. Revised CPA method to compute lyapunov functions for nonlinear systems. *Journal of Mathematical Analysis and Applications*, 410(1):292–306, 2014.

Elmer G Gilbert and K Tin Tan. Linear systems with state and control constraints: The theory and application of maximal output admissible sets. *IEEE Trans. on Auto. Cont.*, 36(9):1008–1020, 1991.

Mahmoud AK Gomaa, Oscar De Silva, George KI Mann, and Raymond G Gosine. Computationally efficient stability-based nonlinear model predictive control design for quadrotor aerial vehicles. *IEEE Trans. on Control Systems Technology*, 2022.

Sergio Grammatico, Franco Blanchini, and Andrea Caiti. Control-sharing and merging control Lyapunov functions. *IEEE Transactions on Automatic Control*, 59(1):107–119, 2013.

Luca Greco, Hugues Mounier, and Maria Bekcheva. An approximate characterisation of the set of feasible trajectories for constrained flat systems. *Automatica*, 144:110484, 2022.

Melissa Greeff. *Flying Flat Out: Fast Multirotor Flight Using Vision-Based Navigation in Real-World Environments*. PhD thesis, University of Toronto (Canada), 2022.

Melissa Greeff and Angela P Schoellig. Flatness-based model predictive control for quadrotor trajectory tracking. In *2018 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 6740–6745. IEEE, 2018.

Melissa Greeff and Angela P Schoellig. Exploiting differential flatness for robust learning-based tracking control using Gaussian processes. *IEEE Control Systems Letters*, 2020.

Melissa Greeff, Adam W Hall, and Angela P Schoellig. Learning a stability filter for uncertain differentially flat systems using gaussian processes. In *60th IEEE Conf.on Decision and Control*, pages 789–794. IEEE, 2021.

Melissa Greeff, SiQi Zhou, and Angela P Schoellig. Fly out the window: Exploiting discrete-time flatness for fast vision-based multirotor flight. *IEEE Robotics and Automation Letters*, 7(2):5023–5030, 2022.

Marcus Greiff. *Nonlinear control of unmanned aerial vehicles: Systems with an attitude*. Lund University, 2021.

Gene Grimm, Michael J Messina, Sezai Emre Tuna, and Andrew R Teel. Model predictive control: for want of a local control lyapunov function, all is not lost. *IEEE Transactions on Automatic Control*, 50(5):546–558, 2005.

Bjarne Grimstad and Henrik Andersson. Relu networks as surrogate models in mixed-integer linear programs. *Computers & Chemical Engineering*, 131:106580, 2019.

Martin Guay, Denis Dochain, Michel Perrier, and Nicolas Hudon. Flatness-based extremum-seeking control over periodic orbits. *IEEE Transactions on Automatic Control*, 52(10):2005–2012, 2007.

Hongyan Guo, Fei Wang, Hong Chen, and Dapeng Guo. Stability control of vehicle with tire blowout using differential flatness based mpc method. In *Proceedings of the 10th world congress on intelligent control and automation*, pages 2066–2071. IEEE, 2012.

Yi Guo, Yi Long, and Weihua Sheng. Global trajectory generation for nonholonomic robots in dynamic environments. In *Proceedings 2007 IEEE International Conference on Robotics and Automation*, pages 1324–1329. IEEE, 2007.

Adam W Hall, Melissa Greeff, and Angela P Schoellig. Differentially flat learning-based model predictive control using a stability, state, and input constraining safety filter. *IEEE Control Systems Letters*, 7:2191–2196, 2023.

Zhichao Han, Yuwei Wu, Tong Li, Lu Zhang, Liuao Pei, Long Xu, Chengyang Li, Changjia Ma, Chao Xu, Shaojie Shen, et al. An efficient spatial-temporal trajectory planner for autonomous vehicles in unstructured environments. *IEEE Transactions on Intelligent Transportation Systems*, 2023.

L Hanel and J Lehner. Enhanced flexibility for steam power plants via model based feedforward control. *IFAC Proceedings Volumes*, 46(6):7–12, 2013.

John Hauser, Shankar Sastry, and George Meyer. Nonlinear control design for slightly non-minimum phase systems: Application to v/stol aircraft. *Automatica*, 28(4):665–679, 1992.

Wilhemus PMH Heemels, Bart De Schutter, and Alberto Bemporad. Equivalence of hybrid dynamical models. *Automatica*, 37(7):1085–1091, 2001.

M. Herceg, M. Kvasnica, C.N. Jones, and M. Morari. Multi-Parametric Toolbox 3.0. In *2013 European Control Conference*, pages 502–510, 2013.

Yoann Hervagault, Ionela Prodan, and Laurent Lefèvre. Motion planning for usvs with communication guarantees: An experimental setup. In *2019 18th European Control Conference (ECC)*, pages 3984–3989. IEEE, 2019.

Rainer Hettich and Kenneth O Kortanek. Semi-infinite programming: theory, methods, and applications. *SIAM review*, 35(3):380–429, 1993.

Andreas Himmel, Sebastian Sager, and Kai Sundmacher. Time-minimal set point transition for nonlinear siso systems under different constraints. *Automatica*, 114:108806, 2020.

Botao Hu and Sandipan Mishra. A time-optimal trajectory generation algorithm for quadrotor landing onto a moving platform. In *2017 American Control Conference (ACC)*, pages 4183–4188. IEEE, 2017.

Tingshu Hu and Zongli Lin. *Control systems with actuator saturation: analysis and design*. Springer Science & Business Media, 2001.

Tingshu Hu and Zongli Lin. Composite quadratic Lyapunov functions for constrained control systems. *IEEE Transactions on Automatic Control*, 48(3):440–450, 2003.

Tingshu Hu, Zongli Lin, and Ben M Chen. Analysis and design for discrete-time linear systems subject to actuator saturation. *Systems & control letters*, 45(2):97–112, 2002a.

Tingshu Hu, Zongli Lin, and Li Qiu. An explicit description of null controllable regions of linear systems with saturating actuators. *Systems & control letters*, 47(1):65–78, 2002b.

Carlos Aguilar Ibanez and O Gutiérrez Frias. Controlling the inverted pendulum by means of a nested saturation function. *Nonlinear Dynamics*, 53:273–280, 2008.

Moad Idrissi, Mohammad Salami, and Fawaz Annaz. A review of quadrotor unmanned aerial vehicles: applications, architectural design and control algorithms. *Journal of Intelligent & Robotic Systems*, 104(2):22, 2022.

Achim Ilchmann and Eugene P Ryan. Universal λ-tracking for nonlinearly-perturbed systems in the presence of noise. *Automatica*, 1994.

Aditya Intwala and Yash Parikh. A review on vertical take off and landing (vtol) vehicles. *International Journal of Innovative Research in Advanced Engineering (IJIRAE)*, 2 (2):187–191, 2015.

Daniel Ioan, Sorin Olaru, Silviu-Iulian Niculescu, Ionela Prodan, and Florin Stoican. Navigation in a multi-obstacle environment. from partition of the space to a zonotopic-based MPC. In *2019 18th European Control Conference (ECC)*, pages 1772–1777. IEEE, 2019.

Daniel Ioan, Ionela Prodan, Sorin Olaru, Florin Stoican, and Silviu-Iulian Niculescu. Mixed-integer programming in motion planning. *Annual Reviews in Control*, 51:65–87, 2021.

Alberto Isidori. *Nonlinear control systems: An introduction*. Springer, 1985.

Bhavini Jeloka, Florentina Nicolau, Adnane Saoud, and Ravi N Banavar. Data-driven control of adaptive cruise control systems using differential flatness and gaussian processes. In *2024 American Control Conference (ACC)*, pages 5094–5099. IEEE, 2024.

Zhong-Ping Jiang, Iven MY Mareels, and Yuan Wang. A lyapunov formulation of the nonlinear small-gain theorem for interconnected iss systems. *Automatica*, 32(8):1211–1215, 1996.

Pablo Jiménez, Federico Thomas, and Carme Torras. 3d collision detection: a survey. *Computers & Graphics*, 25(2):269–285, 2001.

Tor A Johansen. Computation of lyapunov functions for smooth nonlinear systems using convex optimization. *Automatica*, 36(11):1617–1626, 2000.

Martina Joševski and Dirk Abel. Flatness-based trajectory planning for the battery state of charge in hybrid electric vehicles. *IFAC-PapersOnLine*, 49(11):134–140, 2016.

Louis Jouret, Adnane Saoud, and Sorin Olaru. Safety verification of neural-network-based controllers: a set invariance approach. *IEEE Control Systems Letters*, 2023.

Pedro Julian, Jose Guivant, and Alfredo Desages. A parametrization of piecewise linear lyapunov functions via linear programming. *International Journal of Control*, 72(7-8): 702–715, 1999.

Yirmeyahu Kaminski and François Ollivier. Flat singularities of chained systems, illustrated with an aircraft model. *Computational and Applied Mathematics*, 43(3):135, 2024.

Yirmeyahu J Kaminski, Jean Lévine, and François Ollivier. Intrinsic and apparent singularities in differentially flat systems, and application to global motion planning. *Systems & Control Letters*, 113:117–124, 2018.

Christoph Kandler, Steven X Ding, Tim Koenings, Nick Weinhold, and Matthias Schultalbers. A differential flatness based model predictive control approach. In *IEEE Int. Conf. on Cont. Applications*, pages 1411–1416. IEEE, 2012.

Sertac Karaman and Emilio Frazzoli. Optimal kinodynamic motion planning using incremental sampling-based methods. In *49th IEEE conference on decision and control (CDC)*, pages 7681–7687. IEEE, 2010.

Benjamin Karg and Sergio Lucia. Efficient representation and approximation of model predictive control laws via deep learning. *IEEE Transactions on Cybernetics*, 50(9): 3866–3878, 2020.

James Kennedy and Russell Eberhart. Particle swarm optimization. In *Proceedings of ICNN'95-international conference on neural networks*, volume 4, pages 1942–1948. IEEE, 1995.

Nami Kidane, Hisakazu Nakamura, Yuh Yamashita, and Hirokazu Nishitani. Controller design for a nonlinear system with inputs restricted to a direct product of Minkowski balls. In *43rd IEEE Conf. on Decision and Control*, 2004.

Justin Koeln, Trevor J Bird, Jacob Siefert, Justin Ruths, Herschel Pangborn, and Neera Jain. zonolab: A matlab toolbox for set-based control systems analysis using hybrid zonotopes. In *2024 American Control Conference (ACC)*. IEEE, 2024.

Petar Kokotović, Hassan K Khalil, and John O'reilly. *Singular perturbation methods in control: analysis and design*. SIAM, 1999.

Ilya Kolmanovsky and Elmer G Gilbert. Theory and computation of disturbance invariant sets for discrete-time linear systems. *Mathematical problems in engineering*, 4(4): 317–367, 1998.

Ryoya Kubo, Yasuhiro Fujii, and Hisakazu Nakamura. Control lyapunov function design for trajectory tracking problems of wheeled mobile robot. *IFAC-PapersOnLine*, 53(2): 6177–6182, 2020.

Rumit Kumar, Alireza Nemati, Manish Kumar, Rajnikant Sharma, Kelly Cohen, and Franck Cazaurang. Tilting-rotor quadcopter for aggressive flight maneuvers using differential flatness based flight controller. In *Dynamic Systems and Control Conference*, volume 58295, page V003T39A006. American Society of Mechanical Engineers, 2017.

Michael J Kurtz and Michael A Henson. Feedback linearizing control of discrete-time nonlinear systems with input constraints. *International Journal of Control*, 70(4): 603–616, 1998.

Alexander Kurzhanski and István Vályi. *Ellipsoidal calculus for estimation and control*. Springer, 1997.

Michal Kvasnica, Juraj Hledík, Ivana Rauová, and Miroslav Fikar. Complexity reduction of explicit model predictive control via separation. *Automatica*, 49(6):1776–1781, 2013.

Marouane Laaziz, Florentina Nicolau, Malek Ghanes, Jean-Pierre Barbot, Robert Boisliveau, and Nadia Machkour. Fault detection, isolation, and estimation for a three-phase grid connected dc-ac inverter with lcl filters. *IEEE Control Systems Letters*, 2024.

Reza Lavaei and Leila J Bridgeman. Systematic, Lyapunov-based, safe and stabilizing controller synthesis for constrained nonlinear systems. *IEEE Trans. on Automatic Control*, 2023.

Steven M LaValle. *Planning algorithms*. Cambridge University Press, 2006.

Mircea Lazar. Flexible control Lyapunov functions. In *2009 American Control Conference*, pages 102–107. IEEE, 2009.

Mircea Lazar and Andrej Jokic. Synthesis of trajectory-dependent control lyapunov functions by a single linear program. In *International Workshop on Hybrid Systems: Computation and Control*, pages 237–251. Springer, 2009a.

Mircea Lazar and Andrej Jokic. Synthesis of trajectory-dependent control Lyapunov functions by a single linear program. In *International Workshop on Hybrid Systems: Computation and Control*. Springer, 2009b.

Mircea Lazar and Andrej Jokić. On infinity norms as Lyapunov functions for piecewise affine systems. In *13th ACM international conference on Hybrid systems: computation and control*, pages 131–140, 2010.

Yuri S Ledyaev and Eduardo D Sontag. A lyapunov characterization of robust stabilization. *Nonlinear Analysis-Series A Theory and Methods and Series B Real World Applications*, 37(7):813–840, 1999.

Jay H Lee. Model predictive control: Review of the three decades of development. *International Journal of Control, Automation and Systems*, 9:415–424, 2011.

Jean Levine. *Analysis and control of nonlinear systems: A flatness-based approach*. Springer Science & Business Media, 2009.

Jean Lévine. On necessary and sufficient conditions for differential flatness. *Applicable Algebra in Engineering, Communication and Computing*, 22(1):47–90, 2011.

Horacio Leyva, Baltazar Aguirre-Hernández, and Jesús F Espinoza. Stabilization of affine systems with polytopic control value sets. *Journal of Dynamical and Control Systems*, pages 1–13, 2023.

Boqian Li, Shiping Wen, Zheng Yan, Guanghui Wen, and Tingwen Huang. A survey on the control Lyapunov function and control barrier function for nonlinear-affine control systems. *IEEE/CAA Journal of Automatica Sinica*, 2023.

Shenggao Li, Hua Chen, Wei Zhang, and Patrick M Wensing. Quadruped robot hopping on two legs. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 7448–7455. IEEE, 2021.

Xiao Li, Yutong Li, Anouck Girard, and Ilya Kolmanovsky. System-level safety guard: Safe tracking control through uncertain neural network dynamics models. In *6th Annual Learning for Dynamics & Control Conference*, pages 968–979. PMLR, 2024.

Xingchen Li, Xifeng Gao, Wei Zhang, and Lina Hao. Smooth and collision-free trajectory generation in cluttered environments using cubic b-spline form. *Mechanism and Machine Theory*, 169:104606, 2022.

Alexandre Liapounoff. Problème général de la stabilité du mouvement. *Annales de la Faculté des sciences de l'Université de Toulouse pour les sciences mathématiques et les sciences physiques*, 9:203–474, 1907.

Daniel Limón, Teodoro Alamo, Francisco Salas, and Eduardo F Camacho. Input to state stability of min–max mpc controllers for nonlinear systems with bounded uncertainties. *Automatica*, 42(5):797–803, 2006.

Sikang Liu, Michael Watterson, Kartik Mohta, Ke Sun, Subhrajit Bhattacharya, Camillo J Taylor, and Vijay Kumar. Planning dynamically feasible trajectories for quadrotors using safe flight corridors in 3-d complex environments. *IEEE Robotics and Automation Letters*, 2(3):1688–1695, 2017.

Oskar Ljungqvist. *Motion planning and feedback control techniques with applications to long tractor-trailer vehicles*, volume 2070. Linköping University Electronic Press, 2020.

J. Lofberg. Yalmip : a toolbox for modeling and optimization in MATLAB. In *2004 IEEE International Conference on Robotics and Automation*, pages 284–289, 2004. doi: 10.1109/CACSD.2004.1393890.

Christophe Louembet, Franck Cazaurang, Ali Zolghadri, Catherine Charbonnel, and Christelle Pittet. Design of algorithms for satellite slew manoeuver by flatness and collocation. In *2007 American Control Conference*, pages 3168–3173. IEEE, 2007.

Christophe Louembet, Franck Cazaurang, Ali Zolghadri, C Charbonnel, and C Pittet. Path planning for satellite slew manoeuvres: a combined flatness and collocation-based approach. *IET control theory & applications*, 3(4):481–491, 2009.

Hao Lu, Cunjia Liu, Lei Guo, and Wen-Hua Chen. Constrained anti-disturbance control for a quadrotor based on differential flatness. *International Journal of Systems Science*, 48(6):1182–1193, 2017.

Carlos Luis and Jérôme Le Ny. Design of a trajectory tracking controller for a nanoquadcopter. Technical report, Mobile Robotics and Autonomous Systems Laboratory, Polytechnique Montreal, 2016.

Carlos E. Luis, Marijan Vukosavljev, and Angela P. Schoellig. Online trajectory generation with distributed model predictive control for multi-robot motion planning. *IEEE Robot. Autom. Lett.*, 5(2), 2020a. doi: 10.1109/LRA.2020.2964159.

Carlos E Luis, Marijan Vukosavljev, and Angela P Schoellig. Online trajectory generation with distributed model predictive control for multi-robot motion planning. *IEEE Robotics and Automation Letters*, 5(2):604–611, 2020b.

Tom Lyche, Carla Manni, and Hendrik Speleers. Foundations of spline theory: B-splines, spline approximation, and hierarchical. *Splines and PDEs: From Approximation Theory to Numerical Linear Algebra: Cetraro, Italy 2017*, 2219, 2018.

Radhakrishnan Mahadevan, Sunil K Agrawal, and Francis J Doyle III. Differential flatness based nonlinear predictive control of fed-batch bioreactors. *Control Engineering Practice*, 9(8):889–899, 2001.

Vincent Marguet, Florin Stoican, and Ionela Prodan. On the application of the schoenberg quasi-interpolant for complexity reduction in trajectory generation. In *2023 European Control Conference*, 2023.

Vincent Marguet, Cong Khanh Dinh, Florin Stoican, and Ionela Prodan. Indoor formation motion planning using b-splines parametrization and evolutionary optimization. *Control Engineering Practice*, 152:106066, 2024a.

Vincent Marguet, Florin Stoican, and Ionela Prodan. Energy-efficient trajectory planning with b-splines and the schoenberg quasi-interpolant. In *2024 63rd IEEE Conference on Decision and Control (CDC)*. IEEE, 2024b.

Elisha D Markus, John T Agee, and Adisa A Jimoh. Flat control of industrial robotic manipulators. *Robotics and Autonomous systems*, 87:226–236, 2017.

Ph Martin. Endogenous feedbacks and equivalence; systems and networks: Mathematical theory and applications. *Proc. MTNS'93*, 2:343–346, 1994.

Philippe Martin, Santosh Devasia, and Brad Paden. A different look at output tracking: Control of a vtol aircraft. *Automatica*, 32(1):101–107, 1996.

Philippe Martin, Richard M Murray, and Pierre Rouchon. Flat systems: open problems, infinite dimensional extension, symmetries and catalog. In *Advances in the control of nonlinear systems*, pages 33–57. Springer, 2001.

Matlab. MATLAB regression neural network. www.mathworks.com/help/stats/regressionneuralnetwork.html, 2021.

David Q Mayne, James B Rawlings, Christopher V Rao, and Pierre OM Scokaert. Constrained model predictive control: Stability and optimality. *Automatica*, 36(6): 789–814, 2000.

David Q Mayne, María M Seron, and SV Raković. Robust model predictive control of constrained linear systems with bounded disturbances. *Automatica*, 41(2):219–224, 2005.

David Q Mayne, Saša V Raković, Rolf Findeisen, and Frank Allgöwer. Robust output feedback model predictive control of constrained linear systems. *Automatica*, 42(7): 1217–1222, 2006.

David Q Mayne, Erric C Kerrigan, EJ Van Wyk, and Paola Falugi. Tube-based robust nonlinear model predictive control. *International journal of robust and nonlinear control*, 21(11):1341–1353, 2011.

Peter McMullen. On zonotopes. *Transactions of the American Mathematical Society*, 159:91–109, 1971.

Daniel Mellinger and Vijay Kumar. Minimum snap trajectory generation and control for quadrotors. In *2011 IEEE international conference on robotics and automation*, pages 2520–2525. IEEE, 2011.

Ali Mesbah. Stochastic model predictive control: An overview and perspectives for future research. *IEEE Control Systems Magazine*, 36(6):30–44, 2016.

Nathan Michel, Sylvain Bertrand, Sorin Olaru, Giorgio Valmorbida, and Didier Dumur. Design and flight experiments of a tube-based model predictive controller for the ar. drone 2.0 quadrotor. *IFAC-PapersOnLine*, 52(22):112–117, 2019.

Stefan S. Mihai, Florin Stoican, and Bogdan D. Ciubotaru. On the link between explicit MPC and the face lattice of the lifted feasible domain. *IFAC-PapersOnLine*, 2022. ISSN 2405-8963.

Youngjae Min, Spencer M Richards, and Navid Azizan. Data-driven control with inherent lyapunov stability. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pages 6032–6037. IEEE, 2023.

Vincent Morio. *Contribution au développement d'une loi de guidage autonome par platitude. Application à une mission de rentrée atmosphérique.* PhD thesis, Université Sciences et Technologies-Bordeaux I, 2009.

Vincent Morio, Franck Cazaurang, and Jean Lévine. On the computation of *pi*-flat outputs for linear time-delay systems. *arXiv preprint arXiv:0910.3619*, 2009a.

Vincent Morio, Franck Cazaurang, and Philippe Vernis. Flatness-based hypersonic reentry guidance of a lifting-body vehicle. *Control Engineering Practice*, 17(5):588–596, 2009b.

Vincent Morio, Franck Cazaurang, Ali Zolghadri, and Philippe Vernis. A new path planner based on flatness approach application to an atmospheric reentry mission. In *2009 European Control Conference (ECC)*, pages 3021–3026. IEEE, 2009c.

Hugues Mounier and Joachim Rudolph. Flatness-based control of nonlinear delay systems: A chemical reactor example. *International Journal of Control*, 71(5):871–890, 1998.

Hugues Mounier, Silviu-Iulian Niculescu, Arben Cela, and Marcel Stefan Geamanu. Flatness-based longitudinal vehicle control with embedded torque constraint. *IMA Journal of Mathematical Control and Information*, 36(3):729–744, 2019.

Mark W Mueller and Raffaello D'Andrea. A model predictive controller for quadrocopter state interception. In *2013 European Control Conference (ECC)*, pages 1383–1389. IEEE, 2013.

Sayak Mukherjee, Ján Drgoňa, Aaron Tuor, Mahantesh Halappanavar, and Draguna Vrabie. Neural lyapunov differentiable predictive control. In *2022 IEEE 61st Conference on Decision and Control (CDC)*, pages 2097–2104. IEEE, 2022.

Varun Murali, Igor Spasojevic, Winter Guerra, and Sertac Karaman. Perception-aware trajectory generation for aggressive quadrotor flight using differential flatness. In *2019 American Control Conference (ACC)*, pages 3936–3943. IEEE, 2019.

Richard M Murray, Muruhan Rathinam, and Willem Sluis. Differential flatness of mechanical control systems: A catalog of prototype systems. In *ASME international mechanical engineering congress and exposition*, pages 349–357. Citeseer, 1995.

Souransu Nandi and Tarunraj Singh. Differential flatness based run-to-run control of blood glucose for people with type 1 diabetes. In *2019 American Control Conference (ACC)*, pages 4773–4778. IEEE, 2019.

Tiago P Nascimento and Martin Saska. Position and attitude control of multi-rotor aerial vehicles: A survey. *Annual Reviews in Control*, 48:129–146, 2019.

Vesna Nevistic and L Del Re. Feasible suboptimal model predictive control for linear plants with state dependent constraints. In *American Control Conference-ACC'94*, volume 3, pages 2862–2866. IEEE, 1994.

Huu Thien Nguyen, Ngoc Thinh Nguyen, and Ionela Prodan. Notes on the terminal region enlargement of a stabilizing nmpc design for a multicopter. *Automatica*, 159: 111375, 2024.

Ngoc Thinh Nguyen. *Reliable hierarchical control for multicopter systems.* PhD thesis, Université Grenoble Alpes, 2019.

Ngoc Thinh Nguyen and Ionela Prodan. Stabilizing a multicopter using an nmpc design with a relaxed terminal region. *IFAC-PapersOnLine*, 54(6):126–132, 2021.

Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefevre. Multi-layer optimization-based control design for quadcopter trajectory tracking. In *2017 25th Mediterranean Conference on Control and Automation (MED)*, pages 601–606. IEEE, 2017a.

Ngoc Thinh Nguyen, Ionela Prodan, Florin Stoican, and Laurent Lefevre. Reliable nonlinear control for quadcopter trajectory tracking through differential flatness. *IFAC-PapersOnLine*, 50(1):6971–6976, 2017b.

Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefèvre. Effective angular constrained trajectory generation for thrust-propelled vehicles. In *2018 European Control Conference (ECC)*, pages 1833–1838. IEEE, 2018.

Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefevre. A stabilizing NMPC design for thrust-propelled vehicles dynamics via feedback linearization. In *2019 American Control Conference*, pages 2909–2914. IEEE, 2019.

Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefèvre. Flat trajectory design and tracking with saturation guarantees: a nano-drone application. *International Journal of Control*, 93(6):1266–1279, 2020a.

Ngoc Thinh Nguyen, Ionela Prodan, and Laurent Lefèvre. Stability guarantees for translational thrust-propelled vehicles dynamics through nmpc designs. *IEEE Transactions on Control Systems Technology*, 29(1):207–219, 2020b.

Ngoc Thinh Nguyen, Pranav Tej Gangavarapu, Arne Sahrhage, Georg Schildbach, and Floris Ernst. Navigation with polytopes and b-spline path planner. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pages 5695–5701. IEEE, 2023.

Florentina Nicolau and Alessio Iovine. Flatness of interconnected linear systems and applications to electrical systems. *IFAC-PapersOnLine*, 55(40):241–246, 2022.

Florentina Nicolau and Witold Respondek. Flatness of multi-input control-affine systems linearizable via one-fold prolongation. *SIAM Journal on Control and Optimization*, 55(5):3171–3203, 2017.

Florentina Nicolau, Witold Respondek, and Jean-Pierre Barbot. Constructing flat inputs for two-output systems. In *The 23rd International Symposium on Mathematical Theory of Networks and Systems*, 2018.

Florentina Nicolau, Witold Respondek, and Jean-Pierre Barbot. Flat inputs: theory and applications. *SIAM Journal on Control and Optimization*, 58(6):3293–3321, 2020.

Marco Nicotra. *Constrained control of nonlinear systems: The explicit reference governor and its application to unmanned aerial vehicles*. PhD thesis, Université libre de Bruxelles, 2016.

Oladapo Ogunbodede, Souransu Nandi, and Tarunraj Singh. Periodic control of unmanned aerial vehicles based on differential flatness. In *2018 Annual American Control Conference (ACC)*, pages 2681–2686, 2018. doi: 10.23919/ACC.2018.8430793.

Oladapo Ogunbodede, Souransu Nandi, and Tarunraj Singh. Endurance maximizing periodic control of unmanned aerial vehicles. In *2019 American Control Conference (ACC)*, pages 3617–3622. IEEE, 2019a.

Oladapo Ogunbodede, Souransu Nandi, and Tarunraj Singh. Periodic control of unmanned aerial vehicles based on differential flatness. *Journal of Dynamic Systems, Measurement, and Control*, 141(7):071003, 2019b.

Anibal Ollero, Marco Tognon, Alejandro Suarez, Dongjun Lee, and Antonio Franchi. Past, present, and future of aerial robotic manipulators. *IEEE Transactions on Robotics*, 38(1):626–645, 2021.

Joshua Ortiz, Alyssa Vellucci, Justin Koeln, and Justin Ruths. Hybrid zonotopes exactly represent relu neural networks. In *2023 62nd IEEE Conference on Decision and Control (CDC)*, pages 5351–5357. IEEE, 2023.

Gerardo Ortiz-Torres, Pedro Castillo, Felipe DJ Sorcia-Vázquez, Jesse Y Rumbo-Morales, Jorge A Brizuela-Mendoza, Javier De La Cruz-Soto, and Mario Martínez-García. Fault estimation and fault tolerant control strategies applied to vtol aerial vehicles with soft and aggressive actuator faults. *IEEE Access*, 8:10649–10661, 2020.

Jia Pan, Liangjun Zhang, and Dinesh Manocha. Collision-free and smooth trajectory computation in cluttered environments. *The International Journal of Robotics Research*, 31(10):1155–1175, 2012.

Thanh Hung Pham, Ionela Prodan, Denis Genon-Catalot, and Laurent Lefevre. Efficient energy management for an elevator system under a constrained optimization framework. In *2015 19th International Conference on System Theory, Control and Computing (ICSTCC)*, pages 613–618. IEEE, 2015.

Elijah Polak. *Optimization: algorithms and consistent approximations*, volume 124. Springer Science & Business Media, 2012.

Stephen Prajna, Antonis Papachristodoulou, and Fen Wu. Nonlinear control synthesis by sum of squares optimization: A Lyapunov-based approach. In *5th Asian control conference*, volume 1, pages 157–165. IEEE, 2004.

Ionela Prodan, Sorin Olaru, Ricardo Bencatel, Joao Borges de Sousa, Cristina Stoica, and Silviu-Iulian Niculescu. Receding horizon flight control for trajectory tracking of autonomous aerial vehicles. *Control Engineering Practice*, 21(10):1334–1349, 2013.

Ionela Prodan, Sorin Olaru, Fernando ACC Fontes, Fernando Lobo Pereira, João Borges de Sousa, Cristina Stoica Maniu, and Silviu-Iulian Niculescu. Predictive control for path-following. from trajectory generation to the parametrization of the discrete tracking sequences. *Developments in Model-Based Optimization and Control: Distributed Control and Industrial Applications*, pages 161–181, 2015a.

Ionela Prodan, Florin Stoican, Sorin Olaru, and Silviu-Iulian Niculescu. *Mixed-integer representations in control design: Mathematical foundations and applications*. Springer, 2015b.

Ionela Prodan, Florin Stoican, and Christophe Louembet. Necessary and sufficient LMI conditions for constraints satisfaction within a B-spline framework. In *IEEE 58th Conf. on Decision and Control*. IEEE, 2019.

Dimitrios Pylorof and Efstathios Bakolas. Nonlinear control under polytopic input constraints with application to the attitude control problem. In *2015 American Control Conference (ACC)*, pages 4555–4560. IEEE, 2015.

Chen Qian, Rui Chen, Peiyao Shen, Yongchun Fang, Jifu Yan, and Tiefeng Li. Trajectory generation and tracking control for flapping wing robot three-dimensional flight. *IEEE/ASME Transactions on Mechatronics*, 2024.

Sasa V Rakovic, Eric C Kerrigan, Konstantinos I Kouramas, and David Q Mayne. Invariant approximations of the minimal robust positively invariant set. *IEEE Trans. on Auto. Control*, 50(3):406–410, 2005.

S Ramaswamy, TJ Cutright, and HK Qammar. Control of a continuous bioreactor using model predictive control. *Process Biochemistry*, 40(8):2763–2770, 2005.

Hadi Ravanbakhsh and Sriram Sankaranarayanan. Learning control lyapunov functions from counterexamples and demonstrations. *Autonomous Robots*, 43:275–307, 2019.

Abhijeet Ravankar, Ankit A Ravankar, Yukinori Kobayashi, Yohei Hoshino, and Chao-Chung Peng. Path smoothing techniques in robot navigation: State-of-the-art, current and future challenges. *Sensors*, 18(9):3170, 2018.

Stephane Redon, Young J Kim, Ming C Lin, and Dinesh Manocha. Fast continuous collision detection for articulated models. In *ACM Symposium on Solid Modeling and Applications, SM*, pages 145–156, 2004.

Charles Richter, Adam Bry, and Nicholas Roy. Polynomial trajectory planning for aggressive quadrotor flight in dense indoor environments. In *Robotics Research: The 16th International Symposium ISRR*, pages 649–666. Springer, 2016.

R Tyrrell Rockafellar. *Convex analysis*. Princeton University Press, 1997.

Muhammad Zakiyullah Romdlony and Bayu Jayawardhana. Stabilization with guaranteed safety using control Lyapunov-barrier function. *Automatica*, 2016.

Nicholas J Rose. Linear algebra and its applications (gilbert strang). *SIAM Review*, 24 (4):499–501, 1982.

I Michael Ross and Fariba Fahroo. Issues in the real-time computation of optimal control. *Mathematical and computer modelling*, 43(9-10):1172–1188, 2006.

Gauthier Rousseau, Cristina Stoica Maniu, Sihem Tebbani, Mathieu Babel, and Nicolas Martin. Minimum-time b-spline trajectories with corridor constraints. application to cinematographic quadrotor flight plans. *Control Engineering Practice*, 89:190–203, 2019.

Matteo Rubagotti, Luca Zaccarian, and Alberto Bemporad. A Lyapunov method for stability analysis of piecewise-affine systems over non-invariant domains. *International Journal of Control*, 89(5):950–959, 2016.

Ji-Chul Ryu and Sunil K Agrawal. Differentially flat mobile manipulators mounted with an under-actuated vertical arm. In *2010 IEEE International Conference on Robotics and Automation*, pages 5201–5206. IEEE, 2010.

Ali Saberi, Anton A Stoorvogel, and Peddapullaiah Sannuti. *Control of linear systems with regulation and input constraints*. Springer Science & Business Media, 2012.

Pouya Samanipour and Hasan A Poonawala. Stability analysis and controller synthesis using single-hidden-layer relu neural networks. *IEEE Transactions on Automatic Control*, 69(1):202–213, 2023.

Jose Ricardo Sanchez-Ibanez, Carlos J Pérez-del Pulgar, and Alfonso García-Cerezo. Path planning for autonomous mobile robots: A review. *Sensors*, 21(23):7898, 2021.

Fendy Santoso, Matthew A Garratt, and Sreenatha G Anavatti. Hybrid PD-fuzzy and pd controllers for trajectory tracking of a quadrotor unmanned aerial vehicle: Autopilot designs and real-time flight tests. *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, 51(3):1817–1829, 2019.

Kurt Schlacher and Markus Schöberl. Construction of flat outputs by reduction and elimination. *IFAC Proceedings Volumes*, 40(12):693–698, 2007.

Philipp Schubert and Dirk Abel. Flatness-based model predictive payload control for offshore cranes. In *2023 European Control Conference (ECC)*, pages 1–8. IEEE, 2023.

Dena Senior. Qualisys track manager: User manual. *Laboratory Memorandum*, 2004.

Hoseong Seo, Suseong Kim, and H Jin Kim. Locally optimal trajectory planning for aerial manipulation in constrained environments. In *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 1719–1724. IEEE, 2017.

Ahmed Shahin, Hassan Moussa, Ivano Forrisi, Jean-Philippe Martin, Babak Nahid-Mobarakeh, and Serge Pierfederici. Reliability improvement approach based on flatness control of parallel-connected inverters. *IEEE Transactions on Power Electronics*, 32(1):681–692, 2016.

Jinhua She, Kou Miyamoto, Qing-Long Han, Min Wu, Hiroshi Hashimoto, and Qing-Guo Wang. Generalized-extended-state-observer and equivalent-input-disturbance methods for active disturbance rejection: Deep observation and comparison. *IEEE/CAA Journal of Automatica Sinica*, 10(4):957–968, 2022.

Hebertt Sira-Ramírez. From flatness, gpi observers, gpi control and flat filters to observer-based adrc. *Control Theory and Technology*, 16(4):249–260, 2018.

Hebertt Sira-Ramirez and Sunil K Agrawal. *Differentially flat systems*. Crc Press, 2018.

Hebertt Sira-Ramírez, Alberto Luviano-Juárez, Mario Ramirez-Neria, and R Garrido-Moctezuma. Flat filtering: A classical approach to robust control of nonlinear systems. In *2016 American Control Conference (ACC)*, pages 3844–3849. IEEE, 2016.

Hebertt Sira-Ramirez, Eric William Zurita-Bustamante, and Congzhi Huang. Equivalence among flat filters, dirty derivative-based pid controllers, adrc, and integral reconstructor-based sliding mode control. *IEEE Transactions on control systems technology*, 28(5):1696–1710, 2019.

Terrence Skibik, Dominic Liao-McPherson, Torbjørn Cunis, Ilya Kolmanovsky, and Marco M Nicotra. A feasibility governor for enlarging the region of attraction of linear model predictive controllers. *IEEE Transactions on Automatic Control*, 67(10): 5501–5508, 2021.

Terrence Skibik, Dominic Liao-McPherson, and Marco M Nicotra. A terminal set feasibility governor for linear model predictive control. *IEEE Transactions on Automatic Control*, 68(8):5089–5095, 2022.

M Soheil-Hamedani, Majid Zandi, Roghayeh Gavagsaz-Ghoachani, Babak Nahid-Mobarakeh, and Serge Pierfederici. Flatness-based control method: A review of its applications to power systems. In *2016 7th Power Electronics and Drive Systems Technologies Conference (PEDSTC)*, pages 547–552. IEEE, 2016.

Eduardo D Sontag. A lyapunov-like characterization of asymptotic controllability. *SIAM journal on control and optimization*, 21(3):462–471, 1983.

Eduardo D Sontag. A 'universal'construction of Artstein's theorem on nonlinear stabilization. *Systems & control letters*, 13(2), 1989.

Eduardo D Sontag and Hector J Sussmann. Nonlinear output feedback design for linear systems with saturating controls. In *29th IEEE Conference on Decision and Control*, pages 3414–3416. IEEE, 1990.

Tom Robert Vince Steentjes, Mircea Lazar, and Alina Ionela Doban. Construction of continuous and piecewise affine feedback stabilizers for nonlinear systems. *IEEE Transactions on Automatic Control*, 66(9):4059–4068, 2020.

Florin Stoican, Cristian Oară, and Morten Hovd. RPI approximations of the mRPI set characterizing linear dynamics with zonotopic disturbances. In *Developments in Model-Based Optimization and Control*, pages 361–377. Springer, 2015a.

Florin Stoican, Ionela Prodan, and Dan Popescu. Flat trajectory generation for waypoints relaxations and obstacle avoidance. In *2015 23rd Mediterranean Conference on Control and Automation (MED)*, pages 695–700. IEEE, 2015b.

Florin Stoican, Ionela Prodan, Esten Ingar Grøtli, and Ngoc Thinh Nguyen. Inspection trajectory planning for 3d structures under a mixed-integer framework. In *2019 IEEE 15th International Conference on Control and Automation (ICCA)*, pages 1349–1354. IEEE, 2019.

Florin Stoican, Alexandru Postolache, and Ionela Prodan. Nurbs-based trajectory design for motion planning in a multi-obstacle environment. In *2021 European Control Conference (ECC)*, pages 2014–2019. IEEE, 2021.

Florin Stoican, Vincent Marguet, Dan Popescu, Ionela Prodan, and Loretta Ichim. On the energy consumption of a quadcopter navigating in an orchard environment. In *2024 32nd Mediterranean Conference on Control and Automation (MED)*, pages 280–285. IEEE, 2024.

Alimzhan Sultangazin, Luigi Pannocchi, Lucas Fraile, and Paulo Tabuada. Learning to control known feedback linearizable systems from demonstrations. *IEEE Transactions on Automatic Control*, 69(1):189–201, 2023.

Sihao Sun, Angel Romero, Philipp Foehn, Elia Kaufmann, and Davide Scaramuzza. A comparative study of nonlinear MPC and differential-flatness-based control for quadrotor agile flight. *IEEE Transactions on Robotics*, 2022.

Amit Surana. Koopman operator based observer synthesis for control-affine nonlinear systems. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 6492–6499. IEEE, 2016.

Fajar Suryawan. *Constrained trajectory generation and fault-tolerant control based on differential flatness and b-splines*. PhD thesis, School of Electrical Engineering and Computer Science, The University of Newcastle, Australia, 2012.

Ezra Tal and Sertac Karaman. Accurate tracking of aggressive quadrotor trajectories using incremental nonlinear dynamic inversion and differential flatness. *IEEE Transactions on Control Systems Technology*, 29(3):1203–1218, 2020.

Ezra Tal and Sertac Karaman. Global incremental flight control for agile maneuvering of a tailsitter flying wing. *Journal of Guidance, Control, and Dynamics*, 45(12):2332–2349, 2022.

Ezra Tal, Gilhyun Ryou, and Sertac Karaman. Aerobatic trajectory generation for a vtol fixed-wing aircraft using differential flatness. *IEEE Transactions on Robotics*, 2023.

Yoshihiro Tanaka, Masao Fukushima, and Toshihide Ibaraki. A comparative study of several semi-infinite nonlinear programming algorithms. *European Journal of Operational Research*, 36(1):92–100, 1988.

Chin Pei Tang, Patrick T Miller, Venkat N Krovi, Ji-Chul Ryu, and Sunil K Agrawal. Differential-flatness-based planning and control of a wheeled mobile manipulator—theory and experiment. *IEEE/ASME Transactions on Mechatronics*, 16 (4):768–773, 2010.

Emmanuel Tang, Wei Jun Ang, Kian Wee Tan, and Shaohui Foong. Harnessing the differential flatness of monocopter dynamics for the purpose of trajectory tracking in a stable invertible coaxial actuated rotorcraft (sicaro). In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 6145–6151. IEEE, 2024.

Andrew R Teel. Global stabilization and restricted tracking for multiple integrators with bounded controls. *Systems & control letters*, 18(3):165–171, 1992.

Phatiphat Thounthong, Pongsiri Mungporn, Damien Guilbert, Noureddine Takorabet, Serge Pierfederici, Babak Nahid-Mobarakeh, Yihua Hu, Nicu Bizon, Yigeng Huangfu, and Poom Kumam. Design and control of multiphase interleaved boost converters-based on differential flatness theory for pem fuel cell multi-stack applications. *International Journal of Electrical Power and Energy Systems*, 124, 2021.

Cristian Tiriolo and Walter Lucia. A set-theoretic control approach to the trajectory tracking problem for input-output linearized wheeled mobile robots. *IEEE Control Systems Letters*, 2023.

Marco Tognon and Antonio Franchi. *Theory and applications for control of aerial robots in physical interaction through tethers*, volume 140. Springer Nature, 2020.

Iris Ballesteros Tolosana. *Commande Prédictive pour le Véhicule Autonome*. PhD thesis, Université Paris Saclay (COmUE), 2018.

Stephanie Tsuei and Mark B Milam. Trajectory generation for constrained differentially flat systems with time and frequency domain objectives. In *2016 IEEE 55th Conference on Decision and Control (CDC)*, pages 4172–4177. IEEE, 2016.

Tilman Utz and Andreas Kugi. Flatness-based feedforward control design of a system of parabolic pdes based on finite difference semi-discretization. *PAMM*, 12(1):731–732, 2012.

Wannes Van Loock, Goele Pipeleers, and Jan Swevers. Time-optimal path planning for flat systems with application to a wheeled mobile robot. In *9th International Workshop on Robot Motion and Control*, pages 192–196. IEEE, 2013.

Michiel Jacques Van Nieuwstadt. *Trajectory generation for nonlinear control systems*. PhD thesis, California Institute of Technology, 1997.

Subbarao Varigonda, Tryphon T Georgiou, and Prodromos Daoutidis. Numerical solution of the optimal periodic control problem using differential flatness. *IEEE Transactions on Automatic Control*, 49(2):271–275, 2004.

Diederik Verscheure, Bram Demeulenaere, Jan Swevers, Joris De Schutter, and Moritz Diehl. Practical time-optimal trajectory planning for robots: a convex optimization approach. *IEEE Transactions on Automatic Control*, 14:28, 2008.

Richard B Vinter. *Optimal control*. Springer, 2010.

Richard A Vitale. L_p metrics for compact, convex sets. *Journal of Approximation Theory*, 45(3):280–287, 1985.

Andreas Wächter and Lorenz T Biegler. On the implementation of an interior-point filter line-search algorithm for large-scale nonlinear programming. *Mathematical programming*, 106:25–57, 2006.

Rui Wang and Jinkun Liu. Trajectory tracking control of a 6-dof quadrotor uav with input saturation via backstepping. *Journal of the Franklin Institute*, 355(7):3288–3309, 2018.

Zejiang Wang, Jingqiang Zha, and Junmin Wang. Autonomous vehicle trajectory following: A flatness model predictive control approach with hardware-in-the-loop verification. *IEEE Transactions on Intelligent Transportation Systems*, 22(9):5613–5623, 2021. doi: 10.1109/TITS.2020.2987987.

Keith B Wipke, Matthew R Cuddy, and Steven D Burch. Advisor 2.1: A user-friendly advanced powertrain simulation using a combined backward/forward approach. *IEEE transactions on vehicular technology*, 48(6):1751–1761, 1999.

Zhe Wu, Fahad Albalawi, Zhihao Zhang, Junfeng Zhang, Helen Durand, and Panagiotis D Christofides. Control lyapunov-barrier function-based model predictive control of nonlinear systems. *Automatica*, 109:108508, 2019.

Hui Xie and Darryl Veitch. Nested saturation control of multiple vector integrators and its application to motion control of uavs. *International Journal of Robust and Nonlinear Control*, 30(1):246–265, 2020.

Xiaojie Xiu and Ju Zhang. Grid kd tree approach for point location in polyhedral data sets–application to explicit mpc. *International Journal of Control*, 93(4):872–880, 2020.

Yuh Yamashita, Ryosuke Matsukizono, and Koichi Kobayashi. Asymptotic stabilization of nonlinear systems with convex-polytope input constraints by continuous input. *Automatica*, 138:110032, 2022.

Xiaoxia Yang and Yi Huang. Capabilities of extended state observer for estimating uncertainties. In *2009 American control conference*, pages 3700–3705. IEEE, 2009.

Dmitry Yarotsky. Error bounds for approximations with deep relu networks. *Neural Networks*, 94:103–114, 2017.

Ze Yu, Naiming Qi, Zheng Li, Tong Lin, Yuxuan Yao, Jianfeng Wang, and Mingying Huo. Optimization of multi-target continuous dynamic trajectory for unmanned aerial vehicles. *Aerospace Science and Technology*, 150:108958, 2024.

Burak Yüksel, Gabriele Buondonno, and Antonio Franchi. Differential flatness and control of protocentric aerial manipulators with any number of arms and mixed rigid-/elastic-joints. In *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 561–566. IEEE, 2016.

Igyso Zafeiratou, Ionela Prodan, Laurent Lefèvre, and Laurent Piétrac. Meshed dc microgrid hierarchical control: A differential flatness approach. *Electric Power Systems Research*, 180:106133, 2020.

Antonios Zagaris, Hans G Kaper, and Tasso J Kaper. Fast and slow dynamics for the computational singular perturbation method. *Multiscale Modeling & Simulation*, 2 (4):613–638, 2004.

Melanie Nicole Zeilinger, Colin Neil Jones, and Manfred Morari. Real-time suboptimal model predictive control using a combination of explicit mpc and online optimization. *IEEE Transactions on Automatic Control*, 2011.

Jun Zeng, Prasanth Kotaru, Mark W Mueller, and Koushil Sreenath. Differential flatness based path planning with direct collocation on hybrid modes for a quadrotor with a cable-suspended payload. *IEEE Robotics and Automation Letters*, 5(2):3074–3081, 2020.

Jun Zeng, Bike Zhang, and Koushil Sreenath. Safety-critical model predictive control with discrete-time control barrier function. In *2021 American Control Conference (ACC)*, pages 3882–3889. IEEE, 2021.

Xuetao Zhang, Yongchun Fang, Xuebo Zhang, Peiyao Shen, Jingqi Jiang, and Xiang Chen. Attitude-constrained time-optimal trajectory planning for rotorcrafts: Theory and application to visual servoing. *IEEE/ASME Transactions on Mechatronics*, 25 (4):1912–1921, 2020. doi: 10.1109/TMECH.2020.2993617.

Yuhao Zhang and Xiangru Xu. Reachability analysis and safety verification of neural feedback systems via hybrid zonotopes. In *2023 American Control Conference (ACC)*, pages 1915–1921. IEEE, 2023.

Di Zhao, Jayanth Krishnamurthi, Farhan Gandhi, and Sandipan Mishra. Differential flatness based trajectory generation for time-optimal helicopter shipboard landing. In *2018 Annual American Control Conference (ACC)*, pages 2243–2250. IEEE, 2018.